

AFS-KEX: pMAKE-BW and pUAKE-BW

Algorithm Specifications and Supporting Documentation (version 1.0)

Yunlei Zhao, Biming Zhou, Zhixiang Zhao, Chaofeng Guan, Jian Weng,
Songlin Li, Yifan Dong, Boyue Fang, Hanyu Wei, Wenqian Li, Weizhi Ao
Xinsheng Zhang, Wenbo Guo, Jieyu Zheng, Zhenhao Zhang, Xiao Li

June 29, 2026

Table of Contents

1	Introduction	3
1.1	Background	3
1.2	Overview of the Submitted AFS-KEX Proposals	4
1.3	Claimed Characteristics	5
2	Specification and Design Overview of pMAKE-BW and pUAKE-BW	7
2.1	BW-KEM	8
2.2	pMAKE-BW	10
2.3	pUAKE-BW	11
2.4	Security Analysis Overview	12
2.5	Technical Overview and Security Rationale	13
2.6	Parameter Selection	16
2.7	Detailed Discussion on Claimed Characteristics	18
3	Analysis with respect to known attacks	24
3.1	Attacks against the underlying MLWE problem	24
3.2	Attacks exploiting decryption failures	25
4	Efficiency and Performance Evaluation	26
4.1	Reference and AVX2-optimized Benchmarking	26
4.2	ARM Cortex-M4 Benchmarking	28
5	Preliminaries	29
5.1	Cryptographic Primitives	29
5.2	(Quantum-Accessible) Random Oracle Model	30
6	Authenticated Forward-Secure KEM and Its Generic Construction	31
6.1	Authenticated Forward-Secure KEM	32
6.2	Authenticated Forward-Secure PKE	35
6.3	A General Construction of CCA-Secure AFS-KEM from CPA-Secure AFS-PKE	36
6.4	Security Analysis in the Random Oracle Model	37
6.5	Security Analysis in the Quantum-Accessible Random Oracle Model	40
7	AFS-KEM-BW: AFS-KEM Instantiations from BW-KEM	44
7.1	AFS-PKE-BW	44
7.2	AFS-KEM-BW	52
8	The post-eCK-PFS Security Model	55
8.1	Model syntax	55
8.2	Adversarial Capabilities	56
8.3	Match Soundness, Key Secrecy Security, and Freshness Predicates	56
8.4	Explicit Authentication	57
9	Authenticated Key Exchanges based on AFS-KEMs	58
9.1	The pMAKE Protocol and Freshness Predicate F_{pMAKE}	58
9.2	Key Secrecy	60
9.3	Match Soundness	63
9.4	Explicit Authentication	63
10	Full Security Analysis for pUAKE	63
10.1	The pUAKE Protocol and Freshness Predicate F_{pUAKE}	63
10.2	Key Secrecy of the pUAKE protocols	64
10.3	Full Analysis for Lemmas 10.3, 10.4, 10.5, and 10.7	66
10.4	Init-Explicit Authentication Analysis for pUAKE	75
11	Full Security Analysis for pMAKE	77
11.1	Key Secrecy of the pMAKE protocols	77
11.2	Full Analysis for Lemmas 11.1, 11.2, 11.3, and 11.4	80
11.3	Explicit Authentication Analysis for pMAKE	94

1 Introduction

1.1 Background

Post-Quantum Cryptography. Classical public-key cryptographic systems based on RSA [RSA78] and Diffie–Hellman [DH76] face a fundamental threat from quantum computing due to Shor’s algorithm [Sho94]. To address this challenge, the U.S. National Institute of Standards and Technology (NIST) initiated a post-quantum cryptography standardization process in 2016, aiming to standardize public-key encryption, key encapsulation mechanisms (KEMs), and digital signatures resistant to quantum adversaries. In 2024, NIST published its first PQC standards, including ML-KEM, ML-DSA, and SLH-DSA [Nat24b, Nat24a, Nat24c]. In 2025, NIST additionally selected the code-based KEM HQC [MAB⁺25] as a complementary alternative to lattice-based ML-KEM, reflecting an emphasis on cryptographic diversity.

Signature-Free Post-Quantum Authenticated Key Exchange. Authenticated key exchange (AKE) is a fundamental primitive in cryptography that enables two parties to establish a shared session key [BR94]. With the rapid advancement of post-quantum cryptography, designing post-quantum (PQ) secure AKEs has become a central research focus [Nat17, SSW20, SFG25]. Authentication in PQ AKEs can be realized using PQ digital signatures. However, PQ signature schemes such as ML-DSA incur significantly higher communication and computational overhead than PQ KEMs such as ML-KEM at comparable security levels. As a result, KEM-based authentication approaches have emerged as an attractive alternative for constructing PQ AKE protocols and have been widely studied in the literature [BCGP08, FSXY12, FSXY13, HKSU20, LL23, PWZ23, PZ23, BCL⁺26]. Moreover, such approaches have also been explored in practical protocol designs, including PQ WireGuard [HNS⁺21, HKNW25a], KEMTLS [SSW20], and PQ IKE [WS25].

To achieve (perfect) forward secrecy and mutual (resp. unilateral) authentication, KEM-based AKE constructions [FSXY12, HKSU20] typically require **three** (resp. **two**) KEM instances¹ in parallel: one CPA-secure KEM w.r.t. an ephemeral public key and two (resp. one) CCA-secure KEMs w.r.t. the long-term public keys. As a result, such constructions entail the transmission of three (resp. two) PQ KEM ciphertexts, leading to substantial communication overhead in the post-quantum setting, since PQ KEMs have much larger public keys and ciphertexts than their classical counterparts. For example, ML-KEM-512 employs public keys of 800 bytes and ciphertexts of 768 bytes for 128-bit security, whereas the classical Curve25519-based DH-KEM [LHT16] uses public keys and ciphertexts of only 32 bytes. To mitigate this cost, a line of work [XLL⁺18, HKNW25a, BCL⁺26] has studied how to reduce the overall length of two ciphertexts arising in KEM-based AKEs. Recall that each ML-KEM ciphertext consists of a pair $(\mathbf{u}, v)$, where $\mathbf{u}$ is the dominant polynomial vector and v is a polynomial element. The basic idea in prior work is to let two KEM ciphertexts share a common component, thereby reducing the communication bandwidth. In particular, the work of [XLL⁺18] proposes Twin-Kyber, derived from ML-KEM/Kyber [Nat24b, BDK⁺18], which reduces ciphertext size by combining the two v polynomial components into a single polynomial. Subsequent independent works propose more compact constructions derived from ML-KEM, including Maul proposed in [BCL⁺26] and Rebar proposed in [HKNW25a]². Both constructions achieve further ciphertext savings by reusing the dominant polynomial vector $\mathbf{u}$ [KKPP20]. The security of Maul and Rebar is proven under the Hint-MLWE assumption [KLSS23], which is relatively new and not extensively studied compared to the standard MLWE assumption [LS15]. Moreover, their concrete parameter choices in practical instantiations additionally rely on some heuristic assumptions [DJK⁺25, HKNW25a, BCL⁺26]. In particular, all these works [XLL⁺18, HKNW25a, BCL⁺26] require modifying the underlying structure of ML-KEM in accordance with two dependent ciphertexts. Notably, the frameworks of [FSXY12, HKSU20] are also not directly compatible with the ML-KEM standard: the former requires changing how randomness is derived in the Encaps algorithm, while the latter relies on a PKE-level formulation rather than a direct KEM interface.

¹ The protocols in [HKSU20] are based on PKEs rather than KEMs, and apply the Fujisaki–Okamoto transform at the protocol level to reduce computational overhead, while the communication costs remain essentially unchanged.

² The work of [HKNW25a] formulates Rebar in the Reinforced-KEM setting and demonstrates its application to PQ WireGuard.

Limitations and Open Challenges in Signature-Free PQ AKE. We summarize the limitations and open challenges in existing signature-free PQ AKE protocols as follows:

- To the best of our knowledge, no PFS-secure AKE protocol achieves mutual (resp. unilateral) authentication while exchanging only two (resp. one) standard KEM ciphertext(s)³. Here, “standard” refers to native ciphertexts (e.g., those of ML-KEM). This has remained a long-standing open question in the literature.
- For an *efficient* construction of AKE from KEMs, to the best of our knowledge, there is still no lattice-based PFS-secure AKE protocol that achieves computational symmetry. This constitutes another long-standing open problem in lattice-based PQC.⁴ This limitation stems from the asymmetric nature of lattice-based key encapsulation mechanisms. Nevertheless, computational symmetry remains desirable for practical AKE deployment [BMS19], particularly when both roles may run on constrained platforms such as smart cards or sensor nodes, as it can simplify implementation, facilitate dataflow optimization, and reduce code size, circuit area, and reconfiguration overhead.
- Existing analyses of PQ AKEs typically assume that the peer’s identity, and hence the peer’s long-term public key, is fixed and known at or before session initiation. This pre-specified-identity viewpoint is convenient for formal modeling, but it does not reflect most deployed authenticated handshake protocols in practice. As illustrated by Internet Key Exchange (IKE) protocols and their formal analyses [CH98, CK01, CK02, KHN⁺14], a party may be activated with only partial information about its peer, such as an IP address, and learn the authenticated peer identity only during protocol execution. The same phenomenon appears in TLS-style deployments: a client is often invoked with a service name, URL, or IP address, while the authenticated identity and the corresponding long-term public key are obtained and validated from the certificate chain during the handshake [Res18]. Also, even in TLS-style settings, a client usually does not interact with a fixed server, and thus it is not reasonable to assume a client knows in advance the server’s public key before the session begins. Indeed, in TLS, the server’s certificate and public key are sent with ServerHello.

This more general and practically common formulation is known as the *post-specified identity* (post-id) setting [CK02].⁵ To the best of our knowledge, no general PQ AKE protocol has been explicitly designed and formally analyzed in the post-id setting.

1.2 Overview of the Submitted AFS-KEX Proposals

Framework overview. *AFS-KEX* denotes a family of signature-free, post-quantum authenticated key-exchange protocols in the post-specified-identity setting [CK02]. The general framework underlying AFS-KEX, including pMAKE and pUAKE, was recently introduced in work [ZZZ⁺26]. The framework is designed to achieve the following objectives:

1. exchange of at most two standard KEM ciphertexts and two standard KEM public keys,
2. computational symmetry between communicating parties,
3. perfect forward secrecy (PFS),
4. explicit authentication and key confirmation,
5. strong resilience to secret-state exposure,
6. strong resistance to decryption-failure attacks,
7. compatibility with lattice-based KEM standards under the standard MLWE assumption, and

³ We focus on KEM components, which dominate the overall communication cost in the post-quantum setting, and therefore omit symmetric-key components such as MAC tags.

⁴ A trivial yet *inefficient* way to achieve symmetry in lattice-based AKE is to let both parties run two CPA-secure KEMs in parallel. However, this redundant approach further increases both computational and communication overhead.

⁵ In [CK02], parties are assumed to know all users’ long-term public keys in advance, while the intended peer identity is not fixed prior to protocol execution. In contrast, our setting assumes that both the peer identity and the relevant long-term public keys may be unknown at session initiation, capturing a broader class of deployment scenarios.

8. provable security in the random oracle model (ROM) [BR94] and the quantum-accessible random oracle model (QROM) [BDF⁺11] under the post-id eCK-PFS setting.

Core primitive AFS-KEM and BW-KEM instantiation. The central building block of the AFS-KEX framework is an *authenticated forward-secure KEM* (AFS-KEM), which integrates authentication and forward secrecy directly into a KEM abstraction. This is achieved by combining static and ephemeral key material into a composite public-key interface, enabling a unified treatment of authentication and key establishment. Intuitively, an AFS-KEM can be efficiently instantiated from a standard KEM whose public-key structure satisfies a mild homomorphic compatibility property. This property is satisfied by several common post-quantum KEMs, including MLWE-based schemes such as the NIST-standardized ML-KEM [Nat24b], as well as code-based schemes such as HQC [MAB⁺25].

For the concrete instantiation, we realize AFS-KEM using BW-KEM [LZS⁺26], an MLWE-based KEM built around Barnes–Wall lattice codes, which is also a submission to NGCC with proposal number PKCKEM-297000. BW-KEM adopts a scalable nested lattice coding framework as the core error-correcting component, which enables improved concrete security margins, more compact ciphertexts, and significantly lower decryption failure rates. We denote the resulting instantiation by AFS-KEM-BW. This construction is a higher-level composition that directly leverages the underlying BW-KEM primitive. In particular, the encapsulation and decapsulation procedures remain identical to those of BW-KEM. The main modification is the introduction of a composite public key cpk , obtained by combining a long-term public key and an ephemeral public key. The standard BW-KEM encapsulation and decapsulation algorithms are then applied to this derived key material. Specifically, AFS-KEM-BW and BW-KEM can share most of the software code or hardware circuits, significantly simplifying deployment.

Submitted protocol specifications. Based on AFS-KEM-BW, the submitted AFS-KEX-BW suite consists of two concrete protocols: pMAKE-BW and pUAKE-BW ; see Fig. 18 and Fig. 7, respectively. These protocols are obtained by instantiating the generic pMAKE and pUAKE frameworks with AFS-KEM-BW. The primary proposal is the mutually authenticated pMAKE-BW protocol. As a secondary and complementary proposal, this submission also specifies a unilaterally authenticated variant pUAKE-BW intended for settings where only one party is authenticated.

The two submitted protocols are summarized as follows:

- pMAKE-BW is the main submitted protocol. It is a four-round mutually authenticated AKE protocol satisfying all the properties (1)–(7) defined above.
- pUAKE-BW is a secondary submitted protocol. It is a three-round unilaterally authenticated AKE protocol designed to satisfy properties (1), (3), and (5)–(7).

Security model and analysis. Both protocols are analyzed in a post-specified-identity extension of the eCK-PFS framework [LLM07, CF12, BCL⁺26, CK02]. Within this model, we establish key secrecy of pMAKE-BW and pUAKE-BW in both the ROM and the QROM. We also establish explicit authentication for both protocols, following the standard treatment of explicit authentication in authenticated key exchange [SSW20, BCL⁺26].

1.3 Claimed Characteristics

The AFS-KEX-BW algorithm suite, consisting of pMAKE-BW and pUAKE-BW , exhibits the following main characteristics. This subsection provides only a brief overview; detailed discussions are given in Section 2.7.

1. **Compact KEM-based authenticated key exchange.** The pMAKE-BW protocol is a mutually authenticated KEM-based key-exchange protocol with low communication overhead. It transmits only two standard BW-KEM public keys, two standard BW-KEM ciphertexts, two identity strings, and two λ -bit one-time-pad ciphertexts, where λ denotes the target security parameter. Similarly, pUAKE-BW is a unilaterally authenticated variant that transmits only one standard BW-KEM public key, one standard BW-KEM ciphertext, one identity string, and one λ -bit one-time-pad ciphertext. In contrast, straightforward adaptations of existing KEM-based AKE protocols [BCGP08, FSXY12, FSXY13, HKSU20, LL23, PWZ23, PZ23] to the post-id setting with PFS would naturally transmit an additional post-quantum KEM public key and an additional

post-quantum KEM ciphertext to provide the corresponding unilateral or mutual authentication guarantees. This compact structure is particularly valuable in the post-quantum setting, where bandwidth often dominates real-world efficiency.

2. **Round-efficient in the post-id setting.** The submitted protocols match the natural round complexity for KEM-based authenticated key exchange in the post-id setting with perfect forward secrecy. In particular, a direct adaptation of existing KEM-based authenticated key-exchange protocols [BCGP08, FSXY12, FSXY13, HKSU20, LL23, PWZ23, PZ23, BCL⁺26], to support deployment with PFS security in the general and popular setting where the peer’s identity and public key are not fixed and known in advance, leads to four rounds for mutual authentication. Intuitively, the first round conveys the initiator’s static KEM public-key material pk_A ; the second round conveys the responder’s static KEM public-key material pk_B , a KEM ciphertext ct_A corresponding to pk_A , and the responder’s ephemeral KEM public-key material pk_e ; the third round conveys KEM ciphertexts ct_B and ct_e corresponding to the responder’s static and ephemeral KEM public keys, respectively, together with a MAC tag; and the fourth round conveys a final MAC tag. We remark that the fourth round is critical for ensuring perfect forward security (PFS) as well as explicit authentication and key confirmation. Accordingly, pMAKE-BW completes in four rounds, while pUAKE-BW completes in three rounds for unilateral authentication. Thus, AFS-KEX does not increase the round complexity compared with straightforward adaptations of existing KEM-based AKE designs to the general and popular deployment setting.
3. **Compatibility with standard post-quantum KEMs.** The pMAKE and pUAKE constructions are compatible with standard post-quantum KEMs whose public-key structure satisfies mild linearity or homomorphic properties. This includes MLWE-based KEMs such as BW-KEM and ML-KEM, LWE-based KEMs such as FrodoKEM [LBES25], and code-based KEMs such as HQC. For each such underlying KEM, the corresponding AFS-KEM construction can be instantiated directly by reusing the underlying KEM’s encapsulation and decapsulation algorithms. In particular, the submitted protocols pMAKE-BW and pUAKE-BW are instantiated directly from BW-KEM without modifying its underlying encapsulation and decapsulation algorithms. That is, pMAKE-BW and pUAKE-BW can share most of the software code or hardware circuits of BW-KEM, significantly simplifying deployment. This interface-level compatibility facilitates the integration of post-quantum KEMs into authenticated key-exchange protocols, enables reuse of existing cryptographic libraries, and preserves consistency between the KEM layer and the key-exchange layer, thereby supporting incremental, low-friction deployment.
4. **Computational symmetry.** The protocol pMAKE-BW achieves computational symmetry, in the sense that both parties perform the same cryptographic operations. This property is desirable for practical AKE deployment [BMS19], especially when both protocol roles may be implemented on constrained platforms such as smart cards or sensor nodes. It can simplify implementation, facilitate dataflow optimization, and reduce code size, circuit area, and reconfiguration overhead.
5. **Support for post-specified identity setting.** Both protocols support the post-specified-identity setting, where the peer identity and the corresponding long-term public key need not be fixed at session initiation. Instead, they are learned and authenticated during protocol execution, and are then bound to the resulting session transcript. This captures IKE-style deployments in which an initiator may start a session using only the responder’s IP address, while the peer’s authenticated identity and corresponding long-term public key are established only later through the identity and certificate exchange in the handshake [CH98, CK01, CK02, KHN⁺14]. It also captures TLS-style deployments, where a client may initially know only a service name, URL, or IP address, while the authenticated identity and long-term public key are obtained and validated from the certificate chain during the handshake [Res18].
6. **Stronger resilience to secret-state exposure.** The protocol pMAKE-BW provides stronger resilience to (pre-computable) secret-state exposure. Its key derivation uses KEM-derived secret values associated with four KEM public-key components, namely the static and ephemeral public-key components of the initiator and the static and ephemeral public-key components of the responder. This four-way contributory structure gives pMAKE-BW stronger robustness than existing mutually authenticated KEM-based protocols [BCL⁺26, HKNW25a]. Those protocols typically involve only three such KEM public-key components, because only one party contributes

ephemeral KEM public-key material. Therefore, against a passive adversary with secret-state exposure, the session-key secrecy of pMAKE-BW is preserved as long as at least one of the four components remains uncompromised, whereas in the prior designs [BCL⁺26, HKNW25a], compromising the three corresponding secret sources already suffices to recover the session key. Moreover, the dynamic online generation nature of the composite public key makes it immune to KCI-style attacks caused by the leakage of the key encapsulated with the peer’s static public key in existing KEM-based protocols. More details are discussed in Remark 9.1 (page 60).

7. **Stronger resilience to decryption-failure attacks.** The AFS-KEX framework inherits the security properties of an ephemeral KEX: the long-term KEM public-key material is combined with fresh ephemeral public-key material on a per-session basis. Both submitted protocols therefore achieve stronger resilience to decryption-failure attacks (DFAs) than standard KEM-based AKEs. This per-session re-randomization substantially limits key-recovery attacks against ML-KEM that exploit and amplify decryption failures (e.g., [DGJ⁺19, DRV20, BS20, DB22, AWK⁺25]).
8. **Signature-free design.** Authentication in both submitted protocols is achieved without relying on post-quantum digital signatures. Instead, it is realized solely through KEM-based mechanisms. This approach simplifies protocol composition and avoids reliance on post-quantum signature schemes such as ML-DSA, whose performance and bandwidth overheads are typically higher than those of KEM-based mechanisms such as ML-KEM.
9. **Perfect forward secrecy.** Both pMAKE-BW and pUAKE-BW achieve perfect forward secrecy by deriving session keys from fresh ephemeral KEM key material generated for each session. Consequently, session keys established in completed sessions remain indistinguishable from random even if long-term secret keys are compromised at a later time.
10. **Provable security in the ROM and QROM under the MLWE assumption.** The security of both protocols is established in the random-oracle model (ROM) and the quantum random-oracle model (QROM) under the standard MLWE assumption. In addition, both protocols are shown to satisfy explicit authentication and match soundness, following the security frameworks of [SSW20, BCL⁺26].
11. **Compatibility with identity-hiding deployment.** The AFS-KEX design is compatible with optional deployment profiles that provide identity-hiding properties. Identity hiding can be an important feature in privacy-sensitive applications, where exposing the identity string or certificate during the handshake may leak sensitive metadata about the communicating party. In particular, such a profile can be obtained in a common-reference-string (CRS) setting where all users share the same public matrix $\mathbf{A}$, or equivalently the same public seed $\text{seed}_{\mathbf{A}}$ used to generate $\mathbf{A}$. The identity string or certificate can then be transmitted in encrypted form. Therefore, without changing the main AFS-KEM interface, the same design can support identity-hiding credential transmission as an optional deployment profile. We note that the CRS setting is common in ECC-based cryptography, where all users in a system use the same group and the same generator, corresponding to the matrix $\mathbf{A}$ in LWE-based cryptography. Multi-user security in this common- $\mathbf{A}$ setting can be reduced to the underlying MLWE assumption by a standard hybrid argument, with the factor- n loss in advantage for a system of n users.

2 Specification and Design Overview of pMAKE-BW and pUAKE-BW

This section specifies the submitted key-exchange protocols pMAKE-BW and pUAKE-BW, which are constructed directly from BW-KEM. In this specification, we omit the intermediate AFS-KEM-BW construction layer and describe the protocols directly over BW-KEM.

For both protocols, we assume that all party identities are unique and that a public certification authority (CA) binds identities to their corresponding long-term public keys. The CA may be instantiated using standard mechanisms, such as public-key infrastructures (PKI), certificate validation, or trusted public directories. Formally, the CA is modeled as a trusted verification predicate $\text{CA}_{\text{verf}}(\text{id}, \text{pk}) \in \{0, 1\}$, which outputs 1 if and only if pk is the certified long-term public key associated with identity id .

2.1 BW-KEM

BW-KEM [LZS⁺26] introduces a unified framework that integrates structured, low-dimensional Barnes-Wall lattice codes into high-dimensional module-lattice constructions. Unlike conventional approaches that rely on direct modulation or concatenation with classical error-correcting codes, BW-KEM adopts a scalable nested lattice coding scheme as the core error correction mechanism. This design not only enhances error tolerance but also enables a rigorous analysis of the DFR without relying on independence assumptions on noise coefficients. The parameters of the nested lattice code, such as dimension and scaling factors, can be adjusted to accommodate different security levels and application requirements. This flexibility enables fine-grained trade-offs among concrete security margins, ciphertext compactness, error resilience, and computational efficiency.

Notation. Let N , q , k , and λ be positive integers, where k denotes the module rank and λ denotes the security parameter. We define $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$, where $N = 2^f$ such that $x^N + 1$ is the $2N$ -th cyclotomic polynomial.

Let $n = 2^\nu$ and τ be the coding parameters used by the nested-lattice encoding, and assume that n divides N . Define $\mu_0 := \tau n - \frac{n}{4}(\nu - 1)$. Accordingly, the message space is $\mathcal{M} = \{0, 1\}^{\frac{N}{n}\mu_0}$. Let $\hat{q}$ denote the decoding modulus used in PolyDecode.

By default, regular font letters denote elements in $\mathcal{R}_q$, bold lower-case letters denote column vectors, and bold upper-case letters denote matrices. For a column vector $\mathbf{v}$ and a matrix $\mathbf{A}$, we write $\mathbf{v}^T$ and $\mathbf{A}^T$ for their transposes, respectively.

Given any real number x , $\lfloor x \rfloor$ denotes rounding to the closest integer, with ties rounded up. For an even positive integer α , let $r' = r \bmod^\pm \alpha$ denote the unique integer in $(-\alpha/2, \alpha/2]$ such that $\alpha \mid (r - r')$; for an odd positive integer α , the interval is $(-(\alpha - 1)/2, (\alpha - 1)/2]$. For any positive integer α , let $r' = r \bmod^+ \alpha$ denote the unique integer in $[0, \alpha)$ such that $\alpha \mid (r - r')$. We write $r' = r \bmod \alpha$ when the distinction between these two representatives is irrelevant.

For any $f \in \mathbb{Z}_q$, we write $\|f\|_\infty$ for $|f \bmod^\pm q|$. For $w = \sum_{i=0}^{N-1} f_i x^i \in \mathcal{R}_q$, we define $\|w\|_\infty = \max_i \|f_i\|_\infty$. Similarly, for $\mathbf{w} = (w_1, \dots, w_k) \in \mathcal{R}_q^k$, we define $\|\mathbf{w}\|_\infty = \max_i \|w_i\|_\infty$.

For a finite set S , $x \leftarrow S$ means that x is chosen uniformly at random from S . For a probabilistic distribution D , $x \leftarrow D$ denotes sampling x according to D . Given a positive integer η , we define $\mathcal{B}_\eta$ as the centered binomial distribution with parameter η , where $x \leftarrow \mathcal{B}_\eta$ can be sampled as $x = \sum_{i=1}^\eta (a_i - b_i)$ for $(a_1, \dots, a_\eta, b_1, \dots, b_\eta) \leftarrow \{0, 1\}^{2\eta}$. These sampling operations naturally extend coefficient-wise to polynomials and vectors over $\mathcal{R}_q$.

We use deterministic seed-expansion algorithms to generate the objects used by BW-PKE. In the security analysis, this function is treated as a pseudorandom generator (PRG) in the standard model, or modeled as a random oracle. The algorithm `SampleA` expands a seed $\rho \in \{0, 1\}^\lambda$ into a matrix $\mathbf{A} \in \mathcal{R}_q^{k \times k}$. The algorithm `SampleSEη` expands a seed into a pair of vectors in $\mathcal{B}_\eta^k \times \mathcal{B}_\eta^k$. The algorithms `SampleRη` and `SampleEη` expand a seed into, respectively, a vector in $\mathcal{B}_\eta^k$ and a pair in $\mathcal{B}_\eta^k \times \mathcal{B}_\eta$. Unless otherwise specified, all seeds and key strings used by BW-KEM have length λ .

BW-PKE. The BW-PKE scheme is a lattice-based public-key encryption scheme over the ring $\mathcal{R}_q$. Its public key consists of a seed ρ , which deterministically generates the public matrix $\mathbf{A}$, and a vector $\mathbf{t}$ of the form $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where $\mathbf{s}$ and $\mathbf{e}$ are short vectors sampled from a centered binomial distribution. To encrypt a message, the encryption algorithm samples fresh short randomness, computes a noisy linear encoding of the public key, and adds an encoded version of the message before compression. Decryption uses the secret vector $\mathbf{s}$ to remove the main linear term and then decodes the resulting polynomial to recover the message. The concrete algorithms of BW-PKE are given in Fig. 1.

BW-KEM. The key encapsulation mechanism BW-KEM is obtained from BW-PKE using a Fujisaki-Okamoto transform variant of [DHK⁺21]. In addition to the standard key-generation, encapsulation, and decapsulation algorithms, we expose deterministic key-combination interfaces for the construction of AFS-KEX. In particular, `BW-KEM.EKeyGen` takes an existing public key $\mathbf{pk} = (\mathbf{t}, \rho)$ and an explicit seed $\text{seed} \in \{0, 1\}^\lambda$ as input. The seed ρ contained in $\mathbf{pk}$ is used to regenerate the public matrix $\mathbf{A}$, while the explicit input seed is used to generate the ephemeral secret and error vectors.

We use random oracles $\mathbf{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda \times \{0, 1\}^\lambda$ and $\mathbf{H}_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. Thus the encapsulated key, the encryption randomness derived by $\mathbf{H}$, and the implicit-rejection key are all λ -bit strings. The concrete algorithms of BW-KEM are given in Fig. 2.

BW-PKE.KeyGen()	BW-PKE.Enc(pk = (t, ρ), m; r ∈ {0, 1}^λ)	BW-PKE.Dec(sk = s, c = (u, v))
<pre>1 : ρ, σ ←\$ {0, 1}^λ 2 : A := SampleA(ρ) ∈ R_q^{k×k} 3 : (s, e) := SampleSE_{η_1}(σ) ∈ B_{η_1}^k × B_{η_1}^k 4 : t := As + e 5 : return (pk := (t, ρ), sk := s)</pre>	<pre>1 : A := SampleA(ρ) ∈ R_q^{k×k} 2 : r := SampleR_{η_1}(r) ∈ B_{η_1}^k 3 : (e_1, e_2) := SampleE_{η_2}(r) ∈ B_{η_2}^k × B_{η_2}^k 4 : u_0 := A^T r + e_1 5 : v_0 := t^T r + e_2 6 : u := Compress_{q, 2d_u}(u_0) 7 : v := Compress_{q, 2d_v}(v_0 + [PolyEncode(m)]) 8 : return c := (u, v)</pre>	<pre>1 : u' := Decompress_{2d_u, q}(u) 2 : v' := Decompress_{2d_v, q}(v) 3 : w := ⌊ \frac{\hat{q}}{q} (v' - s^T u') ⌋ 4 : m' := PolyDecode(w mod ±\hat{q}) 5 : return m'</pre>
PolyEncode(m = ∑_{j=0}^{N/n-1} m_j x^j ∈ M)	PolyDecode(w = ∑_{j=0}^{N/n-1} w_j x^j ∈ R_{\hat{q}})	
<pre>1 : for j from 0 to \frac{N}{n} - 1 do 2 : m_j := (m_{μ_0 j}, m_{μ_0 j+1}, ⋯, m_{μ_0 j+μ_0-1}) ∈ {0, 1}^{μ_0} 3 : w_j := Encode_{c_{n, τ(q/2^τ)}}(m_j) ∈ C^{n/2} 4 : Decompose w_j into w_j = a_j + b_j i, where a_j, b_j ∈ R^{n/2} 5 : (a_{j,0}, a_{j,1}, ⋯, a_{j,n/2-1}) := a_j, (b_{j,0}, b_{j,1}, ⋯, b_{j,n/2-1}) := b_j 6 : (w_j, w_{j+d}, ⋯, w_{j+(n-1)d}) := (a_{j,0}, b_{j,0}, ⋯, a_{j,n/2-1}, b_{j,n/2-1}) 7 : endfor 8 : w := ∑_{j=0}^{N/n-1} w_j x^j 9 : return w</pre>	<pre>1 : for j from 0 to \frac{N}{n} - 1 do 2 : a_j := (w_j, w_{j+2d}, ⋯, w_{j+(n-2)d}) ∈ R^{n/2} 3 : b_j := (w_{j+d}, w_{j+3d}, ⋯, w_{j+(n-1)d}) ∈ R^{n/2} 4 : x_j := a_j + b_j i ∈ C^{n/2} 5 : m_j := Decode_{c_{n, τ(\hat{q}/2^τ)}}(x_j) ∈ {0, 1}^{μ_0} 6 : (m_{μ_0 j}, m_{μ_0 j+1}, ⋯, m_{μ_0 j+μ_0-1}) := m_j 7 : endfor 8 : m := ∑_{j=0}^{N/n-1} m_j x^j ∈ M 9 : return m</pre>	

Fig. 1. Algorithms for BW-PKE.

BW-KEM.KeyGen()	BW-KEM.EKeyGen(pk = (t, ρ), seed ∈ {0, 1}^λ)
1 : (pk, sk) ← BW-PKE.KeyGen() 2 : z ← \$ {0, 1}^λ 3 : return (pk' := pk, sk' := (sk, z))	1 : A := SampleA(ρ) ∈ R_q^{k×k} 2 : (s_e, e_e) := SampleSE_{η_1}(seed) ∈ B_{η_1}^k × B_{η_1}^k 3 : t_e := As_e + e_e 4 : return (pk_e := (t_e, ρ), sk_e := s_e)
BW-KEM.Encaps(pk)	BW-KEM.Decaps((sk, z), c)
1 : m ← \$ M 2 : (K, r) := H(ID(pk), m) 3 : c ← BW-PKE.Enc(pk, m; r) 4 : return (c, K)	1 : m' := BW-PKE.Dec(sk, c) 2 : (K', r') := H(ID(pk), m') 3 : \tilde{K} := H_1(ID(pk), z, c) 4 : if m' = ⊥ or BW-PKE.Enc(pk, m'; r') ≠ c 5 : return \tilde{K} 6 : else return K'

Fig. 2. Algorithms for BW-KEM and the additional key generation algorithm used by AFS-KEX.

2.2 pMAKE-BW

The pMAKE-BW protocol is a mutually authenticated AKE protocol directly built from BW-KEM. It is the BW-KEM-based instantiation of the generic pMAKE construction. It uses two KEM shared secrets and a pseudorandom function PRF to derive the session key and the seed-protection keys. The protocol flow is depicted in Fig. 3 and proceeds as follows.

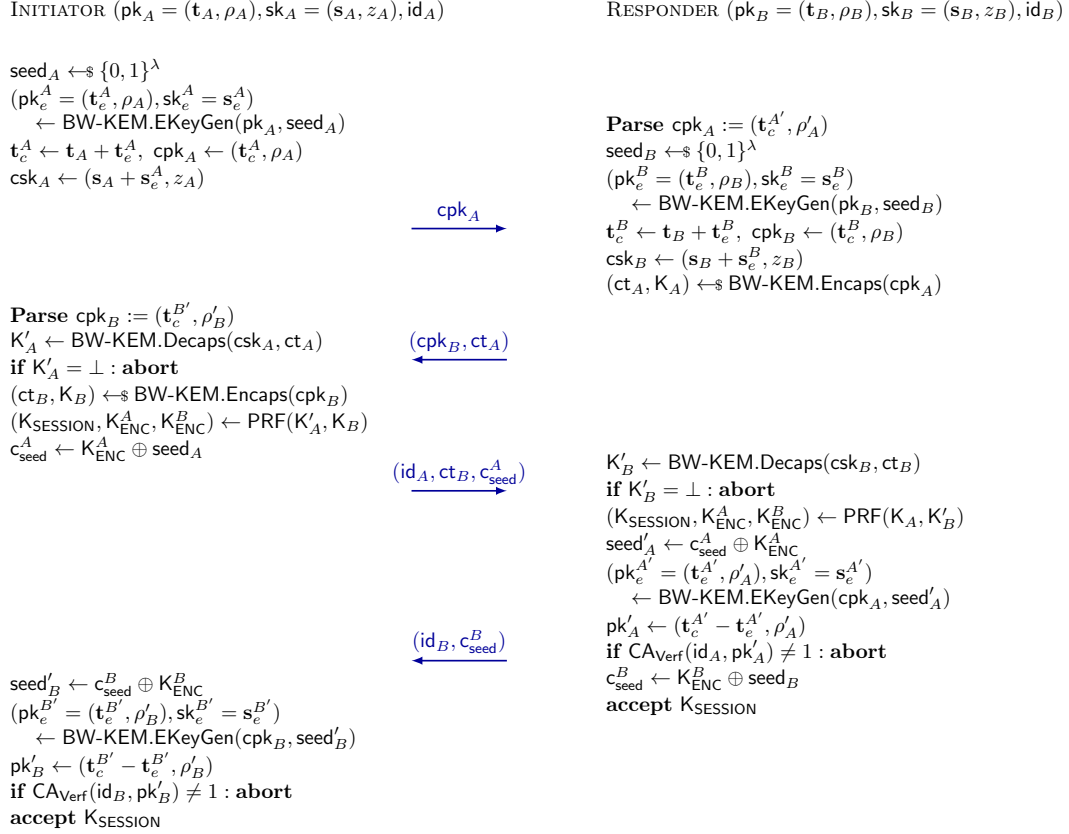


Fig. 3. The pMAKE-BW protocol constructed from BW-KEM.

- *Setup.* The setup algorithm takes the security parameter and the BW-KEM parameter set as input, and outputs the public parameters for pMAKE-BW, including the BW-KEM parameters, the pseudorandom function PRF, and the certification-authority verification algorithm CA_{Verf} .
- *Long-term key generation.* For each party U with a unique identity id_U , the key-generation algorithm samples a random seed and runs BW-KEM.KeyGen to generate (pk_U, sk_U) , where $pk_U = (t_U, \rho_U)$ and $sk_U = (s_U, z_U)$. The public key pk_U is registered and bound to id_U by the certification authority.
- *Round 1 (Initiator $\rightarrow$ Responder).* The initiator A samples $seed_A$ and derives $(pk_e^A, sk_e^A) \leftarrow \text{BW-KEM.EKeyGen}(pk_A, seed_A)$, where $pk_e^A = (t_e^A, \rho_A)$ and $sk_e^A = s_e^A$. It computes $t_c^A = t_A + t_e^A$, $cpk_A = (t_c^A, \rho_A)$, and $csk_A = (s_A + s_e^A, z_A)$, and sends cpk_A to B .
- *Round 2 (Responder $\rightarrow$ Initiator).* Upon receiving $cpk_A = (t_c^{A'}, \rho'_A)$, the responder B samples $seed_B$ and derives $(pk_e^B, sk_e^B) \leftarrow \text{BW-KEM.EKeyGen}(pk_B, seed_B)$, where $pk_e^B = (t_e^B, \rho_B)$ and $sk_e^B = s_e^B$. It computes $t_c^B = t_B + t_e^B$, $cpk_B = (t_c^B, \rho_B)$, and $csk_B = (s_B + s_e^B, z_B)$. It then computes $(ct_A, K_A) \leftarrow \text{BW-KEM.Encaps}(cpk_A)$ and sends (cpk_B, ct_A) to A .
- *Round 3 (Initiator $\rightarrow$ Responder).* Upon receiving $(cpk_B = (t_c^{B'}, \rho'_B), ct_A)$, the initiator decapsulates ct_A using csk_A to obtain K'_A ; if decapsulation fails, it aborts. Otherwise, it computes $(ct_B, K_B) \leftarrow \text{BW-KEM.Encaps}(cpk_B)$ and derives $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K'_A, K_B)$. It sets $c_{\text{seed}}^A \leftarrow K_{\text{ENC}}^A \oplus seed_A$ and sends $(id_A, ct_B, c_{\text{seed}}^A)$ to B .

- *Round 4 (Responder $\rightarrow$ Initiator).* Upon receiving $(\text{id}_A, \text{ct}_B, \text{c}_{\text{seed}}^A)$, the responder decapsulates ct_B using c_{sk_B} to obtain K'_B ; if decapsulation fails, it aborts. Otherwise, it derives $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K_A, K'_B)$. It recovers $\text{seed}'_A \leftarrow \text{c}_{\text{seed}}^A \oplus K_{\text{ENC}}^A$, recomputes $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{BW-KEM.EKeyGen}(\text{cpk}_A, \text{seed}'_A)$ with $\text{pk}_e^{A'} = (\text{t}_e^{A'}, \rho'_A)$, and extracts $\text{pk}'_A = (\text{t}_c^{A'} - \text{t}_e^{A'}, \rho'_A)$. It verifies the binding between id_A and pk'_A using CA_{Verf} ; if verification fails, it aborts. Otherwise, it sets $\text{c}_{\text{seed}}^B \leftarrow K_{\text{ENC}}^B \oplus \text{seed}_B$, accepts K_{SESSION} , and sends $(\text{id}_B, \text{c}_{\text{seed}}^B)$ to A .
- *Initiator acceptance.* Upon receiving $(\text{id}_B, \text{c}_{\text{seed}}^B)$, the initiator recovers $\text{seed}'_B \leftarrow \text{c}_{\text{seed}}^B \oplus K_{\text{ENC}}^B$, recomputes $(\text{pk}_e^{B'}, \text{sk}_e^{B'}) \leftarrow \text{BW-KEM.EKeyGen}(\text{cpk}_B, \text{seed}'_B)$ with $\text{pk}_e^{B'} = (\text{t}_e^{B'}, \rho'_B)$, and extracts $\text{pk}'_B = (\text{t}_c^{B'} - \text{t}_e^{B'}, \rho'_B)$. It verifies the binding between id_B and pk'_B using CA_{Verf} . If verification succeeds, it accepts K_{SESSION} ; otherwise, it aborts.

2.3 pUAKE-BW

The pUAKE-BW protocol is a three-round authenticated key exchange protocol that provides initiator-only authentication in the post-id setting. It is constructed from a single invocation of BW-KEM and a pseudorandom generator PRG. The pseudorandom generator outputs the session key together with a seed-protection key, which is used directly as an XOR mask for the initiator's ephemeral key-generation seed. The protocol flow is depicted in Fig. 4 and proceeds as follows.

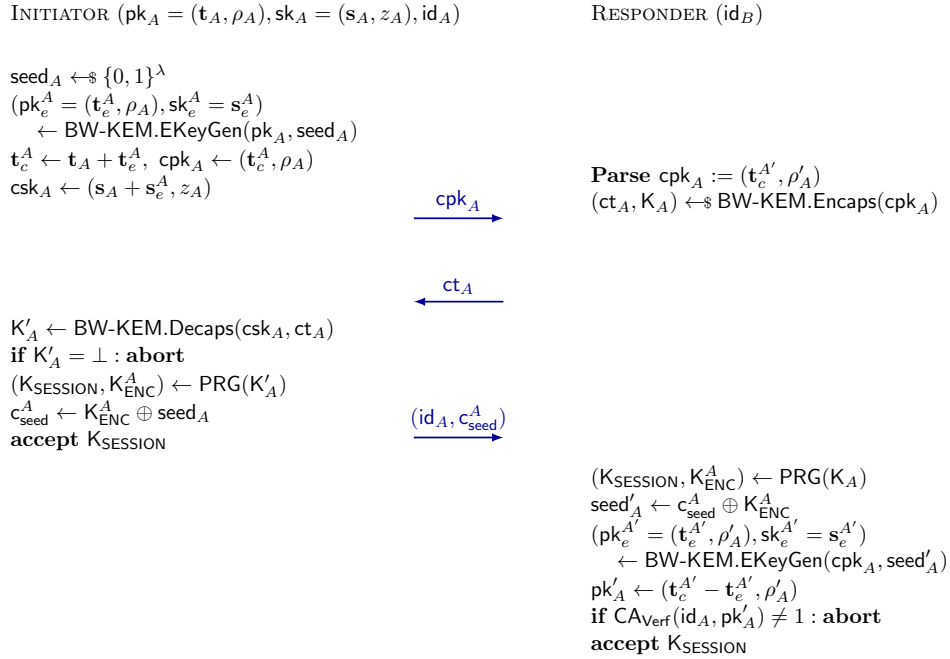


Fig. 4. The pUAKE-BW protocol constructed from BW-KEM.

- *Setup.* The setup algorithm takes the security parameter and the BW-KEM parameter set as input, and outputs the public parameters for pUAKE-BW, including the BW-KEM parameters, the pseudorandom generator PRG, and the certification-authority verification algorithm CA_{Verf} .
- *Long-term key generation.* For each party U with a unique identity id_U , the key-generation algorithm samples a random seed and runs BW-KEM.KeyGen to generate $(\text{pk}_U, \text{sk}_U)$, where $\text{pk}_U = (\text{t}_U, \rho_U)$ and $\text{sk}_U = (\text{s}_U, z_U)$. The public key pk_U is registered and bound to id_U by the certification authority.
- *Round 1 (Initiator $\rightarrow$ Responder).* The initiator A samples seed_A and derives $(\text{pk}_e^A, \text{sk}_e^A) \leftarrow \text{BW-KEM.EKeyGen}(\text{pk}_A, \text{seed}_A)$, where $\text{pk}_e^A = (\text{t}_e^A, \rho_A)$ and $\text{sk}_e^A = \text{s}_e^A$. It computes $\text{t}_c^A = \text{t}_A + \text{t}_e^A$, $\text{cpk}_A = (\text{t}_c^A, \rho_A)$, and $\text{c}_{\text{sk}_A} = (\text{s}_A + \text{s}_e^A, z_A)$, and sends cpk_A to B .

- *Round 2 (Responder $\rightarrow$ Initiator).* Upon receiving $\text{cpk}_A = (\mathbf{t}_c^{A'}, \rho'_A)$, the responder encapsulates under cpk_A to obtain $(\text{ct}_A, K_A) \leftarrow \$ \text{BW-KEM.Encaps}(\text{cpk}_A)$, and sends ct_A to A .
- *Round 3 (Initiator $\rightarrow$ Responder).* Upon receiving ct_A , the initiator decapsulates it using csk_A to obtain K'_A ; if decapsulation fails, it aborts. Otherwise, it derives $(K_{\text{SESSION}}, K_{\text{ENC}}^A) \leftarrow \text{PRG}(K'_A)$, sets $\text{c}_{\text{seed}}^A \leftarrow K_{\text{ENC}}^A \oplus \text{seed}_A$, accepts K_{SESSION} , and sends $(\text{id}_A, \text{c}_{\text{seed}}^A)$ to B .
- *Responder acceptance.* Upon receiving $(\text{id}_A, \text{c}_{\text{seed}}^A)$, the responder derives $(K_{\text{SESSION}}, K_{\text{ENC}}^A) \leftarrow \text{PRG}(K_A)$ and recovers $\text{seed}'_A \leftarrow \text{c}_{\text{seed}}^A \oplus K_{\text{ENC}}^A$. It then recomputes $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{BW-KEM.EKeyGen}(\text{cpk}_A, \text{seed}'_A)$, where $\text{pk}_e^{A'} = (\mathbf{t}_e^{A'}, \rho'_A)$, and extracts $\text{pk}'_A = (\mathbf{t}_c^{A'} - \mathbf{t}_e^{A'}, \rho'_A)$. It verifies the binding between id_A and pk'_A using CA_{Verf} . If verification succeeds, it accepts K_{SESSION} ; otherwise, it aborts.

2.4 Security Analysis Overview

The security analysis of pUAKE-BW and pMAKE-BW follows the modular approach summarized in Fig. 5. Both submitted protocols are obtained by instantiating the general pUAKE and pMAKE authenticated key-exchange frameworks with AFS-KEM-BW. Subsection 2.5 provides the main technical intuition behind the security arguments, while this subsection summarizes the overall structure of the security analysis. Accordingly, the analysis proceeds in three layers.

First, we prove that the underlying AFS-PKE-BW construction satisfies the required CPA-level security notions under the MLWE assumption, namely IND-ECPA, IND-SCPA, and IND-LCK-SCPA. These notions for AFS-PKE are defined in Section 6. The analysis of the AFS-PKE-BW instantiation from BW-PKE is given in Section 7.

Second, we apply the FO_{AFS} transform to lift AFS-PKE security from the CPA level to the CCA level. Concretely, from IND-ECPA, IND-SCPA, and IND-LCK-SCPA security of the underlying AFS-PKE, the transform yields IND-ECPA, IND-SCCA, and IND-LCK-SCCA security of the resulting AFS-KEM in the ROM and QROM. This analysis is given in Section 6.

Third, we recall the general AKE frameworks pUAKE and pMAKE built from an IND-ECPA, IND-SCCA, and IND-LCK-SCCA secure AFS-KEM, and analyze their key secrecy, match soundness, and explicit authentication in the post-eCK-PFS model. The post-eCK-PFS model is recalled in Section 8. The security analysis of pMAKE is given in Section 9, while the corresponding analysis of pUAKE is given in Sections 11 and 10. In particular, the key-secrecy analyses are carried out in both the ROM and the QROM.

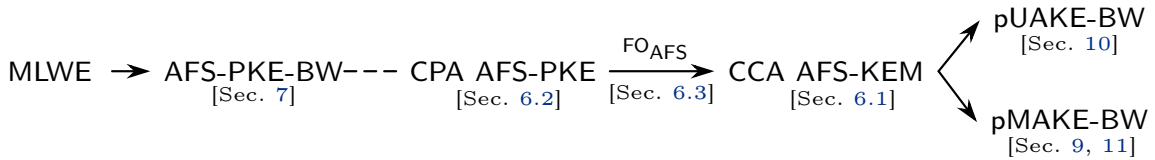


Fig. 5. Overview of the modular security analysis. Dashed edges indicate concrete instantiations of the generic primitive.

Remark 2.1 (Use of public-key identifiers in the concrete instantiation). The FO_{AFS} transform used in the generic security proof and the transform used in the concrete AFS-KEM-BW instantiation differ only in a minor syntactic detail. In the generic description, the composite public key cpk is included in the inputs to the random oracles. In the concrete BW-KEM-based instantiation, this public-key input is represented by a deterministic public-key identifier $\text{ID}(\text{pk})$.

This difference does not affect the security proof. The public-key input in the random-oracle calls is used only as public domain separation, namely to separate random-oracle domains associated with different public keys. Therefore, every random-oracle input involving cpk in the generic proof can be replaced by the corresponding input involving $\text{ID}(\text{cpk})$ in the concrete instantiation. The only required change is to replace the min-entropy assumption on cpk in Theorem 6.9 by the corresponding min-entropy assumption on $\text{ID}(\text{cpk})$.

2.5 Technical Overview and Security Rationale

In this section, we focus on the unilateral authentication protocol **pUAKE** and its concrete instantiation **pUAKE-BW** to illustrate our core techniques. The extension to the mutually authenticated protocol **pMAKE** and its concrete instantiation **pMAKE-BW** follows naturally by composing two instances of unilateral authentication.

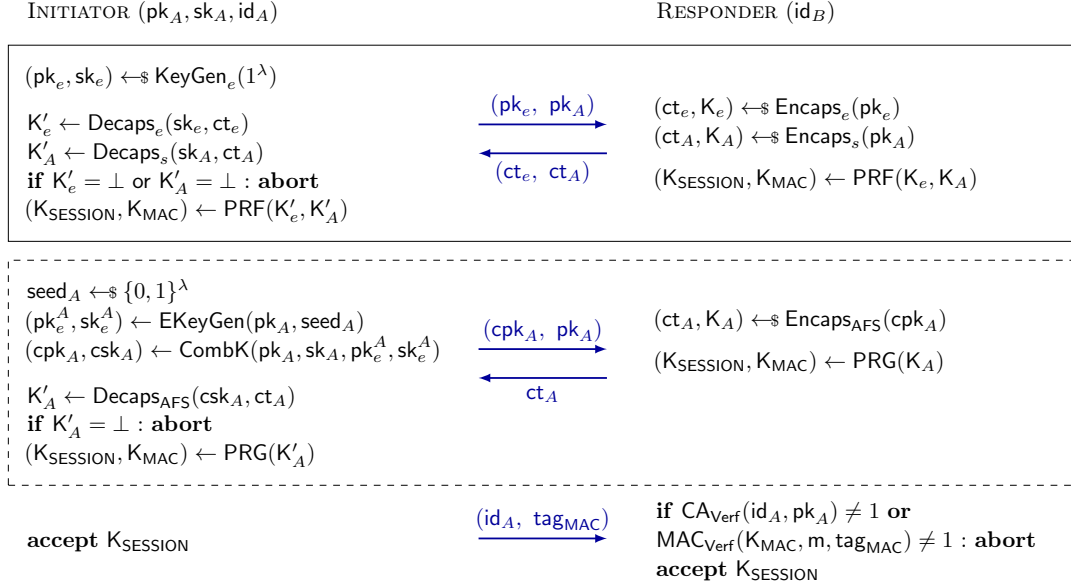


Fig. 6. The $\text{UAKE}_{2\text{KEM}}$ and $\text{NaiveUAKE}_{\text{AFS-KEM}}$ protocols. Elements shown in solid box appear only in $\text{UAKE}_{2\text{KEM}}$, while those in dashed box appear only in $\text{NaiveUAKE}_{\text{AFS-KEM}}$. For simplicity, the MAC computation is omitted. In particular, the authentication tag tag_{MAC} is computed as $\text{MAC}(K_{\text{MAC}}, m)$, where m is derived from the entire transcript up to this point.

Extending Prior Unilaterally Authenticated AKEs to the Post-ID Setting. We briefly recall the standard AKE constructions based on the parallel-KEM paradigm [FSXY12, FSXY13, HKSU20] and explain how it can be adapted to construct unilaterally authenticated AKEs in the post-id setting. In line with most existing AKE constructions, we assume the existence of a trusted certification authority (CA) equipped with a verification algorithm $\text{CA}_{\text{Verf}}(id_A, pk_A)$, which returns 1 if and only if pk_A is a valid public key certified for identity id_A . Then, one can construct a PFS-secure, initiator-authenticated AKE, denoted by $\text{UAKE}_{2\text{KEM}}$, as illustrated in Fig. 6, in which unilateral authentication and PFS⁶ are achieved via two independent KEM instances, KEM_e and KEM_s , executed in parallel. In particular, the responder authenticates the initiator by verifying that $\text{CA}_{\text{Verf}}(id_A, pk_A) = 1$ and that the received MAC tag is valid. When both KEM instances are instantiated with BW-KEM, the techniques of [BCL⁺26, HKNW25b] allow reusing the dominant encapsulation component u in ct_A and ct_e , thereby further reducing the overall communication bandwidth under the Hint-MLWE assumption.

Main Technique I: Authenticated Forward-Secure KEM. The central design principle of our AKE construction is to ensure that the KEM public key itself inherently provides authentication and forward secrecy. Motivated by this goal, we introduce a new cryptographic primitive called the *Authenticated Forward-Secure KEM* (AFS-KEM). An AFS-KEM combines a long-term static key pair with an ephemeral key pair into a single composite key pair (spk, csk) via CombK , which is then used directly by the $\text{Encaps}_{\text{AFS}}$ and $\text{Decaps}_{\text{AFS}}$ algorithms. Intuitively, the static component enforces authentication, while the ephemeral component guarantees forward secrecy of the derived KEM key.

We further define a family of security notions for AFS-KEM, including IND-ECPA, IND-SCCA, and IND-LCK-SCCA. The prefix E (resp. S) denotes that security relies on the secrecy of the ephemeral (resp.

⁶ Perfect forward secrecy (PFS) ensures that a session key remains secure even if the long-term secrets used in its derivation are compromised after the session completes.

static) key component, while allowing compromise of the static (resp. ephemeral) component. The LCK prefix captures whether the composite public key is honestly generated or adversarially malformed, and CPA/CCA specify the availability of a decapsulation oracle. To construct the desired CCA-secure AFS-KEMs, we introduce Authenticated Forward-Secure Public-Key Encryption (AFS-PKE), together with its associated CPA security notions, analogous to those for AFS-KEM. We then propose a Fujisaki–Okamoto (FO) transform [FO99, FO13, HHK17] FO_{AFS} , which upgrades CPA-secure AFS-PKE into a desired CCA-secure AFS-KEM. Once the appropriate security definition for AFS-PKE is established, the FO transform can be applied in a natural and modular way. The resulting security analysis for FO_{AFS} closely follows that of standard FO-based KEM constructions [JZC⁺18, JZM19, BHH⁺19, AHU19, DHK⁺21].

To illustrate the intuition underlying the AFS-KEM construction, consider a simple instantiation based on BW-KEM.⁷ Let $\mathbf{u}_s = \mathbf{A}\mathbf{s}_s + \mathbf{e}_s$ and $\mathbf{u}_e = \mathbf{A}\mathbf{s}_e + \mathbf{e}_e$ denote the static and ephemeral BW-KEM public keys, respectively, with corresponding secret keys $\mathbf{s}_s, \mathbf{s}_e$ and error vectors $\mathbf{e}_s, \mathbf{e}_e$. The pairs $(\mathbf{s}_s, \mathbf{e}_s) = \text{H}(\text{seed}_s)$ and $(\mathbf{s}_e, \mathbf{e}_e) = \text{H}(\text{seed}_e)$ are derived from two independent uniformly random λ -bit seeds via a hash function H . We define the CombK algorithm as component-wise addition. Specifically, the composite public and secret keys are computed as $\text{cpk} := \mathbf{u}_s + \mathbf{u}_e$ ⁸ and $\text{csk} := \mathbf{s}_s + \mathbf{s}_e$. Notably, the resulting composite key pair (cpk, csk) constitutes a valid BW-KEM-style key pair. Therefore, the encapsulation and decapsulation algorithms $\text{Encaps}_{\text{AFS}}$ and $\text{Decaps}_{\text{AFS}}$ are defined identically to the standard BW-KEM encapsulation and decapsulation procedures.

Throughout the following analysis, we assume for simplicity that AFS-KEM is instantiated from BW-KEM as described above.

Naive Designs Based on AFS-KEM Are Insecure. A natural first attempt to construct a unilaterally authenticated AKE from AFS-KEM is illustrated in Fig. 6, which we denote by $\text{NaiveUAKE}_{\text{AFS-KEM}}$. In this design, the initiator sends a composite public key cpk_A , derived from its long-term public key pk_A and an ephemeral public key pk_e^A . Notably, pk_e^A is deterministically generated from an ephemeral seed seed_A via a hash function H . The responder then computes a ciphertext $(\text{ct}_A, K_A) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}_A)$ and derives the session key from the encapsulated key K_A . To enable verification of the initiator’s identity, the initiator additionally transmits its long-term public key pk_A . Since $\text{cpk}_A = \text{pk}_A + \text{pk}_e^A$, as illustrated by our BW-KEM-based instantiation, the initiator may equivalently transmit either $(\text{cpk}_A, \text{pk}_e^A)$ or $(\text{pk}_A, \text{pk}_e^A)$ to its peer.

However, the above construction is insecure. The fundamental issue is that the session key is derived solely from the composite public key cpk_A , while no mechanism enforces that cpk_A is honestly generated. In particular, the adversary may either set cpk_A to an adversarially chosen public key for which it knows the corresponding secret key csk_A , or define a malformed ephemeral public key of the form $\text{pk}_e^A = -\text{pk}_A + \text{pk}'_e$, where $(\text{pk}'_e, \text{sk}'_e)$ is chosen by the adversary. In both cases, the adversary can directly compute the resulting session key and thereby successfully impersonate the initiator. Moreover, there exists subtle small field attack with the naive construction.

These attacks demonstrate that merely transmitting the components of cpk_A is insufficient. Instead, the initiator must additionally provide a *proof*, in some sense, that cpk_A is constructed honestly from its long-term and ephemeral public keys. A generic approach to achieving this is to employ zk-SNARKs [GMR89, BCCT12] to prove the correct formation of the composite public key cpk_A . However, such generic solutions incur prohibitive computational and communication overhead and are therefore unsuitable for post-quantum AKEs.

Main Technique II: Proving cpk via an Encrypted Seed. Our main observation is that this *proof* need not hide the ephemeral secret seed_A from the honest responder, but only from the adversary. Accordingly, we adopt a significantly simpler approach: the initiator directly sends the one-time-pad ciphertext $K_{\text{ENC}}^A \oplus \text{seed}_A$ in the third round, as illustrated by our pUAKE construction in Fig. 7.⁹ Then,

⁷ We omit inessential details such as the auxiliary secret used for implicit rejection in the secret key.

⁸ For completeness, seed_{pk} that is used to generate matrix $\mathbf{A}$ is included as part of the static, ephemeral, and composite public keys. We also omit it here for simplicity.

⁹ This idea is conceptually inspired by identity-concealing authenticated key-exchange protocols [YZ10, YZ13, YZ14, Zha16, Zha20, LPWZ21]. In particular, CAKE [Zha16] protects identity privacy by combining static and ephemeral public keys in a non-malleable manner, while subsequent concealed identity-based AKE protocols [Zha20, LPWZ21] hide the identity-bound ephemeral seed and reveal the corresponding binding only through encrypted seed ciphertexts. We observe that encrypting only a short ephemeral seed via a

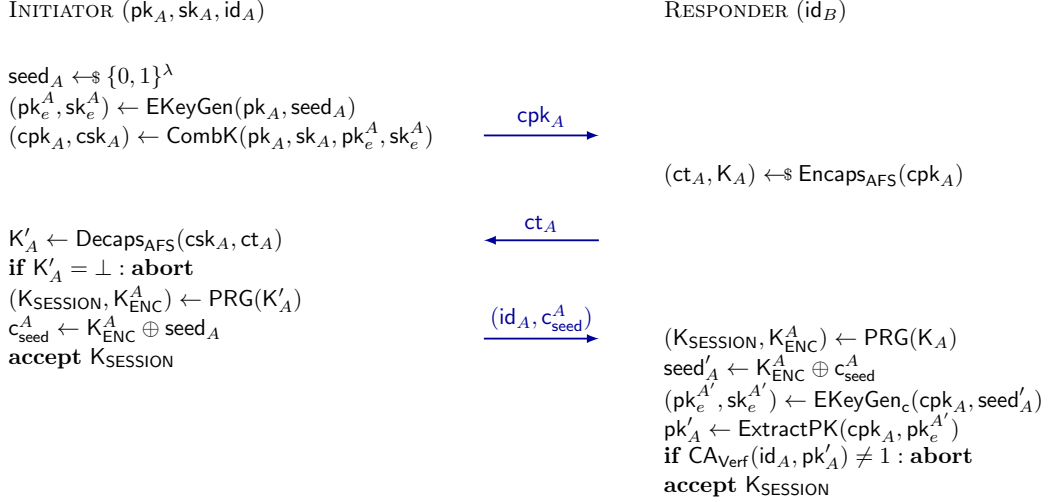


Fig. 7. The pUAKE protocol achieves *initiator-only authentication*.

the responder can locally reconstruct pk'_A from the composite public key cpk_A and the decrypted seed $seed_A$, and then verify the initiator's identity by checking $\text{CA}_{\text{Verf}}(id_A, pk'_A)$. If this verification succeeds, the responder is assured that $cpk_A = pk'_A + pk_e^A$, where $pk_e^A = \text{EKeyGen}(cpk_A, seed_A)$. Moreover, this mechanism allows the initiator to omit explicitly transmitting its long-term public key pk_A , while the resulting one-time-pad ciphertext acts like a MAC: any modification causes the $seed_A$ verification to fail except with negligible probability, thereby providing explicit authentication.

One can readily verify that the above attacks no longer apply to pUAKE. Specifically, when modeling the hash function H as a random oracle, any adversary that does not know the long-term secret key sk_A and outputs a malicious composite public key cpk_A for which it knows the corresponding csk_A cannot find a seed $seed_e$ satisfying $(s_e, e_e) = H(seed_e)$ and $\mathbf{A}s_e + \mathbf{e}_e = cpk_A - pk_A$ except with negligible probability.

We next analyze the security of the pUAKE protocol by focusing on two key attack scenarios that capture the main technical challenges.

Forward Secrecy against a Passive Adversary. We consider a passive adversary that later corrupts the initiator A 's long-term secret key and attempts to compromise a previously established session π of initiator A . In this setting, security relies on the secrecy of the ephemeral seed $seed_A$ in π , and a first attempt is to directly show that the derived KEM key K_A in π and, by the security of PRG, the resulting session key remain computationally indistinguishable from uniform random for the adversary. However, we cannot directly replace K_A with a uniformly random key by relying on the IND-ECPA security of AFS-KEM. The reason is that the adversary observes a ciphertext encrypting $seed_A$ under a symmetric key derived from K_A . Consequently, the reduction in attacking IND-ECPA must simulate the encryption without knowing $seed_A$, thereby introducing a key dependency¹⁰.

To circumvent this key dependency obstacle, we model the pseudorandom generator PRG as a random oracle G and derive the session and encryption keys as $(K_{\text{SESSION}}, K_{\text{ENC}}^A) = G(K_A)$. We first replace $(K_{\text{SESSION}}, K_{\text{ENC}}^A)$ with uniformly random keys, which is indistinguishable to the adversary unless it queries the random oracle at the point K_A . We denote this bad event as **QUERY**. Conditioned on $\neg \text{QUERY}$, the session key is already uniformly random from the adversary's view. It therefore suffices to bound the probability of **QUERY**. To this end, we rely on the perfect secrecy of the one-time-pad to replace the ciphertext c_{seed}^A with an encryption of an independent random seed $\hat{c}_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \hat{seed}_A$, where $\hat{seed}_A \leftarrow \{0, 1\}^\lambda$. This step eliminates the circular dependency, since the encrypted message is now independent of $seed_A$. We can then invoke the IND-ECPA security of AFS-KEM to replace K_A

one-time pad is sufficient for our construction and is much more compact than encrypting the corresponding expanded public-key material, which is particularly valuable in the post-quantum setting.

¹⁰ Key-dependent message (KDM) security, also known as circular security, was first formalized in [BRS02] and is a standard assumption in most fully homomorphic encryption (FHE) constructions. As shown in [BRS02], KDM security can be achieved for public-key encryption schemes in the ROM.

with a uniformly random value, which implies that the probability of QUERY is negligible. Moreover, we lift the above ROM argument to the QROM using the semi-classical oracle techniques of [AHU19].

Impersonation Resistance against an Adaptive Adversary. In this case, the adversary attempts to impersonate a party A without knowing its long-term secret key sk_A . Let π denote the responder session targeted by the adversary and no honest peer session of A exists. Our analysis proceeds as follows. We first rely on the IND-LCK-SCCA security of AFS-KEM to replace the KEM key K_A in session π with a uniformly random value. Consequently, by the pseudorandomness of the PRG, the derived encryption key K_{ENC}^A is also indistinguishable from a uniformly random key to the adversary. As a result, the recovered seed $\text{seed}'_A = K_{\text{ENC}}^A \oplus c_{\text{seed}}^A$ computed by π is uniformly random from the adversary's view, where c_{seed}^A is chosen by the adversary. Consequently, the public key $\text{pk}_e^{A'}$ derived from the uniformly random seed'_A satisfies $\text{pk}_e^{A'} = \text{cpk}_A - \text{pk}_A$ only with negligible probability, and thus the adversary cannot successfully impersonate party A .

In particular, the security notion IND-LCK-SCCA for AFS-KEM defined in Fig. 8 is tailored to the analysis setting considered here. In the IND-LCK-SCCA game, the adversary first outputs a composite public key cpk given an honestly generated static public key pk_s and access to a decapsulation oracle. The challenger computes $(\text{ct}, K_0) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk})$, chooses an independent key $K_1 \leftarrow \mathcal{K}$ and a challenge bit $b \leftarrow \{0, 1\}$, and returns (ct, K_b) to the adversary. The adversary succeeds if it outputs a seed seed_A such that $\text{cpk} = \text{pk}_s + \text{pk}_e$, where $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{cpk}, \text{seed}_A)$, and correctly guesses the challenge bit b . Therefore, the IND-LCK-SCCA security for AFS-KEM allows us to replace the session key K_A in π with a uniformly random KEM key.

To show that AFS-KEM-BW achieves IND-LCK-SCCA security, it suffices to prove that the underlying AFS-PKE-BW scheme satisfies IND-LCK-SCPA security, depicted in Fig. 10, which is the CPA analogue of IND-LCK-SCCA in the PKE setting.

In the IND-LCK-SCPA security analysis for AFS-PKE-BW, we model H as a random oracle and assume that the adversary queries $H(\text{seed})$ just before outputting seed , since this does not change its advantage. We first replace pk_s with a uniformly random value under the MLWE assumption. We then aim to replace the challenge ciphertext ct_b^* with a uniformly random value via an MLWE hybrid argument, where ct_b^* constitutes an MLWE sample relative to $(\mathbf{A} \parallel \text{cpk})$. However, performing this reduction requires embedding the MLWE instance into cpk , which is chosen by the adversary. In particular, we have $\text{cpk} = \text{pk}_s + \mathbf{A}\mathbf{s}_e + \mathbf{e}_e$, where $H(\hat{\text{seed}}) = (\mathbf{s}_e, \mathbf{e}_e)$ for some $\hat{\text{seed}}$ queried by the adversary. To overcome this, the reduction guesses the corresponding $\hat{\text{seed}}$ among the adversary's random-oracle queries and assumes that this is the first query on $\hat{\text{seed}}$. Conditioned on this guess being correct, which occurs with probability $1/q_H$, the reduction can correctly embed the MLWE challenge, and the standard MLWE hybrid argument applies. In the QROM, this guessing argument can be justified using the measure-and-reprogram technique [DFMS19, DFM20].

Analysis in the Post-eCK-PFS Security Model. We provide only a high-level intuition for the security of pUAKE, while the formal analysis additionally relies on carefully defined binding properties of AFS-KEM [CDM24]. The analysis of pMAKE follows the same overall structure but achieves stronger guarantees due to mutual authentication, in which both parties contribute static and ephemeral key material. We analyze both pUAKE and pMAKE in the post-eCK-PFS security model, which extends the classical eCK-PFS model [CF12] to the post-id setting. We further refine explicit authentication into Init-explicit and Resp-explicit notions: both pUAKE and pMAKE achieve Init-explicit authentication, whereas only pMAKE provides Resp-explicit authentication.

2.6 Parameter Selection.

We provide two families of parameter sets for instantiating AFS-KEM. The first family is the BW-KEM-based instantiation, denoted AFS-KEM-BW, which is used for the submitted protocols pMAKE-BW and pUAKE-BW. This is our primary recommendation.

The second family is the ML-KEM-based instantiation, denoted AFS-KEM-ML, and is provided as an alternative instantiation for comparison and compatibility with ML-KEM. It leads to variants such as pMAKE-ML and pUAKE-ML.

Both instantiations follow the same overall AFS construction logic: the long-term and ephemeral key material are combined to obtain a composite public key cpk and the corresponding composite

secret key csk . Encapsulation uses cpk with the underlying KEM encapsulation procedure, while decapsulation uses csk with the corresponding decapsulation procedure, with only the instantiation-specific adaptations to encoding and parameter selection.

Primary BW-KEM-based parameter sets. We provide two groups of parameter sets for AFS-KEM-BW, as shown in Table 1. The first group consists of base parameter sets, which directly inherit the submitted BW-KEM parameters. The second group consists of strengthened parameter sets, which increase the compression parameters to further reduce the decryption-failure probability at the cost of larger ciphertexts. Table 1 reports the main parameters, the security parameter λ , the public-key size $|\text{pk}|$, ciphertext size $|\text{ct}|$, total bandwidth $|\text{pk}| + |\text{ct}|$, and shared key size $|\text{K}|$ in bytes. Here, k denotes the module rank, and λ denotes the bit length of seeds and KEM key strings. The column “(sec.c, sec.q)” gives the estimated classical and quantum security levels. For all parameter sets below, $|\text{K}| = \lambda/8$. In particular, the quantities δ and δ_{AFS} denote the decryption-failure probabilities of the underlying BW-KEM and the corresponding AFS-KEM-BW instantiation, respectively. The gap between them arises because δ is measured for the underlying BW-KEM with a standard key pair, whereas δ_{AFS} accounts for decapsulation under the composite key material of AFS-KEM-BW, where long-term and ephemeral secret terms are combined and the resulting effective noise is larger.

Table 2 specifies the nested-lattice coding parameters used by PolyEncode and PolyDecode in the BW-KEM-based instantiation. For the lowest security level c128, we use the lower-dimensional Barnes–Wall lattice $\text{BW}_8(E_8)$ with $n = 8$ and $\mu_0 = 4$. For the higher security levels, we use BW_{32} with $n = 32$ and $\mu_0 = 32$. In both cases, $\tau = 2$, so the coding lattices are scaled by $q/4$ and $\hat{q}/4$ over the encryption and decoding moduli, respectively. These parameters determine the block structure of the encoding and the message space $\mathcal{M} = \{0, 1\}^{\frac{N}{n}\mu_0}$.

Table 1. Parameter sets of AFS-KEM-BW for pMAKE-BW and pUAKE-BW.

Scheme	Lattice	N	q	k	λ	$(\eta_s, \eta_e, \eta_{\text{ct}})$	d_u	d_v	(sec.c, sec.q)	δ	δ_{AFS}	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $	$ \text{K} $
Base parameter sets															
AFS-KEM-BW-C128	BW_8	256	3329	2	128	(6, 6, 5)	10	4	(131, 119)	$2^{-138.14}$	$2^{-67.98}$	784	768	1552	16
AFS-KEM-BW-C256	BW_{32}	256	3329	4	256	(3, 3, 3)	10	5	(267, 242)	$2^{-209.94}$	$2^{-122.24}$	1568	1440	3008	32
AFS-KEM-BW-C384	BW_{32}	512	3329	3	512	(2, 2, 2)	10	4	(403, 365)	$2^{-216.26}$	$2^{-166.17}$	2368	2176	4544	64
AFS-KEM-BW-C512	BW_{32}	512	3329	4	512	(2, 2, 2)	10	6	(556, 504)	$2^{-232.71}$	$2^{-124.13}$	3136	2944	6080	64
Strengthened parameter sets															
AFS-KEM-BW-C128-s	BW_8	256	3329	2	128	(6, 6, 6)	12	5	(131, 119)	$2^{-171.09}$	$2^{-74.44}$	784	928	1712	16
AFS-KEM-BW-C256-s	BW_{32}	256	3329	4	256	(3, 3, 3)	11	6	(267, 242)	$2^{-277.92}$	$2^{-158.22}$	1568	1600	3168	32
AFS-KEM-BW-C384-s	BW_{32}	512	3329	3	512	(2, 2, 2)	11	5	(403, 365)	$2^{-370.79}$	$2^{-223.72}$	2368	2432	4800	64
AFS-KEM-BW-C512-s	BW_{32}	512	3329	4	512	(2, 2, 2)	11	7	(556, 504)	$2^{-308.56}$	$2^{-168.48}$	3136	3264	6400	64

Table 2. Parameters for $\mathcal{C}_{n,\tau}(\lambda)$.

SecLev	n	τ	μ_0	Coding Lattice	Shaping Lattice
c128	8	2	4	$\text{BW}_8(\frac{q}{4}), \text{BW}_8(\frac{\hat{q}}{4})$	$q \cdot \mathbb{G}^4, \hat{q} \cdot \mathbb{G}^4$
Others	32	2	32	$\text{BW}_{32}(\frac{q}{4}), \text{BW}_{32}(\frac{\hat{q}}{4})$	$q \cdot \mathbb{G}^{16}, \hat{q} \cdot \mathbb{G}^{16}$

The base parameter sets in Table 1 directly inherit the parameters of the submitted BW-KEM specification, registered in NGCC under proposal number PKCKEM-297000, and constitute our primary recommendation. This alignment allows us to use the same parameter sets for BW-KEM and AFS-KEM-BW, and enables pMAKE-BW and pUAKE-BW to be implemented using the standard BW-KEM encapsulation and decapsulation interfaces without modifying the underlying KEM algorithms. Compared with the strengthened variants, the base sets have smaller ciphertexts but larger decryption-failure probabilities. Nevertheless, the resulting error rates remain sufficiently small for practical deployments, as discussed in Subsection 3.2.

The strengthened parameter sets keep the same MLWE dimensions, noise parameters, security estimates, and coding-lattice choices as the corresponding base sets, but increase the compression parameters d_u and d_v . This reduces the decryption-failure probabilities δ and δ_{AFS} at the cost of larger ciphertexts. Therefore, the base sets are recommended for standard-compatible deployment of pMAKE-BW and pUAKE-BW, while the strengthened sets provide more conservative alternatives when lower decryption-failure probabilities are desired.

ML-KEM-based alternative parameter sets. As an alternative, we also provide parameter sets for AFS-KEM-ML, the ML-KEM-based instantiation of AFS-KEM. These parameter sets are provided for comparison and compatibility with ML-KEM, and lead to variants such as pMAKE-ML and pUAKE-ML. They are not our primary recommendation.

The construction AFS-KEM-ML follows the same AFS construction logic as AFS-KEM-BW: the long-term and ephemeral key material are combined to obtain a composite public key cpk and the corresponding composite secret key csk ; encapsulation uses cpk with the ML-KEM encapsulation procedure, while decapsulation uses csk with the ML-KEM decapsulation procedure, with only the necessary adaptations to encoding and parameter selection.

Directly reusing the original ML-KEM parameter sets does not meet the target security levels considered for the corresponding AFS-KEM-ML instantiations. We therefore select adjusted parameters for AFS-KEM-ML-C128, AFS-KEM-ML-C256, AFS-KEM-ML-C384, and AFS-KEM-ML-C512, as shown in Table 3. Compared to the high noise tolerance exhibited by BW-KEM, the original ML-KEM framework is inherently constrained in its parameter selection for AFS-KEM due to the limited error-correction capability of its direct modulation mechanism. For instance, under ML-KEM’s original modulus $q = 3329$, it is difficult to identify suitable parameters for AFS-KEM-ML with low DFRs.

Table 3 reports the main parameters, the security parameter λ , the public-key size $|\text{pk}|$, ciphertext size $|\text{ct}|$, total bandwidth $|\text{pk}| + |\text{ct}|$, and shared-key size $|\text{K}|$ in bytes. Here, k denotes the module rank, λ denotes the bit length of the key strings used by the surrounding AFS-KEM interface, the column “(sec.c, sec.q)” indicates the estimated classical and quantum security strengths in bits, and δ_{AFS} denotes the estimated decryption-failure probability of AFS-KEM-ML.

The rows labeled ML-KEM-128, ML-KEM-192, and ML-KEM-256 are included only as reference direct instantiations using the original ML-KEM parameter sets. Their labels refer to the original ML-KEM security categories. Notably, the quantity δ_{AFS} in these rows denotes the estimated decryption-failure probability of AFS-KEM-ML when instantiated with the original ML-KEM parameter sets, rather than the decryption-failure probability of the original ML-KEM schemes themselves.

Table 3. Parameter sets of AFS-KEM-ML for pMAKE-ML and pUAKE-ML.

Scheme	N	q	k	λ	(η_1, η_2)	(d_u, d_v)	(sec.c, sec.q)	δ_{AFS}	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $	$ \text{K} $
AFS-KEM-ML-C128	256	7681	3	256	(2, 2)	(10, 3)	(162, 147)	2^{-140}	1280	1056	2336	32
AFS-KEM-ML-C256	256	7681	4	256	(5, 3)	(11, 6)	(257, 233)	2^{-88}	1696	1600	3296	32
AFS-KEM-ML-C384	512	7681	3	512	(3, 2)	(11, 5)	(385, 349)	2^{-129}	2528	2432	4960	64
AFS-KEM-ML-C512	512	7681	4	512	(3, 3)	(13, 5)	(533, 484)	2^{-124}	3360	3648	7008	64
ML-KEM-128	256	3329	2	256	(3, 2)	(10, 4)	(117, 108)	2^{-71}	800	768	1568	32
ML-KEM-192	256	3329	3	256	(2, 2)	(10, 4)	(183, 166)	2^{-84}	1184	1088	2272	32
ML-KEM-256	256	3329	4	256	(2, 2)	(11, 5)	(256, 232)	2^{-88}	1568	1568	3136	32

2.7 Detailed Discussion on Claimed Characteristics

In this subsection, we provide a detailed discussion of the claimed characteristics stated in Section 1.3 for the AFS-KEX-BW algorithm suite, which consists of pMAKE-BW and pUAKE-BW. We follow the same order as in Section 1.3 and explain each characteristic in more detail.

Compact KEM-Based Authenticated Key Exchange The pMAKE-BW protocol is a mutually authenticated KEM-based key-exchange protocol with compact communication. It transmits only two

Table 4. Communication-cost comparison for KEM-based AKE protocols. Panel A covers the submitted AFS-KEX protocols under BW-KEM (primary) and adjusted ML-KEM parameters (alternative). Panel B uses the original standard ML-KEM parameter sets for all ML-KEM-derived columns and includes the *Maul*-based DUAKE/DAKE protocols as separate baselines. Highlighted columns correspond to the submitted protocol designs.

Panel A. Submitted AFS-KEX protocols: BW-KEM (primary) and AFS-KEM-ML (alternative)									
Metric	Sec.	Unilateral Authentication				Mutual Authentication			
		BWKEM -UAKE	pUAKE -BW	pUAKE -BW(s)	pUAKE -ML	BWKEM -AKE	pMAKE -BW	pMAKE -BW(s)	pMAKE -ML
Assumption	-	MLWE	MLWE	MLWE	MLWE	MLWE	MLWE	MLWE	MLWE
Comm. (B)	C128	3,120	1,568	1,728	2,368	4,688	3,136	3,456	4,736
	C256	6,048	3,040	3,200	3,328	9,088	6,080	6,400	6,656
	C384	9,152	4,608	4,864	5,024	13,760	9,216	9,728	10,048
	C512	12,224	6,144	6,464	7,072	18,368	12,288	12,928	14,144

Panel B. Unmodified ML-KEM parameter sets and Maul baselines							
Metric	Sec.	Unilateral Authentication			Mutual Authentication		
		MLKEM -UAKE	DUAKE	pUAKE -ML	MLKEM -AKE	DAKE	pMAKE -ML
Assumption	-	MLWE	Hint- MLWE	MLWE	MLWE	Hint- MLWE	MLWE
Comm. (B)	C128	3,168	2,580	1,600	4,768	4,180	3,200
	C256	6,304	5,270	3,168	9,472	8,438	6,336

¹ **Protocol components.** The baseline BWKEM-UAKE (MLKEM-UAKE) and BWKEM-AKE (MLKEM-AKE) follow the standard KEM-based AKE paradigm: the unilateral variant transmits two KEM public keys, two KEM ciphertexts, and one MAC tag, while the mutual variant transmits three KEM public keys, three KEM ciphertexts, and two MAC tags. DUAKE transmits two *Maul* public keys, one *Maul* ciphertext, and one MAC tag; DAKE additionally transmits one ML-KEM public key, one ML-KEM ciphertext, and one MAC tag. All protocols use three rounds for unilateral authentication and four rounds for mutual authentication.

² **Parameter choices.** In Panel A, BWKEM-UAKE, BWKEM-AKE, pUAKE-BW, and pMAKE-BW use the base BW-KEM parameter sets from Table 1, while pUAKE-BW(s) and pMAKE-BW(s) use the strengthened BW-KEM parameter sets from the same table. The pUAKE-ML and pMAKE-ML rows use the adjusted AFS-KEM-ML parameters from Table 3. In Panel B, all ML-KEM-derived columns use the original standard ML-KEM parameter sets: MLKEM-UAKE, MLKEM-AKE, pUAKE-ML, and pMAKE-ML use ML-KEM-512 at C128 and ML-KEM-1024 at C256, rather than the adjusted AFS-KEM-ML parameters used in Panel A. The only non-ML-KEM entries in Panel B are the *Maul*-based DUAKE/DAKE baselines: their parameters are taken from Table 5 of [BCL⁺26], namely *Maul*-512 at C128 and *Maul*-1024 at C256; hence Panel B reports only the C128 and C256 comparison points. Notably, the *Maul* construction is based on ML-KEM.

³ **Accounting convention.** Each MAC tag and each seed encryption c_{seed} is counted as $\lambda/8$ bytes, and identity-string lengths are omitted. In Panel A, $\lambda = 128$ for C128, $\lambda = 256$ for C256, and $\lambda = 512$ for C384 and C512 when using BW-KEM; for AFS-KEM-ML, $\lambda = 256$ for C128 and C256, and $\lambda = 512$ for C384 and C512. In Panel B, $\lambda = 256$ at all reported levels.

BW-KEM public-keys, two standard BW-KEM ciphertexts, two identity strings, and two λ -bit XOR-protected seed values. Concretely, its messages are cpk_A , $(\text{cpk}_B, \text{ct}_A)$, $(\text{id}_A, \text{ct}_B, \text{c}_{\text{seed}}^A)$, and $(\text{id}_B, \text{c}_{\text{seed}}^B)$. Similarly, the unilaterally authenticated variant pUAKE-BW transmits only one BW-KEM public-key, one standard BW-KEM ciphertext, one identity string, and one λ -bit XOR-protected seed value. That is, its messages are cpk_A , ct_A , and $(\text{id}_A, \text{c}_{\text{seed}}^A)$.

In contrast, straightforward adaptations of existing KEM-based AKE protocols [BCGP08, FSXY12, FSXY13, HKSU20, LL23, PWZ23, PZ23] to the same post-id setting with PFS naturally require an additional post-quantum KEM public key and an additional post-quantum KEM ciphertext to provide the corresponding unilateral or mutual authentication guarantees. Our protocols avoid this extra KEM contribution by combining the static and ephemeral components inside the AFS-KEM public-key interface.

This compact structure is particularly valuable in the post-quantum setting, where public keys and ciphertexts are typically much larger than their classical counterparts and bandwidth often dominates real-world efficiency.

Concrete communication costs. Table 4 reports the communication costs of our protocols and several KEM-based AKE baselines at comparable security levels. The table omits identity-string lengths for simplicity. Each MAC tag and each seed encryption c_{seed} is counted as $\lambda/8$ bytes, where λ denotes the security parameter of the corresponding KEM level.

Panel A compares the BW-KEM-based instantiations with the adjusted AFS-KEM-ML instantiations. It includes the direct BW-KEM-based baselines BWKEM-UAKE and BWKEM-AKE, where these two protocols denote straightforward unilateral and mutual KEM-based AKE constructions instantiated with BW-KEM, following the standard parallel two-KEM and three-KEM frameworks in the post-id setting, respectively. It also includes the primary BW-KEM-based protocols pUAKE-BW and pMAKE-BW, their strengthened-parameter variants pUAKE-BW(s) and pMAKE-BW(s), and the adjusted AFS-KEM-ML alternatives pUAKE-ML and pMAKE-ML. The data show that the primary BW-KEM-based instantiations are the most compact among our reported AFS-KEX instantiations. For unilateral authentication, pUAKE-BW is smaller than pUAKE-ML by about 33.8% at C128, 8.3%–8.7% at C256–C384, and 13.1% at C512. The same relative comparison applies to the mutual-authentication protocols, since the mutual variant uses two such KEM contributions.

Panel A also illustrates the bandwidth saving obtained by the AFS-KEX structure itself. Under the base BW-KEM parameter sets, pUAKE-BW reduces communication by about 49.7% compared with BWKEM-UAKE at every listed security level, giving essentially a factor-two saving in the unilateral setting. Similarly, pMAKE-BW reduces communication by 33.0%–33.1% compared with BWKEM-AKE, matching the intuition that the mutual-authentication construction removes roughly one out of three dominant KEM public-key/ciphertext contributions. The strengthened BW-KEM variants remain substantially smaller than the direct BW-KEM-based baselines, while incurring only a modest overhead over the primary BW-KEM-based instantiations.

Panel B gives a separate comparison at the original standard ML-KEM parameter sets. In this panel, all ML-KEM-derived columns use the same unmodified standard parameters, namely ML-KEM-512 at C128 and ML-KEM-1024 at C256. Here UAKE denotes unilateral authenticated key exchange, while DUAKE denotes the Maul-based unilateral AKE baseline from [BCL⁺26]; its mutual-authentication counterpart is denoted by DAKE. This panel therefore isolates the structural bandwidth advantage of the AFS-KEM framework: among the ML-KEM-derived columns, the differences in communication cost come from the protocol design rather than from parameter tuning. The same saving pattern persists in Panel B. For unilateral authentication, pUAKE-ML saves 49.5%–49.7% over MLKEM-UAKE and 38.0%–39.9% over DUAKE. For mutual authentication, pMAKE-ML saves 32.9%–33.1% over MLKEM-AKE and 23.4%–24.9% over DAKE.

These reductions confirm that our AFS-KEX protocols significantly reduce communication bandwidth.

Round-Efficient in the Post-ID Setting The submitted protocols achieve the natural round complexity for KEM-based authenticated key exchange in the general and popular deployment setting with perfect forward secrecy. A direct adaptation of existing KEM-based authenticated key-exchange protocols [BCGP08, FSXY12, FSXY13, HKSU20, LL23, PWZ23, PZ23, BCL⁺26] to support the post-id setting and PFS naturally leads to four rounds for mutual authentication. Intuitively, the first

round conveys the initiator’s static KEM public-key material pk_A ; the second round conveys the responder’s static KEM public-key material pk_B , a KEM ciphertext ct_A corresponding to pk_A , and the responder’s ephemeral KEM public-key material pk_e ; the third round conveys KEM ciphertexts ct_B and ct_e corresponding to the responder’s static and ephemeral KEM public keys, respectively, together with a MAC tag; and the fourth round conveys a final MAC tag. Accordingly, pMAKE-BW completes in four rounds, while pUAKE-BW completes in three rounds for unilateral authentication. Thus, AFS-KEX does not increase the round complexity compared with straightforward adaptations of existing KEM-based AKE designs to the same post-id setting.

Compatibility with Standard Post-Quantum KEMs The pMAKE and pUAKE constructions are compatible with post-quantum KEMs whose key spaces support a mild key-composition operation. Informally, the public-key and secret-key spaces should allow two valid key pairs to be efficiently combined into another valid KEM key pair. This includes MLWE-based KEMs such as BW-KEM and ML-KEM, LWE-based KEMs such as FrodoKEM [LBES25], and code-based KEMs such as HQC. For each such underlying KEM, the corresponding AFS-KEM construction can reuse the native encapsulation and decapsulation algorithms.

In particular, the submitted protocols pMAKE-BW and pUAKE-BW are instantiated directly from BW-KEM without modifying its encapsulation and decapsulation algorithms. The only additional operation is the formation of a session-specific combined public key. For a party U with long-term public key $\text{pk}_U = (\mathbf{t}_U, \rho_U)$ and ephemeral public key $\text{pk}_e^U = (\mathbf{t}_e^U, \rho_U)$, the protocol forms $\text{cpk}_U = (\mathbf{t}_c^U, \rho_U)$ with $\mathbf{t}_c^U = \mathbf{t}_U + \mathbf{t}_e^U$. This value has the same public-key format as a standard BW-KEM public key and can therefore be used as the input to the standard encapsulation algorithm.

This interface-level compatibility is particularly valuable for deployment. By retaining the native KEM interface, the authenticated key-exchange layer can reuse existing post-quantum KEM implementations, including most of the software code and hardware circuitry of BW-KEM. This design also preserves consistency between the KEM layer and the key-exchange layer, thereby simplifying integration into authenticated key-exchange protocols and enabling incremental, low-friction deployment.

Computational Symmetry The protocol pMAKE-BW is computationally symmetric between the two communicating parties. Both the initiator and the responder perform the same dominant KEM operations: each generates one ephemeral key pair, forms one combined public key and secret key, performs one encapsulation and one decapsulation, derives the session and seed-protection keys, and verifies the peer’s certified identity. Thus, the two roles differ only in message order and target public key, not in the type or number of dominant cryptographic operations. More concretely, the responder encapsulates to cpk_A and the initiator encapsulates to cpk_B ; the initiator decapsulates ct_A using csk_A while the responder decapsulates ct_B using csk_B ; Each party also sends one XOR-protected seed value and verifies the validity of the recovered seed.

This symmetry is desirable for practical AKE deployment [BMS19], especially when both protocol roles may run on constrained platforms such as smart cards or sensor nodes. It can simplify implementation, facilitate dataflow optimization, and reduce code size, circuit area, and reconfiguration overhead. Conceptually, it also matches the goal of mutual authentication, since both parties contribute comparable static and ephemeral KEM material to the final session key. This contrasts with many lattice-based KEM-exchange designs in which one role supplies the ephemeral public key while the other role supplies the dominant KEM ciphertext computation.

Support for Post-Specified Identities Both submitted protocols support the post-specified-identity setting, where the peer identity and the corresponding long-term public key need not be fixed at session initiation. Instead, they are learned and authenticated during protocol execution, and are then bound to the resulting session transcript. This is the setting emphasized in the overview of Section 1: the protocol may begin before the peer’s certified public key is known, but the accepting party must still obtain an authenticated binding between the peer identity, the recovered long-term public key, and the established session key.

This setting captures IKE-style deployments, where an initiator may start a session using only the responder’s IP address, while the peer’s authenticated identity and corresponding long-term public

key are established only later through the identity and certificate exchange in the handshake [CH98, CK01, CK02, KHN⁺14]. It also captures TLS-style deployments, where a client may initially know only a service name, URL, or IP address, and obtains and validates the server’s authenticated identity and long-term public key from the certificate chain during the handshake [Res18]. In particular, a TLS client does not generally interact with a fixed server public key known before session activation, the server’s certificate and public key are rather delivered as part of the handshake.

Stronger Resilience to Secret-State Exposure The protocol pMAKE-BW provides stronger resilience to secret-state exposure than existing mutually authenticated KEM-based protocols, because both parties contribute both long-term and fresh ephemeral KEM key material. In pMAKE-BW, the session key is derived from KEM-derived secret values associated with four KEM public-key components: the static and ephemeral public-key components of the initiator and the static and ephemeral public-key components of the responder. The initiator-side KEM secret is K_A or K'_A , and the responder-side KEM secret is K_B or K'_B . Hence, both parties contribute fresh ephemeral KEM material, as well as certified static material, to the final key derivation.

This four-way contributory structure gives pMAKE-BW stronger robustness than existing mutually authenticated KEM-based protocols [BCL⁺26, HKNW25a]. Those protocols typically involve only three such KEM public-key components, because only one party contributes ephemeral KEM public-key material. Therefore, against a passive adversary with secret-state exposure, the session-key secrecy of pMAKE-BW is preserved as long as at least one of the four components remains uncompromised; whereas in the prior designs [BCL⁺26, HKNW25a], compromising the three corresponding secret sources already suffices to recover the session key. This statement should be read in the sense of the formal freshness predicate: exposures that reveal the tested session key itself, or reveal all secret material allowed by the model to determine it, are excluded from the freshness condition.

Additionally, the protocol design avoids reliance on offline precomputed encapsulation keys under static long-term public keys. Since the peer encapsulates to a fresh cpk_U rather than to the long-term pk_U , useful encapsulation values cannot be fully precomputed before the session-specific combined public key is known. This reduces the risk associated with leakage of stored precomputed temporary values. More detailed discussions are given in Remark 9.1 (page 60).

Stronger Resilience to Decryption-Failure Attacks Both submitted protocols achieve stronger resilience to decryption-failure attacks (DFAs) by refreshing the effective KEM decapsulation key on a per-session basis. For a party U , the long-term KEM secret key is combined with a fresh ephemeral secret key to form a combined secret key $\text{csk}_U = \text{sk}_U + \text{sk}_e^U$, and the corresponding encapsulation target cpk_U is the sum of the long-term and ephemeral public-key components. Thus, each encapsulation is performed to a session-specific combined public key rather than directly to a fixed long-term public key.

This per-session re-randomization substantially limits key-recovery attacks against ML-KEM that rely on repeatedly targeting the same long-term KEM public key and accumulating information from decryption failures (e.g., [DGJ⁺19, DRV20, BS20, DB22, AWK⁺25]). Such attacks exploit the fact that, in standard KEM deployments, the decapsulation key remains fixed across sessions, so the adversary can aggregate failure-induced leakage over many ciphertext submissions. In AFS-KEM-based protocols, each session combines the long-term secret key with fresh, independently sampled ephemeral material. Moreover, the AKE flow gives the adversary only a session-level decapsulation attempt rather than a long-lived decryption-failure oracle under the same effective key. Consequently, an adversary targeting a given combined public key can trigger and observe at most one relevant decryption failure under that key in a completed protocol attempt. Even if the adversary actively chooses or manipulates combined public keys across sessions, the resulting ciphertexts target different combined keys, preventing the cumulative leakage required by known failure-amplification attacks.

The submitted construction does not claim to remove the need for a secure and failure-resistant KEM implementation, nor does it claim to eliminate all DFA risks at the implementation level. Rather, it reduces the protocol-level exposure of the long-term KEM public key to repeated decryption-failure amplification. A more detailed analysis of practical error rates and of the concrete security margin against DFAs is given in Subsection 3.2.

Signature-Free Design Authentication in both submitted protocols is achieved without relying on post-quantum digital signatures. Instead of signing the transcript or individual protocol messages, the protocols explicitly authenticate identities through KEM-based mechanisms together with certification-authority verification.

This signature-free design has several advantages. First, it avoids the communication and verification overhead of post-quantum signatures, such as ML-DSA, whose signatures would add non-negligible bandwidth and computation costs compared with a KEM-only design based on ML-KEM. Second, it simplifies protocol composition by relying only on KEM-based mechanisms rather than combining an AKE protocol with an additional signature layer. This can simplify both the software code and the hardware circuits used to realize the AKE protocols. Third, it naturally fits the post-specified-identity setting, because the identity can be revealed only after KEM-derived seed-protection keys have been established and is then bound to the session through the public-key recovery step.

The design still relies on a certification authority to bind identities to long-term public keys. The signature-free claim refers to the online authenticated key-exchange protocol itself: no post-quantum signature generation or verification is required for protocol messages during the protocol execution, even though the surrounding certification infrastructure may use its own authentication mechanisms.

Perfect Forward Secrecy Both pMAKE-BW and pUAKE-BW achieve perfect forward secrecy (PFS) by deriving session keys from fresh KEM key material associated with session-specific combined public keys. Consequently, later compromise of a long-term secret key alone does not directly reveal the session key of a completed fresh session. The essential point is that the decapsulation key used in a completed session includes an ephemeral component that is freshly generated for that session and erased after use, while the encrypted seed message acts like a MAC to support perfect forward secrecy and explicit authentication.

In pMAKE-BW, the session key is derived from two independent KEM shared secrets: $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K'_A, K_B)$ on the initiator side, and $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K_A, K'_B)$ on the responder side. Correctness guarantees $K_A = K'_A$ and $K_B = K'_B$ in matching honest sessions. Since the relevant decapsulation keys are combined secret keys containing ephemeral secret material ($\text{csk}_A = \mathbf{s}_A + \mathbf{s}_e^A$ and $\text{csk}_B = \mathbf{s}_B + \mathbf{s}_e^B$), compromise of only the long-term static keys after session completion is insufficient to recompute the past KEM secrets.

In pUAKE-BW, the session key is derived from a single fresh KEM shared secret: $(K_{\text{SESSION}}, K_{\text{ENC}}^A) \leftarrow \text{PRG}(K'_A)$ on the initiator side, and $(K_{\text{SESSION}}, K_{\text{ENC}}^A) \leftarrow \text{PRG}(K_A)$ on the responder side. Correctness gives $K_A = K'_A$ in matching honest sessions. As in pMAKE-BW, the exact PFS guarantee is subject to the formal freshness restrictions of the security model, especially restrictions on revealing the session key or the relevant ephemeral state. Within those restrictions, later exposure of long-term keys does not enable the adversary to reconstruct the past combined decapsulation key or distinguish the established session key from random.

Provable Security in the ROM and QROM under the MLWE Assumption The key-secrecy security of both pMAKE-BW and pUAKE-BW is established in the random-oracle model (ROM) and the quantum random-oracle model (QROM). Key secrecy requires that the session key of a fresh session be indistinguishable from random. The proof follows the modular route summarized earlier in this section. First, the AFS-PKE-BW instantiation is analyzed under the standard MLWE assumption. Second, the FO_{AFS} transform lifts the required CPA-level properties to the corresponding CCA-level AFS-KEM properties in the ROM and QROM. Third, the generic pMAKE and pUAKE AKE analyses use those AFS-KEM properties, together with the pseudorandomness of PRF or PRG, the deterministic correctness of BW-KEM.EKeyGen, and the certification-authority binding between identities and long-term public keys.

We also show that both protocols satisfy explicit authentication and match soundness. Explicit authentication requires that, except with negligible probability, an honest party accepting a session with a claimed peer identity has a corresponding peer execution. Match soundness rules out inconsistent transcript matching and ensures that matching sessions derive consistent session keys. The analyses of these properties follow the security frameworks of [SSW20, BCL⁺26].

Compatibility with Identity-Hiding Deployment Although identity hiding is not a property claimed by the base submitted protocols, the same AFS-KEX framework can be adapted to support it with a lightweight modification. The basic protocols described above transmit the identity string and certificate material in the clear, because the focus of the submitted specification is authenticated key exchange in the post-specified-identity setting. However, if identity privacy is desired, the credential-transmission step can be encrypted without changing the underlying AFS-KEM mechanism.

The key structural requirement is a common-reference-string (CRS) style setup in which all users share the same public matrix $\mathbf{A}$, or equivalently the same public seed seed_{pk} used to derive $\mathbf{A}$. In the BW-KEM instantiation, a long-term public key has the form $\text{pk}_A = (\mathbf{t}_A, \text{seed}_{\text{pk}})$ and an ephemeral public key has the form $\text{pk}_e^A = (\mathbf{t}_e^A, \text{seed}_{\text{pk}})$ over the same matrix. The transmitted combined public key is then $\text{cpk}_A = (\mathbf{t}_A + \mathbf{t}_e^A, \text{seed}_{\text{pk}})$. Since $\mathbf{t}_e^A$ is freshly generated in each session from an independent ephemeral secret, it acts as a one-time mask of the static public-key component $\mathbf{t}_A$. Consequently, the value cpk_A is a fresh KEM-style public key and does not directly reveal which certified static public key pk_A is being used.

It remains only to avoid sending the identity and certificate in the clear. This can be achieved by replacing the cleartext identity/certificate message with an AEAD ciphertext under a key derived from the KEM secret. For example, in pUAKE-BW the initiator can send an AEAD encryption of $(\text{id}_A, \text{seed}_A)$ under K_{ENC}^A , rather than sending $(\text{id}_A, \text{c}_{\text{seed}}^A)$ in clear/protected separate form. The responder first decapsulates to derive K_{ENC}^A , decrypts the credential ciphertext, and then performs the same reconstruction and certification check as in the base protocol. The mutually authenticated protocol can use the analogous AEAD encryption in both directions. Under this variant, the transcript contains only masked combined public keys, KEM ciphertexts, and encrypted credential material, thereby providing identity hiding against passive observers under the stated CRS assumption and AEAD confidentiality.

3 Analysis with respect to known attacks

3.1 Attacks against the underlying MLWE problem

Since the best known attacks against the MLWE problem do not exploit the underlying lattice structure, the hardness of MLWE can be analyzed in terms of the standard LWE problem. For concrete hardness estimation in most practical lattice-based cryptosystems, the primary attacks considered are two BKZ-based [CN11] strategies, namely the primal and the dual attack.

Primal Attack The primal attack aims to solve the unique-SVP by constructing an instance from the LWE problem. Let $m = (l + 1)n$ denote the number of LWE samples available to the adversary. Given the matrix LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e}) \in \mathbb{Z}_q^{m \times ln} \times \mathbb{Z}_q^m$, the primal attack constructs a lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}_q^{m+kn+1} \mid (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} = \mathbf{0} \pmod{q}\}$ and then applies the BKZ lattice reduction algorithm to find the unique-SVP solution $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$. We refer the reader to [ADPS16] for a detailed analysis of the primal attack.

Dual Attack The dual attack tries to distinguish the matrix LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e}) \in \mathbb{Z}_q^{m \times ln} \times \mathbb{Z}_q^m$ from a uniformly random sample by identifying a short vector $(\mathbf{v}, \mathbf{w})$ in the dual lattice $\Lambda' = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{ln} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \pmod{q}\}$. Suppose such a vector $(\mathbf{v}, \mathbf{w})$ is found. One then computes $z = \mathbf{v}^T \mathbf{b} = \mathbf{v}^T \mathbf{As} + \mathbf{v}^T \mathbf{e} = \mathbf{w}^T \mathbf{s} + \mathbf{v}^T \mathbf{e}$, which is small and approximately Gaussian distributed if $(\mathbf{A}, \mathbf{b})$ corresponds to an LWE sample; otherwise, z is uniformly distributed modulo q . We refer the reader to [Alb17, CMST25] for a detailed analysis of the dual attack.

Following the simple and conservative methodology of the core-SVP hardness [ADPS16], the best known cost of solving the b -dimensional SVP is estimated to be $2^{0.292b}$ in the classical setting and $2^{0.265b}$ in the quantum setting. These cost models are commonly adopted for conservative security estimates of lattice-based schemes. We estimate the classical and quantum core-SVP hardness security of BW-KEM using the Python script from [SAB⁺20, ADPS16, CMST25]. The concrete results are summarized in Table 3.

3.2 Attacks exploiting decryption failures

Analysis of Error Rates in Practice. We suggest the error rates from 2^{-67} to 2^{-74} are reasonable for deploying AFS-KEM-BW-based pMAKE-BW/pUAKE-BW in practice, on the following grounds.

Practical considerations. From a functional and implementation perspective, occasional failures at extremely low rates, such as 2^{-67} per session, can be handled via standard session re-establishment or retransmission mechanisms. Moreover, as an engineering reference point, the IEEE P802.3dj project documents linked from the task-force page specify a bit-error-rate objective of at most 10^{-13} , i.e., approximately $2^{-43.19}$, at the MAC/PLS service interface, after the relevant physical-layer mechanisms such as forward-error correction [IEE]. Thus, 2^{-44} can be viewed as a conservative engineering reference level derived from this residual BER objective. We emphasize that this comparison is not a direct identification of the two error notions. The IEEE P802.3dj objective concerns residual transmission errors at a communication interface, whereas the failure probabilities analyzed here concern cryptographic decapsulation failures in ephemeral key-exchange sessions. Nevertheless, both quantities represent rare functional failures that can be handled by retrying the affected communication or re-establishing the affected session.

This viewpoint is also consistent with Frodo-AKE [BCD⁺16], where decryption failure probabilities ranging from $2^{-41.8}$ to $2^{-33.8}$ are considered acceptable for practical deployment. Therefore, the substantially lower failure probabilities of AFS-KEM-BW, ranging from 2^{-67} to 2^{-74} and lying well below the conservative reference level of 2^{-44} , provide a comfortable margin for practical use in ephemeral KEM-based authenticated key exchange.

Security considerations. Key-recovery attacks against ML-KEM exploiting decryption failures have been extensively studied [DGJ⁺19, DRV20, BS20, DB22, AWK⁺25]. In particular, finding one decryption failure with respect to a fixed long-term key significantly facilitates finding additional failures. Moreover, existing decryption-failure attacks (DFA) require the adversary to target the same secret key and collect thousands of ciphertexts that trigger decapsulation failures, thereby progressively narrowing the search space of the target key. This motivates the use of very low error rates in standard ML-KEM instantiations.

This attack pattern applies naturally to standard KEM-based AKE constructions, where an adversary can establish multiple sessions with the same user and use that user as an oracle revealing whether crafted ciphertexts induce failures. Importantly, all such failure-inducing ciphertexts correspond to the same long-term secret key, enabling the adversary to aggregate leakage across sessions.

In contrast, this attack paradigm appears substantially less applicable to our pMAKE and pUAKE. In our setting, the effective decapsulation key is a combined key $\text{csk} = \text{sk}_s + \text{sk}_e$, where the ephemeral component sk_e is freshly sampled in each session and immediately discarded upon completion. Consequently, for any fixed combined key csk_i , with overwhelming probability the adversary can trigger and observe at most one decryption failure. Notably, in such a failing session, the adversary additionally obtains an OTP encryption of seed_e under a uniformly random implicit-rejection key, which leaks no information about seed_e by the perfect secrecy of the OTP. As a result, a single decryption failure leaks at most one inequality, yielding information that is fundamentally insufficient to recover the full high-dimensional key csk . Furthermore, since the fresh ephemeral component sk_e continuously changes the effective target key across sessions, the adversary is structurally prevented from aggregating leakage across multiple failures, which is a prerequisite for existing decryption-failure attacks.

Even under an extremely pessimistic scenario in which a single decryption failure leaks the entire combined key $\text{csk} = \text{sk}_s + \text{sk}_e$ (which should not occur in practice), the security of the underlying static key remains within standard margins, as assessed in [DDGR20]. In this case, the adversary obtains noisy information on sk_s with noise variance σ_e^2 . Using the leaky-LWE estimator from [DDGR20], we evaluate this setting for ML-KEM-768 and find that such leakage reduces quantum security by approximately 8.5 bits, which remains within standard post-quantum security margins. Notably, this does not contradict our proof, which does not rely on the Hint-MLWE assumption, as the combined secret key csk is never revealed in the security model.

4 Efficiency and Performance Evaluation

This section describes the implementation of our submitted key-exchange scheme and reports performance benchmarks for the reference software implementation, the AVX2-optimized implementation, and the embedded ARM Cortex-M4 implementation. Although the general AFS-KEX framework supports both pMAKE-BW and pUAKE-BW, the submitted implementation realizes pMAKE-BW, the mutually authenticated four-pass protocol instantiated with BW-KEM. This is the variant submitted for evaluation, since it follows the required NGCC-compatible key-exchange interface and is the main proposed scheme. For notational simplicity, throughout this section we use AFS-KEX to denote this concrete implemented scheme, namely pMAKE-BW instantiated with BW-KEM.

NGCC interface and simplified CA verification. The implementation for AFS-KEX are designed for the NGCC key-exchange interface. Under this interface, each party is assumed to have obtained both the peer’s static public key and the peer’s combined public key cpk before the handshake. Consequently, the certification-authority verification $\text{CA}_{\text{verr}}(\text{id}, \text{pk})$ reduces to checking the binding between the claimed identity and the static public key alone. This simplification is consistent with standard NGCC deployment assumptions and the NGCC key-exchange interface.

4.1 Reference and AVX2-optimized Benchmarking

Implementation Overview We first describe the x86 software implementation of the submitted AFS-KEX scheme and report its benchmark results. All measurements follow the NGCC x86 implementation self-evaluation methodology: every parameter set, every implementation version, and the full key-exchange interface are covered.

For each parameter set, the submission provides one platform-independent *reference* implementation written in portable ISO C99, and one *optimized* implementation targeting mainstream 64-bit x86 processors with the AVX2 instruction-set extension; following the NGCC self-evaluation guideline, the optimized implementation is further split into a *performance-optimized* variant and a *resource-optimized* variant. All three versions expose the same NGCC key-exchange programming interface (kex_init , $\text{kex_generate_passi_msg}$, kex_derive_ss) and produce byte-identical known-answer test (KAT) vectors, so the optimized variants are drop-in replacements for the reference code. In the implementation, AFS-KEX provides three parameter sets, AFS-KEX-C128, AFS-KEX-C256, and AFS-KEX-C512, targeting the 128-, 256-, and 512-bit classical security levels, respectively. Each is instantiated directly on top of the corresponding BW-KEM parameter set: the encapsulation and decapsulation routines of BW-KEM are reused without modification, and the protocol layer adds only the composite-public-key construction, which combines a long-term and an ephemeral BW-KEM public key, together with the message-orchestration logic of the post-specified-identity handshake.

Because AFS-KEX reuses BW-KEM as its core primitive, it inherits the same implementation stack: all hashing and extendable-output functionality is instantiated with the SM3-based auxiliary functions supplied by NGCC rather than SHA-3/SHAKE; the reference implementation uses a single-lane scalar SM3 core, whereas the performance-optimized implementation provides an 8-way parallel SM3 engine over 256-bit AVX2 registers together with vectorized PRF/XOF wrappers, and the dominant arithmetic kernels (NTT, inverse NTT, base multiplication), the Barnes-Wall message codec, the centered-binomial noise sampling, and the matrix generation are all vectorized in AVX2 exactly as in the underlying BW-KEM. The protocol-orchestration logic and the composite-public-key construction are implemented in portable C and kept identical across all versions, since they are not on the arithmetic-critical path; the construction recombines long-term and ephemeral public-key material by polynomial addition and reconstructs the peer’s ephemeral public key from the seed recovered from the corresponding seed ciphertext using the underlying key-generation routine.

All experiments were conducted on a single core of an Intel® Xeon® Platinum 8575C processor with 16 GB of main memory, running Ubuntu 22.04.5 LTS (Linux kernel 5.15), with all code compiled by GCC 12.3.0. To ensure comparability, the compilation options follow the NGCC x86 self-evaluation guideline: the reference version uses `-std=c99 -Wpedantic -Wall -Wextra -O2`; the performance-optimized version uses `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer`; and the resource-optimized version uses `-Os -march=x86-64 -mavx2 -flto -fomit-frame-pointer`. Cycle counts for the x86 benchmarks are reported as medians over at least

Table 5. Performance of AFS-KEX for one complete key exchange (init + four rounds + derivation, excluding network time): median cycles and throughput in key exchanges per second. “ref” is the reference implementation, “perf” the performance-optimized AVX2 version, and “res” the resource-optimized AVX2 version.

Instance	Cycles (median)			Throughput (exch./s)		
	ref	perf	res	ref	perf	res
AFS-KEX-C128	1,627,508	389,046	749,298	1,716	7,187	3,636
AFS-KEX-C256	3,807,574	743,118	1,894,836	718	3,764	1,452
AFS-KEX-C512	12,729,024	2,007,954	6,835,822	218	1,390	402

Table 6. Transfer and storage sizes of AFS-KEX (bytes). Identical across the reference and optimized implementations.

Instance	Pub. key	Sec. key	Pass-1	Pass-2	Pass-3	Total msg.	Shared key	Passes
AFS-KEX-C128	1,568	3,170	768	784	16	1,568	16	4
AFS-KEX-C256	3,136	6,338	1,440	1,472	32	2,944	32	4
AFS-KEX-C512	6,272	12,674	2,944	3,008	64	6,016	64	4

1,000 executions. Static code/data/bss sizes are obtained from the `size` utility, and peak RSS is measured using `getrusage`. Following the guideline, the key-exchange performance measures one complete protocol run (initialization, the four message-generation passes, and shared-key derivation), excluding network-transmission time. Functional correctness was verified for every instance and every version by confirming that the initiator and responder derive identical shared secrets; all cases passed.

Performance and Benchmark Results Table 5 reports the median cycle count and the corresponding throughput (key exchanges per second) of one complete mutually-authenticated key exchange, across all three parameter sets and all three implementation versions. The performance-optimized AVX2 implementation accelerates a complete key exchange by roughly 4.2–6.3 $\times$ over the reference implementation, with the largest gains at the higher security levels where the vectorized NTT and base multiplication of the underlying BW-KEM dominate; the resource-optimized variant trades part of this speedup for a smaller code footprint.

The long-term key sizes, per-pass message sizes, total transmitted data, shared-secret size, and number of protocol passes are determined by the protocol specification and are therefore identical across all three implementation versions; they are listed in Table 6. The communication profile reflects the compact, signature-free design of AFS-KEX: the protocol completes in four passes, where the first pass carries a single BW-KEM ciphertext, the second carries one BW-KEM ciphertext plus a λ -bit seed ciphertext, the third carries only a λ -bit seed ciphertext, and the fourth carries no payload (it performs final key confirmation). Crucially, the online handshake messages do not transmit any full BW-KEM public key. Instead, the peer’s ephemeral public-key material is reconstructed locally from the seed recovered from the corresponding λ -bit seed ciphertext. The total online communication therefore consists of two standard BW-KEM ciphertexts and two λ -bit seed ciphertexts, excluding long-term public-key distribution, identity strings, and certificate material. Table 7 reports the static memory footprint (text + data + bss of the self-evaluation binary, as reported by the `size` utility) and the peak resident-set size observed during execution (via `getrusage`); as with the underlying BW-KEM, the performance-optimized version has the largest static footprint because of the AVX2 assembly kernels and the 8-way SM3 engine, whereas the resource-optimized version is the most compact while retaining much of the AVX2 speedup.

Overall, the benchmarks confirm the design goals of AFS-KEX: because the protocol reuses BW-KEM without modifying its encapsulation and decapsulation routines, it directly inherits the AVX2 speedups of the underlying KEM (up to a 6.3 $\times$ acceleration of a complete key exchange) while the composite-public-key construction adds only negligible overhead; the post-specified-identity handshake transmits only two standard BW-KEM ciphertexts and two λ -bit seed ciphertexts, avoiding the transmission of any full KEM public key in the online handshake; and the availability of both a performance-optimized

Table 7. Resource consumption of AFS-KEX (bytes): static memory (text + data + bss) and peak resident-set size (RSS).

Instance	Static memory			Peak RSS		
	ref	perf	res	ref	perf	res
AFS-KEX-C128	44,235	76,100	39,520	2,920,448	2,940,928	2,830,336
AFS-KEX-C256	52,294	80,671	43,925	2,883,584	2,875,392	2,932,736
AFS-KEX-C512	53,414	101,272	72,247	2,875,392	2,830,336	2,883,584

and a resource-optimized AVX2 build allows the same protocol to serve both high-throughput and resource-constrained software platforms without modifying the protocol interface or the test vectors.

4.2 ARM Cortex-M4 Benchmarking

We also develop an optimized ARM Cortex-M4 implementation of AFS-KEX and benchmark it on the STM32F4DISCOVERY board, which is equipped with an ARM Cortex-M4F core running at 24MHz. The firmware is cross-compiled with `arm-none-eabi-gcc` from the GNU ARM Embedded Toolchain and the corresponding `binutils`. The evaluation includes three variants. The `ref` variant is the reference implementation and serves as the baseline for correctness and comparison. The `m4fspeed` variant is optimized for speed on ARM Cortex-M4, while the `m4fstack` variant is optimized for stack usage. For each parameter set, namely AFS-KEX-C128, AFS-KEX-C256, and AFS-KEX-C512, we run the tests on the `stm32f4discovery` platform with 1,000 repetitions for each measurement point. The speed benchmark measures the complete four-pass AFS-KEX flow, including `kex_init_a`, `kex_init_b`, `kex_generate_pass1-kex_generate_pass4`, and `kex_derive_ss_a/kex_derive_ss_b`. All ARM cycle counts and stack values reported below are means over these 1,000 measurements.

The speed results are shown in Table 8. Compared with the portable reference implementation, the ARM Cortex-M4 speed-optimized implementation reduces the total measured cycle count by 2.14 $\times$, 2.34 $\times$, and 3.72 $\times$ for AFS-KEX-C128, AFS-KEX-C256, and AFS-KEX-C512, respectively, where the total is computed as the sum of the two initialization routines, the four pass-generation routines, and the two shared-secret derivation routines. The stack-usage results are shown in Table 9. The ARM Cortex-M4 stack-optimized implementation reduces the maximum observed stack usage by 42.1%, 54.4%, and 55.0% for AFS-KEX-C128, AFS-KEX-C256, and AFS-KEX-C512, respectively, compared with the corresponding reference implementation.

Table 8. Speed of AFS-KEX on ARM Cortex-M4 (Unit: clock cycle).

Parameter Set	Scheme	Init A	Init B	Pass 1	Pass 2	Pass 3	Pass 4	SS A	SS B
AFS-KEX-C128	ref	2,138,494	2,138,284	1,312,393	2,901,694	2,689,193	1,127,869	156	157
	m4fspeed	1,072,017	1,071,926	595,857	1,264,028	1,189,554	545,446	156	157
AFS-KEX-C256	ref	5,129,646	5,127,709	2,781,005	6,197,761	5,882,675	2,533,987	170	171
	m4fspeed	2,313,327	2,312,351	1,150,678	2,459,810	2,424,305	1,133,854	170	171
AFS-KEX-C512	ref	16,179,000	16,181,273	8,587,673	18,872,172	18,121,591	8,032,045	192	193
	m4fspeed	4,516,378	4,516,942	2,252,159	4,847,487	4,765,516	2,207,576	192	193

Table 9. Stack usage of AFS-KEX on ARM Cortex-M4 (Unit: byte).

Parameter Set	Scheme	Init A	Init B	Pass 1	Pass 2	Pass 3	Pass 4	SS A	SS B
AFS-KEX-C128	ref	20,285	20,285	10,395	11,259	16,659	13,156	9	9
	m4fstack	11,738	11,738	4,912	5,011	8,361	7,689	10	9
AFS-KEX-C256	ref	42,056	42,053	20,201	21,802	32,593	27,858	10	9
	m4fstack	19,159	19,159	6,061	6,233	11,901	11,109	10	9
AFS-KEX-C512	ref	82,442	82,444	38,807	42,040	63,669	54,150	9	9
	m4fstack	37,063	37,063	10,920	11,213	22,600	21,055	10	10

5 Preliminaries

Notation. The security parameter is denoted by λ . The set $\{0, \dots, q-1\}$ is denoted by $[q]$. PPT refers to probabilistic polynomial-time algorithms, and QPT refers to quantum polynomial-time algorithms. We use standard asymptotic notation, such as $O(\cdot)$ and $\text{negl}(\cdot)$. All sequences are indexed from 0. For a sequence X , we write $X[i]$ to denote its $i+1$ -th element. For a finite set $\mathcal{X}$, the notation $x \leftarrow \$ \mathcal{X}$ denotes uniform sampling from $\mathcal{X}$, while sampling from a distribution $\mathcal{D}$ is written as $x \leftarrow \$ \mathcal{D}$. The notations $\Pr[P : G]$ and $\Pr[P \mid G]$ denote the probability that the predicate P holds when its variables are instantiated by the execution of program G . The notation $x \triangleq y$ denotes that x is defined to be equal to y . For a randomized algorithm A , we write $y \leftarrow \$ A(x)$; for a deterministic algorithm A , we write $y \leftarrow A(x)$. The expression $x =? y$ evaluates to 1 if $x = y$, and to 0 otherwise. For a Boolean predicate P , we write $[P]$ for its indicator value, i.e., $[P] = 1$ if P holds and $[P] = 0$ otherwise. The size of a set X is denoted by $|X|$. The notation A^H (resp. $A^{[H]}$) denotes that algorithm A is given classical (resp. quantum) access to the oracle H .

5.1 Cryptographic Primitives

Definition 5.1 (Public-Key Encryption). A public-key encryption (PKE) scheme over the message space $\mathcal{M}$ consists of three algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$.

- The key-generation algorithm Gen takes as input the security parameter λ and outputs a key pair (pk, sk) , denoted $(\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}(1^\lambda)$. Usually, we will omit the input of Gen for brevity.
- The encryption algorithm Enc takes as input a public key pk and a message $m \in \mathcal{M}$, and outputs a ciphertext ct , denoted $\text{ct} \leftarrow \$ \text{Enc}(\text{pk}, m)$.
- The decryption algorithm Dec is deterministic and takes as input a secret key sk and a ciphertext ct , and outputs either a message $m' \in \mathcal{M}$ or the failure symbol $\perp$, denoted $m' \leftarrow \text{Dec}(\text{sk}, \text{ct})$.

A PKE scheme is called deterministic (DPKE) if Enc is deterministic.

Correctness. A PKE scheme is said to be δ -correct if

$$\mathbb{E}_{(\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}} \left[\max_{m \in \mathcal{M}} \Pr [\text{Dec}(\text{sk}, \text{ct}) \neq m : \text{ct} \leftarrow \$ \text{Enc}(\text{pk}, m)] \right] \leq \delta,$$

where the expectation is over $(\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}$. A scheme is perfectly correct if $\delta = 0$.

Definition 5.2 (Key Encapsulation Mechanism). A key encapsulation mechanism (KEM) over $\mathcal{K}$ is a tuple of algorithms $(\text{Gen}, \text{Encaps}, \text{Decaps})$.

- $(\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}(1^\lambda)$: the key-generation algorithm takes as input the security parameter λ and outputs a key pair (pk, sk) . We usually omit the input of Gen for brevity.
- $(\text{ct}, K) \leftarrow \$ \text{Encaps}(\text{pk})$: the encapsulation algorithm takes as input a public key pk and outputs a pair (ct, K) , where $K \in \mathcal{K}$ and $\text{ct} \in \mathcal{C}$.
- $K' \leftarrow \text{Decaps}(\text{sk}, \text{ct})$: the decapsulation algorithm takes as input a secret key sk and a ciphertext ct , and outputs either a key $K' \in \mathcal{K}$ or the rejection symbol $\perp$ (if explicit rejection is allowed).

Correctness. A KEM is said to be δ -correct if

$$\Pr [\text{Decaps}(\text{sk}, \text{ct}) \neq K \mid (\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}; (\text{ct}, K) \leftarrow \$ \text{Encaps}(\text{pk})] \leq \delta.$$

Min-Entropy of KEM Public Keys. We say that honestly generated KEM public keys have min-entropy at least k if for any fixed $\hat{\text{pk}} \in \mathcal{PK}$ it holds that $\Pr_{(\text{pk}, \text{sk}) \leftarrow \$ \text{Gen}(1^\lambda)} [\text{pk} = \hat{\text{pk}}] \leq 2^{-k}$.

Definition 5.3 (PRG). A pseudorandom generator (PRG) is a deterministic polynomial-time algorithm $\text{PRG} : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ with $\ell(n) > n$. For an adversary $\mathcal{A}$, its IND-PRG advantage is defined as

$$\text{Adv}_{\text{PRG}, \mathcal{A}}^{\text{IND-PRG}} = \left| \Pr[s \leftarrow \$ \{0, 1\}^n : \mathcal{A}(\text{PRG}(s)) \Rightarrow 1] - \Pr[r \leftarrow \$ \{0, 1\}^{\ell(n)} : \mathcal{A}(r) \Rightarrow 1] \right|.$$

Definition 5.4 (Dual-PRF). A pseudorandom function (PRF) is a keyed function $\text{PRF} : \mathcal{K} \times \mathcal{L} \rightarrow \{0, 1\}^\lambda$. For an adversary $\mathcal{A}$, its PRF advantage is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{PRF}} = \left| \Pr \left[k \leftarrow \mathcal{K} : \mathcal{A}^{\text{PRF}(k, \cdot)} \Rightarrow 1 \right] - \Pr \left[\mathcal{A}^{\mathcal{R}(\cdot)} \Rightarrow 1 \right] \right|,$$

where $\mathcal{R} : \mathcal{L} \rightarrow \{0, 1\}^\lambda$ is a uniformly random function. A PRF satisfies dual-PRF security [Bel06] if both PRF and the swapped function $\text{PRF}'(x, y) = \text{PRF}(y, x)$ are secure PRFs. In our analysis, we assume dual-PRF security for PRF.

5.2 (Quantum-Accessible) Random Oracle Model

We refer the reader to [NC00] for foundational concepts in quantum computation and quantum information. The Random Oracle Model (ROM) [BR93] provides an idealized framework for cryptographic analysis in which a hash function is treated as a publicly accessible random oracle. In this model, an adversary can only obtain the value of a hash function $H(x)$ by querying the random oracle on x , effectively treating H as a truly random function. In the quantum setting, an adversary may evaluate this function on superpositions of inputs. Consequently, the Quantum-Accessible Random Oracle Model (QROM) extends the ROM by allowing quantum adversaries to make superposition queries to the random oracle [BDF⁺11]. In the following, we introduce key technical lemmas in the QROM that will be used throughout this paper.

Lemma 5.5 (Simulating the Random Oracle [Zha12, Theorem 6.1]). Let H be an oracle drawn uniformly at random from the set of $2q$ -wise independent functions. Then, any quantum algorithm making at most q queries to H has zero advantage in distinguishing H from a truly random function.

Lemma 5.6 (Hash Function as a PRF in the QROM [BHH⁺19, Corollary 1]). Let $H : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a quantum-accessible random oracle. Define the keyed function family $F = \{F_k\}_{k \in \mathcal{K}}$ by $F_k(x) := H(k, x)$. Let $\mathcal{A}$ be a quantum IND-PRF adversary that makes q quantum queries to H and any number of queries to a challenge oracle $\mathcal{O}_b$, where $b \leftarrow \{0, 1\}$ and $\mathcal{O}_0(\cdot) = H(k, \cdot)$ for $k \leftarrow \mathcal{K}$, and $\mathcal{O}_1 \leftarrow \mathcal{Y}^{\mathcal{X}}$ is a uniformly random function. Then

$$\left| \Pr[b = b' : b' \leftarrow \mathcal{A}^{\mathcal{O}_b, H}] - \frac{1}{2} \right| \leq \frac{2q}{\sqrt{|\mathcal{K}|}}.$$

Lemma 5.7 (Generic Search Problem [HRS16, Lemma 3]). Let $\gamma \in [0, 1]$ and let Z be a finite set. Define the function $N_1 : Z \rightarrow \{0, 1\}$ such that for each $z \in Z$, $N_1(z) = 1$ with probability $p_z \leq \gamma$, and $N_1(z) = 0$ otherwise. Let N_2 be the constant-zero function: $N_2(z) = 0$ for all z . If a quantum oracle algorithm $\mathcal{A}$ makes at most q queries to N_1 (or N_2), then

$$|\Pr[b = 1 : b \leftarrow \mathcal{A}^{N_1}] - \Pr[b = 1 : b \leftarrow \mathcal{A}^{N_2}]| \leq 8(q + 1)^2 \gamma.$$

Lemma 5.8 (Single-Point Reprogramming in the QROM, adapted from [GHHM21, Proposition 2]). Let X and Y be finite sets. Consider the following two experiments Repro_0 and Repro_1 . First, sample a random oracle $\mathcal{O}_0 \leftarrow \mathcal{Y}^X$ and set $\mathcal{O}_1 := \mathcal{O}_0$. A quantum distinguisher $\mathcal{D}$ is given quantum access to $\mathcal{O}_b$, where $b \in \{0, 1\}$, and may issue one adaptive reprogramming oracle Reprogram . In Reprogram , the experiment samples $x^* \leftarrow X$ and $y^* \leftarrow Y$, sets $\mathcal{O}_1(x^*) := y^*$, and returns x^* to $\mathcal{D}$. If $\mathcal{D}$ makes at most q quantum queries to the random oracle, then

$$|\Pr[\text{Repro}_1^{\mathcal{D}} \Rightarrow 1] - \Pr[\text{Repro}_0^{\mathcal{D}} \Rightarrow 1]| \leq \frac{3}{2} \sqrt{\frac{q}{|X|}}.$$

Ambainis et al. [AHU19] propose a variant of OW2H called the semi-classical oracle $\mathcal{O}_S^{SC}$, which performs a semi-classical measurement only on the output $|f_S(x)\rangle$ but not on the input $|x\rangle$. Here, f_S is the indicator function such that $f_S(x) = 1$ if $x \in S$, and 0 otherwise.

Semi-classical oracle. Roughly speaking, the semi-classical oracle $\mathcal{O}_S^{SC}$ measures only the output $|f_S(x)\rangle$ but not the input $|x\rangle$, where f_S is the indicator function such that $f_S(x) = 1$ if $x \in S$ and 0 otherwise. Formally, for a query to $\mathcal{O}_S^{SC}$ with $\sum_{x,z} a_{x,z} |x\rangle|z\rangle$, $\mathcal{O}_S^{SC}$ performs the following steps:

1. Initialize a single qubit L with $|0\rangle$,
2. Transform $\sum_{x,z} a_{x,z}|x\rangle|z\rangle|0\rangle$ into $\sum_{x,z} a_{x,z}|x\rangle|z\rangle|f_S(x)\rangle$,
3. Measure the qubit L .

After performing this semi-classical measurement, the query state becomes $\sum_{x,z:f_S(x)=y} a_{x,z}|x\rangle|z\rangle$ (non-normalized), where the measurement output is $y \in \{0,1\}$.

Lemma 5.9 (Semi-classical OW2H [AHU19, Theorem 1]). *Let $S \subseteq X$ be random. Let $\mathcal{O}_1, \mathcal{O}_2$ be oracles with domain X and codomain Y such that $\mathcal{O}_1(x) = \mathcal{O}_2(x)$ for any $x \notin S$. Let z be a random bitstring. (The oracles $\mathcal{O}_1, \mathcal{O}_2, S$, and z may have an arbitrary joint distribution D .) Let $\mathcal{O}_S^{SC}$ be an oracle that performs the semi-classical measurements corresponding to the projectors M_y , where $M_y := \sum_{x \in X:f_S(x)=y} |x\rangle\langle x|$ ($y \in \{0,1\}$). Let $\mathcal{O}_2 \setminus S$ (" $\mathcal{O}_2$ punctured on S ") be an oracle that first queries $\mathcal{O}_S^{SC}$ and then $\mathcal{O}_2$. Let $A^{\mathcal{O}_1}(z)$ be an oracle algorithm with query depth q . Let Find denote the event that during the execution of $A^{\mathcal{O}_2 \setminus S}(z)$, $\mathcal{O}_S^{SC}$ outputs 1 during the semi-classical measurements. Then:*

$$\begin{aligned} P_{\text{left}} &:= \Pr[b = 1 : (\mathcal{O}_1, \mathcal{O}_2, S, z) \leftarrow D, b \leftarrow A^{\mathcal{O}_1}(z)] \\ P_{\text{right}} &:= \Pr[b = 1 : (\mathcal{O}_1, \mathcal{O}_2, S, z) \leftarrow D, b \leftarrow A^{\mathcal{O}_2}(z)] \\ P_{\text{find}} &:= \Pr[\text{Find} : (\mathcal{O}_1, \mathcal{O}_2, S, z) \leftarrow D, A^{\mathcal{O}_2 \setminus S}(z)] \end{aligned}$$

Then: $|P_{\text{left}} - P_{\text{right}}| \leq 2\sqrt{(q+1)P_{\text{find}}}$, and $|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}| \leq 2\sqrt{(q+1)P_{\text{find}}}$. The lemma also holds with the bound $\sqrt{(q+1)P_{\text{find}}}$ for an alternative definition of $P_{\text{right}} = \Pr[b = 1 \wedge \neg \text{Find} : (\mathcal{O}_1, \mathcal{O}_2, S, z) \leftarrow D, b \leftarrow A^{\mathcal{O}_2 \setminus S}(z)]$.

Lemma 5.10 (Search in semi-classical oracle [AHU19, Corollary 1]). *Suppose that S and z are independent, and that A is a q -query algorithm. Let $P_{\max} := \max_{x \in X} \Pr[x \in S]$. Then $\Pr[\text{Find} : A^{\mathcal{O}_S^{SC}}(z)] \leq 4q \cdot P_{\max}$.*

Lemma 5.11 ((Single-input) Measure-and-Reprogram, adapted from [DFM20, Theorem 2]). *Let $\mathcal{X}$ and $\mathcal{Y}$ be finite non-empty sets. Let A be an arbitrary oracle quantum algorithm that makes at most q queries to a uniformly random $H : \mathcal{X} \rightarrow \mathcal{Y}$ and outputs some $x \in \mathcal{X}$ together with a (possibly quantum) output z . Let $S^A(\Theta)$ be an oracle algorithm that, on input a uniformly random $\Theta \in \mathcal{Y}$, randomly samples a pair $(i, b) \in ([q] \times \{0,1\}) \cup \{(q,0)\}$, runs A^{H_i} to obtain a (possibly quantum) output z . The oracle H_i is specified as follows: S measures A 's $(i+1)$ -th query input to obtain x , answers A 's ℓ -th H -query using H if $\ell < i+1+b$, and answers it using $H_{x\Theta}$ ($H_{x\Theta}(x) = \Theta$ and $H_{x\Theta}(x') = H(x')$ for all $x' \neq x$) for $\ell \geq i+1+b$. Finally, $S^A(\Theta)$ outputs (x, z) . Then for any fixed $x_0 \in \mathcal{X}$ and any (possibly quantum) predicate V , it holds that*

$$\begin{aligned} \Pr_{\Theta}[x = x_0 \wedge V(x, \Theta, z) : (x, z) \leftarrow S^A(\Theta)] \\ \geq \frac{1}{(2q+1)^2} \Pr_H[x = x_0 \wedge V(x, H(x), z) : (x, z) \leftarrow A^H]. \end{aligned}$$

6 Authenticated Forward-Secure KEM and Its Generic Construction

This section introduces the authenticated forward-secure key encapsulation mechanism, denoted by AFS-KEM, together with its generic construction. We first define the syntax, correctness requirements, and security notions of AFS-KEM. We then introduce the corresponding authenticated forward-secure public-key encryption primitive, denoted by AFS-PKE. Finally, we show how to obtain a CCA-secure AFS-KEM from a CPA-secure AFS-PKE via a Fujisaki–Okamoto-type transform [FO99, FO13, HHK17]. Once the appropriate definition of AFS-PKE is established, the transform can be applied in a natural manner, and the resulting security proof closely follows those for standard KEM constructions [JZC⁺18, JZM19, BHH⁺19, AHU19, DHK⁺21].

6.1 Authenticated Forward-Secure KEM

The AFS-KEM serves as a core cryptographic component of the AKE protocols presented in Section 9. Informally, the AFS-KEM allows one to combine a static key pair (used for authentication) with an ephemeral key pair (which provides additional forward security even if the static public key is compromised) into a single combined KEM key pair. This combined KEM key pair is then used as the key material for the encapsulation and decapsulation algorithms, respectively. We now give the formal definition of AFS-KEM.

Definition 6.1 (Authenticated Forward-Secure KEM). *An authenticated forward-secure key encapsulation mechanism (AFS-KEM) over $\mathcal{K}$ is a tuple of algorithms $\Pi_{\text{AFS-KEM}} = (\text{SKeyGen}, \text{EKeyGen}, \text{EKeyGen}_c, \text{CombPK}, \text{CombSK}, \text{ExtractPK}, \text{Encaps}_{\text{AFS}}, \text{Decaps}_{\text{AFS}})$.*

- $(\text{pk}_s, \text{sk}_s) \leftarrow \text{SKeyGen}(1^\lambda)$: the long-term key-generation algorithm outputs a static key pair $(\text{pk}_s, \text{sk}_s) \in \mathcal{PK}_s \times \mathcal{SK}_s$.
- $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{pk}_s, \text{seed})$: the ephemeral key-generation algorithm takes as input the static public key pk_s and an explicit seed $\text{seed} \leftarrow \{0, 1\}^\lambda$, and deterministically outputs an ephemeral key pair $(\text{pk}_e, \text{sk}_e) \in \mathcal{PK}_e \times \mathcal{SK}_e$.
- $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}_c(\text{cpk}, \text{seed})$: the second ephemeral key-generation algorithm takes as input the composite public key cpk and an explicit seed $\text{seed} \leftarrow \{0, 1\}^\lambda$, and deterministically outputs an ephemeral key pair $(\text{pk}_e, \text{sk}_e) \in \mathcal{PK}_e \times \mathcal{SK}_e$.
- $\text{cpk} \leftarrow \text{CombPK}(\text{pk}_s, \text{pk}_e)$: the public-key combination algorithm takes a static public key pk_s and an ephemeral public key pk_e as input, and outputs a composite public key cpk .
- $\text{csk} \leftarrow \text{CombSK}(\text{sk}_s, \text{sk}_e)$: the secret-key combination algorithm takes a static secret key sk_s and an ephemeral secret key sk_e as input, and outputs a composite secret key csk . For notational convenience, we write $\text{CombK}(\text{pk}_s, \text{sk}_s, \text{pk}_e, \text{sk}_e) := (\text{CombPK}(\text{pk}_s, \text{pk}_e), \text{CombSK}(\text{sk}_s, \text{sk}_e))$.
- $\text{pk}_s \leftarrow \text{ExtractPK}(\text{cpk}, \text{pk}_e)$: the public-key extraction algorithm takes as input the composite public key cpk and the ephemeral public key pk_e , and deterministically outputs the static public key pk_s .
- $(\text{ct}, K) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk})$: the encapsulation algorithm takes as input the composite public key and outputs a ciphertext-key pair (ct, K) , where $\text{ct} \in \mathcal{CT}$ and $K \in \mathcal{K}$.
- $K' \leftarrow \text{Decaps}_{\text{AFS}}(\text{csk}, \text{ct})$: the decapsulation algorithm takes as input the composite secret key and a ciphertext, and outputs either the corresponding key $K' \in \mathcal{K}$ or the rejection symbol $\perp$.

Correctness for AFS-KEM. An AFS-KEM scheme $\Pi_{\text{AFS-KEM}}$ is δ -correct if $\Pr[\text{Decaps}_{\text{AFS}}(\text{csk}, \text{ct}) \neq K] \leq \delta$, where $(\text{pk}_s, \text{sk}_s) \leftarrow \text{SKeyGen}(1^\lambda)$, $\text{seed} \leftarrow \{0, 1\}^\lambda$, $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{pk}_s, \text{seed})$, $\text{cpk} \leftarrow \text{CombPK}(\text{pk}_s, \text{pk}_e)$, $\text{csk} \leftarrow \text{CombSK}(\text{sk}_s, \text{sk}_e)$, and $(\text{ct}, K) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk})$.

Equivalently, we can define δ -correct as a correctness-failure experiment $\text{Exp}_{\text{AFS-KEM}}^{\text{corr}}(1^\lambda)$ that honestly samples all values as above and outputs 1 if and only if $\text{Decaps}_{\text{AFS}}(\text{csk}, \text{ct}) \neq K$. Thus, $\Pi_{\text{AFS-KEM}}$ is δ -correct if and only if $\Pr[\text{Exp}_{\text{AFS-KEM}}^{\text{corr}}(1^\lambda) = 1] \leq \delta$.

We further require AFS-KEM to satisfy the following two perfect correctness properties.

Perfect ExtractPK-Correctness. An authenticated forward-secure KEM $\Pi_{\text{AFS-KEM}}$ is said to satisfy *perfect ExtractPK-correctness* if, for all $\text{pk}_s \in \mathcal{PK}_s$ and $\text{pk}_e \in \mathcal{PK}_e$, it holds that $\text{ExtractPK}(\text{CombPK}(\text{pk}_s, \text{pk}_e), \text{pk}_e) = \text{pk}_s$.

Ephemeral Key Generation Equivalence. An authenticated forward-secure KEM $\Pi_{\text{AFS-KEM}}$ is said to satisfy *Ephemeral Key Generation Equivalence* if, for all $\text{pk}_s \in \mathcal{PK}_s$ and $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{pk}_s, \text{seed})$ from some seed , it holds that $\text{EKeyGen}(\text{pk}_s, \text{seed}) = \text{EKeyGen}_c(\text{CombPK}(\text{pk}_s, \text{pk}_e), \text{seed})$.

We next introduce the following entropy-related notions, which characterize the min-entropy of the combined public key and ciphertext, respectively.

Min-Entropy of the Combined Public Key. An authenticated forward-secure KEM $\Pi_{\text{AFS-KEM}}$ is said to have combined public keys with min-entropy at least k if, for every fixed long-term public key $\text{pk}_s \in \mathcal{PK}_s$ and every $\text{cpk}' \in \mathcal{CPK}$, it holds that $\Pr_{\text{seed} \leftarrow \{0, 1\}^\lambda}[\text{CombPK}(\text{pk}_s, \text{pk}_e) = \text{cpk}'] \leq 2^{-k}$, where $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{pk}_s, \text{seed})$, and the probability is taken over the randomness of seed .

Game IND-SCCA _{AFS-KEM} ($\mathcal{A}$)	Game IND-LCK-SCCA _{AFS-KEM} ($\mathcal{A}$)	Game IND-ECPA _{AFS-KEM} ($\mathcal{A}$)
1 : $(pk_s^*, sk_s^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$ 2 : $seed_e^* \leftarrow \$ \{0, 1\}^\lambda$ 3 : $(pk_e^*, sk_e^*) \leftarrow \text{EKeyGen}(pk_s^*, seed_e^*)$ 4 : $cpk^* \leftarrow \text{CombPK}(pk_s^*, pk_e^*)$ 5 : $csk^* \leftarrow \text{CombSK}(sk_s^*, sk_e^*)$ 6 : $L_e := \{(cpk^*, pk_e^*, sk_e^*, seed_e^*)\}$ 7 : $b \leftarrow \$ \{0, 1\}$ 8 : $(ct^*, K_0^*) \leftarrow \$ \text{Encaps}_{\text{AFS}}(cpk^*)$ 9 : $K_1^* \leftarrow \$ \mathcal{K}$ 10 : $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{EKeyGen}}, \mathcal{O}^{\text{Dec}_s}}$ 11 : $(cpk^*, pk_s^*, seed_e^*, ct^*, K_b^*)$ 12 : return $b' ? = b$	1 : $(pk_s^*, sk_s^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$ 2 : $L_e := \emptyset$ 3 : $cpk^* \leftarrow \mathcal{A}^{\mathcal{O}^{\text{EKeyGen}}, \mathcal{O}^{\text{Dec}_s}}(pk_s^*)$ 4 : $b \leftarrow \$ \{0, 1\}$ 5 : $(ct^*, K_0^*) \leftarrow \$ \text{Encaps}_{\text{AFS}}(cpk^*)$ 6 : $K_1^* \leftarrow \$ \mathcal{K}$ 7 : $(b', seed_e') \leftarrow \mathcal{A}^{\mathcal{O}^{\text{EKeyGen}}, \mathcal{O}^{\text{Dec}_s}}(pk_s^*, ct^*, K_b^*)$ 8 : $(pk_e', sk_e') \leftarrow \text{EKeyGen}_c(cpk^*, seed_e')$ 9 : $pk_s' \leftarrow \text{ExtractPK}(cpk^*, pk_e')$ 10 : if $pk_s' \neq pk_s^*$: abort 11 : return $b' ? = b$	1 : $(pk_s^*, sk_s^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$ 2 : $seed_e^* \leftarrow \$ \{0, 1\}^\lambda$ 3 : $(pk_e^*, sk_e^*) \leftarrow \text{EKeyGen}(pk_s^*, seed_e^*)$ 4 : $cpk^* \leftarrow \text{CombPK}(pk_s^*, pk_e^*)$ 5 : $csk^* \leftarrow \text{CombSK}(sk_s^*, sk_e^*)$ 6 : $b \leftarrow \$ \{0, 1\}$ 7 : $(ct^*, K_0^*) \leftarrow \$ \text{Encaps}_{\text{AFS}}(cpk^*)$ 8 : $K_1^* \leftarrow \$ \mathcal{K}$ 9 : $b' \leftarrow \mathcal{A}(cpk^*, pk_s^*, pk_e^*, sk_s^*, ct^*, K_b^*)$ return $b' ? = b$
$\mathcal{O}^{\text{EKeyGen}}()$	$\mathcal{O}^{\text{Dec}_s}(cpk, ct)$	
1 : $seed \leftarrow \$ \{0, 1\}^\lambda$ 2 : $(pk_e, sk_e) \leftarrow \text{EKeyGen}(pk_s^*, seed)$ 3 : $cpk \leftarrow \text{CombPK}(pk_s^*, pk_e)$ 4 : $L_e \leftarrow L_e \cup \{(cpk, pk_e, sk_e, seed)\}$ 5 : return $(cpk, pk_e, sk_e, seed)$	1 : if $ct = ct^* \wedge cpk = cpk^*$: return $\perp$ 2 : $V := \{(pk_e', sk_e', seed') (cpk, pk_e', sk_e', seed') \in L_e\}$ 3 : if $ V \neq 1$: return $\perp$ 4 : $(pk_e', sk_e', seed') \leftarrow V$ 5 : $csk \leftarrow \text{CombSK}(sk_s^*, sk_e')$ 6 : return $\text{Decaps}_{\text{AFS}}(csk, ct)$	

Fig. 8. IND-SCCA, IND-LCK-SCCA, and IND-ECPA security games for an AFS-KEM scheme

γ -Spreadness of Ciphertexts. An authenticated forward-secure KEM is said to satisfy γ -spreadness if, for any combined public key $cpk \in \mathcal{CPK}$ and any ciphertext $ct' \in \mathcal{CT}$, $\Pr[ct = ct'] \leq 2^{-\gamma}$, where $(ct, K) \leftarrow \$ \text{Encaps}_{\text{AFS}}(cpk)$.

Next, we define the security notions for our AFS-KEM. Specifically, we consider the notions of IND-SCCA, IND-LCK-SCCA, and IND-ECPA security, as shown in Fig. 8. Here, the prefix S (resp. E) indicates that the notion is defined with respect to the static (resp. ephemeral) public key. The LCK qualifier specifies whether the composite public key is formed from an honestly generated ephemeral key or from an adversarially chosen seed. Finally, CPA and CCA indicate whether the adversary is granted access to a decapsulation oracle.

In particular, the *legitimate-chosen-key static chosen-ciphertext* security (IND-LCK-SCCA) captures the following guarantee. Even when a ciphertext is generated under a maliciously chosen composite public key cpk , which is combined from the honest static public key pk_s (which remains hidden from the adversary) and an adversarially generated ephemeral public key $pk_e' = \text{EKeyGen}(cpk, seed')$ for some $seed'$ chosen by the adversary, the encapsulated KEM key K remains computationally indistinguishable from a uniformly random key, given the corresponding ciphertext ct .

Definition 6.2 (IND-SCCA Security of AFS-KEM). For an AFS-KEM scheme, consider the security game IND-SCCA_{AFS-KEM} defined in Fig. 8. The scheme is said to achieve IND-SCCA security if for any polynomial time adversary $\mathcal{A}$ the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{A}) := |\Pr[\text{IND-SCCA}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2}|$.

Definition 6.3 (IND-LCK-SCCA Security of AFS-KEM). For an AFS-KEM scheme, consider the security game IND-LCK-SCCA_{AFS-KEM} defined in Fig. 8. The scheme is said to achieve IND-LCK-SCCA security if for any polynomial time adversary $\mathcal{A}$ the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{A}) := |\Pr[\text{IND-LCK-SCCA}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2}|$.

Definition 6.4 (IND-ECPA Security of AFS-KEM). For an AFS-KEM scheme, consider the security game IND-ECPA_{AFS-KEM} defined in Fig. 8. The scheme is said to achieve IND-ECPA security if for any polynomial time adversary the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{A}) := |\Pr[\text{IND-ECPA}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2}|$.

Game Key-Coll _{AFS-KEM} ($\mathcal{A}$)	Game SPK-Bind _{AFS-KEM} ($\mathcal{A}$)
1 : $g \leftarrow \mathcal{A}()$ 2 : $(pk_s^*, sk_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$ 3 : $seed_e^* \leftarrow \{0, 1\}^\lambda$ 4 : $(pk_e^*, sk_e^*) \leftarrow \text{EKeyGen}(pk_s^*, seed_e^*)$ 5 : $(cpk^*, csk^*) \leftarrow \text{CombPK}(pk_s^*, sk_s^*, pk_e^*, sk_e^*)$ 6 : if $g = 1$: 7 : $cpk' \leftarrow \mathcal{A}(cpk^*, pk_s^*, sk_s^*, seed_e^*)$ 8 : $(ct', K') \leftarrow \text{Encaps}_{\text{AFS}}(cpk')$ 9 : $ct^* \leftarrow \mathcal{A}(cpk^*, pk_s^*, sk_s^*, seed_e^*, ct', K')$ 10 : return $[cpk' \neq cpk^*] \wedge [\text{Decaps}_{\text{AFS}}(csk^*, ct^*) = K']$ 11 : if $g = 2$: 12 : $(ct^*, K^*) \leftarrow \text{Encaps}_{\text{AFS}}(cpk^*)$ 13 : $cpk' \leftarrow \mathcal{A}(cpk^*, pk_s^*, sk_s^*, seed_e^*, ct^*, K^*)$ 14 : $(ct', K') \leftarrow \text{Encaps}_{\text{AFS}}(cpk')$ 15 : $\text{dummy} \leftarrow \mathcal{A}(ct', K')$ 16 : return $[K' = K^*]$ 17 : if $g = 3$: 18 : $(ct^*, K^*) \leftarrow \text{Encaps}_{\text{AFS}}(cpk^*)$ 19 : $ct' \leftarrow \mathcal{A}(cpk^*, pk_s^*, sk_s^*, seed_e^*, ct^*, K^*)$ 20 : return $[ct' \neq ct^*] \wedge [\text{Decaps}_{\text{AFS}}(csk^*, ct') = K^*]$	1 : $(pk_s^*, sk_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$ 2 : $(\overline{pk}_s^*, \overline{sk}_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$ 3 : $seed_e^* \leftarrow \{0, 1\}^\lambda$ 4 : $(pk_e^*, sk_e^*) \leftarrow \text{EKeyGen}(pk_s^*, seed_e^*)$ 5 : $cpk^* \leftarrow \text{CombPK}(pk_s^*, pk_e^*)$ 6 : $seed' \leftarrow \mathcal{A}(cpk^*, seed_e^*, pk_s^*, sk_s^*, \overline{pk}_s^*, \overline{sk}_s^*)$ 7 : $(pk'_e, sk'_e) \leftarrow \text{EKeyGen}_c(cpk^*, seed')$ 8 : $pk'_s \leftarrow \text{ExtractPK}(cpk^*, pk'_e)$ 9 : return $[pk'_s = \overline{pk}_s^*] \vee ([seed' \neq seed_e^*] \wedge [pk'_s = pk_s^*])$
	Game RSPK-Bind _{AFS-KEM} ($\mathcal{A}$) 1 : $(pk_s^*, sk_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$ 2 : $cpk \leftarrow \mathcal{A}(pk_s^*, sk_s^*)$ 3 : $seed_e \leftarrow \{0, 1\}^\lambda$ 4 : $\text{dummy} \leftarrow \mathcal{A}(seed_e)$ 5 : $(pk_e, sk_e) \leftarrow \text{EKeyGen}_c(cpk, seed_e)$ 6 : $pk'_s \leftarrow \text{ExtractPK}(cpk, pk_e)$ 7 : return $[pk'_s = pk_s^*]$

Fig. 9. Security games defining the SPK-Bind, RSPK-Bind, and Key-Coll properties for an AFS-KEM scheme. In particular, in the Key-Coll game, the adversary can choose $g \in \{1, 2, 3\}$. For $g = 2$ in Key-Coll and RSPK-Bind, dummy denotes an output of $\mathcal{A}$ that is discarded by the experiment, as it does not affect the game advantage. This step is included solely to support the simulation in the subsequent AKE security proof.

We now formalize three intrinsic binding properties of AFS-KEM required for our AKE analysis. These notions follow the general binding property for KEM introduced in [CDM24] and the C2PRI property defined in [BCD⁺24]. The first property, static public key binding (SPK-Bind), ensures that a composite public key uniquely determines its underlying static public key, and that adversarially chosen ephemeral seeds cannot be used to extract the original static key. The second property, randomized static public key binding (RSPK-Bind), guarantees that no adversary can use a randomly chosen ephemeral seed to extract the original static public key from a composite key. The third property, key-collision resistance (Key-Coll), ensures that it is computationally infeasible for an adversary to produce key collisions in any of the three attack scenarios captured by the Key-Coll game.

Definition 6.5 (Static Public Key Binding of AFS-KEM). *For an AFS-KEM scheme, consider the security game $\text{SPK-Bind}_{\text{AFS-KEM}}(\mathcal{A})$ defined in Fig. 9. The scheme is said to achieve SPK-Bind security if for any polynomial time adversary $\mathcal{A}$, the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{A}) := \Pr[\text{SPK-Bind}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1]$.*

Definition 6.6 (RSPK-Bind of AFS-KEM). *For an AFS-KEM scheme, consider the security game $\text{RSPK-Bind}_{\text{AFS-KEM}}(\mathcal{A})$ defined in Fig. 9. The scheme is said to achieve RSPK-Bind security if for any polynomial-time adversary $\mathcal{A}$, the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{A}) := \Pr[\text{RSPK-Bind}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1]$.*

Definition 6.7 (Key Collision Security of AFS-KEM). *For an AFS-KEM scheme, consider the security game $\text{Key-Coll}_{\text{AFS-KEM}}(\mathcal{A})$ defined in Fig. 9. We say that the scheme achieves key-collision resistance if for any polynomial time adversary $\mathcal{A}$, the following advantage is negligible in the security parameter λ : $\text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{A}) := \Pr[\text{Key-Coll}_{\text{AFS-KEM}}(\mathcal{A}) \Rightarrow 1]$.*

6.2 Authenticated Forward-Secure PKE

Similar to AFS-KEM, an AFS-PKE combines a static key pair with an ephemeral key pair into a single *composite* public/secret key pair. The composite key material is then used by the encryption and decryption algorithms.

Definition 6.8 (Authenticated Forward-Secure PKE). *An authenticated forward-secure public-key encryption scheme (AFS-PKE) over message space $\mathcal{M}$ is a tuple of algorithms $\Pi_{\text{AFS-PKE}} = (\text{SKeyGen}, \text{EKeyGen}, \text{EKeyGen}_c, \text{CombPK}, \text{CombSK}, \text{ExtractPK}, \text{Enc}_{\text{AFS}}, \text{Dec}_{\text{AFS}})$.*

- $(\text{pk}_s, \text{sk}_s) \leftarrow \text{SKeyGen}(1^\lambda)$: the long-term key-generation algorithm outputs a static key pair $(\text{pk}_s, \text{sk}_s) \in \mathcal{PK}_s \times \mathcal{SK}_s$.
- $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}(\text{pk}_s, \text{seed})$: the ephemeral key-generation algorithm takes as input the static public key pk_s and a seed $\text{seed} \in \{0, 1\}^\lambda$, and deterministically outputs an ephemeral key pair $(\text{pk}_e, \text{sk}_e) \in \mathcal{PK}_e \times \mathcal{SK}_e$.
- $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}_c(\text{cpk}, \text{seed})$: the composite-key-based ephemeral key-generation algorithm takes as input the composite public key cpk and a seed $\text{seed} \in \{0, 1\}^\lambda$, and deterministically outputs an ephemeral key pair $(\text{pk}_e, \text{sk}_e) \in \mathcal{PK}_e \times \mathcal{SK}_e$.
- $\text{cpk} \leftarrow \text{CombPK}(\text{pk}_s, \text{pk}_e)$: the public-key combination algorithm takes as input a static public key pk_s and an ephemeral public key pk_e , and outputs a composite public key cpk .
- $\text{csk} \leftarrow \text{CombSK}(\text{sk}_s, \text{sk}_e)$: the secret-key combination algorithm takes as input a static secret key sk_s and an ephemeral secret key sk_e , and outputs a composite secret key csk . For notational convenience, we write $\text{CombK}(\text{pk}_s, \text{sk}_s, \text{pk}_e, \text{sk}_e) := (\text{CombPK}(\text{pk}_s, \text{pk}_e), \text{CombSK}(\text{sk}_s, \text{sk}_e))$.
- $\text{pk}_s \leftarrow \text{ExtractPK}(\text{cpk}, \text{pk}_e)$: the public-key extraction algorithm takes as input a composite public key cpk and an ephemeral public key pk_e , and deterministically outputs the corresponding static public key pk_s .
- $\text{ct} \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}, m)$: the encryption algorithm takes as input the composite public key cpk and a message $m \in \mathcal{M}$, and outputs a ciphertext $\text{ct} \in \mathcal{CT}$. When necessary, we make the encryption randomness explicit by writing $\text{ct} \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}, m; r)$, where $r \leftarrow \mathcal{R}$ for a randomness space $\mathcal{R}$.
- $m' \leftarrow \text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$: the decryption algorithm takes as input the composite secret key csk and a ciphertext $\text{ct} \in \mathcal{CT}$, and outputs either a message $m' \in \mathcal{M}$ or the rejection symbol $\perp$.

Game IND-S(E)CPA _{AFS-PKE} ($\mathcal{A}$)	Game IND-LCK-SCPA _{AFS-PKE} ($\mathcal{A}$)
1 : $(\text{pk}_s^*, \text{sk}_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$	1 : $(\text{pk}_s^*, \text{sk}_s^*) \leftarrow \text{SKeyGen}(1^\lambda)$
2 : $\text{seed}_e^* \leftarrow \{0, 1\}^\lambda$	2 : $(\text{cpk}^*, m_0, m_1) \leftarrow \mathcal{A}(\text{pk}_s^*)$
3 : $(\text{pk}_e^*, \text{sk}_e^*) \leftarrow \text{EKeyGen}(\text{pk}_s^*, \text{seed}_e^*)$	3 : $b \leftarrow \{0, 1\}$
4 : $\text{cpk}^* \leftarrow \text{CombPK}(\text{pk}_s^*, \text{pk}_e^*)$	4 : $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$
5 : $(m_0, m_1) \leftarrow \mathcal{A}(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*)$	5 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}(\text{pk}_s^*, \text{ct}^*)$
6 : $(m_0, m_1) \leftarrow \mathcal{A}(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_s^*)$	6 : $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_e)$
7 : $b \leftarrow \{0, 1\}$	7 : $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$
8 : $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$	8 : if $\text{pk}'_s \neq \text{pk}_s^*$: abort
9 : $b' \leftarrow \mathcal{A}(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*, \text{ct}^*)$	9 : return $b' ? = b$
10 : $b' \leftarrow \mathcal{A}(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_s^*, \text{ct}^*)$	
11 : return $b' ? = b$	

Fig. 10. The IND-ECPA, IND-SCPA, and IND-LCK-SCPA security games for AFS-PKE. Dashed boxes denote IND-SCPA, and solid boxes denote IND-ECPA.

Correctness for AFS-PKE. An AFS-PKE scheme $\Pi_{\text{AFS-PKE}}$ with message space $\mathcal{M}$ is δ -correct if $\mathbb{E}[\max_{m \in \mathcal{M}} \Pr[\text{Dec}_{\text{AFS}}(\text{csk}, \text{Enc}_{\text{AFS}}(\text{cpk}, m)) \neq m]] \leq \delta$, where the expectation is taken over the

randomness of $(pk_s, sk_s) \leftarrow \$ \text{SKeyGen}(1^\lambda)$, $\text{seed} \leftarrow \$ \{0, 1\}^\lambda$, $(pk_e, sk_e) \leftarrow \text{EKeyGen}(pk_s, \text{seed})$, $\text{cpk} \leftarrow \text{CombPK}(pk_s, pk_e)$, and $\text{csk} \leftarrow \text{CombSK}(sk_s, sk_e)$.

Min-Entropy of the Combined Public Key. An AFS-PKE has combined public keys with min-entropy at least k if, for all $pk_s \in \mathcal{PK}_s$ and $\text{cpk}' \in \mathcal{CPK}$, $\Pr_{\text{seed} \leftarrow \$ \{0, 1\}^\lambda}[\text{CombPK}(pk_s, pk_e) = \text{cpk}'] \leq 2^{-k}$, where pk_e is generated as $(pk_e, sk_e) \leftarrow \text{EKeyGen}(pk_s, \text{seed})$.

We define the security notions IND-ECPA, IND-SCPA, and IND-LCK-SCPA for AFS-PKE, as shown in Fig. 10. These notions parallel the corresponding security definitions of AFS-KEM, but are formulated in the PKE setting without decryption oracle access.

6.3 A General Construction of CCA-Secure AFS-KEM from CPA-Secure AFS-PKE

Based on a CPA-secure AFS-PKE scheme, a CCA-secure AFS-KEM can be constructed via the FO_{AFS} transformation defined below, following the general FO paradigm of [FO99, HHK17, DHK⁺21]. Our FO_{AFS} transformation takes as input an AFS-PKE scheme together with hash functions $H : \{0, 1\}^* \rightarrow \mathcal{K}$, $G : \{0, 1\}^* \rightarrow \mathcal{R}$, and $H_{\text{rej}} : \{0, 1\}^* \rightarrow \mathcal{K}$, and outputs an AFS-KEM as described in Fig. 11. In our FO-transformed construction, the decapsulation algorithm additionally takes the combined public key cpk as input, and cpk is used to derive both the encapsulated key and the implicit-rejection keys.

In particular, the FO_{AFS} transformation enhances the security of the underlying PKE scheme $\Pi_{\text{AFS-PKE}}$ by lifting its IND-LCK-SCPA/IND-SCPA and IND-ECPA security to IND-LCK-SCCA/IND-SCCA and IND-ECPA security, respectively. Theorem 6.9 summarizes the resulting security guarantees of the FO_{AFS} transformation in both the ROM and the QROM. Notably, the proof follows the standard techniques used in prior analyses of FO-based KEM constructions [HHK17, JZC⁺18, JZM19, BHH⁺19, DHK⁺21].

$\text{SKeyGen}(1^\lambda)$	$\text{CombPK}(pk_s, pk_e)$	$\text{Decaps}_{\text{AFS}}(\text{cpk}, \text{csk}, \text{ct})$
1 : $s \leftarrow \$ \{0, 1\}^\lambda$	1 : $\text{cpk} \leftarrow \text{CombPK}'(pk_s, pk_e)$	1 : parse $(\text{csk}', s) \leftarrow \text{csk}$
2 : $(pk'_s, sk'_s) \leftarrow \$ \text{SKeyGen}'(1^\lambda)$	2 : return cpk	2 : $m' \leftarrow \text{Dec}_{\text{AFS}}(\text{csk}', \text{ct})$
3 : $sk_s \leftarrow (sk'_s, s)$, $pk_s \leftarrow pk'_s$	$\text{CombSK}(sk_s, sk_e)$	3 : $K \leftarrow H(\text{cpk}, m')$
4 : return (pk_s, sk_s)	1 : parse $(sk'_s, s) \leftarrow sk_s$	4 : $r \leftarrow G(\text{cpk}, m')$
$\text{EKeyGen}(pk_s, \text{seed})$	2 : $\text{csk}' \leftarrow \text{CombSK}'(sk'_s, sk_e)$	5 : $\tilde{K} \leftarrow H_{\text{rej}}(\text{cpk}, s, \text{ct})$
1 : $(pk_e, sk_e) \leftarrow \text{EKeyGen}'(pk_s, \text{seed})$	3 : $\text{csk} \leftarrow (\text{csk}', s)$	6 : if $m' = \perp$ or
2 : return (pk_e, sk_e)	4 : return csk	$\text{Enc}_{\text{AFS}}(\text{cpk}, m'; r) \neq \text{ct}$:
$\text{EKeyGen}_c(\text{cpk}, \text{seed})$	$\text{Encaps}_{\text{AFS}}(\text{cpk})$	7 : return $\tilde{K}$
1 : $(pk_e, sk_e) \leftarrow \text{EKeyGen}_c'(\text{cpk}, \text{seed})$	1 : $m \leftarrow \$ \mathcal{M}$	8 : return K
2 : return (pk_e, sk_e)	2 : $K \leftarrow H(\text{cpk}, m)$	
$\text{ExtractPK}(\text{cpk}, pk_e)$	3 : $r \leftarrow G(\text{cpk}, m)$	
1 : $pk_s \leftarrow \text{ExtractPK}'(\text{cpk}, pk_e)$	4 : $\text{ct} \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}, m; r)$	
2 : return pk_s	5 : return (ct, K)	

Fig. 11. Our FO_{AFS} transform for constructing an AFS-KEM $\Pi_{\text{AFS-KEM}} = (\text{SKeyGen}, \text{EKeyGen}, \text{EKeyGen}_c, \text{CombPK}, \text{CombSK}, \text{ExtractPK}, \text{Encaps}_{\text{AFS}}, \text{Decaps}_{\text{AFS}})$ from an AFS-PKE scheme $\Pi_{\text{AFS-PKE}} = (\text{SKeyGen}', \text{EKeyGen}', \text{EKeyGen}_c', \text{CombPK}', \text{CombSK}', \text{ExtractPK}', \text{Enc}_{\text{AFS}}, \text{Dec}_{\text{AFS}})$.

Theorem 6.9 (FO_{AFS} transform in the ROM and QROM). Let $\Pi_{\text{AFS-KEM}}$ denote the AFS-KEM scheme obtained by applying the FO_{AFS} transformation of Fig. 11 to an underlying δ -correct AFS-PKE scheme $\Pi_{\text{AFS-PKE}}$ over message space $\mathcal{M}$. Let H , G , and H_{rej} be modeled as random oracles. Assume that the composite public keys generated in $\Pi_{\text{AFS-PKE}}$ have min-entropy at least k . For any adversary $\mathcal{A}$ against the IND-LCK-SCCA security of $\Pi_{\text{AFS-KEM}}$ (resp. its IND-SCCA or IND-ECPA security) that makes at most q_D queries to $\mathcal{O}^{\text{Dec}_s}$, at most q_E queries to $\mathcal{O}^{\text{EKeyGen}}$, and at most q_H , q_G , and $q_{H_{\text{rej}}}$

queries to H , G , and H_{rej} , respectively, there exists an adversary $\mathcal{B}$ such that

$$\begin{aligned}\text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-LCK-SCCA}}(\mathcal{A}) &\leq 2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-LCK-SCPA}}(\mathcal{B}) + \alpha, \\ \text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-SCCA}}(\mathcal{A}) &\leq 2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-SCPA}}(\mathcal{B}) + \alpha, \\ \text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-ECPA}}(\mathcal{A}) &\leq 2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-ECPA}}(\mathcal{B}) + \frac{2(q_H + q_G)}{|\mathcal{M}|}.\end{aligned}$$

If H , G , and H_{rej} are modeled as quantum-accessible random oracles and $\mathcal{A}$ makes at most $q_H, q_G, q_{H_{\text{rej}}}$ (quantum) queries to H , G , and H_{rej} , respectively, then there exists an adversary $\mathcal{B}$ such that

$$\begin{aligned}\text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-LCK-SCCA}}(\mathcal{A}) &\leq 2\sqrt{\hat{q}\left(2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-LCK-SCPA}}(\mathcal{B}) + 4(q_G + q_H + q_D)/|\mathcal{M}|\right)} + \beta, \\ \text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-SCCA}}(\mathcal{A}) &\leq 2\sqrt{\hat{q}\left(2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-SCPA}}(\mathcal{B}) + 4(q_G + q_H + q_D)/|\mathcal{M}|\right)} + \beta, \\ \text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-ECPA}}(\mathcal{A}) &\leq 2\sqrt{\hat{q}_{\text{ECPA}}\left(2 \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-ECPA}}(\mathcal{B}) + 4(q_G + q_H)/|\mathcal{M}|\right)}.\end{aligned}$$

Here $\hat{q} = q_H + q_G + q_D + 1$, $\hat{q}_{\text{ECPA}} = q_H + q_G + 1$, $\alpha = 2(q_H + q_G + q_D)|\mathcal{M}|^{-1} + q_{H_{\text{rej}}}2^{-\lambda} + (q_E + 1)^2 2^{-k} + (q_D + q_G + q_H)\delta$, and $\beta = (q_E + 1)^2 2^{-k} + 2q_{H_{\text{rej}}}2^{-\lambda/2} + 16(q_E + 1)\delta(q_G + q_H + q_D + 1)^2$.

6.4 Security Analysis in the Random Oracle Model.

Security Analysis for IND-LCK-SCCA in the ROM.

Proof. We proceed with a sequence of games, which are given in detail in Fig. 12.

Game G_0 . Game G_0 is the original IND-LCK-SCCA security game. Hence,

$$\text{Adv}_{\Pi_{\text{KEM}}}^{\text{IND-LCK-SCCA}}(\mathcal{A}) = \left| \Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

Game G_1 . In this game, we abort if a collision occurs among the combined public keys cpk in $\mathcal{L}_e$. Conditioned on no abort, each cpk corresponds to at most one entry in the list $\mathcal{L}_e$, and hence we have $|V| \leq 1$ in $\mathcal{O}^{\text{Dec}_s}$. By the k -min-entropy of the combined public key cpk , the collision probability is at most $q_E^2/2^k$. Games G_0 and G_1 are identical until such a collision occurs, therefore,

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq \frac{q_E^2}{2^k}.$$

Game G_2 . We replace H_{rej} inside $\mathcal{O}^{\text{Dec}_s}$ by a new internal random oracle $\bar{H}_{\text{rej}}(\text{cpk}, \text{ct})$ that does not take the secret s^* as input. This modification is indistinguishable to $\mathcal{A}$ unless $\mathcal{A}$ queries the random oracle H_{rej} on an input containing s^* . Since $s^* \leftarrow \{0, 1\}^\lambda$ is hidden from $\mathcal{A}$, we obtain

$$|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]| \leq \frac{q_{H_{\text{rej}}}}{2^\lambda}.$$

Game G_3 . In this game, we simulate the decapsulation oracle $\mathcal{O}^{\text{Dec}_s}$ without using the secret key sk_s^* . Specifically, we only rely on $\bar{H}_{\text{rej}}$ in $\mathcal{O}^{\text{Dec}_s}$ and patch the random oracle H as shown in Fig. 12. Note that if AFS-PKE were perfectly correct, then the random oracle patching would also be perfectly correct, and therefore the two games would look identical in $\mathcal{A}$'s view.

We define the event CORR that $\mathcal{A}$ queries $G(\text{cpk}, m)$ for some tuple $(\text{cpk}, \text{pk}'_e, \text{sk}'_e, \text{seed}') \in \mathcal{L}_e$ such that a correctness error is induced. That is, we have $m \neq \text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$, where $\text{ct} = \text{Enc}_{\text{AFS}}(\text{cpk}, m; G(\text{cpk}, m))$ and $\text{csk} \leftarrow \text{CombSK}'(\text{sk}_s^*, \text{sk}'_e)$. Since there are at most $(q_D + q_G + q_H)$ explicit and implicit queries to G , and AFS-PKE is δ -correct, we have $\Pr[\text{CORR}] \leq (q_D + q_G + q_H)\delta$.

Next, we show that when the CORR event does not occur, G_2 and G_3 are identical. That is $|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\text{CORR}]$. Consider a query $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$ such that there exist $\text{pk}'_e, \text{sk}'_e, \text{seed}'$ with $(\text{cpk}, \text{pk}'_e, \text{sk}'_e, \text{seed}') \in \mathcal{L}_e$. Notably, if such a tuple does not exist, then $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$ return $\perp$ in both G_2 and G_3 . Moreover, let $\text{csk} \leftarrow \text{CombSK}'(\text{sk}_s^*, \text{sk}'_e)$ and define $m' := \text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$ and $\text{ct}' = \text{Enc}_{\text{AFS}}(\text{cpk}, m'; G(\text{cpk}, m'))$. Consider the following cases.

IND-LCK-SCCA Game G_0-G_3	$\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$
1 : $(\text{pk}_s^*, \text{sk}_s^*) \leftarrow \text{SKeyGen}'(1^\lambda)$ 2 : $s^* \leftarrow \{0, 1\}^\lambda$, $\text{L}_e := \emptyset$, $b \leftarrow \{0, 1\}$ 3 : $\text{Oracles} = \{\mathcal{O}^{\text{EKeyGen}}, \mathcal{O}^{\text{Dec}_s}, \text{H}, \text{H}_{\text{rej}}, \text{G}\}$ 4 : $\text{cpk}^* \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pk}_s^*)$ 5 : $m^* \leftarrow \mathcal{M}$ 6 : $\text{K}_0^* \leftarrow \text{H}(\text{cpk}^*, m^*)$ 7 : $r^* \leftarrow \text{G}(\text{cpk}^*, m^*)$ 8 : $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m^*; r^*)$ 9 : $\text{K}_1^* \leftarrow \mathcal{K}$ 10 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pk}_s^*, \text{ct}^*, \text{K}_b^*)$ 11 : $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}'_e(\text{cpk}^*, \text{seed}'_e)$ 12 : $\text{pk}'_s \leftarrow \text{ExtractPK}'(\text{cpk}^*, \text{pk}'_e)$ 13 : if $\text{pk}'_s \neq \text{pk}_s^*$: abort 14 : return $b' ? = b$	1 : if $\text{ct} = \text{ct}^* \wedge \text{cpk} = \text{cpk}^*$: return $\perp$ 2 : $V := \{(\text{pk}'_e, \text{sk}'_e, \text{seed}') \mid (\text{cpk}, \text{pk}'_e, \text{sk}'_e, \text{seed}') \in \text{L}_e\}$ 3 : if $ V \neq 1$: return $\perp$ 4 : $(\text{pk}'_e, \text{sk}'_e, \text{seed}') \leftarrow V$ 5 : $\tilde{K} \leftarrow \text{H}_{\text{rej}}(\text{cpk}, s^*, \text{ct})$ // G_0, G_1 6 : $\tilde{K} \leftarrow \bar{\text{H}}_{\text{rej}}(\text{cpk}, \text{ct})$ // G_2, G_3 7 : return $\tilde{K}$ // G_3 8 : $\text{csk} \leftarrow \text{CombSK}'(\text{sk}_s^*, \text{sk}'_e)$ 9 : $m' \leftarrow \text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$ 10 : $K \leftarrow \text{H}(\text{cpk}, m')$ 11 : $r \leftarrow \text{G}(\text{cpk}, m')$ 12 : if $m' = \perp$ or $\text{Enc}_{\text{AFS}}(\text{cpk}, m'; r) \neq \text{ct}$: 13 : return $\tilde{K}$ 14 : return K 15 :
$\mathcal{O}^{\text{EKeyGen}}()$ 1 : $\text{seed} \leftarrow \{0, 1\}^\lambda$ 2 : $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}'(\text{pk}_s^*, \text{seed})$ 3 : $\text{cpk} \leftarrow \text{CombPK}'(\text{pk}_s^*, \text{pk}_e)$ 4 : if $\exists (\text{cpk}, \cdot, \cdot, \cdot) \in \text{L}_e$: // G_1 5 : $\text{COLL} \leftarrow \text{true}$ // G_1 6 : abort // G_1 7 : $\text{L}_e \leftarrow \text{L}_e \cup \{(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})\}$ 8 : return $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$	$\text{H}(\text{cpk}, m)$ 1 : if $\exists K : (\text{cpk}, m, K) \in \text{L}_H$: return K 2 : $K \leftarrow \mathcal{K}$ 3 : if $\exists \text{pk}'_e, \text{sk}'_e, \text{seed}' : (\text{cpk}, \text{pk}'_e, \text{sk}'_e, \text{seed}') \in \text{L}_e$: // G_3 4 : $\text{ct}' \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}, m; \text{G}(\text{cpk}, m))$ // G_3 5 : $K \leftarrow \bar{\text{H}}_{\text{rej}}(\text{cpk}, \text{ct}')$ // G_3 6 : $\text{L}_H \leftarrow \text{L}_H \cup \{(\text{cpk}, m, K)\}$ 7 : return K
$\text{H}_{\text{rej}}, \text{G}$ 1 : Random oracles	

Fig. 12. Games G_0-G_3 for the proof of IND-LCK-SCCA security in the ROM

- Case 1: $m' = \perp$. H is never called on $m' = \perp$, and hence $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct}) = \bar{\text{H}}_{\text{rej}}(\text{cpk}, \text{ct})$ has the same distribution in both G_2 and G_3 .
- Case 2: $m' \neq \perp \wedge \text{ct} \neq \text{ct}'$. Both games return $\bar{\text{H}}_{\text{rej}}(\text{cpk}, \text{ct})$. The only way for $\mathcal{A}$ to detect a difference between the two games is to make a query $\text{H}(\text{cpk}, m)$ such that $\text{Enc}_{\text{AFS}}(\text{cpk}, m; \text{G}(\text{cpk}, m)) = \text{ct}$. Notably, in G_3 $\text{H}(\text{cpk}, m)$ would return the same key as in $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$, whereas in G_2 the two keys would be independent. Since $\text{ct} \neq \text{ct}'$, we also have $m \neq m'$. Therefore, such a query implies that the CORR event occurs.
- Case 3: $m' \neq \perp \wedge \text{ct} = \text{ct}'$. In Game G_2 we have $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct}) = \text{H}(\text{cpk}, m')$. In Game G_3 we have $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct}) = \text{H}(\text{cpk}, m)$ for all m that deterministically encrypt to the same ct . The only way for $\mathcal{A}$ to detect a difference between the two games is to query H on some value $m \neq m'$ that also deterministically encrypts to the same ct . But this also implies that the CORR event occurs.

Combining all the analyses we have,

$$|\Pr[G_2^A \Rightarrow 1] - \Pr[G_3^A \Rightarrow 1]| \leq (q_D + q_G + q_H)\delta.$$

Game G_4 . We replace K_0^* and r^* with uniformly random values and define the event QUERY that $\mathcal{A}$ queries either $\text{H}(\text{cpk}^*, m^*)$ or $\text{G}(\text{cpk}^*, m^*)$. If QUERY does not occur, then Games G_3 and G_4 are identical. Hence,

$$|\Pr[G_3^A \Rightarrow 1] - \Pr[G_4^A \Rightarrow 1]| \leq \Pr[\text{QUERY} : G_4].$$

In Game G_4 , the bit b is independent of the adversary's view, and thus

$$\Pr[G_4^A \Rightarrow 1] = 1/2.$$

To bound $\Pr[\text{QUERY} : G_4]$, we construct an adversary $\mathcal{B}$ against the IND-LCK-SCPA security of $\Pi_{\text{AFS-PKE}}$. Adversary $\mathcal{B}$ receives pk_s^* from its challenger and forwards it to $\mathcal{A}$. When $\mathcal{A}$ outputs cpk^* , adversary $\mathcal{B}$ submits it together with two random messages $m_0^*, m_1^* \leftarrow \$ \mathcal{M}$ to its challenger and receives a challenge ciphertext ct_b^* . Then adversary $\mathcal{B}$ perfectly simulates Game G_4 for $\mathcal{A}$ by embedding $\text{ct}^* = \text{ct}_b^*$. When $\mathcal{B}$ receives seed'_e from $\mathcal{A}$, $\mathcal{B}$ records that value for later output.

If $\mathcal{A}$ queries $\text{H}(\text{cpk}^*, m_{b'}^*)$ or $\text{G}(\text{cpk}^*, m_{b'}^*)$, adversary $\mathcal{B}$ eventually outputs (b', seed'_e) . Otherwise, $\mathcal{B}$ outputs a random bit and seed'_e . Notably, if Game G_4 does not abort, then seed'_e is a valid output for $\mathcal{B}$. Since m_{1-b}^* is information-theoretically hidden, the probability that $\mathcal{A}$ queries m_{1-b}^* is at most $(q_H + q_G + q_D)/|\mathcal{M}|$. Assume that this is not the case. We have

$$\begin{aligned} \text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-LCK-SCPA}}(\mathcal{B}) + \frac{q_H + q_G + q_D}{|\mathcal{M}|} &\geq \left| \Pr[b = b'] - \frac{1}{2} \right| \\ &= \left| \Pr[\text{QUERY} : G_4] + \frac{1}{2} \Pr[\neg \text{QUERY} : G_4] - \frac{1}{2} \right| \\ &= \frac{1}{2} \Pr[\text{QUERY} : G_4]. \end{aligned}$$

Therefore, $\Pr[\text{QUERY} : G_4] \leq 2 \left(\text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-LCK-SCPA}}(\mathcal{B}) + \frac{q_H + q_G + q_D}{|\mathcal{M}|} \right)$.

Combining all the bounds completes the proof. $\square$

Security Analysis for IND-SCCA in the ROM.

Proof. Since the proof closely follows the analysis of IND-LCK-SCCA in the ROM, we only highlight the differences and omit repeated arguments.

Game G_0 is the original IND-SCCA security game, i.e., $\text{Adv}_{\Pi_{\text{AFS-KEM}}}^{\text{IND-SCCA}}(\mathcal{A}) = |\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2}|$. Games G_1 – G_4 follow the same sequence of transformations as in the IND-LCK-SCCA proof in the ROM. Specifically, in G_1 we abort if a collision for cpk occurs in $\mathcal{L}_e$. Notably, we may have $q_E + 1$ elements in $\mathcal{L}_e$. In Games G_2 and G_3 , we transform the experiment to simulate the oracle $\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$ without knowledge of sk_s^* . Finally, in Game G_4 , we replace $K_0^* = \text{H}(\text{cpk}^*, m^*)$ and $r^* = \text{G}(\text{cpk}^*, m^*)$ with uniformly random values $K_0^* \leftarrow \$ \mathcal{K}$ and $r^* \leftarrow \$ \mathcal{R}$. Recall that the event **QUERY** denotes that the adversary queries $\text{H}(\text{cpk}^*, m^*)$ or $\text{G}(\text{cpk}^*, m^*)$.

It remains to bound $\Pr[\text{QUERY} : G_4]$. We construct a reduction $\mathcal{B}$ against the IND-SCPA security of AFS-PKE as follows. Adversary $\mathcal{B}$ receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*)$ from its challenger, chooses random messages $m_0^*, m_1^* \leftarrow \$ \mathcal{M}$, and submits them to obtain a challenge ciphertext ct_b^* . It then forwards $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*, \text{ct}_b^*, K^*)$ to $\mathcal{A}$ and perfectly simulates the game as G_4 , where K^* is sampled uniformly at random. If $\mathcal{A}$ queries $\text{H}(\text{cpk}^*, m_{b'}^*)$ or $\text{G}(\text{cpk}^*, m_{b'}^*)$ in the game, $\mathcal{B}$ outputs b' and aborts. Otherwise it outputs a random bit. Since m_{1-b}^* is information-theoretically hidden, the probability that it is queried is at most $(q_H + q_G + q_D)/|\mathcal{M}|$. Assume that this is not the case. Similar to the analysis in IND-LCK-SCCA, we have

$$\Pr[\text{QUERY} : G_4] \leq 2 \left(\text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-SCPA}}(\mathcal{B}) + \frac{q_H + q_G + q_D}{|\mathcal{M}|} \right).$$

This completes the proof. $\square$

Security Analysis for IND-ECPA in the ROM.

Proof. Game G_0 is the original IND-ECPA security game, that is,

$$\text{Adv}_{\Pi_{\text{KEM}}}^{\text{IND-ECPA}}(\mathcal{A}) = \left| \Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

Game G_1 . In Game G_1 , we replace $K_0^* = \text{H}(\text{cpk}^*, m^*)$ and $r^* = \text{G}(\text{cpk}^*, m^*)$ with uniformly random values $K_0^* \leftarrow \$ \mathcal{K}$ and $r^* \leftarrow \$ \mathcal{R}$. Notably, r^* is the randomness used to compute the challenge ciphertext ct^* . Let **QUERY** denote the event that the adversary queries $\text{H}(\text{cpk}^*, m^*)$ or $\text{G}(\text{cpk}^*, m^*)$. When **QUERY** does not occur, Games G_0 and G_1 are identical. Therefore,

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\text{QUERY} : G_1].$$

IND-LCK-SCCA Game G_0-G_6	$\mathcal{O}^{\text{Dec}_s}(\text{cpk}, \text{ct})$
1: $(\text{pk}_s^*, \text{sk}_s^*) \leftarrow \mathcal{S} \text{KeyGen}'(1^\lambda)$ 2: $s^* \leftarrow \mathcal{S} \{0, 1\}^\lambda$, $L_e := \emptyset$, $b \leftarrow \mathcal{S} \{0, 1\}$, $\text{cn} \leftarrow 0$ 3: $\text{Oracles} = \{\mathcal{O}^{\text{EKeyGen}}, \mathcal{O}^{\text{Dec}_s}, \text{H}\rangle, \text{H}_{\text{rej}}\rangle, \text{G}\rangle\}$ 4: for $0 \leq i \leq q_E - 1$: // G_1-G_6 5: $\text{seed}^i \leftarrow \mathcal{S} \{0, 1\}^\lambda$ // G_1-G_6 6: $(\text{pk}_e^i, \text{sk}_e^i) \leftarrow \text{EKeyGen}'(\text{pk}_s^*, \text{seed}^i)$ // G_1-G_6 7: $\text{cpk}^i \leftarrow \text{CombPK}'(\text{pk}_s^*, \text{pk}_e^i)$ // G_1-G_6 8: $L_e \leftarrow L_e \cup \{(\text{cpk}^i, \text{pk}_e^i, \text{sk}_e^i, \text{seed}^i)\}$ // G_1-G_6 9: if $\exists i \neq j : \text{cpk}^i = \text{cpk}^j$: abort // G_2-G_6 10: $\text{cpk}^* \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pk}_s^*)$ 11: $m^* \leftarrow \mathcal{S} \mathcal{M}$ 12: $K_0^* \leftarrow \text{H}(\text{cpk}^*, m^*)$ 13: $r^* \leftarrow \text{G}(\text{cpk}^*, m^*)$ 14: $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m^*; r^*)$ 15: $K_1^* \leftarrow \mathcal{S} \mathcal{K}$ 16: $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{\text{Oracles}}(\text{pk}_s^*, \text{ct}^*, K_0^*)$ 17: $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}'_e(\text{cpk}^*, \text{seed}'_e)$ 18: $\text{pk}'_s \leftarrow \text{ExtractPK}'(\text{cpk}^*, \text{pk}'_e)$ 19: if $\text{pk}'_s \neq \text{pk}_s^*$: abort 20: return $b' = b$	1: if $\text{ct} = \text{ct}^* \wedge \text{cpk} = \text{cpk}^*$: return $\perp$ 2: $V \leftarrow \{(\text{pk}'_e, \text{sk}'_e, \text{seed}') \mid (\text{cpk}, \text{pk}'_e, \text{sk}'_e, \text{seed}') \in L_e\}$ 3: if $ V \neq 1$: return $\perp$ 4: $(\text{pk}'_e, \text{sk}'_e, \text{seed}') \leftarrow V$ 5: $\tilde{K} \leftarrow \text{H}_{\text{rej}}(\text{cpk}, s^*, \text{ct})$ // G_0-G_2 6: $\tilde{K} \leftarrow \text{H}_{\text{rej}}(\text{cpk}, \text{ct})$ // G_3-G_6 7: return $\tilde{K}$ // G_5, G_6 8: $\text{csk} \leftarrow \text{CombSK}'(\text{sk}_s^*, \text{sk}'_e)$ 9: $m' \leftarrow \text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$ 10: $K \leftarrow \text{H}(\text{cpk}, m')$ 11: $r \leftarrow \text{G}(\text{cpk}, m')$ 12: if $m' = \perp$ or $\text{Enc}_{\text{AFS}}(\text{cpk}, m'; r) \neq \text{ct}$: 13: return $\tilde{K}$ 14: return K
$\mathcal{O}^{\text{EKeyGen}}()$	$\text{H}(\text{cpk}, m)$
1: $\text{cn} \leftarrow \text{cn} + 1$ // G_1-G_6 2: return $(\text{cpk}^{\text{cn}}, \text{pk}_e^{\text{cn}}, \text{sk}_e^{\text{cn}}, \text{seed}^{\text{cn}})$ // G_1-G_6 3: $\text{seed} \leftarrow \mathcal{S} \{0, 1\}^\lambda$ 4: $(\text{pk}_e, \text{sk}_e) \leftarrow \text{EKeyGen}'(\text{pk}_s^*, \text{seed})$ 5: $\text{cpk} \leftarrow \text{CombPK}'(\text{pk}_s^*, \text{pk}_e)$ 6: $L_e \leftarrow L_e \cup \{(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})\}$ 7: return $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$	1: if $\exists i : \text{cpk} = \text{cpk}^i$: // G_5, G_6 2: $\text{ct}' \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}, m; \text{G}(\text{cpk}, m))$ // G_5, G_6 3: return $\text{H}_{\text{rej}}(\text{cpk}, \text{ct}')$ // G_5, G_6 4: return $\text{H}(\text{cpk}, m)$
	$\text{G}(\text{cpk}, m)$
	1: if $\exists i : \text{cpk} = \text{cpk}^i$: // G_2-G_6 2: if $\exists i : \text{cpk} = \text{cpk}^i$: // G_2-G_6 3: return $\text{G}_i(m)$ // G_2, G_3, G_6 4: return $\text{G}'_i(m)$ // G_4, G_5 5: return $\text{G}(\text{cpk}, m)$
	H_{rej}
	1: Random Oracles

Fig. 13. Games G_0-G_6 for the proof of IND-LCK-SCCA security in the QROM

In Game G_1 , the bit b is independent of the adversary's view, and thus

$$\Pr[G_1^A \Rightarrow 1] = 1/2.$$

It remains to bound $\Pr[\text{QUERY} : G_1]$. We construct a reduction $\mathcal{B}$ against the IND-ECPA security of AFS-PKE as follows. Adversary $\mathcal{B}$ receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_e^*)$ from its challenger, chooses random messages $m_0^*, m_1^* \leftarrow \mathcal{S} \mathcal{M}$, and submits them to obtain a challenge ciphertext ct_b^* . It then forwards $(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_e^*, \text{ct}_b^*, K^*)$ to $\mathcal{A}$ and perfectly simulates Game G_1 , where K^* is sampled uniformly at random. If $\mathcal{A}$ queries $\text{H}(\text{cpk}^*, m_{b'}^*)$ or $\text{G}(\text{cpk}^*, m_{b'}^*)$, $\mathcal{B}$ outputs b' and aborts; otherwise it outputs a random bit. Since m_{1-b}^* is information-theoretically hidden, the probability that it is queried is at most $(q_H + q_G)/|\mathcal{M}|$. Assume that this is not the case. We have

$$\Pr[\text{QUERY} : G_1] \leq 2 \left(\text{Adv}_{\Pi_{\text{AFS-PKE}}}^{\text{IND-ECPA}}(\mathcal{B}) + \frac{q_H + q_G}{|\mathcal{M}|} \right).$$

This completes the proof. $\square$

6.5 Security Analysis in the Quantum-Accessible Random Oracle Model.

Security Analysis for IND-LCK-SCCA in the QROM.

Proof. The detailed descriptions of the games are given below, and an overview of Games G_0 to G_6 is provided in Figure 13.

Game G_0 . Game G_0 is the original IND-LCK-SCCA security game. Hence,

$$\text{Adv}_{\Pi_{\text{KEM}}}^{\text{IND-LCK-SCCA}}(\mathcal{A}) = \left| \Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

Game G_1 . In this game, we precompute all tuples $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$ that will be stored in $\mathbf{L}_e$. Recall that there are at most q_E queries to $\mathcal{O}^{\text{KeyGen}}()$, we denote the resulting tuples by $(\text{cpk}^i, \text{pk}_e^i, \text{sk}_e^i, \text{seed}^i)$ for $0 \leq i \leq q_E - 1$. Moreover, we define $\text{csk}^i = \text{CombSK}(\text{sk}_s^*, \text{sk}_e^i)$. This change is purely conceptual and does not affect the adversary's view. That is,

$$\Pr[G_0^{\mathcal{A}} \Rightarrow 1] = \Pr[G_1^{\mathcal{A}} \Rightarrow 1].$$

Game G_2 . In this game, we abort if a collision occurs among the precomputed combined public keys, that is, if there exist indices $i \neq j$ such that $\text{cpk}^i = \text{cpk}^j$. Conditioned on no abort, each cpk corresponds to at most one entry in $\mathbf{L}_e$, and hence we have $|V| \leq 1$ in $\mathcal{O}^{\text{Dec}_s}$. By the k -min-entropy of the combined public key, the collision probability is at most $q_E^2/2^k$. Moreover, we use an internal random oracle $\mathbf{G}_i(m)$ to simulate $\mathbf{G}(\text{cpk}^i, m)$. Games G_1 and G_2 are identical until such a collision occurs; therefore,

$$|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]| \leq \frac{q_E^2}{2^k}.$$

Game G_3 . In this game, we replace $\mathbf{H}_{\text{rej}}$ inside $\mathcal{O}^{\text{Dec}_s}$ by a new internal random oracle $\bar{\mathbf{H}}_{\text{rej}}(\text{cpk}, \text{ct})$ that does not take the secret s^* as input. Since $s^* \leftarrow_{\$} \{0, 1\}^\lambda$ is hidden from $\mathcal{A}$, Lemma 5.6 implies

$$|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]| \leq \frac{2q_{\mathbf{H}_{\text{rej}}}}{\sqrt{2}^\lambda}.$$

Game G_4 . In this game, for $0 \leq i \leq q_E - 1$, we replace the internal random oracle $\mathbf{G}_i$ by an oracle $\mathbf{G}'_i$ which samples only “good” randomness for any combined public key cpk^i , meaning that no decryption failures can occur when using $\text{Dec}_{\text{AFS}}(\text{csk}^i, \cdot)$ on proper ciphertexts of the form $\text{Enc}_{\text{AFS}}(\text{cpk}^i, \cdot)$. Specifically, for each $0 \leq i \leq q_E - 1$ and $m \in \mathcal{M}$, we define $\mathcal{R}_{\text{BAD}}^i(m) = \{r \in \mathcal{R} : \text{Dec}_{\text{AFS}}(\text{csk}^i, \text{Enc}_{\text{AFS}}(\text{cpk}^i, m)) \neq m\}$. In G_4 , we replace each internal oracle $\mathbf{G}_i$, where $0 \leq i \leq q_E - 1$, by an oracle $\mathbf{G}'_i$ that uniformly samples an output in $\mathcal{R} \setminus \mathcal{R}_{\text{BAD}}^i$. Following the same analysis in the proof of [DHK⁺21, Theorem 3.2, Game 4] and a hybrid argument, we have

$$|\Pr[G_3^{\mathcal{A}} \Rightarrow 1] - \Pr[G_4^{\mathcal{A}} \Rightarrow 1]| \leq 8q_E \delta(q_G + q_H + q_D + 1)^2.$$

Game G_5 . In this game, we modify the behavior of $\mathbf{H}(\text{cpk}, m)$ as follows. For every input such that $\text{cpk} = \text{cpk}^i$ for some $0 \leq i \leq q_E - 1$, we set $\mathbf{H}(\text{cpk}, m) := \bar{\mathbf{H}}_{\text{rej}}(\text{cpk}^i, \text{ct})$, where $\text{ct} := \text{Enc}_{\text{AFS}}(\text{cpk}^i, m; \mathbf{G}'_i(\text{cpk}^i, m))$. Under this modification of $\mathbf{H}$, the oracle $\mathcal{O}^{\text{Dec}_s}$ can be simulated without relying on the secret key sk_s^* . Concretely, upon any query $\mathcal{O}^{\text{Dec}_s}(\text{cpk}^i, \text{ct})$ with $0 \leq i \leq q_E - 1$, the oracle responds with $K = \bar{\mathbf{H}}_{\text{rej}}(\text{cpk}^i, \text{ct})$.

Due to the modification introduced in G_4 , for every tuple (cpk^i, m) , it holds that if $\text{Enc}_{\text{AFS}}(\text{cpk}^i, m; \mathbf{G}(\text{cpk}^i, m)) = \text{ct}$, then we have $\text{Dec}_{\text{AFS}}(\text{csk}^i, \text{ct}) = m$. Consequently, the mapping from (cpk^i, m) to $(\text{cpk}^i, \text{ct})$ is injective. Moreover, since $\bar{\mathbf{H}}_{\text{rej}}$ is never evaluated on such inputs in G_4 , the adversary's view is identical in Games G_4 and G_5 . That is,

$$\Pr[G_4^{\mathcal{A}} \Rightarrow 1] = \Pr[G_5^{\mathcal{A}} \Rightarrow 1].$$

Game G_6 . In this game, we undo the changes in G_4 , that is, we replace each $\mathbf{G}'_i$ by $\mathbf{G}_i$ for $0 \leq i \leq q_E - 1$. Using the same argument as in G_4 , we have

$$|\Pr[G_5^{\mathcal{A}} \Rightarrow 1] - \Pr[G_6^{\mathcal{A}} \Rightarrow 1]| \leq 8q_E \delta(q_G + q_H + q_D + 1)^2.$$

Game G_7 . In this game, we introduce a single random oracle $\mathbf{H} \times \mathbf{G} : \mathcal{CPK} \times \mathcal{M} \rightarrow \mathcal{K} \times \mathcal{R}$ to jointly simulate the random oracles $\mathbf{H}$ and $\mathbf{G}$. That is, we define $(\mathbf{H}(\text{cpk}, m), \mathbf{G}(\text{cpk}, m)) := \mathbf{H} \times \mathbf{G}(\text{cpk}, m)$. Since this change is purely conceptual, we have

$$\Pr[G_6^{\mathcal{A}} \Rightarrow 1] = \Pr[G_7^{\mathcal{A}} \Rightarrow 1].$$

Game G_8 . In this game, we replace $H \times G$ with another oracle $\overline{H \times G}$. In particular, $\overline{H \times G}(x) = (K, r)$ if $x = (\text{cpk}^*, m^*)$, where K, r are randomly chosen from $\mathcal{K}, \mathcal{R}$, and $\overline{H \times G}(x) = H \times G(x)$ for all $x \neq (\text{cpk}^*, m^*)$. Notably, G_8 still uses $H \times G(\text{cpk}^*, m^*)$ to compute K_0^* and r^* . Therefore in Game G_8 , the adversary's view is independent of b , and

$$\Pr[G_8^A \Rightarrow 1] = 1/2.$$

Since we have reprogrammed $H \times G$ in G_8 on (cpk^*, m^*) , by Lemma 5.9 we have

$$|\Pr[G_7^A \Rightarrow 1] - \Pr[G_8^A \Rightarrow 1]| \leq 2\sqrt{(q_H + q_G + q_D + 1) \Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)]}.$$

Here $\mathcal{C}$ runs Game G_8 with adversary $\mathcal{A}$ using the oracle $\overline{H \times G} \setminus (\text{cpk}^*, m^*)$ instead of $H \times G$. In particular, $m^* \leftarrow \$ \mathcal{M}$ is treated as an input to $\mathcal{C}$. Moreover, $\Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}]$ denotes the probability that the oracle $\mathcal{O}_{(\text{cpk}^*, m^*)}^{SC}$ ever outputs 1 during the execution of $\mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)$.

Game G_9 . In this game, we change $c^* = \text{Enc}_{\text{AFS}}(\text{cpk}^*, m^*; r^*)$ to $c^* = \text{Enc}_{\text{AFS}}(\text{cpk}^*, \hat{m}^*; r^*)$, where $\hat{m}^*$ is sampled independently from $\mathcal{M}$. Let $\mathcal{D}$ denote the algorithm with no input that runs Game G_9 with adversary $\mathcal{A}$ and oracle $\overline{H \times G} \setminus (\text{cpk}^*, m^*)$.

To bound $|\Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()]|$, we construct an adversary $\mathcal{B}$ against the IND-LCK-SCPA security of AFS-PKE as follows. Adversary $\mathcal{B}$ receives pk_s^* from its challenger and forwards it to $\mathcal{A}$. When $\mathcal{A}$ outputs cpk^* , adversary $\mathcal{B}$ submits it together with two random messages $m_0^*, m_1^* \leftarrow \$ \mathcal{M}$ to its challenger and receives a challenge ciphertext ct_b^* . Then $\mathcal{B}$ perfectly simulates Game G_9 for $\mathcal{A}$, except that it embeds $\text{ct}^* = \text{ct}_b^*$ and uses $\overline{H \times G} \setminus (\text{cpk}^*, m_0^*)$ to simulate the random oracle instead of $H \times G$. Finally, if FIND occurs, $\mathcal{B}$ outputs 1; otherwise it outputs 0.

It is clear that if $b = 0$, then $\mathcal{B}$ provides a perfect simulation of $\mathcal{C}$, and if $b = 1$, it provides a perfect simulation of $\mathcal{D}$. That is, $\Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)] = \Pr[\mathcal{B} \Rightarrow 1 : b = 0]$ and $\Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()] = \Pr[\mathcal{B} \Rightarrow 1 : b = 1]$. Therefore, we have

$$|\Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()]| \leq 2 \text{Adv}_{\text{AFS-PKE}}^{\text{IND-LCK-SCPA}}(\mathcal{B}).$$

Analysis of $\Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()]$. There is clear independence between the (empty) input of $\mathcal{D}$ and the set S . That is, m^* is independent of the adversary's view. Using Lemma 5.10, we have

$$\Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()] \leq 4(q_G + q_H + q_D)/|\mathcal{M}|.$$

Combining all the bounds completes the proof. $\square$

Security Analysis for IND-SCCA in the QROM.

Proof. Since the proof closely follows the analysis of IND-LCK-SCCA in the QROM, we only highlight the differences here. Game G_0 is the original IND-SCCA security game, i.e.,

$$\text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{A}) = \left| \Pr[G_0^A \Rightarrow 1] - \frac{1}{2} \right|.$$

Games G_1 – G_9 follow the same sequence of transformations as in the IND-LCK-SCCA proof in the QROM. In particular, in this case we have $q_E + 1$ elements in $\mathcal{L}_e$. Thus, we replace the term q_E in the IND-LCK-SCCA proof with $q_E + 1$. The final difference from the IND-LCK-SCCA analysis in this proof is the bound $\Pr[\text{FIND} : \mathcal{C}^{H \times G \setminus (\text{cpk}^*, m^*)}(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{H \times G \setminus (\text{cpk}^*, m^*)}()]$.

To bound this term, we construct an adversary $\mathcal{B}$ against the IND-SCPA security of AFS-PKE as follows. Adversary $\mathcal{B}$ receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*)$ from its challenger, chooses random messages $m_0^*, m_1^* \leftarrow \$ \mathcal{M}$, and submits them to obtain a challenge ciphertext ct_b^* . It then forwards $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*, \text{ct}_b^*, K^*)$ to $\mathcal{A}$ and perfectly simulates the game as in G_9 , where K^* is sampled uniformly at random. Moreover, $\mathcal{B}$ uses $\overline{H \times G} \setminus (\text{cpk}^*, m_0^*)$ to simulate the random oracle instead of $H \times G$. Finally, if FIND occurs, $\mathcal{B}$ outputs 1; otherwise, it outputs 0.

It is clear that if $b = 0$, then $\mathcal{B}$ provides a perfect simulation of $\mathcal{C}$, and if $b = 1$, it provides a perfect simulation of $\mathcal{D}$. That is, $\Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)] = \Pr[\mathcal{B} \Rightarrow 1 : b = 0]$ and $\Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()] = \Pr[\mathcal{B} \Rightarrow 1 : b = 1]$. Therefore, we have

$$\left| \Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()] \right| \leq 2 \text{Adv}_{\text{AFS-PKE}}^{\text{IND-SCPA}}.$$

This completes the proof. $\square$

Security Analysis for IND-ECPA in the QROM.

Proof. Game G_0 is the original IND-ECPA security game, that is,

$$\text{Adv}_{\Pi_{\text{KEM}}}^{\text{IND-ECPA}}(\mathcal{A}) = \left| \Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

Game G_1 . In this game, we introduce a single random oracle $H \times G : \mathcal{CPK} \times \mathcal{M} \rightarrow \mathcal{K} \times \mathcal{R}$ to jointly simulate the random oracles H and G . That is, we define $(H(\text{cpk}, m), G(\text{cpk}, m)) := H \times G(\text{cpk}, m)$. The total number of oracle queries made by the adversary to $H \times G$ is at most $q_H + q_G$. Since this change is purely conceptual, we have

$$\Pr[G_0^{\mathcal{A}} \Rightarrow 1] = \Pr[G_1^{\mathcal{A}} \Rightarrow 1].$$

Game G_2 . In this game, we replace $H \times G$ with another oracle $\overline{H \times G}$. In particular, $\overline{H \times G}(x) = (K, r)$ if $x = (\text{cpk}^*, m^*)$, where K, r are sampled uniformly from $\mathcal{K}, \mathcal{R}$, and $\overline{H \times G}(x) = H \times G(x)$ for all $x \neq (\text{cpk}^*, m^*)$. Notably, Game G_2 still uses $H \times G(\text{cpk}^*, m^*)$ to compute K_0^* and r^* . Therefore, in Game G_2 , the adversary's view is independent of b , and thus

$$\Pr[G_2^{\mathcal{A}} \Rightarrow 1] = 1/2.$$

Since we have reprogrammed $H \times G$ in G_2 on (cpk^*, m^*) , Lemma 5.9 implies $|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]| \leq 2\sqrt{(q_H + q_G + q_D + 1) \Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)]}$. Here $\mathcal{C}$ runs Game G_2 with adversary $\mathcal{A}$ using the oracle $\overline{H \times G} \setminus (\text{cpk}^*, m^*)$ instead of $H \times G$. In particular, $m^* \leftarrow \mathcal{M}$ is treated as an input to $\mathcal{C}$. Moreover, $\Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)]$ denotes the probability that the oracle $\mathcal{O}_{(\text{cpk}^*, m^*)}^{\overline{H \times G}}$ ever outputs 1 during the execution of $\mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)$.

Game G_3 . In this game, we change $c^* = \text{Enc}_{\text{AFS}}(\text{cpk}^*, m^*; r^*)$ to $c^* = \text{Enc}_{\text{AFS}}(\text{cpk}^*, \hat{m}^*; r^*)$, where $\hat{m}^*$ is sampled independently from $\mathcal{M}$. Let $\mathcal{D}$ denote the algorithm with no input that runs Game G_3 with adversary $\mathcal{A}$ and oracle $\overline{H \times G} \setminus (\text{cpk}^*, m^*)$.

To bound $|\Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()]|$, we construct a reduction $\mathcal{B}$ against the IND-ECPA security of AFS-PKE. Adversary $\mathcal{B}$ receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_s^*)$ from its challenger, chooses random messages $m_0^*, m_1^* \leftarrow \mathcal{M}$, and submits them to obtain a challenge ciphertext ct_b^* . It then forwards $(\text{cpk}^*, \text{pk}_s^*, \text{pk}_e^*, \text{sk}_s^*, \text{ct}_b^*, K^*)$ to $\mathcal{A}$ and perfectly simulates Game G_3 , where K^* is sampled uniformly at random. Moreover, $\mathcal{B}$ uses $\overline{H \times G} \setminus (\text{cpk}^*, m_0^*)$ to simulate the random oracle instead of $H \times G$. Finally, if FIND occurs, $\mathcal{B}$ outputs 1; otherwise it outputs 0. It is clear that if $b = 0$, then $\mathcal{B}$ provides a perfect simulation of $\mathcal{C}$, and if $b = 1$, it provides a perfect simulation of $\mathcal{D}$. That is, $\Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)] = \Pr[\mathcal{B} \Rightarrow 1 : b = 0]$ and $\Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()] = \Pr[\mathcal{B} \Rightarrow 1 : b = 1]$. Therefore, we have

$$\left| \Pr[\text{FIND} : \mathcal{C}^{\overline{H \times G}}(\text{cpk}^*, m^*)(m^*)] - \Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()] \right| \leq 2 \text{Adv}_{\text{AFS-PKE}}^{\text{IND-ECPA}}.$$

Analysis of $\Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()]$. There is clear independence between the (empty) input of $\mathcal{D}$ and the set S , and thus m^* is independent of the adversary's view. Using Lemma 5.10, we obtain

$$\Pr[\text{FIND} : \mathcal{D}^{\overline{H \times G}}(\text{cpk}^*, m^*)()] \leq 4(q_G + q_H)/|\mathcal{M}|.$$

This completes the proof. $\square$

7 AFS-KEM-BW: AFS-KEM Instantiations from BW-KEM

In this section, we present the construction AFS-KEM-BW derived from BW-KEM [LZS⁺26] under the standard MLWE assumption. The scheme is obtained by applying the FO_{AFS} transformation to the BW-KEM-based instantiation of authenticated forward-secure PKE, denoted AFS-PKE-BW.

In AFS-KEM-BW, we rely on the Module Learning with Errors (MLWE) assumption, as in BW-KEM.

Definition 7.1 (Module-LWE Problem [LS15]). Let χ be a probability distribution over the ring $\mathcal{R}_q$, and let k, l, q be positive integers. For an adversary $\mathcal{A}$, its advantage against the decisional module learning with errors problem $\text{MLWE}_{\chi, k, l, q}$ is defined as

$$\text{Adv}_{\text{MLWE}_{\chi, k, l, q}}(\mathcal{A}) := |\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u}) = 1]|,$$

where $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times l}$, $\mathbf{s} \leftarrow \chi^l$, $\mathbf{e} \leftarrow \chi^k$, and $\mathbf{u} \leftarrow \mathcal{R}_q^k$.

SKeyGen(1^λ)	EKeyGen($\text{pk}_s, \text{seed}_e$)	EKeyGen _c ($\text{cpk}, \text{seed}_e$)
1 : $\text{seed}_{\text{pk}} \leftarrow \{0, 1\}^\lambda$	1 : $\text{parse}(\text{seed}_{\text{pk}}, \cdot) \leftarrow \text{pk}_s$	1 : $\text{parse}(\text{seed}_{\text{pk}}, \cdot) \leftarrow \text{cpk}$
2 : $\text{seed}_s \leftarrow \{0, 1\}^\lambda$	2 : $\mathbf{A} \leftarrow \text{G}(\text{seed}_{\text{pk}})$	2 : $\mathbf{A} \leftarrow \text{G}(\text{seed}_{\text{pk}})$
3 : $\mathbf{A} \leftarrow \text{G}(\text{seed}_{\text{pk}}) \in \mathcal{R}_q^{k \times k}$	3 : $(\mathbf{s}_e, \mathbf{e}_e) \leftarrow \text{G}'(\text{seed}_e)$	3 : $(\mathbf{s}_e, \mathbf{e}_e) \leftarrow \text{G}'(\text{seed}_e)$
4 : $(\mathbf{s}_s, \mathbf{e}_s) \leftarrow \text{G}'(\text{seed}_s)$	$\in \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$	$\in \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$
$\in \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$	4 : $\mathbf{t}_e = \mathbf{A}\mathbf{s}_e + \mathbf{e}_e \bmod q$	4 : $\mathbf{t}_e = \mathbf{A}\mathbf{s}_e + \mathbf{e}_e \bmod q$
5 : $\mathbf{t}_s = \mathbf{A}\mathbf{s}_s + \mathbf{e}_s \bmod q$	5 : $\text{pk}_e = (\text{seed}_{\text{pk}}, \mathbf{t}_e)$	5 : $\text{pk}_e = (\text{seed}_{\text{pk}}, \mathbf{t}_e)$
6 : $\text{pk}_s = (\text{seed}_{\text{pk}}, \mathbf{t}_s)$	6 : $\text{sk}_e = \mathbf{s}_e$	6 : $\text{sk}_e = \mathbf{s}_e$
7 : $\text{sk}_s = \mathbf{s}_s$	7 : return $(\text{pk}_e, \text{sk}_e)$	7 : return $(\text{pk}_e, \text{sk}_e)$
8 : return $(\text{pk}_s, \text{sk}_s)$	Enc_{AFS} (cpk, m, r)	Dec_{AFS} (csk, ct)
CombSK (sk_s, sk_e)	1 : $\text{parse}(\text{seed}_{\text{pk}}, \mathbf{t}_c) \leftarrow \text{cpk}$	1 : $\mathbf{u}' = \text{Decompress}_q(\text{ct}_1, d_u)$
1 : $\mathbf{s}_c = \text{sk}_s + \text{sk}_e \bmod q$	2 : $\mathbf{A} \leftarrow \text{G}(\text{seed}_{\text{pk}})$	2 : $\mathbf{v}' = \text{Decompress}_q(\text{ct}_2, d_v)$
2 : $\text{csk} = \mathbf{s}_c$	3 : $\mathbf{r} \leftarrow \mathcal{B}_{\eta_1}^k$	3 : $\mathbf{m}' = \text{Decode}(\mathbf{v}' - \mathbf{s}_c^T \mathbf{u}')$
3 : return csk	4 : $(\mathbf{e}_1, \mathbf{e}_2) \leftarrow \mathcal{B}_{\eta_2}^k \times \mathcal{B}_{\eta_2}^k$	4 : return $\mathbf{m}'$
CombPK (pk_s, pk_e)	5 : $\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$	ExtractPK (cpk, pk_e)
1 : $\text{parse}(\text{seed}_{\text{pk}}, \mathbf{t}_s) \leftarrow \text{pk}_s$	6 : $\mathbf{v} = \mathbf{t}_c^T \mathbf{r} + \mathbf{e}_2 + \text{Encode}(m)$	1 : $\text{parse}(\text{seed}_{\text{pk}}, \mathbf{t}_c) \leftarrow \text{cpk}$
2 : $\text{parse}(\text{seed}_{\text{pk}}, \mathbf{t}_e) \leftarrow \text{pk}_e$	7 : $\text{ct}_1 = \text{Compress}_q(\mathbf{u}, d_u)$	2 : $\text{parse}(\text{seed}_{\text{pk}}, \mathbf{t}_e) \leftarrow \text{pk}_e$
3 : $\mathbf{t}_c = \mathbf{t}_s + \mathbf{t}_e \bmod q$	8 : $\text{ct}_2 = \text{Compress}_q(\mathbf{v}, d_v)$	3 : $\mathbf{t}_s = \mathbf{t}_c - \mathbf{t}_e \bmod q$
4 : $\text{cpk} = (\text{seed}_{\text{pk}}, \mathbf{t}_c)$	9 : return $\text{ct} = (\text{ct}_1, \text{ct}_2)$	4 : $\text{pk}_s = (\text{seed}_{\text{pk}}, \mathbf{t}_s)$
5 : return cpk		5 : return pk_s

Fig. 14. Algorithms for AFS-PKE-BW.

7.1 AFS-PKE-BW

We use the notation introduced in Section 2.1. In particular, k denotes the module rank, λ denotes the bit length of seeds and key strings, and the message space is $\mathcal{M} = \{0, 1\}^{\frac{\lambda}{n} \mu_0}$, where μ_0 is defined as in Section 2.1.

Let $\text{G} : \{0, 1\}^* \rightarrow \mathcal{R}_q^*$ and $\text{G}' : \{0, 1\}^* \rightarrow \mathcal{B}_\eta^*$ be extendable-output functions (XOFs) that take bit strings as input and produce outputs of the required lengths and dimensions. In particular, G is used to derive public matrices over $\mathcal{R}_q$, while G' is used to derive short secret and error vectors over the centered binomial distribution. Fig. 14 illustrates the construction of AFS-PKE-BW, derived from the PKE component of BW-KEM.

We define $\text{Compress}_q(x, d) = \lfloor 2^d x / q \rfloor \bmod +2^d$ and $\text{Decompress}_q(x, d) = \lfloor qx / 2^d \rfloor$ for $x \in \mathbb{Z}_q$, and apply these functions coefficient-wise to elements of $\mathcal{R}_q$. Moreover, $\text{Encode}(\cdot)$ and $\text{Decode}(\cdot)$ rely on the scalable nested-lattice coding technique described in the BW-KEM proposal; see Fig. 2.

In BW-KEM.PKE, a public key consists of a polynomial vector and a seed, namely $\text{pk} = (\mathbf{t}, \rho)$ with $\mathbf{t} \in \mathcal{R}_q^k$. Accordingly, key combination reuses the seed and performs coefficient-wise addition modulo q on the polynomial vectors. Formally, for a static public key $\text{pk}_s = (\mathbf{t}_s, \rho)$ and an ephemeral public key $\text{pk}_e = (\mathbf{t}_e, \rho)$ generated with respect to the same seed ρ , the composite public key is defined as $\text{cpk} = (\mathbf{t}_s + \mathbf{t}_e \bmod q, \rho)$, and the composite secret key is defined as $\text{csk} = \text{sk}_s + \text{sk}_e \bmod q$. The original static public key can be efficiently recovered by subtracting the ephemeral public key from the composite public key, yielding an algebraic **ExtractPK** procedure with perfect correctness. This construction critically relies on the additive homomorphism of the underlying MLWE public keys. The detailed description of AFS-PKE-BW is given in Fig. 14.

Theorem 7.2 (Correctness of AFS-PKE-BW). *Let $\mathbf{s}_s, \mathbf{e}_s, \mathbf{s}_e, \mathbf{e}_e, \mathbf{r}, \mathbf{e}_1$, and e_2 be random variables distributed as specified in Fig. 14. Let $\boldsymbol{\epsilon}_u \leftarrow \phi_{d_u}^k$ and $\epsilon_v \leftarrow \phi_{d_v}$, where the distribution ϕ_d^m over $\mathcal{R}_q^m$ is defined as follows: sample $\mathbf{x} \leftarrow \mathcal{R}_q^m$ and output $(\mathbf{x} - \text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d), d)) \bmod \pm q$. For a polynomial f , let $D = N/n$ and define its i -th block as $(f)_i = (f_i, f_{i+D}, f_{i+2D}, \dots, f_{i+(n-1)D})$ for $i \in \{0, 1, \dots, D-1\}$. That is, each block contains n coefficients spaced exactly $D = N/n$ positions apart. Let Err be the error polynomial $(\mathbf{e}_s + \mathbf{e}_e)^T \mathbf{r} - (\mathbf{s}_s + \mathbf{s}_e)^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v$. Define*

$$\delta = 1 - \Pr [\|(\text{Err})_i\|_{q,2} < r_0, \forall i \in \{0, 1, \dots, D-1\}],$$

where $r_0 = \frac{q\sqrt{2n}}{2^{\tau+2}} - \frac{\sqrt{n}}{2} \left(\frac{q}{\hat{q}} + 1 \right)$. Then AFS-PKE-BW is δ -correct.

Proof. From Algorithm $\text{Dec}_{\text{AFS}}(\text{csk}, \text{ct})$, the value of $\mathbf{u}'$ is given by

$$\begin{aligned} \mathbf{u}' &= \text{Decompress}_{2^{d_u}, q}(\text{Compress}_{q, 2^{d_u}}(\mathbf{A}^T \mathbf{r} + \mathbf{e}_1)) \\ &= \mathbf{A}^T \mathbf{r} + \mathbf{e}_1 + \boldsymbol{\epsilon}_u, \end{aligned}$$

for some $\boldsymbol{\epsilon}_u \in \mathcal{R}_q^k$. Similarly, the value of v' is computed as

$$\begin{aligned} v' &= \text{Decompress}_{2^{d_v}, q}(\text{Compress}_{q, 2^{d_v}}(\mathbf{t}_c^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor)) \\ &= (\mathbf{A} \mathbf{s}_c + \mathbf{e}_c)^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor + \epsilon_v, \end{aligned}$$

for some $\epsilon_v \in \mathcal{R}_q$. We may safely assume that $\boldsymbol{\epsilon}_u$ and ϵ_v follow the distributions $\phi_{d_u}^k$ and ϕ_{d_v} , respectively, since they both take the form

$$(\mathbf{x} - \text{Decompress}_{2^d, q}(\text{Compress}_{q, 2^d}(\mathbf{x}))) \bmod \pm q,$$

where $\mathbf{x}$ is pseudorandom under the hardness assumption of the Module-LWE problem.

Denote $\epsilon_1 = \left\lfloor \frac{\hat{q}}{q} (v' - \mathbf{s}_c^T \mathbf{u}') \right\rfloor - \frac{\hat{q}}{q} (v' - \mathbf{s}_c^T \mathbf{u}')$, and $\epsilon_2 = \lfloor \text{PolyEncode}(m) \rfloor - \text{PolyEncode}(m)$. Then, it follows that

$$m' = \text{PolyDecode}_{\mathcal{C}_{n, \tau}(\frac{q}{2^\tau})} \left(\frac{q}{\hat{q}} \left\lfloor \frac{\hat{q}}{q} (v' - \mathbf{s}_c^T \mathbf{u}') \right\rfloor \bmod \pm q \right).$$

We then derive

$$\begin{aligned} \frac{q}{\hat{q}} \left\lfloor \frac{\hat{q}}{q} (v' - \mathbf{s}_c^T \mathbf{u}') \right\rfloor &= v' - \mathbf{s}_c^T \mathbf{u}' + \frac{q}{\hat{q}} \epsilon_1 \\ &= (\mathbf{A} \mathbf{s}_c + \mathbf{e}_c)^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor + \epsilon_v - \mathbf{s}_c^T (\mathbf{A}^T \mathbf{r} + \mathbf{e}_1 + \boldsymbol{\epsilon}_u) + \frac{q}{\hat{q}} \epsilon_1 \\ &= \text{PolyEncode}(m) + \mathbf{e}_c^T \mathbf{r} - \mathbf{s}_c^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v + \frac{q}{\hat{q}} \epsilon_1 + \epsilon_2. \end{aligned}$$

Every n coefficients spaced $D = \frac{N}{n}$ positions apart of $\text{PolyEncode}(m)$ (referred to as a *block*) essentially correspond to a lattice code vector in $\mathcal{C}_{n,\tau}(\frac{q}{2^\tau})$. The deviation from the target lattice point is then bounded by

$$\begin{aligned} & \|(\mathbf{e}_c^T \mathbf{r} - \mathbf{s}_c^T(\mathbf{e}_1 + \mathbf{e}_u) + e_2 + \epsilon_v)_i\|_{q,2} + \frac{q}{\hat{q}} \|(\epsilon_1)_i\|_{q,2} + \|(\epsilon_2)_i\|_{q,2} \\ & \leq \|(\mathbf{e}_c^T \mathbf{r} - \mathbf{s}_c^T(\mathbf{e}_1 + \mathbf{e}_u) + e_2 + \epsilon_v)_i\|_{q,2} + \frac{\sqrt{n}}{2} \left(\frac{q}{\hat{q}} + 1 \right), \end{aligned}$$

since $(\epsilon_1)_i, (\epsilon_2)_i \in (-\frac{1}{2}, \frac{1}{2}]^n$.

According to the decoding theorem in our BW-KEM proposal, decoding succeeds whenever the above deviation is smaller than the packing radius $\frac{q\sqrt{2n}}{2^{\tau+2}}$. Hence, the decryption error probability of AFS-PKE-BW is

$$1 - \Pr \left[\|((\mathbf{e}_s + \mathbf{e}_e)^T \mathbf{r} - (\mathbf{s}_s + \mathbf{s}_e)^T(\mathbf{e}_1 + \mathbf{e}_u) + e_2 + \epsilon_v)_i\|_{q,2} < r_0, \forall i \right],$$

where $r_0 = \frac{q\sqrt{2n}}{2^{\tau+2}} - \frac{\sqrt{n}}{2} \left(\frac{q}{\hat{q}} + 1 \right)$. □

Theorem 7.3 (IND-LCK-SCPA Security of AFS-PKE-BW in the ROM/QROM). *We model the hash functions G and G' as random oracles. For any adversary $\mathcal{A}$ against the IND-LCK-SCPA security of AFS-PKE-BW making at most q_G and $q_{G'}$ queries to G and G' , respectively, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ running in time comparable to that of $\mathcal{A}$ such that*

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-LCK-SCPA}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}_{\eta_1,k,k,q}}^{\text{MLWE}}(\mathcal{B}) + q_{G'} \text{Adv}_{\mathcal{B}_{\eta_1,k+1,k,q}}^{\text{MLWE}}(\mathcal{C}) + q_{G'} 2^{-\lambda}.$$

If the hash functions G and G' are modeled as quantum-accessible random oracles and $\mathcal{A}$ makes at most q_G and $q_{G'}$ quantum queries to G and G' , respectively, then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ running in time comparable to that of $\mathcal{A}$ such that

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-LCK-SCPA}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}_{\eta_1,k,k,q}}^{\text{MLWE}}(\mathcal{B}) + (2q_{G'} + 1)^2 \text{Adv}_{\mathcal{B}_{\eta_1,k+1,k,q}}^{\text{MLWE}}(\mathcal{C}) + 2q_{G'} 2^{-\lambda/2}.$$

Proof sketch. We outline the ROM proof of IND-LCK-SCPA security. We first assume that the adversary queries $G'(\text{seed}'_e)$ before outputting seed'_e . The proof invokes the MLWE assumption to replace the static public key pk_s^* with a uniformly random element. Conditioned on not aborting, the adversary's output cpk^* and seed'_e must satisfy $\text{cpk}^* = (\mathbf{t}_s^* + \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*, \rho^*)$, where $(\mathbf{s}_e^*, \mathbf{e}_e^*) = G'(\text{seed}'_e)$ and $\mathbf{A}$ is generated from ρ^* . In the next game hop, the challenger guesses the index of the random-oracle query on seed'_e among the at most $q_{G'}$ queries to G' . This guess succeeds with probability at least $1/q_{G'}$. Upon a correct guess, the challenger samples $(\mathbf{s}_e^*, \mathbf{e}_e^*)$ in advance and embeds this value into the random oracle at the guessed query. It then samples $\mathbf{t}_c^*$ uniformly, sets $\mathbf{t}_s^* := \mathbf{t}_c^* - (\mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*)$, and uses $\text{cpk}^* = (\mathbf{t}_c^*, \rho^*)$ to compute the challenge ciphertext ct^* . Since $\mathbf{t}_c^*$ is uniformly random, the MLWE assumption allows us to replace the ciphertext component $(\mathbf{u}, v)$ in ct^* with uniformly random values. In the QROM, the guessing step is justified via the measure-and-reprogram technique [DFMS19,DFM20].

Security Analysis in the ROM.

Proof. We proceed with a sequence of games, where G_3 and G_4 are given in detail in Fig. 15.

Game G_0 . Game G_0 is the initial IND-LCK-SCPA security game. Moreover, we assume that the adversary queries $G'(\text{seed}'_e)$ before outputting seed'_e . Notably, this does not change the adversary's advantage. Indeed, for any adversary $\mathcal{A}$ making at most $q_{G'}$ queries to G' , we can construct a random oracle middleman adversary $\mathcal{A}'$ that always queries $G'(\text{seed}'_e)$ before outputting the value seed'_e received from $\mathcal{A}$. Moreover, the combined public key cpk^* output by the adversary must be well-formed, namely $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$, where $\text{seed}_{\text{pk}}^*$ is the seed in pk_s^* used to generate the matrix $\mathbf{A}$. Otherwise, the game aborts, since this would imply $\text{pk}'_s \neq \text{pk}_s^*$. That is,

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-LCK-SCPA}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

IND-LCK-SCPA Game G_3	IND-LCK-SCPA Game G_4
1 : $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times k}$ 2 : $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_s^*) \leftarrow \{0, 1\}^{256} \times \mathcal{R}_q^k$ 3 : $(\text{cpk}^*, m_0, m_1) \leftarrow \mathcal{A}^{G, G'}(\text{pk}_s^*)$ 4 : $b \leftarrow \{0, 1\}$ 5 : $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$ 6 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{G, G'}(\text{pk}_s^*, \text{ct}^*)$ 7 : $\text{parse}(\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*) \leftarrow \text{cpk}^*$ 8 : $(\mathbf{s}'_e, \mathbf{e}'_e) \leftarrow G'(\text{seed}'_e)$ 9 : $\mathbf{t}'_e = \mathbf{A}\mathbf{s}'_e + \mathbf{e}'_e$ 10 : $\mathbf{t}'_s = \mathbf{t}_c^* - \mathbf{t}'_e$ 11 : if $\mathbf{t}'_s \neq \mathbf{t}_s^*$: abort 12 : return $b' ? = b$	1 : $(\text{seed}_{\text{pk}}^*, \mathbf{A}) \leftarrow \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$ 2 : $(\mathbf{s}_e^*, \mathbf{e}_e^*) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$ 3 : $\mathbf{t}_e^* = \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*$ 4 : $\mathbf{t}_c^* \leftarrow \mathcal{R}_q^k$ 5 : $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$ 6 : $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^* - \mathbf{t}_e^*)$ 7 : $\text{cn} = -1$ 8 : $i \leftarrow [q_{G'}]$ 9 : $(m_0, m_1) \leftarrow \mathcal{A}^{G, G'}(\text{pk}_s^*)$ 10 : $b \leftarrow \{0, 1\}$ 11 : $\text{ct}^* \leftarrow \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$ 12 : $b' \leftarrow \mathcal{A}^{G, G'}(\text{pk}_s^*, \text{ct}^*)$ 13 : return $b' ? = b$
$G'(\text{seed})$ 1 : if $\exists \mathbf{s}, \mathbf{e} : (\text{seed}, \mathbf{s}, \mathbf{e}) \in \mathcal{L}_{G'} :$ 2 : return $(\mathbf{s}, \mathbf{e})$ 3 : $(\mathbf{s}, \mathbf{e}) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$ 4 : $\text{cn} \leftarrow \text{cn} + 1$ $// G_4$ 5 : if $\text{cn} = i : (\mathbf{s}, \mathbf{e}) \leftarrow (\mathbf{s}_e^*, \mathbf{e}_e^*)$ $// G_4$ 6 : $\mathcal{L}_{G'} \leftarrow \mathcal{L}_{G'} \cup \{(\text{seed}, \mathbf{s}, \mathbf{e})\}$ 7 : return $(\mathbf{s}, \mathbf{e})$	$G(\text{seed}_{\text{pk}})$ 1 : if $\text{seed}_{\text{pk}} = \text{seed}_{\text{pk}}^* :$ return $\mathbf{A}$ 2 : if $\exists \mathbf{A}' : (\text{seed}_{\text{pk}}, \mathbf{A}') \in \mathcal{L}_G :$ 3 : return $\mathbf{A}'$ 4 : $\mathbf{A}' \leftarrow \mathcal{R}_q^{k \times k}$ 5 : $\mathcal{L}_G \leftarrow \mathcal{L}_G \cup \{(\text{seed}_{\text{pk}}, \mathbf{A}')\}$ 6 : return $\mathbf{A}'$

Fig. 15. Games G_3, G_4 for the IND-LCK-SCPA security of AFS-PKE-BW in the ROM

Game G_1 . In game G_1 , we change how to simulate the random oracle G . Specifically, at the beginning of the game, we first sample $(\text{seed}_{\text{pk}}^*, \mathbf{A}) \leftarrow \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$. Then we define G such that $G(\text{seed}_{\text{pk}}^*) = \mathbf{A}$. This game is the same as G_0 for $\mathcal{A}$, thus

$$\text{Adv}^{G_0}(\mathcal{A}) = \text{Adv}^{G_1}(\mathcal{A}).$$

Game G_2 . In this game, we replace $(\mathbf{s}_s^*, \mathbf{e}_s^*) \leftarrow G'(\text{seed}_s^*)$ with uniformly random $(\mathbf{s}_s^*, \mathbf{e}_s^*) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$. If the adversary never queries $G'(\text{seed}_s^*)$, then Games G_1 and G_2 are identical for the adversary. Since $\text{seed}_s^* \leftarrow \{0, 1\}^\lambda$ is uniformly random from the adversary's perspective, the probability that the adversary queries it is at most $q_{G'}/2^\lambda$. Therefore, we have

$$\left| \text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq \frac{q_{G'}}{2^\lambda}.$$

Game G_3 . In game G_3 , $\mathbf{t}_s^* = \mathbf{A}\mathbf{s}_s^* + \mathbf{e}_s^*$ is sampled uniformly at random. For any adversary that can distinguish between G_2 and G_3 , there exists an adversary $\mathcal{B}$ that can solve the $\text{MLWE}(\mathcal{B}_{\eta_1}, k, k, q)$ problem. Therefore, we have

$$\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k, k, q}^{\text{MLWE}}(\mathcal{B}).$$

Game G_4 . The full description of G_4 is given in Fig. 15. In particular, in G_4 the challenger samples a uniformly random $\mathbf{t}_c^* \leftarrow \mathcal{R}_q^k$ and defines the challenge composite public key as $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$, which is used to compute ct^* . It further samples independent ephemeral secrets $(\mathbf{s}_e^*, \mathbf{e}_e^*)$ and an index $i \leftarrow [q_{G'}]$, and sets $\mathbf{t}_e^* := \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*$ and $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^* - \mathbf{t}_e^*)$. When the adversary makes the $(i+1)$ -th query to G' , the challenger returns $(\mathbf{s}_e^*, \mathbf{e}_e^*)$.

For any adversary $\mathcal{A}$ that wins in Game G_3 , we construct an adversary $\hat{\mathcal{A}}$ attacking the Game G_4 as follows. The adversary $\hat{\mathcal{A}}$ forwards pk_s^* to $\mathcal{A}$. Upon receiving (cpk', m_0, m_1) from $\mathcal{A}$, it forwards (m_0, m_1) to its challenger and obtains the challenge ciphertext ct^* , which is then given to $\mathcal{A}$. All random-oracle queries of $\mathcal{A}$ are answered using $\hat{\mathcal{A}}$'s own oracle access. Finally, when $\mathcal{A}$ outputs (b', seed'_e) , $\hat{\mathcal{A}}$ outputs the bit b' . In particular, when the sampled index $i \leftarrow [q_{G'}]$ in G_4 coincides with the first query of the form $G'(\text{seed}'_e)$ made by $\mathcal{A}$ (which occurs with probability $1/q_{G'}$), the composite public key cpk output by $\mathcal{A}$ is identical to the challenge value cpk^* in the game of $\hat{\mathcal{A}}$, provided that the game of $\mathcal{A}$ does not abort. Therefore, the simulation for $\mathcal{A}$ is perfect, and $\hat{\mathcal{A}}$ succeeds whenever $\mathcal{A}$ succeeds. Consequently, we have

$$\text{Adv}^{G_3}(\mathcal{A}) \leq q_{G'} \cdot \text{Adv}^{G_4}(\hat{\mathcal{A}}).$$

Game G_5 . In Game G_5 , the values $\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$ and $v = \mathbf{t}_c^{*T} \mathbf{r} + e_2$ in the challenge ciphertext ct^* are replaced by uniformly random values $\mathbf{u} \leftarrow \mathcal{R}_q^k$ and $v \leftarrow \mathcal{R}_q$. Note that adding any fixed value $\lfloor \frac{q}{2} \rfloor m$ to v preserves the uniform distribution over $\mathcal{R}_q$. For any adversary that can distinguish Game G_4 from Game G_5 , we can construct an adversary $\mathcal{C}$ against $\text{MLWE}(\mathcal{B}_{\eta_1}, k+1, k, q)$ such that

$$\left| \text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A}) \right| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k+1, k, q}^{\text{MLWE}}(\mathcal{C}).$$

In game G_5 , ct^* and m_b are independent, therefore

$$\text{Adv}^{G_5}(\mathcal{A}) = 0.$$

Combining all bounds above completes the proof. $\square$

Security Analysis in the QROM.

Proof. We proceed with a sequence of games, where G_3, G_4, G_5 are given in detail in Fig. 16.

Game G_0 . Game G_0 is the initial IND-LCK-SCPA security game. Notably, the combined public key cpk^* output by the adversary must be of the form $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$, where $\text{seed}_{\text{pk}}^*$ is the seed in pk_s^* used to generate the matrix $\mathbf{A}$. Otherwise the game aborts, as this would imply $\text{pk}'_s \neq \text{pk}_s^*$. That is,

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-LCK-SCPA}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 . In this game, we modify how the random oracle $\mathbf{G}$ is simulated. That is, we first sample $(\text{seed}_{\text{pk}}, \mathbf{A}) \leftarrow \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$ and a $2q_G$ -wise independent function $\hat{\mathbf{G}} : \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$. Then we define $\mathbf{G}(x) = \hat{\mathbf{G}}(x) + \mathbf{A} - \mathbf{M}$ for every $x \in \{0, 1\}^{256}$, where $\mathbf{M} = \mathbf{G}'(\text{seed}_A)$. Using Lemma 5.5, a $2(q_G + 1)$ -wise independent function $\hat{\mathbf{G}}$ is perfectly indistinguishable from a random oracle to any adversary $\mathcal{A}$ making at most $(q_G + 1)$ oracle queries (including the implicit query $\hat{\mathbf{G}}(\text{seed}_A)$ made by the challenger). Therefore, $\mathbf{G}$ is also perfectly indistinguishable from a random oracle for the adversary $\mathcal{A}$. Notably, we have $\mathbf{G}(\text{seed}_{\text{pk}}) = \mathbf{A}$ and $\mathbf{A}$ is uniformly random sampled in this game and

$$\Pr[G_0^{\mathcal{A}} \Rightarrow 1] = \Pr[G_1^{\mathcal{A}} \Rightarrow 1].$$

Game G_2 . In this game, we replace $(\mathbf{s}_s^*, \mathbf{e}_s^*) \leftarrow \mathbf{G}'(\text{seed}_s^*)$ with uniformly random $(\mathbf{s}_s^*, \mathbf{e}_s^*) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$. Since $\text{seed}_s^* \leftarrow \{0, 1\}^\lambda$ is uniformly random from the adversary's perspective, by Lemma 5.6, we have

$$\left| \text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A}) \right| \leq \frac{q_{G'}}{\sqrt{2\lambda}}.$$

Game G_3 . In game G_3 , $\mathbf{t}_s^* = \mathbf{A}\mathbf{s}_s^* + \mathbf{e}_s^*$ is sampled uniformly at random. For any adversary that can distinguish between G_2 and G_3 , there exists an adversary $\mathcal{B}$ that can solve the $\text{MLWE}(\mathcal{B}_{\eta_1}, k, k, q)$ problem. Therefore, we have

$$\left| \text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A}) \right| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k, k, q}^{\text{MLWE}}(\mathcal{B}).$$

IND-LCK-SCPA Game G_3, G_4	IND-LCK-SCPA Game G_5
1 : $\text{cn} = -1$ // G_4	1 : $\text{cn} = -1$
2 : $(i, b) \leftarrow \mathbb{S}([q_{G'}] \times \{0, 1\}) \cup \{(q, 0)\}$ // G_4	2 : $(i, b) \leftarrow \mathbb{S}([q_{G'}] \times \{0, 1\}) \cup \{(q, 0)\}$
3 : $\mathbf{A} \leftarrow \mathbb{R}_q^{k \times k}$	3 : $(\mathbf{A}, \text{seed}_{\text{pk}}^*) \leftarrow \mathbb{R}_q^{k \times k} \times \{0, 1\}^{256}$
4 : $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_s^*) \leftarrow \{0, 1\}^{256} \times \mathbb{R}_q^k$	4 : $\mathbf{t}_c^* \leftarrow \mathbb{R}_q^k$
5 : $(\mathbf{s}_e^*, \mathbf{e}_e^*) \leftarrow \mathbb{B}_{\eta_1}^k \times \mathbb{B}_{\eta_1}^k$ // G_4	5 : $(\mathbf{s}_e^*, \mathbf{e}_e^*) \leftarrow \mathbb{B}_{\eta_1}^k \times \mathbb{B}_{\eta_1}^k$
6 : $\mathbf{t}_e^* = \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*$ // G_4	6 : $\mathbf{t}_e^* = \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^*$
7 : $(\text{cpk}^*, m_0, m_1) \leftarrow \mathcal{A}^{ \mathcal{G}' , \mathcal{G} }(\text{pk}_s^*)$ // G_3	7 : $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^* - \mathbf{t}_e^*)$
8 : $(\text{cpk}^*, m_0, m_1) \leftarrow \mathcal{A}^{ \mathcal{G}'_i , \mathcal{G} }(\text{pk}_s^*)$ // G_4	8 : $(m_0, m_1) \leftarrow \mathcal{A}^{ \mathcal{G}'_i , \mathcal{G} }(\text{pk}_s^*)$
9 : $b \leftarrow \mathbb{S}\{0, 1\}$	9 : $b \leftarrow \mathbb{S}\{0, 1\}$
10 : $\text{ct}^* \leftarrow \mathbb{S} \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$	10 : $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$
11 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{ \mathcal{G}' , \mathcal{G} }(\text{pk}_s^*, \text{ct}^*)$ // G_3	11 : $\text{ct}^* \leftarrow \mathbb{S} \text{Enc}_{\text{AFS}}(\text{cpk}^*, m_b)$
12 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{ \mathcal{G}'_i , \mathcal{G} }(\text{pk}_s^*, \text{ct}^*)$ // G_4	12 : $(b', \text{seed}'_e) \leftarrow \mathcal{A}^{ \mathcal{G}'_i , \mathcal{G} }(\text{pk}_s^*, \text{ct}^*)$
13 : $\text{parse}(\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*) \leftarrow \text{cpk}^*$	13 : return $b' ? = b$
14 : if $\mathbf{t}_c^* - \mathbf{t}_e^* \neq \mathbf{t}_s^*$: abort // G_4	
15 : return $b' ? = b$ // G_4	
16 : $(\mathbf{s}'_e, \mathbf{e}'_e) \leftarrow \mathcal{G}'(\text{seed}'_e)$	
17 : $\mathbf{t}'_e \leftarrow \mathbf{A}\mathbf{s}'_e + \mathbf{e}'_e$	
18 : if $\mathbf{t}_c^* - \mathbf{t}'_e \neq \mathbf{t}_s^*$: abort	
19 : return $b' ? = b$	
$\mathcal{G}'(\text{seed})$	$\mathcal{G}'_i(\text{seed})$
1 : random oracle	1 : $\text{cn} \leftarrow \text{cn} + 1$
$\mathcal{G}(\text{seed}_{\text{pk}})$	2 : if $\text{cn} \geq i + b \wedge \text{seed} = \text{seed}_{i+1}$:
1 : simulates as G_1	3 : seed_{i+1} is the measurement outcome
	4 : of $\mathcal{A}$'s $(i + 1)$ -th query input register
	5 : return $(\mathbf{s}_e^*, \mathbf{e}_e^*)$
	6 : return $\mathcal{G}'(\text{seed})$

Fig. 16. G_3 - G_5 for the IND-LCK-SCPA security of AFS-PKE-BW in the QROM

Let $C^{|\mathcal{G}'|}$ be an oracle algorithm that perfectly simulates Game G_3 for the adversary $\mathcal{A}$. Internally, it samples a random oracle $\mathcal{G}'$, a bit b , a matrix $\mathbf{A}$, and a static public key $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_s^*)$, and computes the corresponding challenge ciphertext ct^* as G_3 . Let $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$, (seed'_e, b') denote the adversary's output. Let $x = \text{seed}'_e$, $y = (y_0, y_1) = \mathcal{G}'(\text{seed}'_e)$, and $z = (z_1, z_2, z_3, z_4) = (\mathbf{A}, \mathbf{t}_c^* - \mathbf{t}_s^*, b, b')$. C finally outputs (x, z) . Define the predicates $V_1(x, y, z) = [z_1 y_0 + y_1 = ? z_2]$ and $V_2(x, y, z) = [z_3 = ? z_4]$, and let $V := V_1 \wedge V_2$. This predicate V captures the adversary's success condition. Hence, $\text{Adv}^{G_3}(\mathcal{A}) = \sum_{x_0} \Pr_{\mathcal{G}'}[x = x_0 \wedge V(x, \mathcal{G}'(x), z) = 1 : (x, z) \leftarrow C^{|\mathcal{G}'|}] \cdot |1 - 1/2|$.

Let $S^C(\Theta)$ be an oracle algorithm that randomly samples a pair $(i, b) \in ([q] \times \{0, 1\}) \cup \{(q, 0)\}$, runs $C^{|\mathcal{G}'|}$ until the $(i + 1)$ -th query, measures the query input registers to obtain x , continues to run $C^{|\mathcal{G}'|}$ until the $(i + b + 1)$ -th $\mathcal{G}'$ -query, reprograms $\mathcal{G}'$ to $\mathcal{G}'_{x\Theta}$ (where $\mathcal{G}'_{x\Theta}(x) = \Theta$ and $\mathcal{G}'_{x\Theta}(x') = \mathcal{G}'(x')$ for all $x' \neq x$), and runs $C^{|\mathcal{G}'_{x\Theta}|}$ until the end to output z . Let $y = (y_0, y_1) = \mathcal{G}'(x) = \Theta$, and $z = (z_1, z_2, z_3, z_4) = (\mathbf{A}, \mathbf{t}_c^* - \mathbf{t}_s^*, b, b')$. C finally outputs (x, z) . Sampling $\Theta = (\mathbf{s}_e^*, \mathbf{e}_e^*) \leftarrow \mathbb{B}_{\eta_1}^k \times \mathbb{B}_{\eta_1}^k$ and a random oracle $\mathcal{G}'$, $S^C(\Theta)$ perfectly simulates Game G_4 (shown in Fig. 16). And we have $\text{Adv}^{G_4}(\mathcal{A}) = \sum_{x_0} \Pr_{\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S^C(\Theta)] \cdot |1 - 1/2|$.

According to Lemma 5.11, we have $\sum_{x_0} \Pr_{G'}[x = x_0 \wedge V(x, G'(x), z) = 1 : (x, z) \leftarrow C^{G'}] \leq (2q_{G'} + 1)^2 \sum_{x_0} \Pr_{\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S^C(\Theta)]$. Therefore, we have

$$\text{Adv}^{G_3}(\mathcal{A}) \leq (2q_{G'} + 1)^2 \text{Adv}^{G_4}(\mathcal{A}).$$

Game G_5 . In this game, we modify the challenger's behavior as illustrated in Fig. 16. Notably, if Game G_4 does not abort, we have $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_e^* + \mathbf{t}_s^*)$, therefore the challenger can directly use cpk^* to compute the challenge ciphertext ct^* . In this game, we first sample $\mathbf{t}_c^* \leftarrow \mathcal{R}_q^k$ and then set $\mathbf{t}_s^* := \mathbf{t}_c^* - \mathbf{t}_e^*$. We then use $\text{cpk}^* := (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$ to compute the challenge ciphertext ct_b^* .

Any adversary $\mathcal{A}$ that wins Game G_4 can be used to build an adversary $\mathcal{A}'$ that wins Game G_5 with the same advantage. Concretely, the adversary $\mathcal{A}'$ uses $(\text{pk}_s^*, \text{ct}^*)$ and its oracles to simulate Game G_4 for $\mathcal{A}$ and outputs the bit returned by $\mathcal{A}$. The simulation is perfect whenever Game G_4 does not abort, and thus a win of $\mathcal{A}$ in G_4 implies a win of $\mathcal{A}'$ in G_5 . Therefore,

$$\text{Adv}^{G_4}(\mathcal{A}) \leq \text{Adv}^{G_5}(\mathcal{A}).$$

Game G_6 . In Game G_6 , the values $\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$ and $v = \mathbf{t}_c^{*T} \mathbf{r} + e_2$ in the challenge ciphertext ct^* are replaced by uniformly random values $\mathbf{u} \leftarrow \mathcal{R}_q^k$ and $v \leftarrow \mathcal{R}_q$. Note that adding any fixed value $\lfloor \frac{q}{2} \rfloor m$ to v preserves the uniform distribution over $\mathcal{R}_q$. For any adversary that can distinguish Game G_5 from Game G_6 , we can construct an adversary $\mathcal{C}$ against $\text{MLWE}(\mathcal{B}_{\eta_1}, k+1, k, q)$ such that

$$\left| \text{Adv}^{G_5}(\mathcal{A}) - \text{Adv}^{G_6}(\mathcal{A}) \right| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k+1, k, q}^{\text{MLWE}}(\mathcal{C}).$$

In game G_6 , ct^* and m_b are independent, therefore

$$\text{Adv}^{G_6}(\mathcal{A}) = 0.$$

Combining all bounds above completes the proof. $\square$

Theorem 7.4 (IND-S(E)CPA Security of AFS-PKE-BW in the QROM). *We model the hash function G as a quantum-accessible random oracle. For any adversary $\mathcal{A}$ against either the IND-ECPA or the IND-SCPA security of AFS-PKE-BW that makes at most q_G quantum queries to G , there exist adversaries $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ running in time comparable to that of $\mathcal{A}$ such that*

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-S(E)CPA}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k, k, q}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{\mathcal{B}_{\eta_1}, k+1, k, q}^{\text{MLWE}}(\mathcal{C}) + \text{Adv}_{G'}^{\text{IND-PRG}}(\mathcal{D}).$$

The proofs of IND-ECPA and IND-SCPA security for AFS-PKE-BW follow the standard IND-CPA analysis of the underlying BW-KEM.PKE scheme. We give the proof in the QROM, which immediately implies the corresponding same ROM bound.

Proof. Since the proofs of IND-ECPA and IND-SCPA security are similar, we briefly outline the argument for IND-ECPA here. We proceed with a sequence of games, where G_3 - G_5 are given in detail in Fig. 17.

Game G_0 . Game G_0 is the initial IND-ECPA security game. Therefore, we have

$$\text{Adv}_{\text{AFS-PKE-BW}}^{\text{IND-ECPA}}(\mathcal{A}) = \left| \Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

Game G_1 . In this game, we modify how the random oracle G is simulated. That is, we first sample $(\text{seed}_{\text{pk}}, \mathbf{A}) \leftarrow \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$ and a $2q_G$ -wise independent function $G' : \{0, 1\}^{256} \times \mathcal{R}_q^{k \times k}$. Then we define $G(x) = G'(x) + \mathbf{A} - \mathbf{M}$ for every $x \in \{0, 1\}^{256}$, where $\mathbf{M} = G'(\text{seed}_A)$. Using Lemma 5.5, a $2(q_G + 1)$ -wise independent function G' is perfectly indistinguishable from a random oracle to any adversary $\mathcal{A}$ making at most $(q_G + 1)$ oracle queries (including the implicit query $G'(\text{seed}_A)$ made by the challenger). Therefore, G is also perfectly indistinguishable from a random oracle for the adversary $\mathcal{A}$. Notably, we have $G(\text{seed}_{\text{pk}}) = \mathbf{A}$ in this game and

$$\Pr[G_0^{\mathcal{A}} \Rightarrow 1] = \Pr[G_1^{\mathcal{A}} \Rightarrow 1].$$

IND-ECPA Game G_3	IND-ECPA Game G_4 - G_5
1 : $(pk_s^*, sk_s^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$	1 : $(pk_s^*, sk_s^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$
2 : $\text{parse}(\text{seed}_{pk}^*, t_s^*) \leftarrow pk_s^*$	2 : $\text{parse}(\text{seed}_{pk}^*, t_s^*) \leftarrow pk_s^*$
3 : $t_e^* \leftarrow \$ \mathcal{R}_q^k$	3 : $t_c^* \leftarrow \$ \mathcal{R}_q^k$
4 : $pk_e^* = (\text{seed}_{pk}^*, t_e^*)$	4 : $cpk^* = (\text{seed}_{pk}^*, t_c^*)$
5 : $cpk^* = (\text{seed}_{pk}^*, t_s^* + t_e^*)$	5 : $pk_e^* = (\text{seed}_{pk}^*, t_c^* - t_s^*)$
6 : $(m_0, m_1) \leftarrow \mathcal{A}^{(G)}(cpk^*, pk_s^*, pk_e^*, sk_s^*)$	6 : $(m_0, m_1) \leftarrow \mathcal{A}^{(G)}(cpk^*, pk_s^*, pk_e^*, sk_s^*)$
7 : $b \leftarrow \$ \{0, 1\}$	7 : $b \leftarrow \$ \{0, 1\}$
8 : $ct^* \leftarrow \$ \text{Enc}_{\text{AFS}}(cpk^*, m_b)$	8 : $ct^* \leftarrow \$ \text{Enc}_{\text{AFS}}(cpk^*, m_b)$ // G_4
9 : $b' \leftarrow \mathcal{A}^{(G)}(cpk^*, pk_s^*, pk_e^*, sk_s^*, ct^*)$	9 : $u \leftarrow \$ \mathcal{R}_q^k, ct_1 = \text{Compress}_q(u, d_u)$ // G_5
10 : return $b' ? = b$	10 : $v \leftarrow \$ \mathcal{R}_q, ct_2 = \text{Compress}_q(v, d_v)$ // G_5
$G(\text{seed}_{pk})$	11 : $ct^* = (ct_1, ct_2)$ // G_5
1 : simulates as G_1	12 : $b' \leftarrow \mathcal{A}^{(G)}(cpk^*, pk_s^*, pk_e^*, sk_s^*, ct^*)$
	13 : return $b' ? = b$

Fig. 17. G_3 - G_5 for the IND-ECPA security proofs of AFS-PKE-BW.

Game G_2 . In this game, we replace $(s_e^*, e_e^*) \leftarrow G'(\text{seed}_e^*)$ with uniformly random $(s_e^*, e_e^*) \leftarrow \$ \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$. For any adversary $\mathcal{A}$ that can distinguish between Games G_1 and G_2 , we construct an adversary $\mathcal{D}$ against the IND-PRG security of G' . Adversary $\mathcal{D}$ receives y_b , where y_0 is the real PRG output computed from a random seed, and y_1 is a truly random value from the output space of G' . Adversary $\mathcal{D}$ perfectly simulates Game G_2 except that it uses y_b as (s_e^*, e_e^*) to compute (pk_e^*, sk_e^*) . When $b = 0$, $\mathcal{D}$ perfectly simulates G_1 and when $b = 1$, $\mathcal{D}$ perfectly simulates G_2 . That is, $\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathcal{D} \Rightarrow 1 \mid b = 0]$ and $\Pr[G_2^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathcal{D} \Rightarrow 1 \mid b = 1]$. Therefore,

$$|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{G'}^{\text{IND-PRG}}(\mathcal{D}).$$

Game G_3 . In this game, we replace $t_e^* = \mathbf{A}s_e^* + \mathbf{e}_e^*$ in pk_e^* with a uniformly random value $t_e^* \leftarrow \$ \mathcal{R}_q^k$. For any adversary $\mathcal{A}$ that can distinguish between G_2 and G_3 , we can construct an adversary $\mathcal{B}$ that solves the $\text{MLWE}(\mathcal{B}_{\eta_1}, k, k, q)$ problem. Therefore, we have

$$|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k, k, q}^{\text{MLWE}}(\mathcal{B}).$$

Game G_4 . In this game, we re-express several elements of G_3 in an equivalent manner. Specifically, the polynomial vector t_c^* in cpk^* is sampled uniformly at random, and the ephemeral public key is defined as $pk_e^* = (\text{seed}_{pk}^*, t_c^* - t_s^*)$. The distribution of t_e^* and t_c^* in G_3 is identical to that in G_4 . Thus,

$$\Pr[G_3^{\mathcal{A}} \Rightarrow 1] = \Pr[G_4^{\mathcal{A}} \Rightarrow 1].$$

Game G_5 . In this game, the values $\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$ and $v = t_c^{*T} \mathbf{r} + e_2$ in the challenge ciphertext ct^* are replaced by uniformly random $\mathbf{u} \leftarrow \$ \mathcal{R}_q^k$ and $v \leftarrow \$ \mathcal{R}_q$ (adding any fixed $\lfloor \frac{q}{2} \rfloor m$ to v preserves uniformity). Similarly, for any adversary that can distinguish between G_4 and G_5 , we can construct an adversary $\mathcal{C}$ against the $\text{MLWE}(\mathcal{B}_{\eta_1}, k+1, k, q)$ problem. That is, we have

$$|\Pr[G_4^{\mathcal{A}} \Rightarrow 1] - \Pr[G_5^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{\mathcal{B}_{\eta_1}, k+1, k, q}^{\text{MLWE}}(\mathcal{C}).$$

In this game, ct^* and m_b are independent. Therefore,

$$\Pr[G_5^{\mathcal{A}} \Rightarrow 1] = \frac{1}{2}.$$

Combining all bounds above completes the proof. $\square$

7.2 AFS-KEM-BW

Based on AFS-PKE-BW, the AFS-KEM-BW is obtained by applying our FO_{AFS} transform (shown in Fig. 11). The transform upgrades the CPA security of AFS-PKE-BW to the corresponding CPA/CCA security of AFS-KEM-BW. For AFS-KEM-BW, the properties of perfect ExtractPK-correctness, ephemeral key generation equivalence, min-entropy of the combined public key, and γ -spreadness¹¹ hold directly. The analyses of the SPK-Bind, RSPK-Bind, and Key-Coll security of AFS-KEM-BW given in Theorems 7.5, 7.6, and 7.7.

Theorem 7.5 (SPK-Bind Security of AFS-KEM-BW in the ROM/QROM). *Let $q_{G'}$ be the number of queries made by the adversary to the random oracle G' , and let $p_{\eta_1} := \max_x \Pr[\text{CBD}_{\eta_1} = x] = \frac{\binom{2\eta_1}{\eta_1}}{2^{2\eta_1}}$. For any PPT adversary $\mathcal{A}$ in the ROM, we have*

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{SPK-Bind}}(\mathcal{A}) \leq 2^{-\lambda} + (q_{G'} + 1)p_{\eta_1}^{kn}.$$

For any QPT adversary $\mathcal{A}$ making at most $q_{G'}$ quantum queries to G' in the QROM, we have

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{SPK-Bind}}(\mathcal{A}) \leq 2^{-\lambda} + O((q_{G'} + 1)^2 p_{\eta_1}^{kn}).$$

Proof. Recall that in the SPK-Bind security game, the adversary receives $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and outputs a seed seed' . Then, the challenger computes $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}')$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. The adversary wins if either $\text{pk}'_s = \overline{\text{pk}}_s^*$, or $\text{seed}' \neq \text{seed}_e^*$ and $\text{pk}'_s = \text{pk}_s^*$.

First, consider the case where $\text{pk}'_s = \overline{\text{pk}}_s^*$. By the construction of AFS-KEM-BW, the static public key contains the seed component seed_{pk} used to generate the matrix $\mathbf{A}$. Moreover, the seed component seed'_{pk} in pk'_s is the same as the $\text{seed}_{\text{pk}}^*$ in the static public key pk_s^* . Therefore, for pk'_s to equal $\overline{\text{pk}}_s^*$, it is necessary that $\text{seed}_{\text{pk}}^* = \overline{\text{seed}}_{\text{pk}}^*$, where $\overline{\text{seed}}_{\text{pk}}^*$ denotes the corresponding seed component contained in $\overline{\text{pk}}_s^*$. Since $\text{seed}_{\text{pk}}^*$ and $\overline{\text{seed}}_{\text{pk}}^*$ are independently sampled uniform λ -bit strings, this event occurs with probability at most $2^{-\lambda}$.

Second, consider the case where $\text{seed}' \neq \text{seed}_e^*$ and $\text{pk}'_s = \text{pk}_s^*$. Write $(\mathbf{s}_e^*, \mathbf{e}_e^*) = G'(\text{seed}_e^*)$, $(\mathbf{s}'_e, \mathbf{e}'_e) = G'(\text{seed}')$, and $\mathbf{A} = G(\text{seed}_{\text{pk}}^*)$. By construction, $\text{cpk}^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_c^*)$ with $\mathbf{t}_c^* = \mathbf{t}_s^* + \mathbf{t}_e^* \bmod q$, where $\mathbf{t}_e^* = \mathbf{A}\mathbf{s}_e^* + \mathbf{e}_e^* \bmod q$. On input seed' , the challenger computes $\mathbf{t}'_e = \mathbf{A}\mathbf{s}'_e + \mathbf{e}'_e \bmod q$ and extracts $\text{pk}'_s = (\text{seed}_{\text{pk}}^*, \mathbf{t}'_s)$ with $\mathbf{t}'_s = \mathbf{t}_c^* - \mathbf{t}'_e \bmod q$. Thus, $\text{pk}'_s = \text{pk}_s^*$ implies $\mathbf{t}'_s = \mathbf{t}_s^*$, and hence $\mathbf{t}'_e = \mathbf{t}_e^* \bmod q$. Equivalently, $\mathbf{A}\mathbf{s}'_e + \mathbf{e}'_e = \mathbf{t}_e^* \bmod q$. Thus, the second case can occur only if a fresh seed $\text{seed}' \neq \text{seed}_e^*$ generates the same ephemeral public-key component as seed_e^* .

For every fresh seed $\text{seed}' \neq \text{seed}_e^*$, the pair $(\mathbf{s}'_e, \mathbf{e}'_e)$ is sampled independently according to G' . In particular, $\mathbf{e}'_e \leftarrow \mathcal{B}_{\eta_1}^k$ consists of kn independent coefficients sampled from CBD_{η_1} . Hence, by the definition of $p_{\eta_1} := \max_x \Pr[\text{CBD}_{\eta_1} = x]$, we have $\max_{\mathbf{e}_0} \Pr_{\mathbf{e}'_e \leftarrow \mathcal{B}_{\eta_1}^k}[\mathbf{e}'_e = \mathbf{e}_0] = p_{\eta_1}^{kn}$. Therefore, for the fixed values of $\mathbf{A}$ and $\mathbf{t}_e^*$, we have

$$\begin{aligned} & \Pr_{(\mathbf{s}'_e, \mathbf{e}'_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k} [\mathbf{A}\mathbf{s}'_e + \mathbf{e}'_e = \mathbf{t}_e^* \bmod q] \\ &= \sum_{\mathbf{s} \in \mathcal{B}_{\eta_1}^k} \Pr_{\mathbf{s}'_e \leftarrow \mathcal{B}_{\eta_1}^k} [\mathbf{s}'_e = \mathbf{s}] \cdot \Pr_{\mathbf{e}'_e \leftarrow \mathcal{B}_{\eta_1}^k} [\mathbf{A}\mathbf{s} + \mathbf{e}'_e = \mathbf{t}_e^* \bmod q] \\ &= \sum_{\mathbf{s} \in \mathcal{B}_{\eta_1}^k} \Pr_{\mathbf{s}'_e \leftarrow \mathcal{B}_{\eta_1}^k} [\mathbf{s}'_e = \mathbf{s}] \cdot \Pr_{\mathbf{e}'_e \leftarrow \mathcal{B}_{\eta_1}^k} [\mathbf{e}'_e = \mathbf{t}_e^* - \mathbf{A}\mathbf{s} \bmod q] \\ &\leq \sum_{\mathbf{s} \in \mathcal{B}_{\eta_1}^k} \Pr_{\mathbf{s}'_e \leftarrow \mathcal{B}_{\eta_1}^k} [\mathbf{s}'_e = \mathbf{s}] \cdot p_{\eta_1}^{kn} \\ &= p_{\eta_1}^{kn}. \end{aligned}$$

Thus, for each fresh seed $\text{seed}' \neq \text{seed}_e^*$, the probability that it generates the same ephemeral public-key component as seed_e^* is at most $p_{\eta_1}^{kn}$. Since the adversary makes at most $q_{G'}$ queries to G' in

¹¹ γ -spreadness reduces to that of BW-KEM, since the combined public key cpk follows the same structure as a valid BW-KEM public key.

the ROM and outputs one final seed, a union bound over at most $q_{G'} + 1$ candidate seeds shows that the second case occurs with probability at most $(q_{G'} + 1)p_{\eta_1}^{kn}$. Combining the two cases, in the ROM we obtain

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{SPK-Bind}}(\mathcal{A}) \leq 2^{-\lambda} + (q_{G'} + 1)p_{\eta_1}^{kn}.$$

QROM Analysis. In the QROM, the same target-search event is bounded by the standard quantum search bound [BBBV97], replacing the second term by $O((q_{G'} + 1)^2 p_{\eta_1}^{kn})$. Combining the two cases, in the QROM we obtain

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{SPK-Bind}}(\mathcal{A}) \leq 2^{-\lambda} + O((q_{G'} + 1)^2 p_{\eta_1}^{kn}).$$

□

Theorem 7.6 (RSPK-Bind Security of AFS-KEM-BW in the ROM/QROM). *Let $q_{G'}$ be the number of queries made by the adversary to the random oracle G' , and let $p_{\eta_1} := \max_x \Pr[\text{CBD}_{\eta_1} = x] = \frac{\binom{2\eta_1}{\eta_1}}{2^{2\eta_1}}$. For any PPT adversary $\mathcal{A}$ in the ROM, we have*

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{RSPK-Bind}}(\mathcal{A}) \leq (q_{G'} + 1)2^{-\lambda} + p_{\eta_1}^{kn}.$$

For any QPT adversary $\mathcal{A}$ making at most $q_{G'}$ quantum queries to G' , we have

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{RSPK-Bind}}(\mathcal{A}) \leq p_{\eta_1}^{kn} + \frac{3}{2}\sqrt{(q_{G'} + 1)2^{-\lambda}}.$$

Proof. Note that revealing seed_e to the adversary does not give the adversary any additional advantage. Therefore, we may equivalently analyze a game in which seed_e is sampled after cpk is fixed and is not given to the adversary.

First, write $\text{pk}_s^* = (\text{seed}_{\text{pk}}^*, \mathbf{t}_s^*)$ and $\text{cpk} = (\text{seed}_{\text{pk}}, \mathbf{t}_c)$, where cpk is output by the adversary. Let $\mathbf{A} = G(\text{seed}_{\text{pk}})$, and write $(\mathbf{s}_e, \mathbf{e}_e) = G'(\text{seed}_e)$, where $\text{seed}_e \leftarrow \{0, 1\}^\lambda$. Since EKeyGen_c computes $\mathbf{t}_e = \mathbf{A}\mathbf{s}_e + \mathbf{e}_e \bmod q$, and ExtractPK outputs $\text{pk}'_s = (\text{seed}_{\text{pk}}, \mathbf{t}_c - \mathbf{t}_e \bmod q)$, the condition $\text{pk}'_s = \text{pk}_s^*$ first implies $\text{seed}_{\text{pk}} = \text{seed}_{\text{pk}}^*$, and then implies $\mathbf{A}\mathbf{s}_e + \mathbf{e}_e = \mathbf{t}_c - \mathbf{t}_s^* \pmod{q}$.

We now bound the probability of this event in the ROM. Let S denote the set of inputs to G' queried by the adversary before it outputs cpk . Since seed_e is sampled uniformly after cpk has been fixed, we have $\Pr[\text{seed}_e \in S] \leq (q_{G'} + 1)2^{-\lambda}$. Conditioned on $\text{seed}_e \notin S$, the value $G'(\text{seed}_e)$ is fresh and independent of cpk . Thus, $(\mathbf{s}_e, \mathbf{e}_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$ independently of the fixed target $\mathbf{t}_c - \mathbf{t}_s^*$. As in the second case of the proof of Theorem 7.5, for any fixed $\mathbf{A}$ and any fixed target $\mathbf{t}$, we have $\Pr_{(\mathbf{s}_e, \mathbf{e}_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k}[\mathbf{A}\mathbf{s}_e + \mathbf{e}_e = \mathbf{t} \bmod q] \leq p_{\eta_1}^{kn}$. Applying this bound to $\mathbf{t} = \mathbf{t}_c - \mathbf{t}_s^*$, we obtain $\Pr[\text{pk}'_s = \text{pk}_s^*] \leq q_{G'}2^{-\lambda} + p_{\eta_1}^{kn}$. Therefore, in the ROM we have

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{RSPK-Bind}}(\mathcal{A}) \leq (q_{G'} + 1)2^{-\lambda} + p_{\eta_1}^{kn}.$$

QROM Analysis. We proceed in two steps. First, after the adversary has output cpk , we reprogram G' at the point seed_e so that $G'(\text{seed}_e)$ induces an independent fresh sample $(\tilde{\mathbf{s}}_e, \tilde{\mathbf{e}}_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$. By Lemma 5.8, applied with $X = \{0, 1\}^\lambda$, we have $|X| = 2^\lambda$. Hence, for any adversary making at most $q_{G'} + 1$ oracle queries, where the additional 1 accounts for the implicit G' query induced by $\text{SKeyGen}(1^\lambda)$, the distinguishing advantage caused by this reprogramming is at most

$$\frac{3}{2}\sqrt{(q_{G'} + 1)2^{-\lambda}}.$$

Next, we bound the success probability in the modified experiment. In this experiment, the value $(\tilde{\mathbf{s}}_e, \tilde{\mathbf{e}}_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k$ is sampled independently of the adversary's already fixed target $\mathbf{t}_c - \mathbf{t}_s^*$. By the same argument as above, for any fixed $\mathbf{A}$ and any fixed target $\mathbf{t}$, we have

$$\Pr_{(\mathbf{s}_e, \mathbf{e}_e) \leftarrow \mathcal{B}_{\eta_1}^k \times \mathcal{B}_{\eta_1}^k}[\mathbf{A}\mathbf{s}_e + \mathbf{e}_e = \mathbf{t} \bmod q] \leq p_{\eta_1}^{kn}.$$

Applying this bound to $\mathbf{t} = \mathbf{t}_c - \mathbf{t}_s^*$, the adversary wins in the modified experiment with probability at most $p_{\eta_1}^{kn}$. Combining the analysis above, we obtain

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{RSPK-Bind}}(\mathcal{A}) \leq p_{\eta_1}^{kn} + \frac{3}{2}\sqrt{(q_{G'} + 1)2^{-\lambda}}.$$

□

Theorem 7.7 (Key-Coll Security of AFS-KEM-BW in the ROM/QROM). Suppose that H , G , and H_{rej} are modeled as random oracles. Let q_H , q_G , and $q_{H_{\text{rej}}}$ be the number of queries made by the adversary to H , G , and H_{rej} , respectively. For any PPT adversary $\mathcal{A}$ in the ROM, we have

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{Key-Coll}}(\mathcal{A}) \leq \frac{q_H + q_{H_{\text{rej}}} + 2}{2^\lambda}.$$

For any QPT adversary $\mathcal{A}$ making at most q_H , q_G , and $q_{H_{\text{rej}}}$ quantum queries to H , G , and H_{rej} , respectively, in the QROM, we have

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{Key-Coll}}(\mathcal{A}) \leq \frac{(q_H + 1)^2 + (q_{H_{\text{rej}}} + 1)^2}{2^\lambda}.$$

Proof. The proof of Theorem 7.7 is established by analyzing the adversary's advantage through a case-by-case discussion for $g \in \{1, 2, 3\}$.

Part $g = 1$: In this case, the experiment computes $(\text{ct}', K') \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}')$ for a public key $\text{cpk}' \neq \text{cpk}^*$ output by $\mathcal{A}$. The adversary wins only if it subsequently outputs a ciphertext ct^* such that $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}^*) = K'$. Note that the key returned by $\text{Decaps}_{\text{AFS}}$ is derived either from the random oracle H or from the implicit rejection oracle H_{rej} . We bound the advantage of $\mathcal{A}$ by considering the following two cases.

- **Case 1: A second-preimage event for H .** In this case, the key returned by $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}^*)$ is derived from H . Since $\text{cpk}' \neq \text{cpk}^*$ and the input to H binds the combined public key, the H -input used in this decapsulation is distinct from the one that generated K' during $\text{Encaps}_{\text{AFS}}(\text{cpk}')$. Hence the adversary can win only if it finds a second preimage associated with cpk^* whose output equals the target key K' . In the ROM, this event occurs with probability at most $(q_H + 1)/2^\lambda$, where the additional $+1$ accounts for the internal H -evaluation performed by the decapsulation algorithm in the experiment. In the QROM, by the standard quantum search bound [Zha19, Corollary 2], this probability is bounded by $(q_H + 1)^2/2^\lambda$.
- **Case 2: A target-preimage event for H_{rej} .** In this case, the key returned by $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}^*)$ is derived from H_{rej} . Therefore, the adversary can win only if it finds an input to H_{rej} whose output equals the target key K' . In the ROM, this event occurs with probability at most $(q_{H_{\text{rej}}} + 1)/2^\lambda$. In the QROM, by the standard quantum search bound, this probability is bounded by $(q_{H_{\text{rej}}} + 1)^2/2^\lambda$.

Part $g = 2$: In this case, the experiment first computes $(K^*, \text{ct}^*) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. The adversary then outputs a public key cpk' , and the experiment computes $(K', \text{ct}') \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}')$. The adversary wins only if $K' = K^*$. In this case, both keys are derived from the random oracle H . Since each encapsulation samples an independent $m \leftarrow \{0, 1\}^\lambda$, the two H -inputs coincide only if $\text{cpk}' = \text{cpk}^*$ and the same message is sampled, which occurs with probability at most $2^{-\lambda}$.

Otherwise, the two H -inputs are distinct. A win in this case yields a second-preimage event for H , since two distinct H -inputs produce the same target key K^* . In the ROM, this event occurs with probability at most $(q_H + 1)/2^\lambda$. In the QROM, by the standard quantum search bound, this probability is bounded by $(q_H + 1)^2/2^\lambda$.

Part $g = 3$: In this case, the experiment computes $(K^*, \text{ct}^*) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. The adversary wins only if it outputs a ciphertext $\text{ct}' \neq \text{ct}^*$ such that $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}') = K^*$. We bound the advantage of $\mathcal{A}$ by considering the following three cases.

- **Case 1: Coincidence of the H -inputs.** Suppose that the key returned by $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}')$ is derived from H , and that the decapsulation procedure recovers the same message as in $(K^*, \text{ct}^*) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. Since the re-encryption randomness is deterministically derived from G using the same combined public key and message, the re-encryption check reconstructs the same ciphertext ct^* . Since the corresponding re-encryption check in $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}')$ is valid in this case, we have $\text{ct}' = \text{ct}^*$, contradicting the winning condition $\text{ct}' \neq \text{ct}^*$. Hence this case cannot occur.
- **Case 2: A second-preimage event for H .** In this case, the key returned by $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}')$ is derived from H , but the decapsulation procedure recovers a message different from the one used in $(K^*, \text{ct}^*) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. Hence the corresponding H -input is distinct from the one that generated K^* in $\text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. Therefore, the adversary can win only if it finds a second preimage for H whose output equals the target key K^* . In the ROM, this event occurs with

probability at most $(q_H + 1)/2^\lambda$, where the additional $+1$ accounts for the internal H -evaluation performed by the decapsulation algorithm in the experiment. In the QROM, by the standard quantum search bound, this probability is bounded by $(q_H + 1)^2/2^\lambda$.

- **Case 3: A target-preimage event for H_{rej} .** In this case, the key returned by $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}')$ is derived from H_{rej} . Therefore, the adversary can win only if it finds an input to H_{rej} whose output equals the target key K^* . In the ROM, this event occurs with probability at most $(q_{H_{\text{rej}}} + 1)/2^\lambda$. In the QROM, by the standard quantum search bound, this probability is bounded by $(q_{H_{\text{rej}}} + 1)^2/2^\lambda$.

Finally, the total advantage of $\mathcal{A}$ is bounded by

$$\text{Adv}_{\text{AFS-KEM-BW}}^{\text{Key-Coll}}(\mathcal{A}) = \max\{\text{Adv}_{\mathcal{A}}^{q=1}, \text{Adv}_{\mathcal{A}}^{q=2}, \text{Adv}_{\mathcal{A}}^{q=3}\}$$

□

8 The post-eCK-PFS Security Model

The eCK-PFS variant [CF12] of the eCK model [LLM07] captures a strictly broader class of attack scenarios than the CK and CK^+ models [CK01, CK02, Kra05]. In this section, we define the **post-eCK-PFS** model, which extends the eCK-PFS framework [CF12, BCL⁺26] to the *post-specified peer* setting [CK02]. In contrast to the classical eCK-PFS framework, our model does not assume that the peer identity and the peer static public key of a session are fixed or known at the time of session activation. Instead, the peer identity and static public key may be learned as the protocol execution evolves.

8.1 Model syntax

The set U denotes the identities of honest participants in the system. Each identity $S \in U$ is associated with a certified static KEM public key pk_S and the corresponding private key sk_S , which form a static key pair $(\text{pk}_S, \text{sk}_S) \leftarrow \$ \text{SKeyGen}(1^\lambda)$ generated during the setup phase. Each party may participate in multiple, possibly concurrent, instances of the protocol, each of which is referred to as a session. Each session is uniquely identified by a session identifier, $\pi \in U \times \mathbb{N}$, represented as a pair (U, n) that denotes the n -th local session of participant U . Each session π maintains the following collection of session-specific information.

- **actor** $\in U$ denotes the party running the protocol instance.
- **peer** $\in U \cup \{*\}$ denotes the identity of the intended communication partner. The value $*$ indicates that the peer is unknown and may be determined during protocol execution. This value is set upon receiving the peer's identity.
- **role** $\in \{\text{Init}, \text{Resp}\}$ denotes whether the initiator or the responder owns the session.
- **key** $\in (\mathcal{K} \cup \{\perp\})$ denotes the session key computed by the actor. It is initially set to $\perp$ and updated upon session acceptance.
- **tr** denotes the session transcript, consisting of a finite, time-ordered sequence of messages sent and received by the actor. Here, $\text{tr}[i]$ denotes the set of messages exchanged in the $(i + 1)$ -th round of the protocol execution. We let **sent** and **rec** denote the subsequences of messages sent and received by the actor, respectively.¹²
- **eph**¹³ denotes the set of ephemeral values associated with a session that may be leaked to the adversary.
- **status** denotes the current state of a session, which is either **running**, **accepted**, or **rejected**.

¹² Note that **rec** is determined by the adversary via the **send** oracle defined later, whereas **sent** is generated according to the protocol specification.

¹³ The precise contents of **eph** may vary across protocols and capture the specific ephemeral secret values that are allowed to be revealed to the adversary.

Session Relations. For a session identified by π , we write $\pi.X$ as shorthand for the value of the session variable X . Two accepted sessions π and π' are said to be **Match** if $\pi.\text{role} \neq \pi'.\text{role}$ and $\pi.\text{tr} = \pi'.\text{tr}$. That is, two matching sessions have opposite roles and identical message transcripts.

We next introduce two auxiliary notions that capture partial agreement between sessions based on the first three protocol messages. A session π is said to be a **FirstMatch** session of π' if $\pi.\text{role} \neq \pi'.\text{role}$ and $\pi.\text{tr}[i] = \pi'.\text{tr}[i]$ for all $i \in \{0, 1\}$. A session π is said to be a **PartialMatch** session of π' if $\pi.\text{role} \neq \pi'.\text{role}$ and the two sessions agree on all KEM-related messages exchanged during the first three rounds of the protocol. Formally, for every $i \in \{0, 1, 2\}$, the KEM-related components (i.e., the KEM public key and ciphertext) of $\pi.\text{tr}[i]$ and $\pi'.\text{tr}[i]$ are identical.

8.2 Adversarial Capabilities

Following [CF12, BCL⁺26], we consider a (quantum) polynomial-time adversary with full control over the communication network. In particular, the adversary may arbitrarily intercept, modify, inject, delay, or drop messages. Consequently, all communication between protocol participants is mediated by the adversary, and even two honest parties can establish a session only via adversarially controlled message delivery.

To formalize the adversary's ability in a game-based security experiment, the adversary is given a set of oracles defined as follows. The first two queries, **NewSession** and **Send**, model the adversary's complete control over the communication network. The queries **RevealSK**, **RevealEph**, and **Corrupt** capture the adversary's ability to compromise participants and obtain secret information. Finally, the **Test** query models the adversary's challenge of distinguishing a real session key from a random one.

- **NewSession**(U, V, role): Creates a new session π with owner $\pi.\text{actor} \leftarrow U$, intended peer $\pi.\text{peer} \leftarrow V$, and role $\pi.\text{Role} \leftarrow \text{role}$. The peer identity may be unspecified, in which case $V = *$.
- **Send**(π, m): Sends a message m to session π . If the session π has not been created via **NewSession**, the query returns $\perp$. Otherwise, the **Send** oracle executes the protocol on behalf of $\pi.\text{actor}$ which receives m , updates the session state accordingly, and returns the response message and the updated session status $\pi.\text{status}$.
- **RevealSK**(π): If the session has not accepted (i.e., $\pi.\text{status} \neq \text{accepted}$), the query returns $\perp$. Otherwise, it reveals the session key $\pi.\text{key}$ to the adversary.
- **RevealEph**(π): If $\pi.\text{status} \in \{\text{accepted}, \text{rejected}\}$, the query returns $\perp$, reflecting the assumption that all ephemeral secret information is securely erased upon completion of the protocol.¹⁴ Otherwise, the query reveals the ephemeral secret state $\pi.\text{eph}$ to the adversary.
- **Corrupt**(U): Reveals the static secret key sk_U of party U . The time at which party U is corrupted is recorded.
- **Test**(π): This query may be issued at most once. A session queried by **Test** is called a test session and is denoted by π^{Test} . If π has not accepted (i.e., $\pi.\text{status} \neq \text{accepted}$), the query returns $\perp$. Otherwise, the **Test** oracle has a uniformly random bit b , which is fixed throughout the game. If $b = 0$, it returns the real session key $\pi.\text{key}$. If $b = 1$, it samples a uniformly random key $\text{key} \leftarrow \mathcal{K}$ and returns key.

8.3 Match Soundness, Key Secrecy Security, and Freshness Predicates

Match Soundness. Match soundness is defined via a security game in which the adversary faces no restrictions on its queries, and wins if it produces either (i) two matching completed sessions that derived different session keys, or (ii) three distinct completed sessions sharing the same message transcript.

Key Secrecy Security. To define key secrecy security, we consider an indistinguishability game in which the adversary attempts to distinguish a real session key from a uniformly random key, given access to the oracle queries defined above. To prevent trivial wins, the adversary's capabilities must be restricted, and such restrictions may vary across different protocols. Following [BCL⁺26], we abstract

¹⁴ We assume that all randomness used by the protocol is securely erased upon completion of the protocol, regardless of whether the session accepts or rejects.

these restrictions by a freshness predicate F , which takes as input the adversary's query transcript and outputs true or false. For example, a simple freshness predicate F requires that the adversary has not issued any `RevealSK` query to the tested session π_{test} or to any matching session of π_{test} . Formally, F outputs true if this condition holds, and false otherwise. We now give a formal definition of F-Key Secrecy (F-KS) security. In the F-key secrecy game, the tested session is required to satisfy the freshness predicate F .

Definition 8.1 (F-KS Security). *Let Π be a key-exchange protocol among a set of parties $\mathcal{U}$, and let F be a freshness predicate defined over the adversary's query transcript. For any polynomial-time adversary $\mathcal{A}$, consider the experiment $G_{\Pi, \mathcal{A}}^{\text{F-KS}}$ defined as follows:*

1. **Setup.** *The challenger generates all static key pairs (pk_U, sk_U) for all identities $U \in \mathcal{U}$ and samples a uniformly random bit b (which will be used to answer `Test` queries). The public keys are provided to $\mathcal{A}$.*
2. **Query.** *The adversary has access to the oracles `NewSession`, `Send`, `RevealSK`, `RevealEph`, `Corrupt`, and `Test`.*
3. **Stop.** *At some point, $\mathcal{A}$ halts and outputs a guess b' for b .*
4. **Finalize.** *The adversary wins the game if the tested session π_{test} satisfies the predicate F and $b' = b$. Otherwise, the adversary loses the game, and the game $G_{\Pi, \mathcal{A}}^{\text{F-KS}}$ outputs a random bit.*

We say Π satisfies F-KS security if, for all PPT adversaries $\mathcal{A}$ the advantage $\text{Adv}_{\Pi}^{\text{F-KS}}(\mathcal{A})$ is negligible in the security parameter λ :

$$\text{Adv}_{\Pi}^{\text{F-KS}}(\mathcal{A}) = \left| \Pr[G_{\Pi}^{\text{F-KS}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2} \right|.$$

Notation for Freshness Predicates Following [BCL⁺26], we introduce auxiliary notation to simplify the formalization of the freshness predicates for our protocols `pUAKE` and `pMAKE` in later sections. All predicates defined in this subsection are Boolean-valued and are evaluated with respect to the adversary's query transcript.

Let $\pi.\text{Init}$ (resp. $\pi.\text{Resp}$) denote the initiator (responder) party of session π . More formally, $\pi.\text{Init}$ (resp. $\pi.\text{Resp}$) denotes the party $\pi.\text{actor}$ if $\pi.\text{role} = \text{Init}$ (resp. $\pi.\text{role} = \text{Resp}$), and $\pi.\text{peer}$ otherwise. For a tested session π^{Test} and $X \in \mathcal{A}, \mathcal{P}, \mathcal{I}, \mathcal{R}$, Let $\text{party}(X)$ denote the parties $\pi^{\text{Test}}.\text{actor}$, $\pi^{\text{Test}}.\text{peer}$, $\pi^{\text{Test}}.\text{Init}$, and $\pi^{\text{Test}}.\text{Resp}$, respectively. For $Y \in \{\text{Match}, \text{FirstMatch}, \text{PartialMatch}\}$, we write $\text{Set}(Y)$ for the set of sessions that stand in relation Y to the test session π^{Test} . Using these notations, we define the following freshness predicates:

- Y is true if and only if $\text{Set}(Y)$ is non-empty.
- $\text{Cor}X$ is true if and only if party X has been corrupted, that is, if the adversary has issued a `Corrupt`(X) query.
- $\text{Cor}X\text{before}T$ (resp. $\text{Cor}X\text{after}T$) is true if and only if party X is corrupted before (resp. after) the test session π^{Test} accepts.
- AisInit is true if and only if the role of party $\text{party}(A)$ is `Init`.
- AisResp is true if and only if the role of party $\text{party}(A)$ is `Resp`.
- $\text{Sk}T$ is true if and only if the adversary has issued a `RevealSK`(π^{Test}) query.
- $\text{Sk}M$ is true if and only if there exists a session $s \in \text{Set}(\text{Match})$ such that the adversary has issued a `RevealSK`(s) query.
- $\text{Eph}T$ is true if and only if the adversary has issued a `RevealEph`(π^{Test}) query.
- $\text{Eph}Y$ is true if and only if there exists a session $s \in \text{Set}(Y)$ such that the adversary has issued a `RevealEph`(s) query.

8.4 Explicit Authentication

The Explicit Authentication property captures the requirement that whenever a session π accepts and its intended peer is not corrupted, there exists a unique honest matching session for π . Note that if π

sends the final message of the exchange, then even if the intended peer is not corrupted, this does not guarantee that the final message sent by π is actually received by an honest matching session.

Following [BCL⁺26], we consider two variants of this notion, namely *Init-explicit authentication* and *Resp-explicit authentication*, which are defined based on the notion of malicious acceptance.

Definition 8.2 (Z-Malicious Acceptance). *Let $Z \in \{\text{Init}, \text{Resp}\}$. For a session π , we define the predicate MalAcc_Z to be true if and only if all of the following conditions hold:*

1. $\pi.\text{role} = Z$ and $\pi.\text{status} = \text{accepted}$;
2. $\text{Corrupt}(\pi.\text{peer})$ was not queried before π accepts;
3. there does not exist an honest session π' with $\pi'.\text{actor} = \pi.\text{peer}$ such that
 - if sessions with role Z send the final message of the protocol, then π and π' share the same message transcript except possibly for the final message sent by π and received by π' , and
 - otherwise, π and π' share the same message transcript and π' is completed. Note that in this case, the sessions are matching.

Definition 8.3 (X-Explicit Authentication). *Let $X \in \{\text{Init}, \text{Resp}\}$, and let $\bar{X}$ denote the unique role in $\{\text{Init}, \text{Resp}\}$ such that $\bar{X} \neq X$. Let Π be a key-exchange protocol among a set of parties $\mathcal{U}$. For any probabilistic polynomial-time adversary $\mathcal{A}$, consider the experiment $G_{\Pi, \mathcal{A}}^{X\text{-EA}}$ defined as follows:*

1. **Setup.** The challenger generates all static key pairs $(\text{pk}_U, \text{sk}_U)$ for all identities $U \in \mathcal{U}$ and provides the public keys to $\mathcal{A}$.
2. **Query.** The adversary has access to the oracles NewSession , Send , RevealSK , RevealEph , and Corrupt .
3. **Stop.** At some point, $\mathcal{A}$ halts and outputs a session π .
4. **Finalize.** The adversary wins the experiment if $\text{MalAcc}_{\bar{X}}(\pi) = \text{true}$.

We say Π satisfies X -explicit authentication if for all polynomial time adversaries $\mathcal{A}$, the probability of the adversary winning the experiment is negligible in the security parameter λ :

$$\text{Adv}_{\Pi}^{X\text{-EA}}(\mathcal{A}) = \Pr[G_{\Pi}^{X\text{-EA}}(\mathcal{A}) \Rightarrow 1].$$

9 Authenticated Key Exchanges based on AFS-KEMs

In this section, we formalize and analyze our mutually authenticated protocol **pMAKE** (Fig. 18) and unilaterally authenticated protocol **pUAKE** (Fig. 7). Both protocols are constructed from AFS-KEM schemes that satisfy IND-ECPA, IND-SCCA and IND-LCK-SCCA security, and are analyzed in the post-eCK-PFS model. Moreover, we require that AFS-KEM satisfies perfect **ExtractPK**-correctness and ephemeral-key-generation equivalence. In particular, we show that both protocols achieve match soundness, key secrecy, and explicit authentication. We focus primarily on the security of **pMAKE** here, while the full analysis of **pUAKE** is deferred to Section 10.

Post-id Setting and Certification Authority. Both AKE protocols are analyzed in the post-id setting, where parties neither possess each other's static public keys nor know the peer identity before protocol execution. We further assume that all party identities are unique and that a public certification authority (CA)¹⁵ binds identities to their corresponding static public keys. Formally, the CA is modeled as a trusted verification predicate $\text{CA}_{\text{verf}}(\text{id}, \text{pk}) \in \{0, 1\}$, which outputs 1 if and only if pk is the certified static public key of id .

9.1 The pMAKE Protocol and Freshness Predicate $\mathbf{F}_{\text{pMAKE}}$.

The **pMAKE** protocol is a four-round authenticated key exchange protocol that provides mutual authentication in the post-id setting. It is constructed from two invocations of AFS-KEM and a pseudorandom function PRF. The protocol flow is depicted in Fig. 18 and proceeds as follows.

¹⁵ Such a certification authority can be instantiated via standard mechanisms, including public-key infrastructures (PKI), certificate validation, or trusted public directories.

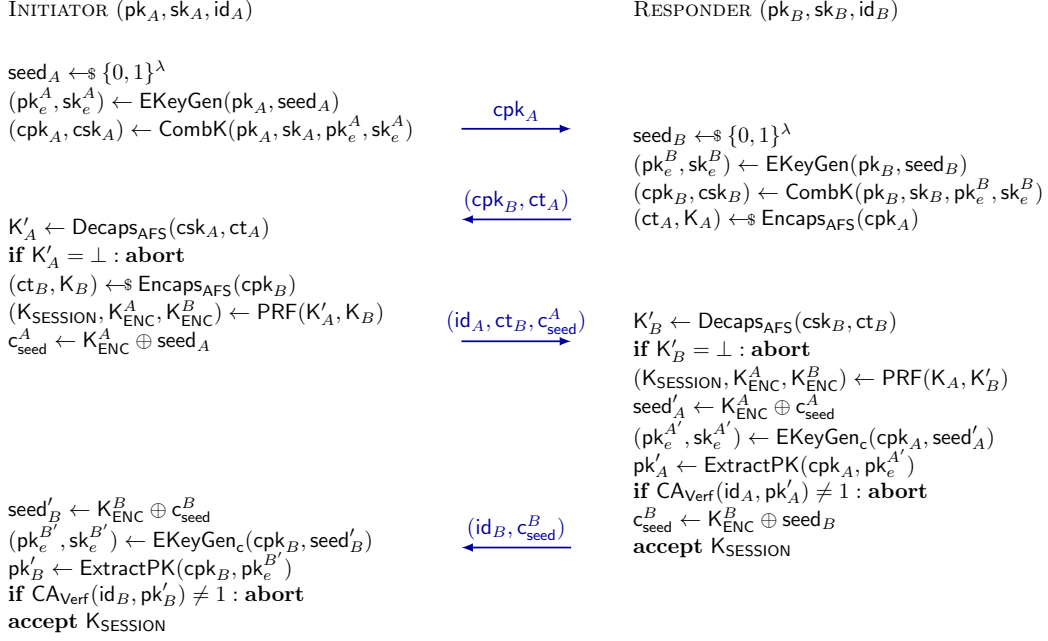


Fig. 18. The pMAKE protocol.

- *Long-term key generation and registration.* Before any protocol execution, each party U runs the static key-generation algorithm of the underlying AFS-KEM, $(\text{pk}_U, \text{sk}_U) \leftarrow \text{SKeyGen}(1^\lambda)$, to obtain a long-term key pair. The public key pk_U is registered together with the identity id_U at the certification authority. The certification authority maintains the binding between identities and certified long-term public keys and provides the verification predicate $\text{CA}_{\text{Verf}}(\text{id}, \text{pk})$. For an honestly registered party U , it holds that $\text{CA}_{\text{Verf}}(\text{id}_U, \text{pk}_U) = 1$. During protocol execution, a party need not know the peer's long-term public key in advance. Instead, the peer's claimed identity is verified only after a candidate long-term public key has been extracted from the received composite public key.
- *Round 1 (Initiator $\rightarrow$ Responder).* The initiator A samples a fresh random value seed_A and derives a corresponding ephemeral key pair $(\text{pk}_e^A, \text{sk}_e^A)$ using $\text{EKeyGen}(\text{pk}_A, \text{seed}_A)$. It then combines the long-term and ephemeral keys via CombK to obtain a composite key pair $(\text{cpk}_A, \text{csk}_A)$, and sends cpk_A to the responder B .
- *Round 2 (Responder $\rightarrow$ Initiator).* Upon receiving cpk_A , the responder B samples a fresh random value seed_B and derives a corresponding ephemeral key pair $(\text{pk}_e^B, \text{sk}_e^B)$ using $\text{EKeyGen}(\text{pk}_B, \text{seed}_B)$. It then combines the long-term and ephemeral keys via CombK to obtain $(\text{cpk}_B, \text{csk}_B)$. The responder encapsulates to cpk_A using AFS-KEM to obtain (ct_A, K_A) , and sends $(\text{cpk}_B, \text{ct}_A)$ to the initiator.
- *Round 3 (Initiator $\rightarrow$ Responder).* Upon receiving $(\text{cpk}_B, \text{ct}_A)$, the initiator decapsulates ct_A using csk_A to obtain K'_A . If decapsulation fails, the initiator aborts. Otherwise, it encapsulates to cpk_B using AFS-KEM to obtain (ct_B, K_B) . It then derives the session key K_{SESSION} together with two seed-protection keys K_{ENC}^A and K_{ENC}^B by computing $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K'_A, K_B)$. The initiator protects seed_A as $c_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \text{seed}_A$, and sends $(\text{id}_A, \text{ct}_B, c_{\text{seed}}^A)$ to the responder.
- *Round 4 (Responder $\rightarrow$ Initiator).* Upon receiving $(\text{id}_A, \text{ct}_B, c_{\text{seed}}^A)$, the responder decapsulates ct_B using csk_B to obtain K'_B . If decapsulation fails, the responder aborts. Otherwise, it derives $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B) \leftarrow \text{PRF}(K_A, K'_B)$. The responder recovers seed'_A as $\text{seed}'_A = K_{\text{ENC}}^A \oplus c_{\text{seed}}^A$, reconstructs $\text{pk}_e^{A'}$ via $\text{EKeyGen}_c(\text{cpk}_A, \text{seed}'_A)$, and extracts pk'_A using $\text{ExtractPK}(\text{cpk}_A, \text{pk}_e^{A'})$. It then verifies the binding between id_A and pk'_A using the certification authority. If the verification fails, the responder aborts. Otherwise, it protects seed_B as $c_{\text{seed}}^B = K_{\text{ENC}}^B \oplus \text{seed}_B$, accepts K_{SESSION} , and sends $(\text{id}_B, c_{\text{seed}}^B)$ to the initiator.

Upon receiving $(\text{id}_B, \text{c}_{\text{seed}}^B)$, the initiator recovers seed'_B as $\text{seed}'_B = \text{K}_{\text{ENC}}^B \oplus \text{c}_{\text{seed}}^B$, reconstructs $\text{pk}_e^{B'}$ via $\text{EKeyGen}_c(\text{cpk}_B, \text{seed}'_B)$, and extracts pk'_B using $\text{ExtractPK}(\text{cpk}_B, \text{pk}_e^{B'})$. It then verifies the binding between id_B and pk'_B using the certification authority. If the verification succeeds, the initiator accepts $\text{K}_{\text{SESSION}}$; otherwise, it aborts.

Leakage through RevealEph. Our analysis of the pMAKE protocol incorporates a RevealEph query to capture ephemeral-state leakage. In particular, the security model considers leakage only to ephemeral secrets that can be precomputed offline. For an initiator session, the query reveals the randomness seed_A used in EKeyGen , which uniquely determines the ephemeral secret key sk_e^A . Analogously, for a responder session, the query reveals the randomness seed_B used in EKeyGen and hence the corresponding ephemeral secret key sk_e^B . We assume the randomness used for encapsulation is securely erased after computation.

Remark 9.1 (Precomputation and State Leakage). In many existing KEM-based AKE protocols in the pre-specified-peer setting, the peer’s long-term public key is known before session activation. Hence, an implementation may be tempted to precompute encapsulation outputs offline, e.g., $(\text{ct}_A, \text{K}_A) \leftarrow \text{Encaps}(\text{pk}_A)$, and store K_A for later use in an actual protocol execution, thereby reducing online computation. This implementation practice should be avoided: storing the precomputed key K_A for later use effectively turns it into local ephemeral state that may be exposed by RevealEph. If this state is leaked, an adversary may use it to complete a session while impersonating the intended peer, leading to a KCI-style state-leakage attack.

Our construction avoids this pitfall by tying the key computation to a fresh, session-specific cpk . Since cpk is generated during the protocol execution and is unavailable before session activation, the encapsulation output cannot be precomputed offline. Thus, the encapsulated key is bound to the current session, ruling out KCI-style attacks based on the leakage of retained precomputed keys.

This protection can be further strengthened by deriving the ephemeral secret key from a session-specific auxiliary string aux , e.g., $\text{sk}_e \leftarrow H(\text{seed}_e, \text{aux})$, where aux may include the current transcript prefix, role labels, nonces, and timestamps. In this case, the corresponding ephemeral public key and combined public key are also bound to aux and therefore cannot be precomputed before the current session context is fixed. Moreover, after recovering seed_e , the peer can recompute sk_e using the same aux and verify that the received cpk is consistent with the session-bound ephemeral key material. Hence, any accepted cpk is tied to the current session context rather than to retained precomputed state.

This peer-verifiable protection is not directly compatible with the NAXOS-style technique used in eCK-secure AKE protocols [LLM07, FSXY12, FSXY13]. In NAXOS-style KEM-based constructions [FSXY12, FSXY13], the KEM encapsulation randomness is derived by mixing ephemeral secret values with the party’s long-term secret key. Although a public session-specific string aux could also be included in such a derivation, the resulting binding is only locally enforced by the sender. Since the peer does not know the sender’s long-term secret key or the ephemeral secret values entering the derivation, it cannot recompute the derived randomness from the aux . Consequently, the peer cannot verify that the received ciphertext was honestly generated using the current session context.

9.2 Key Secrecy

Key secrecy with perfect forward secrecy is achieved for both pMAKE and pUAKE in the ROM and QROM, where the PRG and PRF are modeled as (quantum-accessible) random oracles. The analysis mainly relies on the IND-ECPA, IND-SCCA, and IND-LCK-SCCA security of AFS-KEM, together with its Key-Coll, SPK-Bind, and RSPK-Bind security.

We next define the freshness predicate F_{pMAKE} for pMAKE and prove that pMAKE achieves F_{pMAKE} -key secrecy in Theorem 9.3. The corresponding freshness predicate F_{pUAKE} and the analysis for pUAKE are deferred to Section 10 (Definition 10.1, Theorems 10.2, 10.6).

PartialMatch. As defined in Sec. 8.1, two sessions π and π' are said to be PartialMatch if they agree on the KEM-related components of the first three protocol messages. Concretely, in the pMAKE protocol, sessions π and π' are PartialMatch if $(\pi.\text{cpk}_A, \pi.\text{cpk}_B, \pi.\text{ct}_A, \pi.\text{ct}_B) = (\pi'.\text{cpk}_A, \pi'.\text{cpk}_B, \pi'.\text{ct}_A, \pi'.\text{ct}_B)$, where $\pi.X$ denotes the value X in session π .

Definition 9.2 (Freshness Predicate for pMAKE). Using the notation from Sec. 8.3, we define the freshness predicate F_{pMAKE} as a disjunction of the cases $\text{FM}_1, \dots, \text{FM}_6$, each capturing an admissible pattern of adversarial capabilities. In the subsequent security analysis, for the cases FM_1 – FM_4 , we show that the adversary’s advantage in winning the key-secrecy game is negligible. The cases FM_5 and FM_6 are handled by showing that these events occur only with negligible probability.

Specifically, the analyses of cases FM_1 – FM_4 denote the scenarios where a `PartialMatch` session of π^{Test} exists, and as long as at least one of the secret materials $(\text{sk}_A, \text{sk}_B, \text{seed}_A, \text{seed}_B)$ in the tested session π^{Test} and its `PartialMatch` session is not revealed to the adversary, the session key remains secure. The analyses of cases FM_5 and FM_6 indicate that if the adversary does not corrupt the peer of π^{Test} before π^{Test} accepts, then a `PartialMatch` session of π^{Test} must exist.

$$\begin{aligned}\text{FM}_1 &\triangleq \overline{\text{CorA}} \wedge \text{PartialMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FM}_2 &\triangleq \overline{\text{CorP}} \wedge \text{PartialMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FM}_3 &\triangleq \overline{\text{EphT}} \wedge \text{PartialMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FM}_4 &\triangleq \overline{\text{EphPartialMatch}} \wedge \text{PartialMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FM}_5 &\triangleq \text{CorPafterT} \wedge \overline{\text{PartialMatch}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FM}_6 &\triangleq \overline{\text{CorP}} \wedge \overline{\text{PartialMatch}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}.\end{aligned}$$

We define

$$F_{\text{pMAKE}} \triangleq \bigvee_{i=1}^6 \text{FM}_i.$$

Theorem 9.3 (F_{pMAKE} -Key Secrecy of pMAKE in the ROM and QROM). Let there be at most N parties and at most S sessions activated in the F_{pMAKE} -KS game. Suppose that the underlying AFS-KEM is δ -correct, satisfies γ -spreadness, and satisfies perfect `ExtractPK`-correctness and ephemeral-key-generation equivalence. We model the PRF as a random oracle G , to which the adversary makes at most q_G queries. Then, for any PPT adversary $\mathcal{A}$ attacking the F_{pMAKE} -KS security of pMAKE, there exist PPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, constructed below and running in time comparable to that of $\mathcal{A}$, such that

$$\begin{aligned}\text{Adv}_{\text{pMAKE}}^{F_{\text{pMAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &\quad + NS \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + 2(S+1) \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) \\ &+ 4(2NS+1) \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \\ &+ S \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + 8 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) \\ &+ 8(S+1) \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \frac{4q_G}{|\mathcal{K}|} + 16\delta \\ &+ (8N^2 + 29N + 20) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \end{aligned} \right) \end{aligned} \right).\end{aligned}$$

If we model PRF as a quantum-accessible random oracle G , to which the adversary makes at most q_G quantum queries, then, for any QPT adversary $\mathcal{A}$ attacking the F_{pMAKE} -KS security of pMAKE, there exist QPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that

$$\begin{aligned}\text{Adv}_{\text{pMAKE}}^{F_{\text{pMAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &\quad + NS \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + 2(S+1) \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 4 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \\ &+ S \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + 8(S+1) \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &+ (5N+4) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 16\delta \end{aligned} \right) \end{aligned} \right) \\ &\quad + 4NS^2 \left(2\sqrt{(q_G+1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_G}{|\mathcal{K}|} + \beta \right)} + \beta \right),\end{aligned}$$

where $\beta = N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$. Here k_1 and k_2 are the min-entropy bounds on the honest static and combined public keys of AFS-KEM, respectively. Furthermore, $|\mathcal{K}|$ denotes the cardinality of the AFS-KEM encapsulated key space.

Proof sketch. The full proof is given in Section 11.1. We present only a proof sketch here. We assume that all honest users hold distinct static public keys, that all honest sessions derive distinct combined public keys cpk , including cpk_A and cpk_B , and that AFS-KEM is perfectly correct. These assumptions follow from the high min-entropy of the static public keys and the combined public keys, together with the δ -correctness of AFS-KEM. Under these assumptions, a session cannot have more than one PartialMatch or Match session.

According to the definition of F_{pMAKE} (Definition 9.2), we analyze the six disjoint freshness cases FM_1 – FM_6 . Let π denote the tested session, and let π' denote its unique PartialMatch session, if it exists. By symmetry of the protocol, we assume without loss of generality that $\pi.\text{role} = \text{Resp}$. The case $\pi.\text{role} = \text{Init}$ follows by a similar argument.

Analysis for Cases FM_1 and FM_2 in the standard model. The analyses of Cases FM_1 and FM_2 are similar. We therefore focus on Case FM_2 . Accordingly, we consider the setting where $\pi.\text{role} = \text{Resp}$ and $\pi.\text{peer} = P^*$ is uncorrupted. First, by the SPK-Bind security of AFS-KEM (Definition 6.5), we have $\pi'.\text{actor} = P^*$ and if π' accepts, then π' must be the unique Match session of π . At a high level, cpk provides a binding to both its static public key pk_s and its ephemeral seed seed_e . Any attempt by the adversary to change the identity id (hence the corresponding static public key) or c_{seed} (which changes the recovered seed_e since the OTP encryption is deterministic under a fixed key) will make the cpk verification fail. Then we guess the sessions π and π' , as well as the peer identity $\pi.\text{peer}$, denoted by P^* , which succeeds with probability $1/(NS^2)$. We then rely on the IND-SCCA security of AFS-KEM to replace the KEM key K_A in sessions π and π' with a uniformly random value, provided that the adversary does not know the static secret key sk_{P^*} . Notably, the decapsulation oracle is used to simulate all other sessions $\hat{\pi} \notin \{\pi, \pi'\}$ with $\hat{\pi}.\text{actor} = P^*$, without knowing sk_{P^*} . Next, by the security of the PRF, we replace K_{SESSION} , K_{ENC}^A , and K_{ENC}^B in sessions π and π' with independent uniformly random keys. Therefore, now at most one accepted session derives the same session key as π , namely the matching session, while any other accepted session derives an independent session key. By the definition of FM_2 , the session key of the matching session of π cannot be revealed to the adversary. Thus, the adversary's advantage in this case is negligible.

Analysis for Cases FM_3 and FM_4 in the ROM. In these two cases, we model the PRF as a random oracle G . The analyses of Cases FM_3 and FM_4 are similar, so we focus on Case FM_4 here. That is, we consider the setting where the seed_A in π' is not leaked to the adversary and $\pi.\text{role} = \text{Resp}$. We first guess the sessions π and π' , as well as the identity of $\pi'.\text{actor}$, which succeeds with probability $1/(NS^2)$. First, by the key-collision security of AFS-KEM (Definition 6.7), except with negligible probability there exists no other honestly generated session $\hat{\pi} \notin \{\pi, \pi'\}$ that derives the same KEM key K_A^{Test} , where K_A^{Test} denotes the value of K_A in π and π' . We then replace K_{SESSION} , K_{ENC}^A , and K_{ENC}^B in sessions π and π' with the same uniformly random values. We define the event QUERY as the adversary querying G with input $(K_A^{\text{Test}}, \cdot)$. Conditioned on $\neg \text{QUERY}$, this replacement is indistinguishable to the adversary. Then, the SPK-Bind security of AFS-KEM implies that at most one accepted session derives the same session key as π , namely the matching session, whose key is unrevealed by definition. Consequently, the adversary's advantage in this case is negligible.

It therefore suffices to bound the probability of the event QUERY. Since the OTP key K_{ENC}^A is uniform from the adversary's view, perfect secrecy of the OTP allows us to replace c_{seed}^A in π' with an OTP encryption of an independent random seed. Then, by the IND-ECPA security of AFS-KEM, we replace the KEM key K_A^{Test} in π and π' with a uniformly random value. After these hybrids, the value K_A^{Test} is uniformly random and independent of the adversary's view, and hence $\Pr[\text{QUERY}]$ is negligible.

Analysis for Cases FM_3 and FM_4 in the QROM. We lift the ROM argument for Cases FM_3, FM_4 to the QROM using the semi-classical oracle technique of [AHU19].

Analysis for Cases FM_5 and FM_6 in the standard model. Since the arguments for Cases FM_5 and FM_6 are analogous, we focus on Case FM_6 . We show that the event FM_6 occurs only with negligible probability. We first guess the tested session π and its peer identity $\pi.\text{peer}$, denoted by P^* , which succeeds with probability $1/(NS)$. Based on the IND-LCK-SCCA security of AFS-KEM, we can replace the KEM key K_A in session π and in the (unique, if exists) session $\bar{\pi}$ satisfying $\pi.\text{cpk}_A = \bar{\pi}.\text{cpk}_A$ and $\pi.\text{ct}_A = \bar{\pi}.\text{ct}_A$ with the same uniformly random value. The reduction proceeds as follows. It receives a challenge public key pk_s , which is embedded as pk_{P^*} . When session π receives cpk_A , the reduction

forwards it to its challenger and obtains a challenge ciphertext ct together with a key K_b , which are then embedded as $K_A = K_b$ and $\text{ct}_A = \text{ct}$ in π and $\bar{\pi}$ (if exists). All other sessions with actor P^* are simulated using the decapsulation oracle.

Notably, if such a session $\bar{\pi}$ exists and $\bar{\pi}$ is not the **PartialMatch** session of π , then by the key-collision security of AFS-KEM we have $\pi.K_B \neq \bar{\pi}.K_B$. We then replace K_{SESSION} , K_{ENC}^A , and K_{ENC}^B in session π and in its **PartialMatch** session (if exists) with the same uniformly random values, while assigning an independent uniformly random key to the session $\bar{\pi}$ (if exists) that is not the **PartialMatch** session of π . Since only the **PartialMatch** session shares the same encryption keys as π , if such a **PartialMatch** session does not exist, then these encryption keys are never used in any other session. Therefore, the seed $\text{seed} = K_{\text{ENC}} \oplus c_{\text{seed}}$ recovered by π is uniformly random from the adversary's view by the perfect secrecy of the OTP. We can then rely on the RSPK-Bind security of the underlying AFS-KEM scheme (Definition 6.6) to show that right now the session π accepts only with negligible probability. Consequently, Case FM_6 can occur only with negligible probability.

9.3 Match Soundness.

Match soundness holds for both pUAKE and pMAKE. The first condition follows directly from the correctness of AFS-KEM. For the second condition, the min-entropy of the combined public key ensures that two distinct sessions derive the same composite public key cpk only with negligible probability. Moreover, the γ -spreadness property of AFS-KEM ensures that two distinct sessions that receive valid cpk (resp. cpk_A , cpk_B) from their peer produce the same encapsulation ciphertext only with negligible probability.

9.4 Explicit Authentication.

Init-explicit authentication is achieved by both pMAKE and pUAKE, while pMAKE additionally provides Resp-explicit authentication. The arguments establishing explicit authentication for pMAKE closely follow the analysis of Cases FM_5 and FM_6 . The only difference lies in the final step, where we further show that if a session π accepts, then the unique existing **PartialMatch** session must satisfy all requirements of the explicit-authentication property. For example, to prove Init-explicit authentication for pMAKE, consider any responder session π that accepts and whose peer has not been corrupted before π accepts. In the final step, we show that the **PartialMatch** session of π must additionally have the same third-round message as π . The formal analyses of Explicit Authentication are deferred to Section 11.3 for pMAKE and Section 10.4 for pUAKE.

10 Full Security Analysis for pUAKE

10.1 The pUAKE Protocol and Freshness Predicate $\mathbf{F}_{\text{pUAKE}}$.

The pUAKE protocol is a three-round authenticated key exchange protocol that provides initiator-only authentication in the post-id setting. It is constructed from a single AFS-KEM and a pseudorandom generator (PRG). The protocol flow is depicted in Fig. 7 and proceeds as follows.

- *Long-term key generation and registration.* Before any protocol execution, each party U runs the static key-generation algorithm of the underlying AFS-KEM, $(\text{pk}_U, \text{sk}_U) \leftarrow \text{SKeyGen}(1^\lambda)$, to obtain a long-term key pair. The public key pk_U is registered together with the identity id_U at the certification authority. The certification authority maintains the binding between identities and certified long-term public keys and provides the verification predicate $\text{CA}_{\text{Verf}}(\text{id}, \text{pk})$. For an honestly registered party U , it holds that $\text{CA}_{\text{Verf}}(\text{id}_U, \text{pk}_U) = 1$.
- *Round 1 (Initiator $\rightarrow$ Responder).* The initiator A samples a fresh random value seed_A and derives a corresponding ephemeral key pair $(\text{pk}_e^A, \text{sk}_e^A)$ using EKeyGen . It then combines the long-term and ephemeral keys via CombK to obtain a composite key pair $(\text{cpk}_A, \text{csk}_A)$, and sends cpk_A to the responder B .
- *Round 2 (Responder $\rightarrow$ Initiator).* Upon receiving cpk_A , the responder encapsulates using the AFS-KEM to obtain (ct_A, K_A) and sends ct_A to the initiator. The initiator decapsulates ct_A using csk_A to obtain K'_A and derives the session key K_{SESSION} together with an encryption key K_{ENC}^A via a PRG. The initiator A then accepts the session key K_{SESSION} .

- *Round 3 (Initiator $\rightarrow$ Responder)*. The initiator encrypts seed_A under K_{ENC}^A by computing $c_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \text{seed}_A$, and sends $(\text{id}_A, c_{\text{seed}}^A)$ to the responder. The responder recovers $\text{seed}'_A = K_{\text{ENC}}^A \oplus c_{\text{seed}}^A$ and reconstructs $\text{pk}_e^{A'}$ via EKeyGen_c . It then extracts pk'_A using ExtractPK and verifies the binding between id_A and pk'_A using the certification authority (CA). If the verification succeeds, the responder accepts the session key K_{SESSION} .

Leakage through RevealEph. Our analysis of the pUAKE protocol accounts for an RevealEph query that may reveal the following information. For an initiator session, the query reveals the randomness seed_A used in EKeyGen , which implies knowledge of the ephemeral secret key sk_e^A . For a responder session, no information is revealed. In particular, we assume that the randomness used for encapsulation is securely erased immediately after its computation.

Definition 10.1 (Freshness Predicate for pUAKE). *Using the notion in Sec. 8.3, we define the freshness predicate F_{pUAKE} as the following five cases, each corresponding to an admissible adversarial capability pattern. All pairs of cases are disjoint by construction. In the subsequent security analysis, the case FU_5 will be handled by showing that the event FU_5 occurs with negligible probability, whereas for FU_1 – FU_4 we show that the adversary’s advantage in winning the key secrecy game is negligible.*

$$\begin{aligned} \text{FU}_1 &\triangleq \text{AisInit} \wedge \overline{\text{Corl}} \wedge \text{FirstMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FU}_2 &\triangleq \text{AisResp} \wedge \overline{\text{Corl}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FU}_3 &\triangleq \text{AisInit} \wedge \text{Corl} \wedge \overline{\text{EphT}} \wedge \text{FirstMatch} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FU}_4 &\triangleq \text{AisResp} \wedge \text{Corl} \wedge \text{FirstMatch} \wedge \overline{\text{EphFirstMatch}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}, \\ \text{FU}_5 &\triangleq \text{AisResp} \wedge \text{Corl after T} \wedge \overline{\text{FirstMatch}} \wedge \overline{\text{SkT}} \wedge \overline{\text{SkM}}. \end{aligned}$$

We define

$$F_{\text{pUAKE}} \triangleq \bigvee_{i=1}^5 \text{FU}_i.$$

10.2 Key Secrecy of the pUAKE protocols

Theorem 10.2 (F_{pUAKE} -Key Secrecy of pUAKE in the ROM). *Let there be at most N parties and at most S sessions activated in the F_{pUAKE} -KS game. Suppose that the underlying AFS-KEM is δ -correct, satisfies γ -spreadness, and satisfies perfect ExtractPK -correctness and ephemeral-key-generation equivalence. We model the PRG as a random oracle G , to which the adversary makes at most q_G queries. Then, for any PPT adversary $\mathcal{A}$ against the F_{pUAKE} -KS security of pUAKE, there exist PPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, constructed below and running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{F_{\text{pUAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &\quad + NS \left(4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) \right) \\ &\quad + S \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + 4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) \right. \\ &\quad \left. + 4S \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + 4(N+1) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right) \\ &\quad \left. + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + \frac{2q_G}{|\mathcal{K}|} + 5\delta \right). \end{aligned}$$

where k_1 and k_2 are min-entropy bounds on honest static and combined public keys of AFS-KEM, respectively, and $|\mathcal{K}|$ is the cardinality of the AFS-KEM encapsulated key space.

Proof. First, we assume that all honest initiators have distinct long-term public keys, and that all honest initiator sessions compute distinct cpk_A . Moreover, we assume that every responder session that receives an honest cpk_A from some honest initiator session, computes a distinct ct_A . Since the min-entropy of the public key pk_A is at least k_1 , the honestly combined public key cpk_A has min-entropy at least k_2 , and the ciphertexts of AFS-KEM satisfy γ -spreadness, this will add $\frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma}$ to the

final bound. Under these assumptions, it is impossible for a session to have more than one **FirstMatch** or **Match** session.

We assume that the tested session satisfies the F_{pUAKE} throughout the experiment, as otherwise the adversary loses the indistinguishability game. To bound $\text{Adv}_{\text{pUAKE}}^{F_{\text{pUAKE}}\text{-KS}}(\mathcal{A})$, we partition the event that the adversary wins into five cases according to the definition $F_{\text{pUAKE}} = \bigvee_{i=1}^5 \text{FU}_i$ (Definition 10.1), which yields

$$\text{Adv}_{\text{pUAKE}}^{F_{\text{pUAKE}}\text{-KS}}(\mathcal{A}) \leq \sum_{i=1}^5 \text{Adv}_{\text{pUAKE}}^{\text{FU}_i\text{-KS}}.$$

We now give security reductions for each subcase $\text{Adv}_{\text{pUAKE}}^{\text{FU}_i\text{-KS}}(\mathcal{A})$ with $i \in \{1, 2, 3, 4, 5\}$, where throughout we assume that the tested session satisfies FU_i . Otherwise the adversary trivially loses the corresponding indistinguishability game. The corresponding reductions are stated in Lemmas 10.3, 10.4, and 10.5. For readability, we defer the proofs of these lemmas to Subsection 10.3.

Lemma 10.3. *With the notations and assumptions of the proof of Theorem 10.2, there exist adversaries $\mathcal{B}_1$, $\mathcal{C}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_1\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + \delta \right)$$

Lemma 10.4. *With the notations and assumptions of the proof of Theorem 10.2, there exist adversaries $\mathcal{B}_2$, $\mathcal{C}$, $\mathcal{D}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_2\text{-KS}}(\mathcal{A}) \leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + (N+1)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + S\delta \right)$$

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_5\text{-KS}}(\mathcal{A}) \leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + S\delta \right)$$

Lemma 10.5. *With the notations and assumptions of the proof of Theorem 10.2, and with PRG modeled as a random oracle $\mathcal{G}$ to which the adversary makes at most $q_{\mathcal{G}}$ queries, there exist adversaries $\mathcal{B}_3$, $\mathcal{C}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} + \delta \right) \\ &\quad + NS^2 \left(2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right). \\ \text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} + \delta \right) \\ &\quad + NS^2 \left(2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right). \end{aligned}$$

Combining all the above lemmas completes the proof. $\square$

Theorem 10.6 (F_{pUAKE} -Key Secrecy of pUAKE in the QROM). *Let there be at most N parties and at most S sessions activated in the F_{pUAKE} -KS game. Suppose that the underlying AFS-KEM is δ -correct, satisfies γ -spreadness, and satisfies perfect ExtractPK-correctness and ephemeral-key-generation equivalence. We model the PRG as a quantum-accessible random oracle $\mathcal{G}$, to which the adversary makes at most $q_{\mathcal{G}}$ (quantum) queries. Then, for any QPT adversary $\mathcal{A}$ attacking the F_{pUAKE} -KS security of pUAKE, there exist QPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, constructed below and running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{F_{\text{pUAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^{\gamma}} \\ &\quad + NS \left(4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 2 \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) \right. \\ &\quad \left. + S \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + 5\delta \right. \right. \\ &\quad \left. \left. + 4S \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + 4(N+1) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right) \right. \\ &\quad \left. + 4 \sqrt{(q_{\mathcal{G}}+1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{4q_{\mathcal{G}}}{|\mathcal{K}|} \right)} \right). \end{aligned}$$

where k_1 and k_2 are the min-entropy bounds on the honest static and combined public keys of AFS-KEM, respectively. Furthermore, $|\mathcal{K}|$ denotes the cardinality of the AFS-KEM encapsulated key space.

Proof. This proof follows the same argument as that of Theorem 10.2, except for Lemma 10.5. Lemma 10.5 is the only lemma that relies on the random oracle model, while all other cases are analyzed in the standard model. Below, we extend Lemma 10.5 to the QROM setting. For readability, the proof of Lemma 10.7 is deferred to Subsection 10.3.

Lemma 10.7 (QROM counterpart of Lemma 10.5). *With the notations and assumptions of the proof of Theorem 10.6, and with PRG modeled as a quantum-accessible random oracle $\mathbf{G}$ to which the adversary makes at most $q_{\mathbf{G}}$ (quantum) queries, there exist QPT adversaries $\mathcal{B}_3$, $\mathcal{C}$, and $\mathcal{D}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned}\text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \sqrt{(q_{\mathbf{G}} + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{4q_{\mathbf{G}}}{|\mathcal{K}|} \right)} + \beta \right), \\ \text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \sqrt{(q_{\mathbf{G}} + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{4q_{\mathbf{G}}}{|\mathcal{K}|} \right)} + \beta \right),\end{aligned}$$

where $\beta = 2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N + 1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \delta$.

Combining all the above analyses completes the proof. $\square$

10.3 Full Analysis for Lemmas 10.3, 10.4, 10.5, and 10.7

Lemma 10.3. *With the notations and assumptions of the proof of Theorem 10.2, there exist adversaries $\mathcal{B}_1$, $\mathcal{C}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_1\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + (N + 1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + \delta \right)$$

Proof. We assume that the tested session satisfies FU_1 , otherwise the adversary trivially loses the game.

Game G_0 . This game corresponds to the original game for $\text{Adv}_{\text{pUAKE}}^{\text{FU}_1\text{-KS}}(\mathcal{A})$, that is,

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_1\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (Guessing $\pi^{\text{Test}}.\text{actor}$, π^{Test} , and $\pi^{\text{FirstMatch}}$). In game G_1 , we first guess the actor identity of the test session, and further guess the test session π^{Test} and its corresponding (unique) FirstMatch session $\pi^{\text{FirstMatch}}$. If any of the guesses is incorrect, the game is aborted. Concretely, we sample P^* uniformly from the N parties, and sample two session instances π, π' uniformly from the S sessions, respectively. If any of the guesses is incorrect (i.e., $P^* \neq \pi^{\text{Test}}.\text{actor}$, or $\pi \neq \pi^{\text{Test}}$, or $\pi' \neq \pi^{\text{FirstMatch}}$), the game is aborted. The probability of guessing correctly is $1/(NS^2)$, and therefore

$$\text{Adv}^{G_0}(\mathcal{A}) \leq NS^2 \text{Adv}^{G_1}(\mathcal{A}).$$

Game G_2 (Random K_A). In this game, we replace the KEM key K_A in the tested session π and its (unique) FirstMatch session π' with an independently uniform key $\tilde{k} \leftarrow \$ \mathcal{K}$. For any adversary $\mathcal{A}$ that can distinguish G_1 from G_2 , we construct an adversary $\mathcal{B}_1$ against the IND-SCCA security for AFS-KEM. Adversary $\mathcal{B}_1$ interacts with its AFS-KEM challenger and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*, \text{ct}^*, K_b^*)$, where K_b^* equals the real encapsulated key if $b = 0$ and a uniform random key if $b = 1$. It then simulates the view of $\mathcal{A}$ as in G_2 , except for the following changes.

- $\mathcal{B}_1$ sets pk_s^* as the long-term public key of P^* , and generates long-term key pairs $(\text{pk}_U, \text{sk}_U)$ honestly for all other parties. It embeds $(\text{cpk}^*, \text{ct}^*)$ into the tested session π and its corresponding FirstMatch session π' , and uses K_b^* as the KEM key in both sessions. Using seed_e^* and K_b^* , $\mathcal{B}_1$ can simulate both sessions according to the protocol specification.

- For each non-tested session $\hat{\pi} \neq \pi$ of P^* with $\hat{\pi}.\text{role} = \text{Init}$, $\mathcal{B}_1$ independently queries $\mathcal{O}^{\text{EKeyGen}}()$ to obtain a fresh tuple $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$ for the session and sets the first protocol message to be cpk . Upon receiving the corresponding KEM ciphertext ct for $\hat{\pi}$, $\mathcal{B}_1$ queries $\mathcal{O}^{\text{Dec}}(\text{cpk}, \text{ct})$ to obtain the KEM key K_A and continues the simulation of that session honestly. Notably, we have $\text{cpk} \neq \text{cpk}^*$ by the assumption that all honestly generated cpk_A are distinct.

When $b = 0$, $\mathcal{B}_1$ perfectly simulates G_1 , as long as the KEM ciphertext received by π is always correctly decapsulated. In G_1 , π performs the decapsulation of the KEM ciphertext, which could potentially differ from the encapsulated key. In the simulation, $\mathcal{B}_1$ always uses the real encapsulated key. This difference is quantified by the δ -correctness of the underlying AFS-KEM. That is, $|\Pr[\mathcal{B}_1 \Rightarrow 1 : b = 0] - \Pr[G_1^A \Rightarrow 1]| \leq \delta$. When $b = 1$, $\mathcal{B}_1$ perfectly simulates G_2 , that is $\Pr[\mathcal{B}_1 \Rightarrow 1 : b = 1] = \Pr[G_2^A \Rightarrow 1]$. Therefore, we have

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \delta.$$

Game G_3 (Random K_{SESSION} , K_{ENC}^A). In this game, we replace the session keys K_{SESSION} in π and π' with the same independently sampled uniform key $\hat{K}_{\text{SESSION}}$. Moreover, we replace the keys K_{ENC}^A in π and π' with the same independently sampled uniform key $\hat{K}_{\text{ENC}}^A$. For any adversary that can distinguish G_2 from G_3 , we construct an adversary $\mathcal{Z}$ against the IND-PRG security of the PRG as follows. $\mathcal{Z}$ receives a value y_b from its challenger. If $b = 0$, then $y_b = \text{PRG}(k)$, where $k \leftarrow \mathcal{K}$. If $b = 1$, then y_b is uniformly chosen from the output space of the PRG. Adversary $\mathcal{Z}$ then perfectly simulates the game for $\mathcal{A}$ as G_2 , except that it uses y_b as the pair $(K_{\text{SESSION}}, K_{\text{ENC}}^A)$ in the sessions π and π' . Note that when $b = 0$, the simulation is identical to G_2 , and when $b = 1$, it is identical to G_3 . That is $\Pr[\mathcal{Z} \Rightarrow 1 : b = 0] = \Pr[G_2^A \Rightarrow 1]$ and $\Pr[\mathcal{Z} \Rightarrow 1 : b = 1] = \Pr[G_3^A \Rightarrow 1]$. Therefore, we have

$$|\text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A})| \leq \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}).$$

Game G_4 (Peer acceptance implies a matching session). In this game, we abort if π' accepts, but π' is not the matching session of π , which we define as the BAD event. Since games G_3 and G_4 are identical unless the BAD event occurs, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq \Pr[\text{BAD} : G_3].$$

Next, we analyze $\Pr[\text{BAD} : G_3]$. If BAD occurs in G_3 , then the adversary has tampered with the third message flow between π and π' . Specifically, the pair $(\text{id}_{P^*}, \text{c}_{\text{seed}})$ sent by π differs from $(\text{id}', \text{c}'_{\text{seed}})$ received by π' . We consider the following two cases:

- **Case 1:** If $\text{id}_{P^*} \neq \text{id}'$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that it first samples B^* uniformly from among the N parties and aborts if $B^* \neq \text{id}'$. Notably, $\mathcal{C}$ guesses correctly with probability $1/N$. Then $\mathcal{C}$ sets $\text{pk}_{P^*} = \text{pk}_s^*$ and $\text{pk}_{B^*} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π . When π' receives a pair $(\text{id}', \text{c}'_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c'_{seed} and outputs the corresponding seed'_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}', \text{pk}'_s) = 1$ and $\text{pk}_{\text{id}'} = \overline{\text{pk}}_s^*$, we conclude that $\text{pk}'_s = \overline{\text{pk}}_s^*$ by the security of CA. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2:** If $\text{id}_{P^*} = \text{id}'$ and $\text{c}_{\text{seed}} \neq \text{c}'_{\text{seed}}$, we also construct an adversary $\mathcal{C}^{16}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that $\mathcal{C}$ sets $\text{pk}_{P^*} = \text{pk}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π . When π' receives a valid pair $(\text{id}_{P^*}, \text{c}'_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c'_{seed} and outputs the corresponding seed'_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{c}'_{\text{seed}} \neq \text{c}_{\text{seed}}$, we have $\text{pk}'_s = \text{pk}_s^*$ and $\text{seed}'_e \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case, then $\mathcal{C}$ also wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2}] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

¹⁶ We reuse the notation $\mathcal{C}$ for simplicity.

Combining the two cases, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Analysis of G_4 . In G_4 , the session key K_{SESSION} of the tested session π is uniformly sampled, and there may be one accepted session (which must be the matching session) that uses the same session key K_{SESSION} . Note that all session keys in other accepted sessions are independent of K_{SESSION} right now. Since we do not allow the adversary to reveal the session key in the matching session of π , we have

$$\text{Adv}^{G_4}(\mathcal{A}) = 0.$$

Combining all the bounds derived above completes the proof. $\square$

Lemma 10.4. *With the notations and assumptions of the proof of Theorem 10.2, there exist adversaries $\mathcal{B}_2$, $\mathcal{C}$, $\mathcal{D}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_2\text{-KS}}(\mathcal{A}) \leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + (N+1)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + S\delta \right)$$

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_5\text{-KS}}(\mathcal{A}) \leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + S\delta \right)$$

Proof. In this lemma, we analyze both $\text{Adv}_{\text{pUAKE}}^{\text{FU}_2\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pUAKE}}^{\text{FU}_5\text{-KS}}(\mathcal{A})$, since their proofs follow a similar structure. We first present a complete proof for Case FU_2 , and then show how to adapt the argument to Case FU_5 .

Game G_0 . This game corresponds to the original game for $\text{Adv}_{\text{pUAKE}}^{\text{FU}_2\text{-KS}}(\mathcal{A})$, that is,

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_2\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (Guessing $\pi^{\text{Test}}.\text{peer}$, π^{Test}). In game G_1 , we first guess the peer identity of the test session, and further guess the test session π^{Test} . If either guess is incorrect, the game is aborted. Concretely, we sample P^* uniformly from the N parties, and sample one session instance π uniformly from the at most S sessions. If the guess is incorrect (i.e., $P^* \neq \pi^{\text{Test}}.\text{peer}$ or $\pi \neq \pi^{\text{Test}}$), the game is aborted. The probability of guessing correctly is $1/(NS)$, therefore

$$\text{Adv}^{G_0}(\mathcal{A}) \leq NS \cdot \text{Adv}^{G_1}(\mathcal{A}).$$

Game G_2 (Random K_A). In this game, we replace the KEM key K_A in the tested session π with an independently uniform key $\tilde{k} \leftarrow \mathcal{K}$. If the unique FirstMatch session π' of π exists, we also replace the K_A in π' with the same key $\tilde{k}$. For any adversary $\mathcal{A}$ that can distinguish G_1 from G_2 , we construct an adversary $\mathcal{B}_2$ against the IND-LCK-SCCA security of AFS-KEM as follows. Adversary $\mathcal{B}_2$ first receives pk_s^* from its challenger, then sets pk_s^* as the long-term public key of P^* , and generates long-term key pairs $(\text{pk}_U, \text{sk}_U)$ honestly for all other parties. It then simulates the view of $\mathcal{A}$ as in G_2 , except for the following changes.

- For each session $\hat{\pi}$ of P^* with $\hat{\pi}.\text{role} = \text{Init}$, $\mathcal{B}_2$ independently queries $\mathcal{O}^{\text{EKeyGen}}$ to obtain a fresh tuple $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$ for the session and sets the first message of the session to be cpk . When π receives its cpk' , $\mathcal{B}_2$ sets $\text{cpk}^* = \text{cpk}'$ and forwards it to its challenger. After $\mathcal{B}_2$ receives $(\text{pk}_s^*, \text{ct}^*, K_b^*)$ from its challenger, $\mathcal{B}_2$ embeds ct^* into the π and uses K_b^* as the KEM key K_A in π . If there exists a session of P^* that is the FirstMatch session of π and we denote this session as π' , $\mathcal{B}_2$ also sets K_b^* as the KEM key K_A in π' . Moreover, upon P^* receiving the corresponding KEM ciphertext ct for $\hat{\pi} \neq \pi'$ with $\hat{\pi}.\text{role} = \text{Init}$, $\mathcal{B}_2$ queries the decryption oracle $\mathcal{O}^{\text{Dec}}(\text{cpk}, \text{ct})$, where cpk is the composite public key embedded into session $\hat{\pi}$ via $\mathcal{O}^{\text{EKeyGen}}$, to obtain the KEM key K_A . It then continues the simulation of session $\hat{\pi}$ honestly.
- When π receives $(\text{id}_{P^*}, \text{c}_{\text{seed}}^{P^*})$, $\mathcal{B}_2$ decrypts $\text{c}_{\text{seed}}^{P^*}$ using K_b^* to obtain seed_{P^*} and outputs seed_{P^*} to its challenger. Since π is an accepted session, we have $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}_{P^*}') = 1$, where $\text{pk}_{P^*}' \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{P^*})$ and $(\text{pk}_e^{P^*}, \cdot) = \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_{P^*})$. Therefore, we have $\text{pk}_{P^*}' = \text{pk}_s^*$ and seed_{P^*} is a valid output for $\mathcal{B}_2$.

When $b = 0$, adversary $\mathcal{B}_2$ perfectly simulates Game G_1 , except for the case where the ciphertext received by π' (if it exists) is not correctly decapsulated. In G_1 , and if π' exists, π' performs the decapsulation of the ciphertext, which could potentially differ from the encapsulated key. In the simulation, $\mathcal{B}_2$ always uses the real corresponding encapsulated key in π' . This difference can be bounded by the δ -correctness of the underlying AFS-KEM. Specifically, we embed the correctness-failure experiment of AFS-KEM into the guessed session π' and the corresponding session π , where π' is guessed uniformly from among the S sessions with success probability $1/S$. Conditioned on the event that the guess is correct, the simulation is perfect, and whenever the above discrepancy occurs, the embedded correctness-failure experiment outputs 1. That is, $|\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 0] - \Pr[G_1^A \Rightarrow 1]| \leq S\delta$. When $b = 1$, $\mathcal{B}_2$ perfectly simulates G_2 , that is, $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 1] = \Pr[G_2^A \Rightarrow 1]$. Therefore, we have

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + S\delta.$$

Game G_3 (Random K_{SESSION} , K_{ENC}^A). In this game, we replace the keys K_{SESSION} and K_{ENC}^A in π with independently sampled uniform keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. If the unique FirstMatch session π' of π exists, we also replace the K_{SESSION} and K_{ENC}^A in π' with the same keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. For any PPT adversary that can distinguish G_2 from G_3 , we construct an adversary $\mathcal{Z}$ against the IND-PRG security of the PRG as follows. $\mathcal{Z}$ receives a value y_b from its challenger. If $b = 0$, then $y_b = \text{PRG}(k)$, where k is uniformly chosen from $\mathcal{K}$. If $b = 1$, then y_b is uniformly chosen from the output space of the PRG. Adversary $\mathcal{Z}$ then perfectly simulates G_2 for $\mathcal{A}$, except that it uses y_b as the pair $(K_{\text{SESSION}}, K_{\text{ENC}}^A)$ in the sessions π and π' (if it exists). Note that when $b = 0$, the simulation is identical to G_2 , and when $b = 1$, it is identical to G_3 . Therefore, we have

$$|\text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A})| \leq \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}).$$

Game G_4 (Peer acceptance implies a matching session). In this game, we abort if the unique FirstMatch session π' of π exists and π' accepts, but π' is not the matching session of π ; we denote this event by BAD. Since games G_3 and G_4 are identical unless the BAD event occurs, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq \Pr[\text{BAD} : G_3].$$

Next, we analyze $\Pr[\text{BAD} : G_3]$. Note that if the BAD event occurs, then the adversary has tampered with the third message sent by π' . Specifically, the pair $(\text{id}', c'_{\text{seed}})$ sent by π' differs from $(\text{id}_{P^*}, c_{\text{seed}})$ received by π . We consider the following two cases:

- **Case 1:** If $\text{id}_{P^*} \neq \text{id}'$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that it first samples B^* uniformly from among the N parties and π' uniformly from among the S sessions, aborting if $B^* \neq \text{id}'$ or $\pi' \neq \pi^{\text{FirstMatch}}$. Thus, $\mathcal{C}$ guesses correctly with probability $1/(NS)$. Then $\mathcal{C}$ sets $\text{pk}_{B^*} = \text{pk}_s^*$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π' . When π receives a valid pair $(\text{id}_{P^*}, c_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c_{seed} and outputs the corresponding seed_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$, we have $\text{pk}'_s = \overline{\text{pk}}_s^*$ by the security of CA. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1}] \leq NS \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2:** If $\text{id}_{P^*} = \text{id}'$ and $c_{\text{seed}} \neq c'_{\text{seed}}$, we also construct an adversary $\mathcal{C}^{17}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that it first samples π' uniformly from among the S sessions, aborting if $\pi' \neq \pi^{\text{FirstMatch}}$. Thus, $\mathcal{C}$ guesses correctly with probability $1/S$. It then sets $\text{pk}_{P^*} = \text{pk}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π' . When π receives a valid pair $(\text{id}_{P^*}, c_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c_{seed} and outputs the corresponding seed_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $c_{\text{seed}} \neq c'_{\text{seed}}$, we have $\text{pk}'_s = \text{pk}_s^*$ and $\text{seed}_e \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ also wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2}] \leq S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

¹⁷ We reuse the notation $\mathcal{C}$ for simplicity.

Combining the two cases, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq (N+1)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Analysis of G_4 . In G_4 , the session key K_{SESSION} of the tested session π is uniformly random for $\mathcal{A}$, and there may be one accepted session (which must be the matching session) that uses the same session key K_{SESSION} and we do not allow the adversary to reveal this session key for the matching session of π . Note that all session keys in other accepted sessions are independent of K_{SESSION} . Therefore, we have

$$\text{Adv}^{G_4}(\mathcal{A}) = 0.$$

Combining all the bounds derived above completes the proof for case FU_2 .

Proof for Case FU_5 . In this case, we show that the event FU_5 occurs only with negligible probability. Notably, the Init-Explicit Authentication analysis for pUAKE is essentially identical to the analysis of FU_5 , except for Game G'_5 . Here, in G'_5 we abort if the FirstMatch session of the tested session does not exist. In contrast, in the Init-Explicit Authentication analysis (Theorem 10.8), we abort if the corresponding Match session does not exist.

Game G'_0 . This game corresponds to the original game for $\text{Adv}_{\text{pUAKE}}^{\text{FU}_5\text{-KS}}(\mathcal{A})$, that is,

$$\text{Adv}_{\text{pUAKE}}^{\text{FU}_5\text{-KS}}(\mathcal{A}) = \text{Adv}^{G'_0}(\mathcal{A}).$$

Game G'_1 (Guessing $\pi^{\text{Test}}.\text{peer}$, π^{Test}). This is the same game as G_1 for FU_2 . That is, we sample P^* uniformly from the N parties, and sample one session instance π uniformly from the at most S sessions. If the guess is incorrect (i.e., $P^* \neq \pi^{\text{Test}}.\text{peer}$ or $\pi \neq \pi^{\text{Test}}$), the game is aborted. The probability of guessing correctly is $1/(NS)$; therefore,

$$\text{Adv}^{G'_0}(\mathcal{A}) \leq NS \cdot \text{Adv}^{G'_1}(\mathcal{A}).$$

Game G'_2 (New win condition). In this game, we change the condition for the adversary to win. An instance of G'_2 is won by the adversary $\mathcal{A}$ if and only if π^{Test} accepts and nothing makes the instance of the game invalid (as per the conditions of G'_1). We stop the game at the moment when π^{Test} accepts. Note that in G'_2 , the Test query is not requested for π^{Test} , and in all valid instances, P^* remains uncorrupted. For G'_1 to be won, any instance of G'_1 needs to satisfy the win conditions of G'_2 . Thus, we have

$$\text{Adv}^{G'_1}(\mathcal{A}) \leq \text{Adv}^{G'_2}(\mathcal{A}).$$

Game G'_3 (Random K_A). In this game, we replace the KEM key K_A in the tested session π with an independently uniform key $\tilde{k} \leftarrow \$\mathcal{K}$. If the unique FirstMatch session π' of π exists, we also replace the K_A in π' with the same key $\tilde{k} \leftarrow \$\mathcal{K}$. To study the difference in advantage between G'_2 and G'_3 , we use the same argument as in Game G_2 for the case of FU_2 . This argument shows that we can construct an adversary $\mathcal{B}_2$ against the IND-LCK-SCCA security of AFS-KEM such that

$$|\text{Adv}^{G'_2}(\mathcal{A}) - \text{Adv}^{G'_3}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_2) + S\delta.$$

Game G'_4 (Random K_{SESSION} , K_{ENC}^A). In this game, we replace the keys K_{SESSION} and K_{ENC}^A in π with independently sampled uniform keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. If the unique FirstMatch session π' of π exists, we also replace the K_{SESSION} and K_{ENC}^A in π' with the same keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. To study the difference in advantage between G'_3 and G'_4 , we use the same argument as in Game G_3 for the case of FU_2 . Thus, we can construct an adversary $\mathcal{Z}$ against the IND-PRG security of the PRG such that

$$|\text{Adv}^{G'_3}(\mathcal{A}) - \text{Adv}^{G'_4}(\mathcal{A})| \leq \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}).$$

Game G'_5 (FirstMatch session). In this game, we define the event that the FirstMatch session $\pi^{\text{FirstMatch}}$ of the test session π does not exist as the BAD event, and abort if BAD occurs. Note that whenever the FU_5 event occurs in this game, the game aborts. Therefore,

$$\text{Adv}^{G'_5}(\mathcal{A}) = 0.$$

It is evident that $|\text{Adv}^{G'_4}(\mathcal{A}) - \text{Adv}^{G'_5}(\mathcal{A})| \leq \Pr[\text{BAD} : G'_4]$. Next, we analyze $\Pr[\text{BAD} : G'_4]$. In Game G'_4 , the key K_{ENC}^A of the tested session π is uniformly random from the adversary's view. Since the test session π accepts without any FirstMatch session, the key K_{ENC}^A is never used in any other session. Therefore, $\text{seed}'_A = K_{\text{ENC}}^A \oplus c_{\text{seed}}^A$ is uniformly random from the adversary's view. In this case, we can construct an adversary $\mathcal{D}$ against the RSPK-Bind security of AFS-KEM as follows. Specifically, $\mathcal{D}$ receives (pk_s^*, sk_s^*) and perfectly simulates the game for $\mathcal{A}$ as in G'_4 , except that it sets $pk_{P^*} = pk_s^*$ and outputs the composite key cpk_A in session π to its challenger. After $\mathcal{D}$ receives seed_e , it defines $K_{\text{ENC}}^A = \text{seed}_e \oplus c_{\text{seed}}^A$ in session π , where c_{seed}^A is received in π . Let $(pk'_e, sk'_e) = \text{EKeyGen}_c(cpk_A, \text{seed}_e)$ and $pk'_s = \text{ExtractPK}(cpk_A, pk'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, pk'_s) = 1$ and $pk_{P^*} = pk_s^*$, we have $pk'_s = pk_s^*$ by the security of CA. Therefore, if the BAD event occurs in this case, then $\mathcal{D}$ wins the RSPK-Bind game. Consequently, we have $\Pr[\text{BAD} : G'_4] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$ and

$$|\text{Adv}^{G'_4}(\mathcal{A}) - \text{Adv}^{G'_5}(\mathcal{A})| \leq \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}).$$

Combining all the bounds derived above completes the proof for case FU_5 . $\square$

Lemma 10.5. *With the notations and assumptions of the proof of Theorem 10.2, and with PRG modeled as a random oracle G to which the adversary makes at most q_G queries, there exist adversaries $\mathcal{B}_3$, $\mathcal{C}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{q_G}{|\mathcal{K}|} + \delta \right) \\ &\quad + NS^2 \left(2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right). \\ \text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{q_G}{|\mathcal{K}|} + \delta \right) \\ &\quad + NS^2 \left(2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \right). \end{aligned}$$

Proof. We present a unified analysis for Case FU_3 and Case FU_4 by considering $\text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$. It follows that $\text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$.

Game G_0 . This game corresponds to the original game for $\text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$. We use the π, π' to denote the tested session and its unique FirstMatch session, respectively. Moreover, we denote by π^{Init} (resp. π^{Resp}) the initiator (resp. responder) session among π and π' . Thus,

$$\text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (KEM-Correctness). In this game, we abort if the decapsulation of ct_A in π^{Init} does not equal the corresponding encapsulated key K_A in π^{Resp} . This abort event can be bounded by the δ -correctness of AFS-KEM. Specifically, the reduction first guesses π^{Init} and π^{Resp} uniformly from among the S sessions, and guesses $\pi^{\text{Init. actor}}$ uniformly from among the N parties. Notably, the guess is correct with probability $1/(NS^2)$. Conditioned on the event that the guess is correct, the reduction can perfectly embed the correctness-failure experiment of AFS-KEM into the guessed sessions π^{Init} and π^{Resp} . Whenever the abort event occurs, the embedded correctness-failure experiment outputs 1. By the δ -correctness of AFS-KEM, we have

$$|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq NS^2 \delta.$$

Game G_2 . In this game, for notational convenience, let K^{Test} denote the encapsulated key K_A used in session π . We define the COLL event as the existence of a session $\hat{\pi} \notin \{\pi, \pi'\}$ such that $\hat{\pi}.K_A = K^{\text{Test}}$. If the COLL event occurs, we abort the game. Since games G_1 and G_2 are identical unless the COLL event occurs, we have

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq \Pr[\text{COLL} : G_1].$$

Next we analysis the $\Pr[\text{COLL} : G_1]$ by considering the following two cases:

- **Case 1:** $\hat{\pi}.\text{role} = \text{Init}$. We construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}$ first chooses $g = 1$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*)$ from its challenger. Adversary $\mathcal{F}$ guesses the sessions $\hat{\pi}, \pi, \pi'$ and the party $\hat{\pi}.\text{actor}$ by sampling $\hat{\pi}, \pi, \pi'$ uniformly from the S sessions and sampling $\hat{\pi}.\text{actor}$ uniformly from the N parties. Notably, the probability that $\mathcal{F}$ guesses correctly is at least $1/(NS^3)$. Adversary $\mathcal{F}$ then perfectly simulates Game G_1 for $\mathcal{A}$, except that it sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair for $\hat{\pi}.\text{actor}$ and embeds $(\text{cpk}^*, \text{seed}_e^*)$ into the session $\hat{\pi}$. Then, $\mathcal{F}$ forwards the cpk in the session π and π' to its challenger. After receiving the corresponding (ct', K') from its challenger, $\mathcal{F}$ continues the simulation by embedding $(\text{ct}_A = \text{ct}', K_A = K')$ into π and π' . Finally, when session $\hat{\pi}$ receives its ciphertext ct'_A , $\mathcal{F}$ outputs ct'_A to its challenger. Notably, $\mathcal{F}$ perfectly simulates Game G_1 . If the event $[\text{COLL} : \text{Case 1}]$ occurs, then we have $\text{cpk} \neq \text{cpk}^*$ by the assumption that all honestly generated cpk values are distinct (as stated at the beginning of Theorem 10.2) and $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}'_A) = K'$. Therefore, adversary $\mathcal{F}$ wins the Key-Coll game, and hence $\Pr[\text{COLL} : \text{Case 1}] \leq NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.
- **Case 2:** $\hat{\pi}.\text{role} = \text{Resp}$. We also construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}$ first chooses $g = 2$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*, K^*, \text{ct}^*)$ from its challenger. Adversary $\mathcal{F}$ first guesses the sessions $\hat{\pi}, \pi^{\text{Init}}, \pi^{\text{Resp}}$, and the party $\pi^{\text{Init}}.\text{actor}$ by sampling $\hat{\pi}, \pi^{\text{Init}}$, and π^{Resp} uniformly from the S sessions and sampling $\pi^{\text{Init}}.\text{actor}$ uniformly from the N parties. Notably, the probability that $\mathcal{F}$ guesses correctly is at least $1/(NS^3)$. Adversary $\mathcal{F}$ then perfectly simulates Game G_1 for $\mathcal{A}$, except that it sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair for $\pi^{\text{Init}}.\text{actor}$ and sets $(\text{cpk}_A, \text{seed}_A) = (\text{cpk}^*, \text{seed}_e^*)$ in the session π^{Init} , and embeds $(\text{ct}_A, K_A) = (\text{ct}^*, K^*)$ into the session π^{Init} and π^{Resp} . When session $\hat{\pi}$ receives its composite public key cpk' , $\mathcal{F}$ forwards cpk' to its challenger and receives the corresponding (ct', K') . It then embeds (ct', K') into the session $\hat{\pi}$. Notably, $\mathcal{F}$ perfectly simulates Game G_1 for this case. If the event $[\text{COLL} : \text{Case 2}]$ occurs, then we have $K' = K^*$. Thus, adversary $\mathcal{F}$ wins the Key-Coll game, and therefore $\Pr[\text{COLL} : \text{Case 2}] \leq NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.

Combining the two cases, we obtain

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq 2NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}^{18}).$$

Game G_3 (Guessing Actor, π^{Test} , and $\pi^{\text{FirstMatch}}$). In Game G_3 , we guess the test session π^{Test} and its corresponding FirstMatch session $\pi^{\text{FirstMatch}}$, and further guess the identity of $\pi^{\text{Test}}.\text{actor}$ (resp. $\pi^{\text{FirstMatch}}.\text{actor}$) if $\pi^{\text{Test}}.\text{role} = \text{Init}$ (resp. $\pi^{\text{Test}}.\text{role} = \text{Resp}$). If any of these guesses is incorrect, the game is aborted. Concretely, we sample a party P^* uniformly from the N parties and sample two session instances π and π' uniformly from the S sessions. The game aborts if any of the following conditions holds: (i) $\pi \neq \pi^{\text{Test}}$, (ii) $\pi' \neq \pi^{\text{FirstMatch}}$, (iii) $\pi^{\text{Test}}.\text{role} = \text{Init}$ and $P^* \neq \pi^{\text{Test}}.\text{actor}$, or (iv) $\pi^{\text{Test}}.\text{role} = \text{Resp}$ and $P^* \neq \pi^{\text{FirstMatch}}.\text{actor}$. The probability of guessing correctly is $1/(NS^2)$, and therefore

$$\text{Adv}^{G_2}(\mathcal{A}) \leq NS^2 \cdot \text{Adv}^{G_3}(\mathcal{A}).$$

Game G_4 (Peer acceptance implies a matching session). In this game, we define the BAD event as the event that the FirstMatch session π' of π accepts but is not the matching session of π . If the BAD event occurs, we abort the game. If the BAD event does not occur, games G_3 and G_4 are identical. Therefore, we have $|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq \Pr[\text{BAD} : G_3]$. Importantly, after this game hop, the session π^{Resp} accepts the session key if and only if it is the matching session of π^{Init} , otherwise π^{Resp} aborts.

Next, we analyze $\Pr[\text{BAD} : G_3]$. Note that if the BAD event occurs, then the pair $(\text{id}, \text{c}_{\text{seed}})$ sent by π^{Init} differs from $(\text{id}', \text{c}'_{\text{seed}})$ received by π^{Resp} , and both π^{Init} and π^{Resp} accept the session key. We consider the following two cases:

- **Case 1:** If $\text{id} \neq \text{id}'$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives a challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that it first samples B^* uniformly from among the N parties and aborts if $B^* \neq \pi^{\text{Resp}}.\text{peer}$. Notably, $\mathcal{C}$ guesses correctly with probability $1/N$. Then $\mathcal{C}$ sets

¹⁸ We reuse the notation $\mathcal{F}$ for simplicity.

$\text{pk}_{P^*} = \text{pk}_s^*$ and $\text{pk}_{B^*} = \overline{\text{pk}_s^*}$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π^{Init} . When π^{Resp} receives a valid pair $(\text{id}' = B^*, \text{c}'_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c'_{seed} and outputs the corresponding seed'_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(B^*, \text{pk}'_s) = 1$ and $\text{pk}_{B^*} = \overline{\text{pk}_s^*}$, we conclude that $\text{pk}'_s = \overline{\text{pk}_s^*}$. Therefore, if the BAD event occurs and $\mathcal{C}$ guesses correctly, then adversary $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

- **Case 2:** If $\text{id} = \text{id}'$ and $\text{c}_{\text{seed}} \neq \text{c}'_{\text{seed}}$, we also construct an adversary $\mathcal{C}^{19}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}_s^*}, \overline{\text{sk}_s^*})$ and perfectly simulates Game G_3 for $\mathcal{A}$, except that it sets $\text{pk}_{P^*} = \text{pk}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π^{Init} . When π^{Resp} receives a valid pair $(\text{id}_{P^*}, \text{c}'_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c'_{seed} and outputs the corresponding seed'_e to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{c}'_{\text{seed}} \neq \text{c}_{\text{seed}}$, we have $\text{pk}'_s = \text{pk}_s^*$ and $\text{seed}'_e \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case, then $\mathcal{C}$ also wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2}] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Combining the two cases, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq (N + 1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G_5 (random $\text{K}_{\text{SESSION}}, \text{K}_{\text{ENC}}^A$). In this game, we replace the $\text{K}_{\text{SESSION}}, \text{K}_{\text{ENC}}^A$ in π, π' with independent uniformly random values. Define QUERY as the event that $\mathcal{A}$ queries $\mathcal{G}$ on input K^{Test} , where K^{Test} denotes the encapsulated key K_A in sessions π and π' . Then, G_4 and G_5 are identical unless the event QUERY happens, and we have

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq \Pr[\text{QUERY} : G_5].$$

At this point, the session key $\text{K}_{\text{SESSION}}$ of the tested session π is uniformly random from the view of $\mathcal{A}$, and there may be one accepted session (which must be the matching session) that uses the same session key $\text{K}_{\text{SESSION}}$. Note that all session keys in other accepted sessions are independent of $\text{K}_{\text{SESSION}}$. Since we do not allow the adversary to reveal the session key of the matching session of π , we have

$$\text{Adv}^{G_5}(\mathcal{A}) = 0.$$

Game G_6 (Randomized $\text{c}_{\text{seed}}^{P^*}$). In this game, we modify the computation of $\text{c}_{\text{seed}}^{P^*} = \text{K}_{\text{ENC}}^{P^*} \oplus \text{seed}_{P^*}$ in π^{Init} by replacing it with $\hat{\text{c}}_{\text{seed}}^{P^*} = \text{K}_{\text{ENC}}^{P^*} \oplus \hat{\text{seed}}_{P^*}$, where $\hat{\text{seed}}_{P^*} \leftarrow \{0, 1\}^\lambda$ is sampled uniformly and independently of seed_{P^*} . Importantly, after Game G_4 , the session π^{Resp} accepts the session key if and only if it is the matching session of π^{Init} , otherwise π^{Resp} aborts. Since $\text{K}_{\text{ENC}}^{P^*}$ is uniformly random and independent of the adversary's view in Game G_5 , the values $\text{K}_{\text{ENC}}^{P^*} \oplus \text{seed}_{P^*}$ and $\text{K}_{\text{ENC}}^{P^*} \oplus \hat{\text{seed}}_{P^*}$ are identically distributed from the adversary's view. Hence, games G_5 and G_6 are identical from the adversary's view, and therefore

$$\Pr[\text{QUERY} : G_5] = \Pr[\text{QUERY} : G_6].$$

Game G_7 (Random K^{Test}). In this game, we replace the KEM key K^{Test} in π and π' with a uniformly random key. To bound $|\Pr[\text{QUERY} : G_6] - \Pr[\text{QUERY} : G_7]|$, we can construct an IND-ECPA adversary $\mathcal{B}$ against AFS-KEM. Specifically, $\mathcal{B}$ receives $(\text{pk}_s^*, \text{pk}_e^*, \text{cpk}^*, \text{sk}_s^*, \text{ct}^*, \text{K}_b^*)$ from its challenger. Adversary $\mathcal{B}$ honestly simulates the view of $\mathcal{A}$ as in G_6 , except that it sets pk_s^* as the public key of P^* and embeds cpk^* and ct^* into the sessions π and π' , respectively. Moreover, $\mathcal{B}$ uses K_b^* as the KEM key K^{Test} in both π and π' . When $b = 0$, $\mathcal{B}$ perfectly simulates game G_6 , and when $b = 1$, $\mathcal{B}$ perfectly simulates game G_7 . That is, $\Pr[\mathcal{B} \Rightarrow 1 : b = 0] = \Pr[\text{QUERY} : G_6]$ and $\Pr[\mathcal{B} \Rightarrow 1 : b = 1] = \Pr[\text{QUERY} : G_7]$. Therefore, we have

$$|\Pr[\text{QUERY} : G_6] - \Pr[\text{QUERY} : G_7]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3).$$

¹⁹ We reuse the notation $\mathcal{C}$ for simplicity.

Analysis of $\Pr[\text{QUERY} : G_7]$. Note that in this game the key K^{Test} is independent of ct and cpk in session π and π' , and is uniformly random from the adversary's view. It follows that

$$\Pr[\text{QUERY} : G_7] \leq \frac{q_G}{|\mathcal{K}|}.$$

Combining all the bounds derived above completes the proof. $\square$

Lemma 10.7 (QROM counterpart of Lemma 10.5). *With the notations and assumptions of the proof of Theorem 10.6, and with PRG modeled as a quantum-accessible random oracle $\mathcal{G}$ to which the adversary makes at most q_G (quantum) queries, there exist QPT adversaries $\mathcal{B}_3$, $\mathcal{C}$, and $\mathcal{D}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \sqrt{(q_G + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{4q_G}{|\mathcal{K}|} \right)} + \beta \right), \\ \text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) &\leq NS^2 \left(2 \sqrt{(q_G + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_3) + \frac{4q_G}{|\mathcal{K}|} \right)} + \beta \right), \end{aligned}$$

where $\beta = 2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + (N + 1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \delta$.

Proof. We also present a unified analysis for Case FU_3 and Case FU_4 by considering $\text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$. It follows that $\text{Adv}_{\text{pUAKE}}^{\text{FU}_3\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pUAKE}}^{\text{FU}_4\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pUAKE}}^{(\text{FU}_3 \vee \text{FU}_4)\text{-KS}}(\mathcal{A})$.

Game $G_0 - G_4$. The games $G_0 - G_4$ are the same as the $G_0 - G_4$ in proof of Lemma 10.5. Notably, we use π and π' to denote the tested session and its (unique) **FirstMatch** session, respectively. We further denote by π^{Init} (resp. π^{Resp}) the initiator (resp. responder) session among π and π' . K^* denotes the encapsulated key K_A shared by π and π' .

Game G_5 (Random $K_{\text{SESSION}}, K_{\text{ENC}}^A$). In this game, the random oracle $\mathcal{G}$ accessed by $\mathcal{A}$ is replaced by an oracle $\mathcal{G}'$ where $\mathcal{G}'(x) = \mathcal{G}(x)$ if $x \neq K^*$, and $\mathcal{G}'(K^*) = y^*$, where y^* is randomly sampled from the output space of the random oracle $\mathcal{G}$. Notably, the challenger still uses $\mathcal{G}(K^*)$ in π and π' to compute $(K_{\text{SESSION}}, K_{\text{ENC}}^A)$.

Let $z = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, K^*)$ where $(\text{pk}^*, \text{sk}^*) \leftarrow \text{SKeyGen}(1^\lambda)$, $\text{seed}^* \leftarrow \{0, 1\}^\lambda$, $(\text{pk}_e^*, \text{sk}_e^*) \leftarrow \text{EKeyGen}(\text{pk}^*, \text{seed}^*)$, $\text{cpk}^* \leftarrow \text{CombPK}(\text{pk}^*, \text{pk}_e^*)$, and $(\text{ct}^*, K^*) \leftarrow \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. Let $A_{G_i}^{(O)}(z)$ be an oracle algorithm that perfectly simulates the game G_i for $\mathcal{A}$, except that it first uses the random oracle O to simulate the random oracle accessed by $\mathcal{A}$. Moreover, $\mathcal{A}$ uses pk^* as the long-term public key of π^{Init} .actor, and embeds $\text{seed}^*, \text{ct}^*, K^*$ into the sessions π^{Init} and π^{Resp} , and finally returns 1 if $\mathcal{A}$ wins the game. Thus, we have $\text{Adv}^{G_4}(\mathcal{A}) = \Pr[A_{G_4}^{\mathcal{G}}(z) \Rightarrow 1]$ and $\text{Adv}^{G_5}(\mathcal{A}) = \Pr[A_{G_4}^{\mathcal{G}'}(z) \Rightarrow 1]$.

Lemma 5.9 states that $|\Pr[A_{G_4}^{\mathcal{G}}(z) \Rightarrow 1] - \Pr[A_{G_4}^{\mathcal{G}'}(z) \Rightarrow 1]| \leq 2\sqrt{(q_G + 1) \Pr[\text{FIND} : A_{G_4}^{\mathcal{G}' \setminus K^*}(z)]}$. Here $\Pr[\text{FIND} : A_{G_4}^{\mathcal{G}' \setminus K^*}(z)]$ denotes the probability that the oracle $\mathcal{O}_{K^*}^{\text{SC}}$ ever outputs 1 during the execution of $A_{G_4}^{\mathcal{G}' \setminus K^*}(z)$. Note that by definition we have $\Pr[\text{FIND} : A_{G_4}^{\mathcal{G}' \setminus K^*}(z)] = \Pr[\text{FIND} : A_{G_5}^{\mathcal{G}' \setminus K^*}(z)]$, therefore

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq 2\sqrt{(q_G + 1) \Pr[\text{FIND} : A_{G_5}^{\mathcal{G}' \setminus K^*}(z)]}.$$

At this point, the session key K_{SESSION} of the tested session π is uniformly random from the view of $\mathcal{A}$, and there may be one accepted session (which must be the matching session) that uses the same session key K_{SESSION} . Note that all session keys in other accepted sessions are independent of K_{SESSION} . Since we do not allow the adversary to reveal the session key of the matching session of π , we have

$$\text{Adv}^{G_5}(\mathcal{A}) = 0.$$

It remains to analyze $\Pr[\text{FIND} : A_{G_5}^{\mathcal{G}' \setminus K^*}(z)]$.

Game G_6 . This game is identical to G_5 , except that the random oracle accessed by $\mathcal{A}$ is replaced with $\mathcal{G}' \setminus K^*$. By the definition of $\mathcal{A}$, we have

$$\Pr[\text{FIND} : A_{G_5}^{\mathcal{G}' \setminus K^*}(z)] = \Pr[\text{FIND} : A_{G_6}^{\mathcal{G}' \setminus K^*}(z)].$$

Game G_7 (Randomized $c_{\text{seed}}^{P^*}$). In this game, we modify the computation of $c_{\text{seed}}^{P^*} = K_{\text{ENC}}^{P^*} \oplus \text{seed}_{P^*}$ in π^{Init} by replacing it with $\hat{c}_{\text{seed}}^{P^*} = K_{\text{ENC}}^{P^*} \oplus \hat{\text{seed}}_{P^*}$, where $\hat{\text{seed}}_{P^*} \leftarrow \$ \{0, 1\}^\lambda$ is sampled uniformly and independently of seed_{P^*} . Importantly, after Game G_4 , the session π^{Resp} accepts the session key if and only if it is the matching session of π^{Init} , otherwise π^{Resp} aborts. In particular, π^{Resp} need not recover the seed from the received value $c_{\text{seed}}^{P^*}$ in order to decide whether to accept or abort.

Since $K_{\text{ENC}}^{P^*}$ is derived from the hidden value $G(K^*)$, while $\mathcal{A}$ only has access to $G' \setminus K^*$, the key $K_{\text{ENC}}^{P^*}$ is uniformly random and independent of the adversary's view in the execution of $A_{G_6}^{G' \setminus K^*}(z)$. Therefore, the values $K_{\text{ENC}}^{P^*} \oplus \text{seed}_{P^*}$ and $K_{\text{ENC}}^{P^*} \oplus \hat{\text{seed}}_{P^*}$ are identically distributed from the adversary's view. Hence, the executions $A_{G_6}^{G' \setminus K^*}(z)$ and $A_{G_7}^{G' \setminus K^*}(z)$ induce identical distributions on all variables relevant to the event **FIND**. Therefore,

$$\Pr[\text{FIND} : A_{G_6}^{G' \setminus K^*}(z)] = \Pr[\text{FIND} : A_{G_7}^{G' \setminus K^*}(z)].$$

Game G_8 (Random K^*). Game G_8 is identical to Game G_7 , except that the KEM key K^* used in sessions π and π' is replaced by a uniformly random key $\hat{K}^*$, and the random oracle $G' \setminus K^*$ that the adversary accessed is replaced with $G' \setminus \hat{K}^*$.

Let $\hat{z} = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, \hat{K}^*)$. To bound $|\Pr[\text{FIND} : A_{G_7}^{G' \setminus K^*}(z)] - \Pr[\text{FIND} : A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})]|$, we construct an IND-ECPA adversary $\mathcal{B}$ against AFS-KEM as follows. Specifically, $\mathcal{B}$ receives $(\text{pk}_s^*, \text{pk}_e^*, \text{cpk}^*, \text{sk}_s^*, \text{ct}^*, K_b^*)$ from its challenger. Adversary $\mathcal{B}$ honestly simulates the view of $\mathcal{A}$ as $A_{G_7}^{G' \setminus K^*}(z)$, except that it sets pk_s^* as the public key of P^* and embeds cpk^* and ct^* into sessions π and π' , respectively. Moreover, $\mathcal{B}$ uses K_b^* as the KEM key K in both π and π' , it samples a quantum random oracle G' and uses the oracle $G' \setminus K_b^*$ to answer adversary (quantum) RO queries. If the **FIND** event occurs, $\mathcal{B}$ returns 1. When $b = 0$, adversary $\mathcal{D}$ perfectly simulates Game for $\mathcal{A}$ as $A_{G_7}^{G' \setminus K^*}(z)$, whereas when $b = 1$, it perfectly simulates Game as $A_{G_7}^{G' \setminus \hat{K}^*}(z)$. That is $\Pr[\mathcal{D} \Rightarrow 1 : b = 0] = \Pr[\text{FIND} : A_{G_7}^{G' \setminus K^*}(z)]$ and $\Pr[\mathcal{D} \Rightarrow 1 : b = 1] = \Pr[\text{FIND} : A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})]$. Therefore,

$$|\Pr[\text{FIND} : A_{G_7}^{G' \setminus K^*}(z)] - \Pr[\text{FIND} : A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}).$$

Analysis of $\Pr[\text{FIND} : A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})]$. Recall that $\hat{z} = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, \hat{K}^*)$, and that $A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})$ still uses the value $G(\hat{K}^*)$ to derive K_{SESSION} and K_{ENC}^A for the sessions π and π' , while the adversary $\mathcal{A}$ does not access G . Therefore, we define an algorithm $\hat{A}_{G_8}^{G' \setminus \hat{K}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)$ that is identical to $A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})$, except that it samples K_{SESSION} and K_{ENC}^A uniformly at random for π and π' (instead of computing them via $G(\hat{K}^*)$). It is immediate that $\hat{A}_{G_8}^{G' \setminus \hat{K}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*)$ and $A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})$ induce identical views for $\mathcal{A}$. Hence, $\Pr[\text{FIND} : A_{G_8}^{G' \setminus \hat{K}^*}(\hat{z})] = \Pr[\text{FIND} : \hat{A}_{G_8}^{G' \setminus \hat{K}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)]$.

Note that the test key $\hat{K}^*$ is independent of $\text{pk}^*, \text{sk}^*, \text{seed}^*$, and is uniformly random from the adversary's view. By Lemma 5.10, it follows that

$$\Pr[\text{FIND} : \hat{A}_{G_8}^{G' \setminus \hat{K}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)] \leq \frac{4q_G}{|\mathcal{K}|}.$$

Combining all the bounds derived above completes the proof. $\square$

10.4 Init-Explicit Authentication Analysis for pUAKE

Theorem 10.8. *Let there be at most N parties and at most S sessions activated in the Init-EA game. Suppose that the underlying AFS-KEM is δ -correct, γ -spreadness, and satisfies perfect ExtractPK-correctness and ephemeral-key-generation equivalence. Then, for any polynomial-time adversary $\mathcal{A}$ attacking the Init-EA security of pUAKE, there exist polynomial-time adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$, and $\mathcal{Z}$, constructed below and running times are about that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pUAKE}}^{\text{Init-EA}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &\quad + NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}) + \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}) + S\delta \right. \\ &\quad \left. + (N+1)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right), \end{aligned}$$

where k_1 and k_2 are the min-entropy bounds on the honest static and combined public keys of AFS-KEM, respectively.

Proof. First, we assume that all honest initiators have distinct long-term public keys, and that all honest initiator sessions compute distinct cpk_A . Moreover, we assume that every responder session that receives an honest cpk_A from some honest initiator session computes a distinct ct_A . Since the min-entropy of the public key pk_A is at least k_1 , the honestly combined public key cpk_A has min-entropy at least k_2 , and the ciphertexts of AFS-KEM satisfy γ -spreadness, this will add $\frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma}$ to the final bound. Under these assumptions, it is impossible for a session to have more than one **FirstMatch** or **Match** session. We assume the adversary outputs a π that satisfies $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$. otherwise, the adversary trivially loses the game.

Game G_0 . This game corresponds to the original game with the above assumption for $\text{Adv}_{\text{pUAKE}}^{\text{Init-EA}}(\mathcal{A})$, that is,

$$\text{Adv}_{\text{pUAKE}}^{\text{Init-EA}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (Guessing $\pi.\text{peer}$, π). In this game, we guess the session π output by the adversary and its $\pi.\text{peer}$, and abort if the guess is incorrect. Formally, we sample P^* uniformly from the N parties, and sample one session instance π uniformly from at most S sessions. If the guess is incorrect (i.e., $P^* \neq \pi.\text{peer}$ or $\pi \neq \pi$), where π is the session output by the adversary, the game is aborted. The probability of guessing correctly is $1/(NS)$; therefore,

$$\text{Adv}^{G_0}(\mathcal{A}) \leq NS \cdot \text{Adv}^{G_1}(\mathcal{A}).$$

Game G_2 (New win condition). In this game, we change the condition for the adversary to win. An instance of G_2 is won by the adversary $\mathcal{A}$ if and only if π accepts where $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$. We stop the game at the moment when π accepts. Note that in G_2 , P^* remains uncorrupted. For G_1 to be won, any instance of G_1 needs to satisfy the win conditions of G_2 . Thus, we have

$$\text{Adv}^{G_1}(\mathcal{A}) \leq \text{Adv}^{G_2}(\mathcal{A}).$$

Game G_3 (Random K_A). In this game, we replace the KEM key K_A in the session π with an independently uniform key $\tilde{k} \leftarrow \$\mathcal{K}$. If the unique **FirstMatch** session π' of π exists, we also replace the K_A in π' with the same key $\tilde{k} \leftarrow \$\mathcal{K}$. To study the difference in advantage between G_2 and G_3 , we use the same argument as in Game G_2 in Lemma 10.4 for the case of FU_2 . This argument shows that we can construct an adversary $\mathcal{B}$ against the IND-LCK-SCCA security of AFS-KEM such that

$$|\text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}) + S\delta.$$

Game G_4 (Random K_{SESSION} , K_{ENC}^A). In this game, we replace the keys K_{SESSION} and K_{ENC}^A in π with independently sampled uniform keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. If the unique **FirstMatch** session π' of π exists, we also replace the K_{SESSION} and K_{ENC}^A in π' with the same keys $\hat{K}_{\text{SESSION}}$ and $\hat{K}_{\text{ENC}}^A$. To study the difference in advantage between G_3 and G_4 , we use the same argument as in Game G_3 in Lemma 10.4 for the case of FU_2 . Thus, we can construct an adversary $\mathcal{Z}$ against the IND-PRG security of the PRG such that

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq \text{Adv}_{\text{PRG}}^{\text{IND-PRG}}(\mathcal{Z}).$$

Game G_5 (Matching session). In this game, we define the event that the matching session π^{Match} of the test session π does not exist as the **BAD** event, and abort if **BAD** occurs. Note that the adversary can win only if **BAD** occurs and Game G_5 aborts whenever **BAD** occurs, we have

$$\text{Adv}^{G_5}(\mathcal{A}) = 0.$$

It is evident that $|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq \Pr[\text{BAD} : G_4]$. Next, we analyze $\Pr[\text{BAD} : G_4]$. In Game G_4 , the key K_{ENC}^A of the test session π is uniformly random from the adversary's view. Moreover,

if there exists a **FirstMatch** session of π , denoted by π' , then π' uses the same encryption key K_{ENC}^A . We analyze the event **BAD** by considering the following cases.

Case 1: π' does not exist. Since the session π accepts without any **FirstMatch** session, the key K_{ENC}^A is never used in any other session. Therefore, $\text{seed}'_A = K_{\text{ENC}}^A \oplus c_{\text{seed}}^A$ in π is uniformly random from the adversary's view. In this case, we can construct an adversary $\mathcal{D}$ against the RSPK-Bind security of AFS-KEM as follows. Specifically, $\mathcal{D}$ receives (pk_s^*, sk_s^*) and perfectly simulates the game for $\mathcal{A}$ as in G_4 , except that it sets $pk_{P^*} = pk_s^*$ and outputs the composite key cpk_A in session π to its challenger. After $\mathcal{D}$ receives seed_e from its challenger, it defines $K_{\text{ENC}}^A = \text{seed}_e \oplus c_{\text{seed}}^A$ in session π , where c_{seed}^A is received in π . Let $(pk'_e, sk'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}_A, \text{seed}_e)$ and $pk'_s = \text{ExtractPK}(\text{cpk}_A, pk'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, pk'_s) = 1$ and $pk_{P^*} = pk_s^*$, we have $pk'_s = pk_s^*$ by the security of CA. Therefore, if the **BAD** event occurs in this case, then $\mathcal{D}$ wins the RSPK-Bind game. Hence, $\Pr[\text{BAD} : \text{Case 1}] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$.

Case 2: π' exists. In this case, the session π' must accept. The **BAD** event implies that the adversary has tampered with the third message sent by π' . Let $(\text{id}', c'_{\text{seed}})$ be the message sent by π' , and let $(\text{id}_{P^*}, c_{\text{seed}})$ be the message received by π , where $(\text{id}', c'_{\text{seed}}) \neq (\text{id}_{P^*}, c_{\text{seed}})$. We further distinguish the following subcases.

- **Case 2.a:** If $\text{id}_{P^*} \neq \text{id}'$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, pk_s^*, sk_s^*, \overline{pk}_s^*, \overline{sk}_s^*)$ and perfectly simulates Game G_4 for $\mathcal{A}$, except that it first samples B^* uniformly from among the N parties and π' uniformly from among the S sessions, aborting if $B^* \neq \text{id}'$ or $\pi' \neq \pi^{\text{FirstMatch}}$. Thus, $\mathcal{C}$ guesses correctly with probability $1/(NS)$. Then $\mathcal{C}$ sets $pk_{B^*} = pk_s^*$ and $pk_{P^*} = \overline{pk}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π' . When π receives a valid pair $(\text{id}_{P^*}, c_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c_{seed} and outputs the corresponding seed_e to its challenger. Let $(pk'_e, sk'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_e)$ and $pk'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, pk'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, pk'_s) = 1$ and $pk_{P^*} = \overline{pk}_s^*$, we have $pk'_s = \overline{pk}_s^*$ by the security of CA. Therefore, if the **BAD** event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2.a}] \leq NS \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2.b:** If $\text{id}_{P^*} = \text{id}'$ and $c_{\text{seed}} \neq c'_{\text{seed}}$, we also construct an adversary $\mathcal{C}^{20}$ against the SPK-Bind security of AFS-KEM as follows. $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, pk_s^*, sk_s^*, \overline{pk}_s^*, \overline{sk}_s^*)$ and perfectly simulates Game G_4 for $\mathcal{A}$, except that it first samples π' uniformly from among the S sessions, aborting if $\pi' \neq \pi^{\text{FirstMatch}}$. Thus, $\mathcal{C}$ guesses correctly with probability $1/S$. It then sets $pk_{P^*} = pk_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π' . When π receives a valid pair $(\text{id}_{P^*}, c_{\text{seed}})$, $\mathcal{C}$ honestly decrypts c_{seed} and outputs the corresponding seed_e to its challenger. Let $(pk'_e, sk'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_e)$ and $pk'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, pk'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, pk'_s) = 1$ and $c_{\text{seed}} \neq c'_{\text{seed}}$, we have $pk'_s = pk_s^*$ and $\text{seed}_e \neq \text{seed}_e^*$. Therefore, if the **BAD** event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ also wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2.b}] \leq S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Combining all the cases above, we conclude that

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq (N+1)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}).$$

Combining all the bounds derived above completes the proof. $\square$

11 Full Security Analysis for pMAKE

11.1 Key Secrecy of the pMAKE protocols.

Theorem 9.3 (F_{pMAKE} -Key Secrecy of pMAKE in the ROM and QROM). *Let there be at most N parties and at most S sessions activated in the F_{pMAKE} -KS game. Suppose that the underlying AFS-KEM is δ -correct, satisfies γ -spreadness, and satisfies perfect ExtractPK-correctness and ephemeral-key-generation equivalence. We model the PRF as a random oracle G , to which the adversary makes at most q_G queries. Then, for any PPT adversary $\mathcal{A}$ attacking the F_{pMAKE} -KS security of pMAKE, there*

²⁰ We reuse the notation $\mathcal{C}$ for simplicity.

exist PPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, constructed below and running in time comparable to that of $\mathcal{A}$, such that

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{F}_{\text{pMAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &+ NS \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + 2(S+1) \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) \\ &+ 4(2NS+1) \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \\ &+ S \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + 8 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) \\ &+ 8(S+1) \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \frac{4q_G}{|\mathcal{K}|} + 16\delta \\ &+ (8N^2 + 29N + 20) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) \end{aligned} \right) \end{aligned} \right). \end{aligned}$$

If we model PRF as a quantum-accessible random oracle G , to which the adversary makes at most q_G quantum queries, then, for any QPT adversary $\mathcal{A}$ attacking the $\text{F}_{\text{pMAKE}}\text{-KS}$ security of pMAKE, there exist QPT adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}, \mathcal{D}, \mathcal{F}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{F}_{\text{pMAKE}}\text{-KS}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} \\ &+ NS \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + 2(S+1) \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 4 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \\ &+ S \left(\begin{aligned} &4 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + 8(S+1) \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &+ (5N+4) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 16\delta \end{aligned} \right) \end{aligned} \right) \\ &+ 4NS^2 \left(2\sqrt{(q_G+1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_G}{|\mathcal{K}|} + \beta \right)} + \beta \right), \end{aligned}$$

where $\beta = N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$. Here k_1 and k_2 are the min-entropy bounds on the honest static and combined public keys of AFS-KEM, respectively. Furthermore, $|\mathcal{K}|$ denotes the cardinality of the AFS-KEM encapsulated key space.

Proof. First, we assume that all honest users have distinct long-term public keys, and that all honest sessions compute distinct combined public keys cpk_A for initiator sessions and cpk_B for responder sessions. Moreover, we assume that every honest responder session that receives an honestly generated cpk_A from some honest initiator session computes a distinct ciphertext ct_A . Similarly, we assume that every honest initiator session that receives an honestly generated cpk_B from some honest responder session computes a distinct ciphertext ct_B .

Since the long-term public keys have min-entropy at least k_1 , the combined public keys generated honestly from a fixed long-term public key have min-entropy at least k_2^{21} , and the ciphertexts of AFS-KEM satisfy γ -spreadness, these assumptions contribute an additive term $\frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma}$ to the final security bound. Notably, under these assumptions, no session can have more than one **PartialMatch** session or more than one **Match** session.

We assume that the tested session satisfies F_{pMAKE} throughout the experiment; otherwise, the adversary loses the indistinguishability game. To bound $\text{Adv}_{\text{pMAKE}}^{\text{F}_{\text{pMAKE}}\text{-KS}}(\mathcal{A})$, we partition the winning event into six cases according to $\text{F}_{\text{pMAKE}} = \bigvee_{i=1}^6 \text{FM}_i$ (Definition 9.2), which yields

$$\text{Adv}_{\text{pMAKE}}^{\text{F}_{\text{pMAKE}}\text{-KS}}(\mathcal{A}) \leq \sum_{i=1}^6 \text{Adv}_{\text{pMAKE}}^{\text{FM}_i\text{-KS}}(\mathcal{A}).$$

We now give security reductions for each subcase $\text{Adv}_{\text{pMAKE}}^{\text{FM}_i\text{-KS}}(\mathcal{A})$ with $i \in \{1, 2, 3, 4, 5, 6\}$, where throughout we assume that the tested session satisfies FM_i (otherwise the adversary trivially loses the corresponding game). The corresponding reductions are stated in Lemmas 11.1, 11.2, and 11.3. For readability, we defer the proofs of these lemmas to Subsection 11.2.

²¹ Note that in the definition of the min-entropy of the combined public key, the long-term public key pk_s is fixed, and the min-entropy is taken over honestly generated combined public keys derived from pk_s .

Lemma 11.1. *With the notation and assumptions of Theorem 9.3, there exist adversaries $\mathcal{B}_1$, $\mathcal{C}$, and $\mathcal{Z}$ running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_1\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2(N+1) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 2\delta \right).$$

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_2\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + (3N+2) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 2\delta \right).$$

Lemma 11.2. *With the notations and assumptions of the proof of Theorem 9.3, and with PRF modeled as a random oracle $\mathcal{G}$ to which the adversary makes at most $q_{\mathcal{G}}$ queries, there exist adversaries $\mathcal{B}_2$, $\mathcal{C}$, $\mathcal{D}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) &\leq 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} \right) + 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 4N^2S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 4N(N+1)(N+2)S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}). \end{aligned}$$

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) &\leq 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} \right) + 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 4N^2S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 4N(N+1)(N+2)S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}). \end{aligned}$$

Lemma 11.3. *With the notation and assumptions of Theorem 9.3, there exist adversaries $\mathcal{B}_3$, $\mathcal{D}$, $\mathcal{F}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_5\text{-KS}}(\mathcal{A}) &\leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \\ &\quad + 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \delta \right). \end{aligned}$$

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_6\text{-KS}}(\mathcal{A}) &\leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \\ &\quad + 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \delta \right). \end{aligned}$$

Combining all the above analyses completes the proof in the ROM.

Proof in the QROM. This QROM proof follows the same structure as the ROM proof, except for Lemma 11.2, where the function $G = \text{PRF}$ is modeled as a random oracle and therefore must be lifted to the QROM. For readability, we defer the proof of Lemma 11.4 to Subsection 11.2.

Lemma 11.4 (QROM counterpart of Lemma 11.2). *With the notation and assumptions of Theorem 9.3, and modeling PRF as a quantum-accessible random oracle $\mathcal{G}$ to which the adversary makes at most $q_{\mathcal{G}}$ quantum queries, there exist QPT adversaries $\mathcal{B}_2$, $\mathcal{C}$, $\mathcal{D}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) &\leq 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 2NS^2 \left(2\sqrt{(q_{\mathcal{G}}+1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_{\mathcal{G}}}{|\mathcal{K}|} + \beta \right)} + \beta \right). \end{aligned}$$

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) &\leq 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 2NS^2 \left(2\sqrt{(q_{\mathcal{G}}+1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_{\mathcal{G}}}{|\mathcal{K}|} + \beta \right)} + \beta \right). \end{aligned}$$

where $\beta = N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Combining all the above analyses completes the proof in the QROM. $\square$

11.2 Full Analysis for Lemmas 11.1, 11.2, 11.3, and 11.4

Lemma 11.1. *With the notation and assumptions of Theorem 9.3, there exist adversaries $\mathcal{B}_1$, $\mathcal{C}$, and $\mathcal{Z}$ running in time comparable to that of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_1\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2(N+1) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 2\delta \right).$$

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_2\text{-KS}}(\mathcal{A}) \leq NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + (3N+2) \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 2\delta \right).$$

Proof. We begin by providing a formal analysis for case FM_1 and assume that the tested session satisfies FM_1 (otherwise, the adversary trivially loses the corresponding game). We then derive the proof for Case FM_2 , which follows the same structure as Case FM_1 except for G'_4 .

Game G_0 . This game corresponds to the original game for $\text{Adv}_{\text{pMAKE}}^{\text{FM}_1\text{-KS}}(\mathcal{A})$, thus,

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_1\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (KEM Correctness). In this game, we abort if the decapsulation of ct_A or ct_B in π^{Test} does not equal the corresponding encapsulated key. Let ERR_A and ERR_B denote the abort events caused by ct_A and ct_B , respectively. We first bound $\Pr[\text{ERR}_A]$ using the δ -correctness of AFS-KEM. The reduction guesses π^{Test} and $\pi^{\text{PartialMatch}}$ uniformly from S sessions, and guesses $\pi^{\text{Init. actor}}$ uniformly from N parties. Notably, the guess is correct with probability at least $1/(NS^2)$. Conditioned on this guess being correct, the reduction can perfectly embed the δ -correctness experiment of AFS-KEM into the guessed sessions. Therefore, by the δ -correctness of AFS-KEM, we have $\Pr[\text{ERR}_A] \leq NS^2\delta$. By a similar argument, we have $\Pr[\text{ERR}_B] \leq NS^2\delta$. By a union bound, $\Pr[\text{ERR}_A \vee \text{ERR}_B] \leq 2NS^2\delta$. Since games G_0 and G_1 differ only if $\text{ERR}_A \vee \text{ERR}_B$ occurs, it follows that

$$|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq 2NS^2\delta.$$

Game G_2 (Peer acceptance implies a matching session). In this game, we define the BAD event as the event that the PartialMatch session π' accepts but is not the matching session of π . If the BAD event occurs, we abort the game immediately. Note that when the BAD event does not occur, Games G_1 and G_2 are identical from the adversary's view. Therefore,

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq \Pr[\text{BAD} : G_1].$$

Next, we analyze $\Pr[\text{BAD} : G_1]$. Let π^{Init} (resp. π^{Resp}) be the initiator (resp. responder) session among π, π' . If BAD occurs in G_1 , then both π^{Init} and π^{Resp} accept, but the tuple $(\text{id}_A, \text{c}_{\text{seed}}^A, \text{id}_B, \text{c}_{\text{seed}}^B)$ maintained in π^{Init} is different from the tuple $(\text{id}'_A, \text{c}'_{\text{seed}}^A, \text{id}'_B, \text{c}'_{\text{seed}}^B)$ maintained in π^{Resp} . We consider the following four cases.

- **Case 1:** If $\text{id}_A \neq \text{id}'_A$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_1 for $\mathcal{A}$, except that it first guesses the sessions π^{Init} and π^{Resp} uniformly over the S sessions, and guesses the identities id_A and id'_A uniformly over the N parties. Notably, $\mathcal{C}$ guesses correctly with probability at least $1/(N^2S^2)$. Then $\mathcal{C}$ sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and $\text{pk}_{\text{id}'_A} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into the session π^{Init} . When π^{Resp} receives $(\text{id}'_A, \text{ct}_B, \text{c}'_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts $\text{c}'_{\text{seed}}^A$ and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_e(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{Verf}}(\text{id}'_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}'_A} = \overline{\text{pk}}_s^*$, we have $\text{pk}'_A = \overline{\text{pk}}_s^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1}] \leq N^2S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2:** If $\text{id}_B \neq \text{id}'_B$, the analysis is similar to Case 1. We also construct an adversary $\mathcal{C}^{22}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple

²² We reuse the notation $\mathcal{C}$ for simplicity in these four cases.

$(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_1 for $\mathcal{A}$, except that it first randomly guesses $\pi^{\text{Init}}, \pi^{\text{Resp}}, \text{id}_B$, and id'_B , with success probability at least $1/(N^2 S^2)$. Then $\mathcal{C}$ sets $\text{pk}_{\text{id}'_B} = \text{pk}_s^*$ and $\text{pk}_{\text{id}_B} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_B = \text{seed}_e^*$ and $\text{cpk}_B = \text{cpk}^*$ into the session π^{Resp} . When π^{Init} receives $(\text{id}'_B, \text{c}'_{\text{seed}}^B)$, $\mathcal{C}$ honestly decrypts $\text{c}'_{\text{seed}}^B$ and outputs the corresponding seed'_B to its challenger. Let $(\text{pk}_e^{B'}, \text{sk}_e^{B'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_B)$ and $\text{pk}'_B \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{B'})$. Since $\text{CA}_{\text{Verf}}(\text{id}_B, \text{pk}'_B) = 1$ and $\text{pk}_{\text{id}_B} = \overline{\text{pk}}_s^*$, we have $\text{pk}'_B = \overline{\text{pk}}_s^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ also wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2}] \leq N^2 S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

- **Case 3:** If $\text{id}_A = \text{id}'_A$ and $\text{c}_{\text{seed}}^A \neq \text{c}'_{\text{seed}}^A$, we also construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_1 for $\mathcal{A}$, except that it first randomly guesses $\pi^{\text{Init}}, \pi^{\text{Resp}}$, and id_A , with success probability at least $1/(NS^2)$. $\mathcal{C}$ then sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π^{Init} . When π^{Resp} receives a pair $(\text{id}_A, \text{ct}_B, \text{c}'_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts $\text{c}'_{\text{seed}}^A$ and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{Verf}}(\text{id}_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}_A} = \text{pk}_s^*$, we have $\text{pk}'_A = \text{pk}_s^*$. Moreover, since $\text{c}_{\text{seed}}^A \neq \text{c}'_{\text{seed}}^A$ and the OTP key used to recover the seed is the same, we have $\text{seed}'_A \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 3}] \leq NS^2 \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 4:** If $\text{id}_B = \text{id}'_B$ and $\text{c}_{\text{seed}}^B \neq \text{c}'_{\text{seed}}^B$, the analysis is analogous to Case 3. We construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_1 for $\mathcal{A}$, except that it first randomly guesses $\pi^{\text{Init}}, \pi^{\text{Resp}}$, and id_B , with success probability at least $1/(NS^2)$. $\mathcal{C}$ sets $\text{pk}_{\text{id}_B} = \text{pk}_s^*$ and embeds $\text{seed}_B = \text{seed}_e^*$ and $\text{cpk}_B = \text{cpk}^*$ into π^{Resp} . When π^{Init} receives $(\text{id}_B, \text{c}'_{\text{seed}}^B)$, $\mathcal{C}$ honestly decrypts $\text{c}'_{\text{seed}}^B$ and outputs the corresponding seed'_B to its challenger. Let $(\text{pk}_e^{B'}, \text{sk}_e^{B'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_B)$ and $\text{pk}'_B \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{B'})$. Since $\text{CA}_{\text{Verf}}(\text{id}_B, \text{pk}'_B) = 1$ and $\text{pk}_{\text{id}_B} = \text{pk}_s^*$, we have $\text{pk}'_B = \text{pk}_s^*$. Moreover, we have $\text{seed}'_B \neq \text{seed}_e^*$ since $\text{c}_{\text{seed}}^B \neq \text{c}'_{\text{seed}}^B$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 4}] \leq NS^2 \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Combining the above cases, we have

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq 2(N+1)NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G_3 (Guessing $\pi^{\text{Test.actor}}$, π^{Test} , and $\pi^{\text{PartialMatch}}$). In game G_3 , we first guess the actor identity of the test session, and further guess the test session π^{Test} and its corresponding (unique) PartialMatch session $\pi^{\text{PartialMatch}}$. If any of the guesses is incorrect, the game is aborted. Concretely, we sample P^* uniformly from the N parties, and sample two session instances π, π' uniformly from the at most S sessions, respectively. If any of the guesses is incorrect (i.e., $P^* \neq \pi^{\text{Test.actor}}$, or $\pi \neq \pi^{\text{Test}}$, or $\pi' \neq \pi^{\text{PartialMatch}}$), the game is aborted. The probability of guessing correctly is $1/(NS^2)$, and therefore

$$\text{Adv}^{G_0}(\mathcal{A}) \leq NS^2 \text{Adv}^{G_3}(\mathcal{A}).$$

Game G_4 (Random K_A or K_B). In this game, if $\pi.\text{role} = \text{Init}$ (resp. $\pi.\text{role} = \text{Resp}$), we replace the KEM key K_A (resp. K_B) in the tested session π and its (unique) PartialMatch session π' with an independently uniform key $k \leftarrow \mathcal{K}$.

For any adversary $\mathcal{A}$ that can distinguish G_3 from G_4 , we construct an adversary $\mathcal{B}_1$ against the IND-SCCA security of AFS-KEM. Adversary $\mathcal{B}_1$ interacts with its AFS-KEM challenger and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{seed}_e^*, \text{ct}^*, K_b^*)$, where K_b^* equals the real encapsulated key if $b = 0$ and a uniform random key if $b = 1$. It then simulates the view of $\mathcal{A}$ as in G_4 , except that

- $\mathcal{B}_1$ sets pk_s^* as the long-term public key of P^* . It embeds $(\text{cpk}_A, \text{ct}_A) = (\text{cpk}^*, \text{ct}^*)$ (resp. $(\text{cpk}_B, \text{ct}_B) = (\text{cpk}^*, \text{ct}^*)$) into the tested session π and its PartialMatch session π' when $\pi.\text{role} = \text{Init}$ (resp. $\pi.\text{role} = \text{Resp}$), and uses K_b^* as the KEM key K_A (resp. K_B) in both sessions π and π' . Using seed_e^* and K_b^* , $\mathcal{B}_1$ can honestly simulate both sessions.

- For each non-tested session $\hat{\pi} \neq \pi$ of P^* , $\mathcal{B}_1$ independently queries $\mathcal{O}^{\text{KeyGen}}()$ to obtain a fresh tuple $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$ for the session, and sets the cpk_A message of $\hat{\pi}$ to be cpk (resp. sets the cpk_B message to be cpk) when $\hat{\pi}.\text{role} = \text{Init}$ (resp. $\hat{\pi}.\text{role} = \text{Resp}$). Upon receiving the corresponding KEM ciphertext ct for $\hat{\pi}$, $\mathcal{B}_1$ queries $\mathcal{O}^{\text{Decaps}}(\text{cpk}, \text{ct})$ to obtain the KEM key K_A (resp. K_B) when $\hat{\pi}.\text{role} = \text{Init}$ (resp. $\hat{\pi}.\text{role} = \text{Resp}$), and continues the simulation of that session honestly.

When $b = 0$, $\mathcal{B}_1$ perfectly simulates G_3 , and when $b = 1$, $\mathcal{B}_1$ perfectly simulates G_4 . That is, $\Pr[\mathcal{B}_1 \Rightarrow 1 : b = 0] = \Pr[G_3^A \Rightarrow 1]$ and $\Pr[\mathcal{B}_1 \Rightarrow 1 : b = 1] = \Pr[G_4^A \Rightarrow 1]$. Therefore, we have

$$|\text{Adv}^{G_3}(\mathcal{A}) - \text{Adv}^{G_4}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1).$$

Game G_5 (Random K_{SESSION} , K_{ENC}^A , K_{ENC}^B). In this game, we replace the keys $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$ in the sessions π and π' with independently sampled uniform keys $\hat{K}_{\text{SESSION}}, \hat{K}_{\text{ENC}}^A, \hat{K}_{\text{ENC}}^B$, respectively.

For any adversary $\mathcal{A}$ that can distinguish G_4 from G_5 , we construct an adversary $\mathcal{Z}$ against the dual-PRF security of the underlying PRF. Adversary $\mathcal{Z}$ is given oracle access to $O_b(\cdot)$, where O_0 implements the real PRF and O_1 implements a uniformly random function. Adversary $\mathcal{Z}$ simulates the view of $\mathcal{A}$ exactly as in Game G_4 , except that it computes $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B)$ for the sessions π and π' by querying its oracle $O_b(\cdot)$. Notably, if $\pi.\text{role} = \text{Init}$, $\mathcal{Z}$ queries the corresponding $O_b(K_B)$; otherwise, it queries the corresponding $O_b(K_A)$ in π and π' . When $b = 0$, the oracle O_0 corresponds to the real PRF, and the simulation is identical to Game G_4 . When $b = 1$, the oracle O_1 is a truly random function, and the simulation is identical to Game G_5 . Thus, $\Pr[\mathcal{Z} \Rightarrow 1 \mid b = 0] = \Pr[G_4^A \Rightarrow 1]$ and $\Pr[\mathcal{Z} \Rightarrow 1 \mid b = 1] = \Pr[G_5^A \Rightarrow 1]$. Therefore, we obtain

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}).$$

Analysis of G_5 . In G_5 , the session key K_{SESSION} of the tested session π is uniformly random for $\mathcal{A}$, and there may be one accepted session (which must be the matching session) that uses the same session key K_{SESSION} , and we do not allow the adversary to reveal this session key for the matching session of π . Note that all session keys in other accepted sessions are independent of K_{SESSION} . Therefore, we have

$$\text{Adv}^{G_5}(\mathcal{A}) = 0.$$

Combining all the bounds derived above completes the proof for case FM_1 . $\square$

Proof of Case FM_2 . This proof follows the same structure as that for case FM_1 , except for G'_4 . We only highlight the differences below. Notably, we assume that the tested session satisfies FM_2 .

Game G'_0 . This game corresponds to the original game defining $\text{Adv}_{\text{pMAKE}}^{\text{FM}_2\text{-KS}}(\mathcal{A})$. Thus,

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_2\text{-KS}}(\mathcal{A}) = \text{Adv}^{G'_0}(\mathcal{A}).$$

Game G'_1 (KEM-Correctness). In this game, we abort if the decapsulation of ct_A or ct_B in π^{Test} does not equal the corresponding encapsulated key. Similar to the analysis of Game G_1 in case FM_1 , we have

$$|\text{Adv}^{G'_0}(\mathcal{A}) - \text{Adv}^{G'_1}(\mathcal{A})| \leq 2NS^2\delta.$$

Game G'_2 (Peer acceptance implies a matching session). In this game, we define the BAD event as the event that the PartialMatch session π' accepts but is not the matching session of π . If the BAD event occurs, we abort the game immediately. The analysis is identical to that of Game G_2 in case FM_1 , yielding

$$|\text{Adv}^{G'_1}(\mathcal{A}) - \text{Adv}^{G'_2}(\mathcal{A})| \leq 2(N+1)NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G'_3 (Guessing $\pi^{\text{Test}}.\text{peer}$, π^{Test} , and $\pi^{\text{PartialMatch}}$). In game G'_3 , we first guess the peer identity of the test session, and further guess the test session π^{Test} and its corresponding (unique) PartialMatch session $\pi^{\text{PartialMatch}}$. If any of the guesses is incorrect, the game is aborted. Concretely, we sample P^* uniformly from the N parties, and sample two session instances π, π' uniformly from the at

most S sessions, respectively. If any of the guesses is incorrect, i.e., $P^* \neq \pi^{\text{Test}}.\text{peer}$, $\pi \neq \pi^{\text{Test}}$, or $\pi' \neq \pi^{\text{PartialMatch}}$, the game is aborted. The probability of guessing correctly is $1/(NS^2)$. Therefore,

$$\text{Adv}^{G'_2}(\mathcal{A}) \leq NS^2 \text{Adv}^{G'_3}(\mathcal{A}).$$

Game G'_4 . In this game, we define the flag event as $\pi^{\text{PartialMatch}}.\text{actor} \neq P^*$. If the flag event occurs, the game aborts immediately. In particular, after this game hop, we must have $\pi^{\text{PartialMatch}}.\text{actor} = P^*$, where P^* is not corrupted by the adversary. Since G'_3 and G'_4 are identical unless the flag event occurs, we have

$$|\text{Adv}^{G'_3}(\mathcal{A}) - \text{Adv}^{G'_4}(\mathcal{A})| \leq \Pr[\text{flag}].$$

If the flag event occurs, we construct an adversary $\mathcal{C}$ against the static public key binding security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives a challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G'_3 for $\mathcal{A}$, except that it samples B^* uniformly from N parties and aborts if $B^* \neq \pi^{\text{PartialMatch}}.\text{actor}$. Notably, $\mathcal{C}$ guesses correctly with probability $1/N$.

Then $\mathcal{C}$ sets $\text{pk}_{B^*} = \text{pk}_s^*$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ (resp. $\text{seed}_B = \text{seed}_e^*$ and $\text{cpk}_B = \text{cpk}^*$) into $\pi^{\text{PartialMatch}}$ if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). When π receives an OTP ciphertext c , $\mathcal{C}$ honestly decrypts c and outputs the corresponding seed_e' to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}_e')$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Ver}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$, we conclude that $\text{pk}'_s = \overline{\text{pk}}_s^*$. Therefore, if the flag event occurs and $\mathcal{C}$ guesses correctly, adversary $\mathcal{C}$ wins its static public key binding game. Hence,

$$|\text{Adv}^{G'_3}(\mathcal{A}) - \text{Adv}^{G'_4}(\mathcal{A})| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G'_5 (Random K_A or K_B). In this game, if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$), we replace the KEM key K_A (resp. K_B) in the tested session π and its (unique) PartialMatch session π' with an independently uniform key $\hat{k} \leftarrow \mathcal{K}$. Similarly to the analysis of Game G_4 for case FM_1 , it suffices to interchange the conditions $\pi.\text{role} = \text{Init}$ and $\pi.\text{role} = \text{Resp}$ in G_4 to obtain the analysis here. Hence, we can construct an adversary $\mathcal{B}_1$ against the IND-SCCA security of AFS-KEM such that

$$|\text{Adv}^{G'_4}(\mathcal{A}) - \text{Adv}^{G'_5}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-SCCA}}(\mathcal{B}_1).$$

Game G'_6 (Random K_{SESSION} , K_{ENC}^A , K_{ENC}^B). In this game, we replace the keys K_{SESSION} , K_{ENC}^A , K_{ENC}^B in the sessions π and π' with independently sampled uniform keys $\hat{K}_{\text{SESSION}}$, $\hat{K}_{\text{ENC}}^A$, $\hat{K}_{\text{ENC}}^B$, respectively. Similar to the analysis of Game G_5 in case FM_1 , we obtain

$$|\text{Adv}^{G'_5}(\mathcal{A}) - \text{Adv}^{G'_6}(\mathcal{A})| \leq \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}).$$

Analysis of G'_6 . In Game G'_6 , the session key K_{SESSION} of the tested session π is uniformly random from the adversary's view. Therefore,

$$\text{Adv}^{G'_6}(\mathcal{A}) = 0.$$

Combining all the bounds above completes the proof for case FM_2 . $\square$

Lemma 11.3. *With the notation and assumptions of Theorem 9.3, there exist adversaries $\mathcal{B}_3$, $\mathcal{D}$, $\mathcal{F}$, and $\mathcal{Z}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_5\text{-KS}}(\mathcal{A}) &\leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \\ &\quad + 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \delta \right). \\ \text{Adv}_{\text{pMAKE}}^{\text{FM}_6\text{-KS}}(\mathcal{A}) &\leq NS \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \\ &\quad + 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) + \delta \right). \end{aligned}$$

Proof. In this lemma, we provide a unified analysis by considering $\text{Adv}_{\text{pMAKE}}^{(\text{FM}_5 \vee \text{FM}_6)\text{-KS}}(\mathcal{A})$ and showing that the event $\text{FM}_5 \vee \text{FM}_6$ occurs only with negligible probability. In particular, it holds that $\text{Adv}_{\text{pMAKE}}^{\text{FM}_5\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_5 \vee \text{FM}_6)\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pMAKE}}^{\text{FM}_6\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_5 \vee \text{FM}_6)\text{-KS}}(\mathcal{A})$. The proof of explicit authentication for pMAKE (Theorem 11.5) largely follows the analysis of the cases $\text{FM}_5 \vee \text{FM}_6$, except for Game G_6 . In Game G_6 here, we show that a PartialMatch session of the tested session must exist, whereas in the proof of Theorem 11.5, we show that a matching session must exist.

Game G_0 . This game corresponds to the original game for $\text{Adv}_{\text{pMAKE}}^{\text{FM}_5 \vee \text{FM}_6\text{-KS}}(\mathcal{A})$ and we assume that the freshness condition $\text{FM}_5 \vee \text{FM}_6$ holds for the tested session. Let π denote the tested session in the game. Let π' denote the (unique, if it exists) session such that $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) = (\pi.\text{cpk}_A, \pi.\text{ct}_A)$ (resp. $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) = (\pi.\text{cpk}_B, \pi.\text{ct}_B)$) if $\pi.\text{role} = \text{Resp}$ (resp. if $\pi.\text{role} = \text{Init}$). Notably, we have

$$\text{Adv}_{\text{pMAKE}}^{\text{FM}_5 \vee \text{FM}_6\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Game G_1 (Abort if Collision in K_A or K_B). In this game, we define the COLL event as follows: π' exists and $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) \neq (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ (resp. $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) \neq (\pi.\text{cpk}_A, \pi.\text{ct}_A)$) but $\pi.K_B = \pi'.K_B$ (resp. $\pi.K_A = \pi'.K_A$) when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$), then the COLL event occurs. If the COLL event occurs, we abort the game. Since Games G_0 and G_1 are identical unless the COLL event occurs, we have

$$|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq \Pr[\text{COLL} : G_0].$$

For simplicity, we consider the case $\pi.\text{role} = \text{Resp}$. If the COLL event occurs in this case, then $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) \neq (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ while $\pi.K_B = \pi'.K_B$. We consider the following two subcases.

- **Case 1:** $\pi'.\text{cpk}_B \neq \pi.\text{cpk}_B$. We construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}$ first chooses $g = 1$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*)$ from its challenger. It then perfectly simulates Game G_0 for $\mathcal{A}$, except that $\mathcal{F}$ first guesses the identity of $\pi.\text{actor}$ uniformly from the N users and guesses the session π', π uniformly from the S sessions. Notably, the probability that $\mathcal{F}$ guesses both correctly is $1/(NS^2)$. Adversary $\mathcal{F}$ then sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair of $\pi.\text{actor}$, and embeds $(\text{cpk}_B = \text{cpk}^*, \text{seed}_B = \text{seed}_e^*)$ in session π . Next, $\mathcal{F}$ outputs the composite public key cpk'_B in session π' and forwards it to its challenger. Upon receiving the corresponding (ct', K') from the challenger, $\mathcal{F}$ continues the simulation by embedding $(\text{ct}_B = \text{ct}', K_B = K')$ into session π' . Finally, when session π receives a ciphertext ct'_B , $\mathcal{F}$ outputs ct'_B to its challenger. If the event [COLL : Case 1] occurs, then $\text{cpk}'_B \neq \text{cpk}^*$ and $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}'_B) = K_B$. Hence, adversary $\mathcal{F}$ wins the Key-Coll game, and we have $\Pr[\text{COLL} : \text{Case 1}] \leq NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.
- **Case 2:** $\pi'.\text{cpk}_B = \pi.\text{cpk}_B$ but $\pi'.\text{ct}_B \neq \pi.\text{ct}_B$. We again construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}^{23}$ chooses $g = 3$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*, K^*, \text{ct}^*)$ from its challenger. As before, $\mathcal{F}$ guesses the identity of $\pi.\text{actor}$ uniformly from the N users and guesses the session π', π uniformly from the S sessions, with success probability $1/(NS^2)$. It sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair of $\pi.\text{actor}$, sets $\text{cpk}_B = \text{cpk}^*$, $\text{seed}_B = \text{seed}_e^*$ in session π , and embeds $K_B = K^*, \text{ct}_B = \text{ct}^*$ into session π' . When session π receives a ciphertext $\text{ct}'_B \neq \text{ct}^*$, adversary $\mathcal{F}$ outputs ct'_B to its challenger. If the event [COLL : Case 2] occurs, then $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}'_B) = K^*$, and thus $\mathcal{F}$ wins the Key-Coll game. Therefore, $\Pr[\text{COLL} : \text{Case 2}] \leq NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.

Combining the two subcases, we obtain $\Pr[\text{COLL} : \pi.\text{role} = \text{Resp}] \leq 2NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$. By a symmetric analysis, the same bound holds for the case $\pi.\text{role} = \text{Init}$, that is, $\Pr[\text{COLL} : \pi.\text{role} = \text{Init}] \leq 2NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$. Therefore, we have

$$|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq 4NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}).$$

Game G_2 (KEM Correctness). In this game, we define the event ERR_1 to occur if the decapsulation of ct_A (resp. ct_B) in π' (if it exists) when $\pi.\text{role} = \text{Resp}$ (resp. when $\pi.\text{role} = \text{Init}$) does not equal the corresponding encapsulated key. Moreover, if π' is further to be a PartialMatch session of π , we define

²³ For simplicity, we reuse the notation $\mathcal{F}$ for the adversary.

the event ERR_2 to occur if the decapsulation of ct_B (resp. ct_A) in π' (if it exists) when $\pi.\text{role} = \text{Resp}$ (resp. when $\pi.\text{role} = \text{Init}$) does not equal the corresponding encapsulated key.

We first bound $\Pr[\text{ERR}_1]$ using the δ -correctness of AFS-KEM. The reduction guesses the identity of $\pi'.$ actor and the sessions π and π' uniformly at random, with success probability at least $1/(NS^2)$, and embeds the δ -correctness experiment into the corresponding KEM instance $(\text{cpk}_A, \text{ct}_A)$ (resp. $(\text{cpk}_B, \text{ct}_B)$) of π and π' when $\pi.\text{role} = \text{Resp}$ (resp. when $\pi.\text{role} = \text{Init}$). Since AFS-KEM is δ -correct, it follows that $\Pr[\text{ERR}_1] \leq NS^2\delta$. A similar analysis applies to ERR_2 , yielding $\Pr[\text{ERR}_2] \leq NS^2\delta$. By a union bound, games G_{A2} and G_{A3} differ with probability at most $2NS^2\delta$. Therefore,

$$|\text{Adv}^{G_2}(\mathcal{A}) - \text{Adv}^{G_3}(\mathcal{A})| \leq 2NS^2\delta.$$

Game G_3 (Guessing $\pi^{\text{Test}}.\text{peer}$, π^{Test}). In game G_2 , we first guess the peer identity of the test session, and further guess the test session π^{Test} . If either guess is incorrect, the game is aborted. Concretely, we sample P^* uniformly from the N parties, and sample one session instance π uniformly from the at most S sessions. If the guess is incorrect (i.e., $P^* \neq \pi^{\text{Test}}.\text{peer}$ or $\pi \neq \pi^{\text{Test}}$), the game is aborted. The probability of guessing correctly is $1/(NS)$, therefore

$$\text{Adv}^{G_2}(\mathcal{A}) \leq NS \cdot \text{Adv}^{G_3}(\mathcal{A}).$$

Game G_4 (New win condition). In this game, we change the condition for the adversary to win. An instance of G_4 is won by the adversary $\mathcal{A}$ if and only if the tested session π accepts and nothing makes the instance of the game invalid (as per the conditions of G_3). We stop the game at the moment when π accepts. Note that in G_4 , the **Test** query is not requested for π , and in all valid instances and P^* remains uncorrupted. It is evident that any adversary that wins the game G_3 must also win the game G_4 . Thus, we have

$$\text{Adv}^{G_3}(\mathcal{A}) \leq \text{Adv}^{G_4}(\mathcal{A}).$$

Game G_5 (Random K_A or K_B). In this game, we replace the KEM key K_A (resp. K_B) if $\pi.\text{role} = \text{Resp}$ (resp. if $\pi.\text{role} = \text{Init}$) in the tested session π with an independently uniform key $\tilde{k} \leftarrow \$ \mathcal{K}$. Recall that π' denotes the (unique, if it exists) session such that $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) = (\pi.\text{cpk}_A, \pi.\text{ct}_A)$ (resp. $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) = (\pi.\text{cpk}_B, \pi.\text{ct}_B)$) if $\pi.\text{role} = \text{Resp}$ (resp. if $\pi.\text{role} = \text{Init}$). If such π' exists, we also replace the corresponding KEM key K_A (resp. K_B) in π' with the same key $\tilde{k}$.

For any adversary $\mathcal{A}$ that can distinguish G_4 from G_5 , we construct an adversary $\mathcal{B}_3$ against the IND-LCK-SCCA security of AFS-KEM. Adversary $\mathcal{B}_3$ first receives pk_s^* from its challenger, sets pk_s^* as the long-term public key of P^* , and generates long-term key pairs $(\text{pk}_U, \text{sk}_U)$ honestly for all other parties. It then simulates the view of $\mathcal{A}$ as in G_5 , except for the following changes.

- For each session $\hat{\pi}$ of P^* with $\hat{\pi}.\text{role} = \text{Init}$ (resp. Resp), $\mathcal{B}_3$ independently queries $\mathcal{O}^{\text{EKeyGen}}$ to obtain a fresh tuple $(\text{cpk}, \text{pk}_e, \text{sk}_e, \text{seed})$ for the session, and sets $\text{cpk}_A = \text{cpk}$ (resp. $\text{cpk}_B = \text{cpk}$). When tested session π receives the corresponding composite public key cpk' , $\mathcal{B}_3$ sets $\text{cpk}^* = \text{cpk}'$ and forwards it to its challenger. After $\mathcal{B}_3$ receives $(\text{pk}_s^*, \text{ct}^*, K_b^*)$ from its challenger, it embeds $\text{ct}_A = \text{ct}^*$ (resp. $\text{ct}_B = \text{ct}^*$) into the tested session π if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$), and uses K_b^* as the corresponding KEM key K_A (resp. K_B) in π . If the π' session exists, then $\mathcal{B}_3$ also sets K_b^* as the corresponding KEM key K_A (resp. K_B) in π' if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). Moreover, when P^* receives a KEM ciphertext ct for a session $\hat{\pi} \neq \pi'$, the adversary $\mathcal{B}_3$ queries the corresponding decryption oracle $\mathcal{O}^{\text{Dec}}(\text{cpk}, \text{ct})$, where cpk is the composite public key embedded into session $\hat{\pi}$ via $\mathcal{O}^{\text{EKeyGen}}$, to obtain the KEM key K_A (resp. K_B) if $\hat{\pi}.\text{role} = \text{Init}$ (resp. $\hat{\pi}.\text{role} = \text{Resp}$). It then continues the simulation of session $\hat{\pi}$ honestly.
- When π receives $(\text{id}_{P^*}, \text{c}_{\text{seed}}^{P^*})$, $\mathcal{B}_3$ decrypts $\text{c}_{\text{seed}}^{P^*}$ using K_b^* to obtain seed_{P^*} and outputs seed_{P^*} to its challenger. Since π is an accepted session, we have $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}_{P^*}') = 1$, where $\text{pk}_{P^*}' \leftarrow \text{ExtractPK}(\text{cpk}', \text{pk}_e^{P^*})$ and $(\text{pk}_e^{P^*}, \cdot) = \text{EKeyGen}_c(\text{cpk}', \text{seed}_{P^*})$. Therefore, $\text{pk}_{P^*}' = \text{pk}_s^*$ and seed_{P^*} is a valid output for $\mathcal{B}_3$.

When $b = 0$, adversary $\mathcal{B}_3$ perfectly simulates Game G_4 , i.e., $\Pr[\mathcal{B}_3 \Rightarrow 1 : b = 0] = \Pr[G_4^A \Rightarrow 1]$. When $b = 1$, $\mathcal{B}_3$ perfectly simulates G_5 , i.e., $\Pr[\mathcal{B}_3 \Rightarrow 1 : b = 1] = \Pr[G_5^A \Rightarrow 1]$. Therefore,

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq 2\text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}_3).$$

Game G_6 (Random $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$). In this game, we replace the keys $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$ in the session π with independently sampled uniform keys $\hat{K}_{\text{SESSION}}, \hat{K}_{\text{ENC}}^A, \hat{K}_{\text{ENC}}^B$, respectively. If π' exists and is the **PartialMatch** session of π , we use the same uniform keys $\hat{K}_{\text{SESSION}}, \hat{K}_{\text{ENC}}^A, \hat{K}_{\text{ENC}}^B$ as in π . Otherwise, if π' exists but is not a **PartialMatch** session of π , we use independently sampled uniform keys $\bar{K}_{\text{SESSION}}, \bar{K}_{\text{ENC}}^A, \bar{K}_{\text{ENC}}^B$ in π' . Notably, after Game G_1 , if π' exists but is not a **PartialMatch** session of π , then we must have $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) \neq (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ (resp. $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) \neq (\pi.\text{cpk}_A, \pi.\text{ct}_A)$), but $\pi.K_B \neq \pi'.K_B$ (resp. $\pi.K_A \neq \pi'.K_A$) when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$).

For any adversary $\mathcal{A}$ that can distinguish G_5 from G_6 , we construct an adversary $\mathcal{Z}$ against the dual-PRF security of the underlying PRF. Adversary $\mathcal{Z}$ is given oracle access to $O_b(\cdot)$, where O_0 implements the real PRF and O_1 implements a uniformly random function. Adversary $\mathcal{Z}$ simulates the view of $\mathcal{A}$ exactly as in Game G_5 , except that it computes $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B)$ for the sessions π and π' (if it exists) by querying its oracle $O_b(K_B)$ (resp. $O_b(K_A)$) if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). When $b = 0$, the oracle O_0 corresponds to the real PRF, and the simulation is identical to Game G_5 . When $b = 1$, the oracle O_1 is a truly random function, and the simulation is identical to Game G_6 . Thus, $\Pr[\mathcal{Z} \Rightarrow 1 \mid b = 0] = \Pr[G_5^{\mathcal{A}} \Rightarrow 1]$ and $\Pr[\mathcal{Z} \Rightarrow 1 \mid b = 1] = \Pr[G_6^{\mathcal{A}} \Rightarrow 1]$. Therefore, we obtain

$$|\text{Adv}^{G_5}(\mathcal{A}) - \text{Adv}^{G_6}(\mathcal{A})| \leq \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}).$$

Game G_7 (PartialMatch session). In this game, we define **BAD** as the event that the **PartialMatch** session π^{Match} of the test session π does not exist. If **BAD** occurs, we abort the game immediately. At this point, whenever the $\text{FM}_5 \vee \text{FM}_6$ event occurs, the game aborts. Therefore,

$$\text{Adv}^{G_7}(\mathcal{A}) = 0.$$

It is evident that $|\text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A})| \leq \Pr[\text{BAD} : G_6]$. We next analyze $\Pr[\text{BAD} : G_6]$. In G_6 , the keys $\hat{K}_{\text{ENC}}^A$ and $\hat{K}_{\text{ENC}}^B$ of the tested session π are uniform from the adversary's view. Since π accepts without any **FirstMatch** session, these keys are never used in any other session. Therefore, $\text{seed}'_A = \hat{K}_{\text{ENC}}^A \oplus \text{c}_{\text{seed}}^A$ (resp. $\text{seed}'_B = \hat{K}_{\text{ENC}}^B \oplus \text{c}_{\text{seed}}^B$) computed in π is uniform from the adversary's view when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$).

We first consider the case $\pi.\text{role} = \text{Resp}$. We construct an adversary $\mathcal{D}$ against the RSPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{D}$ receives $(\text{pk}_s^*, \text{sk}_s^*)$ and perfectly simulates Game G_6 for $\mathcal{A}$, except that it sets $\text{pk}_{P^*} = \text{pk}_s^*$. It outputs the composite public key cpk_A in session π to its challenger. After receiving seed_e from the challenger, $\mathcal{D}$ defines $K_{\text{ENC}}^A = \text{seed}_e \oplus \text{c}_{\text{seed}}^A$, where c_{seed}^A is received in π . Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}_A, \text{seed}_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}_A, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{pk}_{P^*} = \text{pk}_s^*$, we have $\text{pk}'_s = \text{pk}_s^*$ by the security of CA. Therefore, if the **BAD** event occurs in this case, then $\mathcal{D}$ wins the RSPK-Bind game.

The analysis case $\pi.\text{role} = \text{Init}$ is similar. In this case, $\mathcal{D}$ outputs the composite public key cpk_B in session π to its challenger. After receiving seed_e from the challenger, $\mathcal{D}$ defines $K_{\text{ENC}}^B = \text{seed}_e \oplus \text{c}_{\text{seed}}^B$, where c_{seed}^B is received in π . Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}_B, \text{seed}_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}_B, \text{pk}'_e)$. Again, since $\text{CA}_{\text{Verf}}(\text{id}_{P^*}, \text{pk}'_s) = 1$ and $\text{pk}_{P^*} = \text{pk}_s^*$, we have $\text{pk}'_s = \text{pk}_s^*$ by the security of CA. Therefore, if the **BAD** event occurs in this case, then $\mathcal{D}$ wins the RSPK-Bind game. Consequently, $\Pr[\text{BAD} : G_6] \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$. Thus,

$$|\text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}).$$

Combining all the bounds derived above completes the proof. $\square$

Lemma 11.2. *With the notations and assumptions of the proof of Theorem 9.3, and with PRF modeled as a random oracle $\mathcal{G}$ to which the adversary makes at most $q_{\mathcal{G}}$ queries, there exist adversaries $\mathcal{B}_2, \mathcal{C}, \mathcal{D}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) &\leq 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} \right) + 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 4N^2S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 4N(N+1)(N+2)S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}). \\ \text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) &\leq 2NS^2 \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{q_{\mathcal{G}}}{|\mathcal{K}|} \right) + 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 4N^2S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 4N(N+1)(N+2)S^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}). \end{aligned}$$

Proof. We provide a unified analysis for Case FM_3 and Case FM_4 , as their proofs are essentially identical. We assume that the tested session satisfies $\text{FM}_3 \vee \text{FM}_4$; otherwise, the adversary trivially loses the game. In particular, it holds that $\text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$.

Game G_0 . This game corresponds to the original game defining $\text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$. Thus,

$$\text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Case Distinction. Let π denote the tested session and let π' denote its (unique) `PartialMatch` session. We further write π^{Init} and π^{Resp} to denote the initiator and responder sessions among $\{\pi, \pi'\}$, respectively. Next, we consider the following two cases in Game G_0 . Case A (denoted by G_A): the adversary never issues `RevealEph`(π^{Init}). Case B (denoted by G_B): the adversary never issues `RevealEph`(π^{Resp}). Accordingly, we can bound the adversary's success probability by

$$\text{Adv}^{G_0}(\mathcal{A}) \leq \text{Adv}^{G_A}(\mathcal{A}) + \text{Adv}^{G_B}(\mathcal{A}).$$

Case A: the adversary never issues `RevealEph`(π^{Init}).

Game G_{A1} (KEM Correctness). In this game, we define ERR_1 as the event that the decapsulation of ct_A in π^{Init} does not equal the corresponding encapsulated key, and define ERR_2 analogously for ct_B in π^{Resp} . If either event occurs, we abort. We bound $\Pr[\text{ERR}_1]$ by the δ -correctness of AFS-KEM. The reduction randomly guesses π^{Init} , π^{Resp} , and $\pi^{\text{Init.actor}}$, with success probability at least $1/(NS^2)$, and embeds the correctness-failure experiment into $(\text{cpk}_A, \text{ct}_A)$. Hence $\Pr[\text{ERR}_1] \leq NS^2\delta$. Similarly, $\Pr[\text{ERR}_2] \leq NS^2\delta$. By a union bound, we have

$$|\text{Adv}^{G_A}(\mathcal{A}) - \text{Adv}^{G_{A1}}(\mathcal{A})| \leq 2NS^2\delta.$$

Game G_{A2} . In this game, let $\text{K}_A^{\text{Test}} = \pi^{\text{Resp}}.\text{K}_A$. We define the `COLL` event as the existence of a session $\hat{\pi} \notin \{\pi, \pi'\}$ such that $\hat{\pi}.\text{K}_A = \text{K}_A^{\text{Test}}$. Game G_{A2} is identical to Game G_{A1} , except that the game aborts whenever the `COLL` event occurs. Since Games G_{A1} and G_{A2} are identical unless the `COLL` event occurs, we have

$$|\text{Adv}^{G_{A1}}(\mathcal{A}) - \text{Adv}^{G_{A2}}(\mathcal{A})| \leq \Pr[\text{COLL} : G_{A1}].$$

If the `COLL` event occurs, we consider the following two cases.

- **Case 1:** $\hat{\pi}.\text{role} = \text{Init}$. We construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}$ first chooses $g = 1$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*)$ from its challenger. Adversary $\mathcal{F}$ guesses the sessions $\hat{\pi}$, π^{Init} , and π^{Resp} uniformly over the S sessions, and guesses $\hat{\pi}.\text{actor}$ uniformly over the N parties. The probability that $\mathcal{F}$ guesses correctly is at least $1/(NS^3)$.

Adversary $\mathcal{F}$ then perfectly simulates Game G_{A1} for $\mathcal{A}$, except that it sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair for $\hat{\pi}.\text{actor}$ and embeds $(\text{cpk}_A, \text{seed}_A) = (\text{cpk}^*, \text{seed}_e^*)$ into session $\hat{\pi}$. It then outputs the cpk_A received by π^{Resp} to its challenger. After receiving (ct', K') from the challenger, $\mathcal{F}$ embeds $(\text{ct}_A = \text{ct}', \text{K}_A = K')$ into π^{Init} and π^{Resp} . Finally, when session $\hat{\pi}$ receives its ciphertext ct'_A , $\mathcal{F}$ outputs ct'_A to its challenger.

Note that $\mathcal{F}$ perfectly simulates Game G_{A1} . If the event $[\text{COLL} : \text{Case 1}]$ occurs, then $\text{cpk} \neq \text{cpk}^*$ since all honestly generated cpk values are distinct, and $\text{Decaps}_{\text{AFS}}(\text{csk}^*, \text{ct}'_A) = K'$. Therefore, $\mathcal{F}$ wins the Key-Coll game, and hence $\Pr[\text{COLL} : \text{Case 1}] \leq NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.

- **Case 2:** $\hat{\pi}.\text{role} = \text{Resp}$. We again construct an adversary $\mathcal{F}$ against the Key-Coll game for AFS-KEM as follows. Adversary $\mathcal{F}$ first chooses $g = 2$ and receives a tuple $(\text{cpk}^*, \text{pk}_s^*, \text{sk}_s^*, \text{seed}_e^*, K^*, \text{ct}^*)$ from its challenger. It guesses the sessions $\hat{\pi}$, π^{Init} , and π^{Resp} uniformly over the S sessions, and guesses $\pi^{\text{Init.actor}}$ uniformly over the N parties. The probability of guessing correctly is at least $1/(NS^3)$. Adversary $\mathcal{F}$ then perfectly simulates Game G_{A1} for $\mathcal{A}$, except that it sets $(\text{pk}_s^*, \text{sk}_s^*)$ as the long-term key pair for $\pi^{\text{Init.actor}}$, embeds $(\text{cpk}_A, \text{seed}_A) = (\text{cpk}^*, \text{seed}_e^*)$ into session π^{Init} , and embeds $(\text{ct}_A, \text{K}_A) = (\text{ct}^*, K^*)$ into session π^{Resp} . When session $\hat{\pi}$ receives its composite public key cpk'_A , $\mathcal{F}$

forwards cpk'_A to its challenger and receives (ct', K') . It then embeds $(\text{ct}_A = \text{ct}', K_A = K')$ into session $\hat{\pi}$.

Adversary $\mathcal{F}$ perfectly simulates Game G_{A1} for this case. If the event $[\text{COLL} : \text{Case 2}]$ occurs, then $K' = K^*$. Thus, $\mathcal{F}$ wins the Key-Coll game, and therefore $\Pr[\text{COLL} : \text{Case 2}] \leq NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.

Combining the two cases, we obtain

$$|\text{Adv}^{G_{A1}}(\mathcal{A}) - \text{Adv}^{G_{A2}}(\mathcal{A})| \leq 2NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}).$$

Game G_{A3} (Guessing π^{Test} , $\pi^{\text{PartialMatch}}$, and Actor). In this game, we guess the test session π , its corresponding PartialMatch session π' , and the identity of π^{Init} .actor. Concretely, we sample two session instances π and π' uniformly from the S sessions, and sample a party P^* uniformly from the N parties. The game aborts if $\pi \neq \pi^{\text{Test}}$ or $\pi' \neq \pi^{\text{PartialMatch}}$, or $\pi^{\text{Test}}.\text{role} = \text{Init}$ and $P^* \neq \pi^{\text{Test}}.\text{actor}$, or if $\pi^{\text{Test}}.\text{role} = \text{Resp}$ and $P^* \neq \pi^{\text{PartialMatch}}.\text{actor}$. The probability of guessing correctly is $1/(NS^2)$. Therefore,

$$\text{Adv}^{G_{A2}}(\mathcal{A}) \leq NS^2 \cdot \text{Adv}^{G_{A3}}(\mathcal{A}).$$

Game G_{A4} (Random K_{SESSION} , K_{ENC}^A , K_{ENC}^B). In this game, we replace K_{SESSION} , K_{ENC}^A , and K_{ENC}^B in the guessed sessions π and π' with uniformly random values. Define QUERY as the event that $\mathcal{A}$ queries G on input $(K_A^{\text{Test}}, K_B^{\text{Test}})$, where K_A^{Test} and K_B^{Test} denote the encapsulated keys K_A and K_B in sessions π and π' , respectively. Notably, after Game G_{A2} , every other session $\hat{\pi} \notin \{\pi, \pi'\}$ satisfies $\hat{\pi}.K_A \neq K_A^{\text{Test}}$. Then Games G_{A3} and G_{A4} are identical unless the event QUERY occurs, and we have

$$|\text{Adv}^{G_{A3}}(\mathcal{A}) - \text{Adv}^{G_{A4}}(\mathcal{A})| \leq \Pr[\text{QUERY} : G_{A4}].$$

Game G_{A5} (Abort on BAD). In this game, we define the event BAD as follows. The event BAD occurs if either (i) the responder session π^{Resp} accepts the session key but the tuple $(\text{id}_A, \text{c}_{\text{seed}}^A)$ sent by π^{Init} is different from $(\text{id}'_A, \text{c}'_{\text{seed}}^A)$ received by π^{Resp} , or (ii) the initiator session π^{Init} accepts the session key but π^{Resp} is not the matching session of π^{Init} . If the event BAD occurs, we abort the game. Notably, after this game hop, π^{Resp} accepts the session key only if it receives the honest third message from π^{Init} , and π^{Init} accepts the session key only if π^{Resp} is its matching session.

At this point, the session key K_{SESSION} of the tested session π is uniformly random from the adversary's view. Moreover, there may be one accepted session using the same session key, but such a session must be the matching session of π . All session keys held by other accepted sessions are independent of K_{SESSION} . Therefore,

$$\text{Adv}^{G_{A5}}(\mathcal{A}) = 0.$$

Since Games G_{A4} and G_{A5} are identical unless the BAD event occurs, we have $|\text{Adv}^{G_{A4}}(\mathcal{A}) - \text{Adv}^{G_{A5}}(\mathcal{A})| \leq \Pr[\text{BAD} : G_{A4}]$. Similarly, $|\Pr[\text{QUERY} : G_{A4}] - \Pr[\text{QUERY} : G_{A5}]| \leq \Pr[\text{BAD} : G_{A4}]$. Next, we analyze $\Pr[\text{BAD} : G_{A4}]$. We consider the following cases.

Case 1: The responder session π^{Resp} accepts, but the tuple $(\text{id}_A = P^*, \text{c}_{\text{seed}}^A)$ sent by π^{Init} is different from the tuple $(\text{id}'_A, \text{c}'_{\text{seed}}^A)$ received by π^{Resp} . We consider the following two subcases.

- **Case 1.a:** If $\text{id}_A \neq \text{id}'_A$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_{A4} for $\mathcal{A}$, except that it first guesses the id'_A uniformly over the N parties. Notably, $\mathcal{C}$ guesses correctly with probability at least $1/N$. Then $\mathcal{C}$ sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and $\text{pk}_{\text{id}'_A} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into the session π^{Init} . When π^{Resp} receives $(\text{id}'_A, \text{ct}_B, \text{c}'_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts $\text{c}'_{\text{seed}}^A$ and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{verf}}(\text{id}'_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}'_A} = \overline{\text{pk}}_s^*$, we have $\text{pk}'_A = \overline{\text{pk}}_s^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1.a}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

- **Case 1.b:** If $\text{id}_A = \text{id}'_A$ and $c_{\text{seed}}^A \neq c'_{\text{seed}}^A$, we also construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_{A4} for $\mathcal{A}$. $\mathcal{C}$ then sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π^{Init} . When π^{Resp} receives $(\text{id}_A, \text{ct}_B, c'_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts c'_{seed}^A and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}'_e)$. Since $\text{CA}_{\text{Verf}}(\text{id}_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}_A} = \text{pk}_s^*$, we have $\text{pk}'_A = \text{pk}_s^*$. Moreover, since $c_{\text{seed}}^A \neq c'_{\text{seed}}^A$ and the OTP key used to recover the seed is the same, we have $\text{seed}'_A \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case, then $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 1.b}] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Case 2: The initiator session π^{Init} accepts but π^{Resp} is not the matching session of π^{Init} . First, suppose that π^{Resp} aborts, i.e., it does not accept its session key, and denote this event by $[\text{BAD} : \text{Case 2, abort}]$. Then the corresponding encryption key $\hat{K}_{\text{ENC}}^B$ is uniform from the adversary's view and is never used in any other session. Thus, the value $\text{seed}'_B = \hat{K}_{\text{ENC}}^B \oplus c_{\text{seed}}^B$ computed in π^{Init} is uniform from the adversary's view. In this case, we construct an adversary $\mathcal{D}$ against the RSPK-Bind security of AFS-KEM as follows.

Adversary $\mathcal{D}$ receives $(\text{pk}_s^*, \text{sk}_s^*)$ and perfectly simulates Game G_{A4} for $\mathcal{A}$, except that it first guesses the peer identity B^* of π^{Init} , with success probability at least $1/N$. Then $\mathcal{D}$ sets $\text{pk}_{B^*} = \text{pk}_s^*$ and outputs the composite public key cpk_B in session π^{Init} to its challenger. After receiving seed_e from the challenger, $\mathcal{D}$ defines $K_{\text{ENC}}^B = \text{seed}_e \oplus c_{\text{seed}}^B$, where c_{seed}^B is received in π^{Init} . Let $(\text{pk}'_e, \text{sk}'_e) \leftarrow \text{EKeyGen}_c(\text{cpk}_B, \text{seed}_e)$ and $\text{pk}'_s \leftarrow \text{ExtractPK}(\text{cpk}_B, \text{pk}'_e)$. Since π^{Init} accepts, the extracted public key is accepted for the peer identity B^* , i.e., $\text{CA}_{\text{Verf}}(\text{id}_{B^*}, \text{pk}'_s) = 1$. Since $\text{pk}_{B^*} = \text{pk}_s^*$, we have $\text{pk}'_s = \text{pk}_s^*$ by the security of CA. Therefore, if the BAD event occurs in this case and $\mathcal{D}$ guesses correctly, then $\mathcal{D}$ wins the RSPK-Bind game. Consequently, $\Pr[\text{BAD} : \text{Case 2, abort}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$.

It remains to consider the case where π^{Resp} accepts. In this case, BAD implies that the adversary has tampered with either the third or the fourth protocol message between π^{Init} and π^{Resp} , but both sessions accept. Specifically, the tuple $(\text{id}_A, c_{\text{seed}}^A, \text{id}_B, c_{\text{seed}}^B)$ maintained by π^{Init} differs from $(\text{id}'_A, c'_{\text{seed}}^A, \text{id}'_B, c'_{\text{seed}}^B)$ maintained by π^{Resp} . We consider the following four subcases.

- **Case 2.a:** If $\text{id}_A \neq \text{id}'_A$, then similarly to Case 1.a, we can construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM such that $\Pr[\text{BAD} : \text{Case 2.a}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2.b:** If $\text{id}_B \neq \text{id}'_B$, then a similar SPK-Bind reduction to that in Case 1.a applies. Notably, in this case, the reduction needs to guess both id_B and id'_B . Hence, $\Pr[\text{BAD} : \text{Case 2.b}] \leq N^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2.c:** If $\text{id}_A = \text{id}'_A$ and $c_{\text{seed}}^A \neq c'_{\text{seed}}^A$, then the same SPK-Bind reduction as in Case 1.b applies. Hence, $\Pr[\text{BAD} : \text{Case 2.c}] \leq \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2.d:** If $\text{id}_B = \text{id}'_B$ and $c_{\text{seed}}^B \neq c'_{\text{seed}}^B$, then the same SPK-Bind reduction as in Case 1.b applies. Notably, in this case, the reduction needs to guess id_B . Hence, $\Pr[\text{BAD} : \text{Case 2.d}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

For simplicity, we reuse the notation $\mathcal{C}$ for the SPK-Bind reductions above. Combining the analysis for Case 1 and Case 2, we obtain $\Pr[\text{BAD} : G_{A4}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$. Therefore,

$$|\text{Adv}^{G_{A4}}(\mathcal{A}) - \text{Adv}^{G_{A5}}(\mathcal{A})| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

$$|\Pr[\text{QUERY} : G_{A4}] - \Pr[\text{QUERY} : G_{A5}]| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Recall that $\text{Adv}^{G_{A5}}(\mathcal{A}) = 0$. It remains to analyze $\Pr[\text{QUERY} : G_{A5}]$.

Game G_{A6} (Randomized c_{seed}^A). In this game, we modify the computation of $c_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \text{seed}_A$ in π^{Init} by replacing it with $\hat{c}_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \hat{\text{seed}}_A$, where $\hat{\text{seed}}_A \leftarrow \{0,1\}^\lambda$ is sampled uniformly and independently of seed_A . Importantly, after Game G_{A5} , the responder session π^{Resp} accepts the session key only if it receives the honest third message from π^{Init} ; otherwise, it aborts. Since K_{ENC}^A is uniformly random and independent of the adversary's view in Game G_{A5} , the values $K_{\text{ENC}}^A \oplus \text{seed}_A$ and $K_{\text{ENC}}^A \oplus \hat{\text{seed}}_A$ are identically distributed from the adversary's view. Hence, Games G_{A5} and G_{A6}

are identical from the adversary's view. Therefore,

$$\Pr[\text{QUERY} : G_{A5}] = \Pr[\text{QUERY} : G_{A6}].$$

Game G_{A7} (Random K_A^{Test}). In this game, we replace the KEM key K_A^{Test} in π and π' with a uniformly random key. To bound $|\Pr[\text{QUERY} : G_{A6}] - \Pr[\text{QUERY} : G_{A7}]|$, we construct an IND-ECPA adversary $\mathcal{B}_2$ against AFS-KEM. Specifically, $\mathcal{B}_2$ receives $(pk_s^*, pk_e^*, cpk^*, sk_s^*, ct^*, K_b^*)$ from its challenger. Adversary $\mathcal{B}_2$ honestly simulates the view of $\mathcal{A}$ as in Game G_{A6} , using the party P^* guessed in Game G_{A3} . It sets pk_s^* as the public key of P^* and embeds $cpk_A = cpk^*$ in π^{Init} and $ct_A = ct^*$ in π^{Resp} . Moreover, $\mathcal{B}_2$ uses K_b^* as the KEM key K_A in both π and π' . When $b = 0$, $\mathcal{B}_2$ perfectly simulates Game G_{A6} , and when $b = 1$, $\mathcal{B}_2$ perfectly simulates Game G_{A7} . That is, $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 0] = \Pr[\text{QUERY} : G_{A6}]$ and $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 1] = \Pr[\text{QUERY} : G_{A7}]$. Therefore,

$$|\Pr[\text{QUERY} : G_{A6}] - \Pr[\text{QUERY} : G_{A7}]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2).$$

Analysis of $\Pr[\text{QUERY} : G_{A7}]$. In this game, the key K_A^{Test} is independent of ct and cpk in sessions π and π' , and is uniformly random from the adversary's view. It follows that

$$\Pr[\text{QUERY} : G_{A7}] \leq \frac{q_G}{|\mathcal{K}|}.$$

Combining all the bounds derived above completes the proof of Case A.

Case B: the adversary never issues $\text{RevealEph}(\pi^{\text{Resp}})$.

The analysis of Case B is analogous to that of Case A. We therefore only provide a proof sketch highlighting the differences.

Game G_{B1} (KEM Correctness). In this game, we abort if the decapsulation of ct_A or ct_B in π^{Init} does not equal the corresponding encapsulated key. By an argument analogous to that for Game G_{A1} , we obtain $|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_{B1}}(\mathcal{A})| \leq 2NS^2\delta$.

Game G_{B2} (Key Collision). In this game, let $K_B^{\text{Test}} = \pi^{\text{Resp}}.K_B$. We define the event COLL as the existence of a session $\hat{\pi} \notin \{\pi, \pi'\}$ such that $\hat{\pi}.K_B = K_B^{\text{Test}}$. Game G_{B2} is identical to Game G_{B1} , except that it aborts whenever COLL occurs. By an argument analogous to that for Game G_{A2} , we have $|\text{Adv}^{G_{B1}}(\mathcal{A}) - \text{Adv}^{G_{B2}}(\mathcal{A})| \leq 2NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F})$.

Game G_{B3} (Guessing π^{Test} , $\pi^{\text{PartialMatch}}$, and Actor). In this game, we guess the test session π , its corresponding PartialMatch session π' , and the identity of $\pi^{\text{Resp}}.\text{actor}$. Concretely, we sample two session instances π and π' uniformly from the S sessions, and sample a party P^* uniformly from the N parties. The game aborts if $\pi \neq \pi^{\text{Test}}$ or $\pi' \neq \pi^{\text{PartialMatch}}$, or $\pi^{\text{Test}}.\text{role} = \text{Resp}$ and $P^* \neq \pi^{\text{Test}}.\text{actor}$, or if $\pi^{\text{Test}}.\text{role} = \text{Init}$ and $P^* \neq \pi^{\text{PartialMatch}}.\text{actor}$. Equivalently, P^* is a guess for $\pi^{\text{Resp}}.\text{actor}$. The probability of guessing correctly is $1/(NS^2)$. Therefore,

$$\text{Adv}^{G_{B2}}(\mathcal{A}) \leq NS^2 \cdot \text{Adv}^{G_{B3}}(\mathcal{A}).$$

Game G_{B4} (Random $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$). This game applies the same modification as in Game G_{A4} . Specifically, we replace $K_{\text{SESSION}}, K_{\text{ENC}}^A$, and K_{ENC}^B in the guessed sessions π and π' with independently sampled uniform values. Define QUERY as the event that $\mathcal{A}$ queries G on input $(K_A^{\text{Test}}, K_B^{\text{Test}})$, where K_A^{Test} and K_B^{Test} denote the encapsulated keys K_A and K_B in sessions π and π' , respectively. Then Games G_{B3} and G_{B4} are identical unless QUERY occurs. Therefore, $|\text{Adv}^{G_{B3}}(\mathcal{A}) - \text{Adv}^{G_{B4}}(\mathcal{A})| \leq \Pr[\text{QUERY} : G_{B4}]$.

Game G_{B5} (Abort on BAD). This game applies the same modification as in Game G_{A5} . We define the event BAD as follows. The event BAD occurs if either (i) the responder session π^{Resp} accepts the session key but the tuple $(id_A, c_{\text{seed}}^A)$ sent by π^{Init} is different from $(id'_A, c'_{\text{seed}}^A)$ received by π^{Resp} , or (ii) the initiator session π^{Init} accepts the session key but π^{Resp} is not the matching session of π^{Init} . If BAD occurs, the game aborts. By an argument analogous to that for Game G_{A5} , we obtain $|\text{Adv}^{G_{B4}}(\mathcal{A}) - \text{Adv}^{G_{B5}}(\mathcal{A})| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$. Similarly, $|\Pr[\text{QUERY} : G_{B4}] - \Pr[\text{QUERY} : G_{B5}]| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

At this point, the session key K_{SESSION} of the tested session π is uniformly random from the adversary's view. Moreover, there may be one accepted session using the same session key, but such a session must be the matching session of π . All session keys held by other accepted sessions are independent of K_{SESSION} . Therefore, $\text{Adv}^{G_{B5}}(\mathcal{A}) = 0$. It remains to bound $\Pr[\text{QUERY} : G_{B5}]$.

Game G_{B6} (Randomized c_{seed}^B). In this game, we modify the computation of $c_{\text{seed}}^B = K_{\text{ENC}}^B \oplus \text{seed}_B$ in π^{Resp} by replacing it with $\hat{c}_{\text{seed}}^B = K_{\text{ENC}}^B \oplus \hat{\text{seed}}_B$, where $\hat{\text{seed}}_B \leftarrow \{0, 1\}^\lambda$ is sampled uniformly and independently of seed_B . Importantly, after Game G_{B5} , the initiator session π^{Init} accepts the session key only if π^{Resp} is its matching session; otherwise, π^{Init} aborts. Since K_{ENC}^B is uniformly random and independent of the adversary's view in Game G_{B5} , the values $K_{\text{ENC}}^B \oplus \text{seed}_B$ and $K_{\text{ENC}}^B \oplus \hat{\text{seed}}_B$ are identically distributed from the adversary's view. Hence, Games G_{B5} and G_{B6} are identical from the adversary's view. Therefore, $\Pr[\text{QUERY} : G_{B5}] = \Pr[\text{QUERY} : G_{B6}]$.

Game G_{B7} (Random K_B^{Test}). In this game, we replace the KEM key K_B^{Test} in π and π' with a uniformly random key. By an argument analogous to that for Game G_{A7} , using the party P^* guessed in Game G_{B3} , we have

$$|\Pr[\text{QUERY} : G_{B6}] - \Pr[\text{QUERY} : G_{B7}]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2).$$

Analysis of $\Pr[\text{QUERY} : G_{B7}]$. In this game, the key K_B^{Test} is uniformly random and independent of ct and cpk in sessions π and π' . Therefore, we have

$$\Pr[\text{QUERY} : G_{B7}] \leq \frac{q_G}{|\mathcal{K}|}.$$

Combining all the bounds derived above completes the proof of Case B. $\square$

Lemma 11.4 (QROM counterpart of Lemma 11.2). *With the notation and assumptions of Theorem 9.3, and modeling PRF as a quantum-accessible random oracle $\mathcal{G}$ to which the adversary makes at most q_G quantum queries, there exist QPT adversaries $\mathcal{B}_2, \mathcal{C}, \mathcal{D}$, and $\mathcal{F}$, running in time comparable to that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) &\leq 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 2NS^2 \left(2\sqrt{(q_G + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_G}{|\mathcal{K}|} + \beta \right)} + \beta \right). \\ \text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) &\leq 4NS^2\delta + 4NS^3 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \\ &\quad + 2NS^2 \left(2\sqrt{(q_G + 1) \left(2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2) + \frac{4q_G}{|\mathcal{K}|} + \beta \right)} + \beta \right). \end{aligned}$$

where $\beta = N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N + 1)(N + 2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Proof. Similar to the ROM analysis, we consider $\text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$ and assume that the tested session satisfies $\text{FM}_3 \vee \text{FM}_4$; otherwise, the adversary trivially loses the game. In particular, it holds that $\text{Adv}_{\text{pMAKE}}^{\text{FM}_3\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$ and $\text{Adv}_{\text{pMAKE}}^{\text{FM}_4\text{-KS}}(\mathcal{A}) \leq \text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$. In the following, we mainly highlight the games where the analysis deviates from the ROM proof (Lemma 11.2).

Game G_0 . This game corresponds to the original game defining $\text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A})$. Thus,

$$\text{Adv}_{\text{pMAKE}}^{(\text{FM}_3 \vee \text{FM}_4)\text{-KS}}(\mathcal{A}) = \text{Adv}^{G_0}(\mathcal{A}).$$

Case Distinction. Let π denote the tested session and let π' denote its (unique) `PartialMatch` session. We further write π^{Init} and π^{Resp} to denote the initiator and responder sessions among $\{\pi, \pi'\}$, respectively. Next, we consider the following two cases in Game G_0 . Case A (denoted by G_A):

the adversary never issues $\text{RevealEph}(\pi^{\text{Init}})$. Case B (denoted by G_B): the adversary never issues $\text{RevealEph}(\pi^{\text{Resp}})$. Accordingly, we can bound the adversary's success probability by

$$\text{Adv}^{G_0}(\mathcal{A}) \leq \text{Adv}^{G_A}(\mathcal{A}) + \text{Adv}^{G_B}(\mathcal{A}).$$

Case A: the adversary never issues $\text{RevealEph}(\pi^{\text{Init}})$.

Game $G_{A1} - G_{A3}$. The games $G_{A1} - G_{A3}$ are the same as Games $G_{A1} - G_{A3}$ in the proof of Lemma 11.2.

Game G_{A4} (Random $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$). In this game, the random oracle G accessed by $\mathcal{A}$ is replaced by an oracle G' defined as follows. Let $\bar{G}$ be an independently sampled random oracle, and let K_A^{Test} denote the K_A value in the sessions π and π' . For each $K_B \in \mathcal{K}$, we define $G'(K_A^{\text{Test}}, K_B) = \bar{G}(K_A^{\text{Test}}, K_B)$ and set $G'(x) = G(x)$ for all other inputs x . Notably, the challenger still uses $G(K_A^{\text{Test}}, \cdot)$ in π and π' to compute $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B)$.

Let $z = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, K^*)$ where $(\text{pk}^*, \text{sk}^*) \leftarrow \$ \text{SKeyGen}(1^\lambda)$, $\text{seed}^* \leftarrow \$ \{0, 1\}^\lambda$, $(\text{pk}_e^*, \text{sk}_e^*) \leftarrow \text{EKeyGen}(\text{pk}^*, \text{seed}^*)$, $\text{cpk}^* \leftarrow \text{CombPK}(\text{pk}^*, \text{pk}_e^*)$, and $(\text{ct}^*, K^*) \leftarrow \$ \text{Encaps}_{\text{AFS}}(\text{cpk}^*)$. Sample an independent random oracle $\bar{G}$, and define G' as above. Define $S^* = \{(K^*, K_B) \mid K_B \in \mathcal{K}\}$. For every $x \in S^*$, define $G(x) = \bar{G}(x)$ and set $G(x) = G'(x)$ for all other inputs $x \notin S^*$. Let $A_{G_{A4}}^{[O]}(z)$ be an oracle algorithm that perfectly simulates the game G_{A4} for $\mathcal{A}$, except that it first uses the random oracle O to simulate the random oracle accessed by $\mathcal{A}$. Moreover, $\mathcal{A}$ uses pk^* as the long-term public key of π^{Init} .actor, and embeds $\text{seed}_A = \text{seed}^*$, $\text{ct}_A = \text{ct}^*$, and $K_A = K^*$ into the sessions π^{Init} and π^{Resp} , and finally returns 1 if $\mathcal{A}$ wins the game. Thus, we have $\text{Adv}^{G_{A3}}(\mathcal{A}) = \Pr[A_{G_{A4}}^G(z) \Rightarrow 1]$ and $\text{Adv}^{G_{A4}}(\mathcal{A}) = \Pr[A_{G_{A4}}^{G'}(z) \Rightarrow 1]$.

Lemma 5.9 states that $|\Pr[A_{G_{A4}}^G(z) \Rightarrow 1] - \Pr[A_{G_{A4}}^{G'}(z) \Rightarrow 1]| \leq 2\sqrt{(q_G + 1) \Pr[\text{FIND} : A_{G_{A4}}^{G' \setminus S^*}(z)]}$. Here $\Pr[\text{FIND} : A_{G_{A4}}^{G' \setminus S^*}(z)]$ denotes the probability that the oracle $\mathcal{O}_{S^*}^{G'}$ ever outputs 1 during the execution of $A_{G_{A4}}^{G' \setminus S^*}(z)$. Therefore,

$$|\text{Adv}^{G_{A3}}(\mathcal{A}) - \text{Adv}^{G_{A4}}(\mathcal{A})| \leq 2\sqrt{(q_G + 1) \Pr[\text{FIND} : A_{G_{A4}}^{G' \setminus S^*}(z)]}.$$

Game G_{A5} (Abort on BAD). This game applies the same modification as in Game G_{A5} of Lemma 11.2. We define the event BAD as follows: BAD occurs if either (i) the responder session π^{Resp} accepts but $(\pi^{\text{Init}}.\text{id}_A, \pi^{\text{Init}}.\text{c}_{\text{seed}}^A) \neq (\pi^{\text{Resp}}.\text{id}_A, \pi^{\text{Resp}}.\text{c}_{\text{seed}}^A)$, or (ii) the initiator session π^{Init} accepts but π^{Resp} is not the matching session of π^{Init} . If the event BAD occurs, the game aborts. At this point, the session key K_{SESSION} of the tested session π is uniformly random from the view of $\mathcal{A}$, and we have

$$\text{Adv}^{G_{A5}}(\mathcal{A}) = 0.$$

If the BAD event does not occur, games G_{A4} and G_{A5} are identical. Therefore, we have $|\text{Adv}^{G_{A4}}(\mathcal{A}) - \text{Adv}^{G_{A5}}(\mathcal{A})| \leq \Pr[\text{BAD} : G_{A4}]$. Similarly, we have $|\Pr[\text{FIND} : A_{G_{A4}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{A5}}^{G' \setminus S^*}(z)]| \leq \Pr[\text{BAD} : G_{A4}]$. By an argument analogous to that for Game G_{A5} in Lemma 11.2, we obtain $\Pr[\text{BAD} : G_{A4}] \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$. Therefore,

$$|\text{Adv}^{G_{A4}}(\mathcal{A}) - \text{Adv}^{G_{A5}}(\mathcal{A})| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}),$$

$$|\Pr[\text{FIND} : A_{G_{A4}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{A5}}^{G' \setminus S^*}(z)]| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G_{A6} (Randomized c_{seed}^A). In this game, we modify the computation of $c_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \text{seed}_A$ in π^{Init} by replacing it with $\hat{c}_{\text{seed}}^A = K_{\text{ENC}}^A \oplus \hat{\text{seed}}_A$, where $\hat{\text{seed}}_A \leftarrow \$ \{0, 1\}^\lambda$ is sampled uniformly and independently of seed_A . Importantly, after Game G_{A5} , the responder session π^{Resp} accepts the session key only if it receives the honest third message from π^{Init} , otherwise π^{Resp} aborts. Since K_{ENC}^A is derived from the hidden value $G(K_A^{\text{Test}}, K_B^{\text{Test}})$, while $\mathcal{A}$ only has access to $G' \setminus S^*$, the key K_{ENC}^A is uniformly random and independent of the adversary's view in the execution of $A_{G_{A5}}^{G' \setminus S^*}(z)$. Therefore, the values $K_{\text{ENC}}^A \oplus \text{seed}_A$ and $K_{\text{ENC}}^A \oplus \hat{\text{seed}}_A$ are identically distributed from the adversary's view. Hence, the

executions $A_{G_{A5}}^{G' \setminus S^*}(z)$ and $A_{G_{A6}}^{G' \setminus S^*}(z)$ induce identical distributions on all variables relevant to the event FIND. Therefore,

$$\Pr[\text{FIND} : A_{G_{A5}}^{G' \setminus S^*}(z)] = \Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus S^*}(z)].$$

Game G'_{A6} (Random K^*). It remains to bound $\Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus S^*}(z)]$. In this game, we replace the K^* in z and S^* for $A_{G_{A6}}^{G' \setminus S^*}(z)$ with a uniformly random $\hat{K}^*$. That is, let $\hat{z} = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, \hat{K}^*)$ and $\hat{S}^* = \{(\hat{K}^*, K_B) \mid K_B \in \mathcal{K}\}$. To bound $|\Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})]|$, we construct an IND-ECPA adversary $\mathcal{B}_2$ against AFS-KEM as follows. Specifically, $\mathcal{B}_2$ receives $(\text{pk}_s^*, \text{pk}_e^*, \text{cpk}^*, \text{sk}_s^*, \text{ct}^*, K_b^*)$ from its challenger. Adversary $\mathcal{B}_2$ honestly simulates the view of $\mathcal{A}$ as $A_{G_{A6}}^{G' \setminus S^*}(z)$, except that it sets pk_s^* as the public key of P^* and embeds $\text{cpk}_A = \text{cpk}^*$ and $\text{ct}_A = \text{ct}^*$ into sessions π and π' , respectively. Moreover, $\mathcal{B}_2$ uses K_b^* as the KEM key K_A in both π and π' , samples a quantum random oracle G' , and uses the oracle $G' \setminus S_b^*$ to answer adversary (quantum) RO queries, where $S_b^* = \{(K_b^*, K_B) \mid K_B \in \mathcal{K}\}$. If the FIND event occurs, $\mathcal{B}_2$ returns 1. When $b = 0$, we have $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 0] = \Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus S^*}(z)]$. When $b = 1$, we have $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 1] = \Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})]$. Therefore,

$$|\Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECPA}}(\mathcal{B}_2).$$

Analysis of $\Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})]$. Recall that $\hat{z} = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, \hat{K}^*)$, and that $A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})$ still uses the value $G(\hat{K}^*, \cdot)$ to derive $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$ for the sessions π and π' , while the adversary $\mathcal{A}$ does not access G . Therefore, we define an algorithm $\hat{A}_{G_{A6}}^{G' \setminus \hat{S}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)$ that is identical to $A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})$, except that it samples $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$ uniformly at random for π and π' (instead of computing them via $G(\hat{K}^*, \cdot)$). It is immediate that we have $\hat{A}_{G_{A6}}^{G' \setminus \hat{S}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*)$ and $A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})$ induce identical views for $\mathcal{A}$. Hence, $\Pr[\text{FIND} : A_{G_{A6}}^{G' \setminus \hat{S}^*}(\hat{z})] = \Pr[\text{FIND} : \hat{A}_{G_{A6}}^{G' \setminus \hat{S}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)]$. Note that the test key $\hat{S}^*$ is independent of $\text{pk}^*, \text{sk}^*, \text{seed}^*$, and is uniformly random from the adversary's view. By Lemma 5.10, it follows that

$$\Pr[\text{FIND} : \hat{A}_{G_{A6}}^{G' \setminus \hat{S}^*}(\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*)] \leq \frac{4q_G}{|\mathcal{K}|}.$$

Combining all the bounds derived above completes the proof of Case A.

Case B: the adversary never issues $\text{RevealEph}(\pi^{\text{Resp}})$. In Case B, we can obtain the same bound as in Case A. The analysis of Case B closely follows that of Case A. For completeness, we provide a proof sketch that highlights the only points where the two cases differ.

Game $G_{B1} - G_{B3}$. The games $G_{B1} - G_{B3}$ are identical to the corresponding games $G_{B1} - G_{B3}$ in the ROM analysis (Lemma 11.2) of Case B.

Game G_{B4} (Random $K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B$). This game applies a similar modification as in Game G_{A4} , except that we replace K_{TEST}^A with K_{TEST}^B . More specifically, let K_{TEST}^B denote the encapsulated key K_B in sessions π and π' . We define a modified random oracle G' as follows. Let $\bar{G}$ be an independently sampled random oracle. For every $K_A \in \mathcal{K}$, define $G'(K_A, K_{\text{TEST}}^B) = \bar{G}(K_A, K_{\text{TEST}}^B)$, and set $G'(x) = G(x)$ for all other inputs x . Notably, the challenger still uses $G(\cdot, K_{\text{TEST}}^B)$ in sessions π and π' to compute $(K_{\text{SESSION}}, K_{\text{ENC}}^A, K_{\text{ENC}}^B)$.

Let $z = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, K^*)$ be defined analogously as in Game G_{A4} . Define $S^* = \{(K_A, K^*) \mid K_A \in \mathcal{K}\}$. For every $x \in S^*$, define $G(x) = \bar{G}(x)$ and set $G(x) = G'(x)$ for all other inputs $x \notin S^*$. Let $A_{G_{B4}}^{(O)}(z)$ be an oracle algorithm that perfectly simulates the game G_{B4} for $\mathcal{A}$, except that it first uses the random oracle O to simulate the random oracle accessed by $\mathcal{A}$. Moreover, $\mathcal{A}$ uses pk^* as the long-term public key of $\pi_{\text{Resp}}^{\text{actor}}$, and embeds $\text{seed}_B = \text{seed}^*$, $\text{ct}_B = \text{ct}^*$, and $K_B = K^*$ into the sessions π^{Init} and π^{Resp} , and finally returns 1 if $\mathcal{A}$ wins the game. Thus, we have $\text{Adv}^{G_{B3}}(\mathcal{A}) = \Pr[A_{G_{B4}}^G(z) \Rightarrow 1]$ and $\text{Adv}^{G_{B4}}(\mathcal{A}) = \Pr[A_{G_{B4}}^{G'}(z) \Rightarrow 1]$. Using Lemma 5.9 with the same argument as in Case A, we have

$$|\text{Adv}^{G_{B3}}(\mathcal{A}) - \text{Adv}^{G_{B4}}(\mathcal{A})| \leq 2\sqrt{(q_G + 1) \Pr[\text{FIND} : A_{G_{B4}}^{G' \setminus S^*}(z)]}.$$

Game G_{B5} (Abort on BAD). This game applies the same modification as in Game G_{A5} . We define the event BAD as follows: BAD occurs if either (i) the responder session π^{Resp} accepts but $(\pi^{\text{Init}}.\text{id}_A, \pi^{\text{Init}}.\text{c}_{\text{seed}}^A) \neq (\pi^{\text{Resp}}.\text{id}_A, \pi^{\text{Resp}}.\text{c}_{\text{seed}}^A)$, or (ii) the initiator session π^{Init} accepts but π^{Resp} is not the matching session of π^{Init} . If the event BAD occurs, the game aborts. At this point, the session key K_{SESSION} of the tested session π is uniformly random from the view of $\mathcal{A}$, and we have

$$\text{Adv}^{G_{B5}}(\mathcal{A}) = 0.$$

Moreover, by an argument analogous to that for Game G_{A5} , we obtain

$$|\text{Adv}^{G_{B4}}(\mathcal{A}) - \text{Adv}^{G_{B5}}(\mathcal{A})| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 3N(N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}),$$

$$|\Pr[\text{FIND} : A_{G_{B4}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{B5}}^{G' \setminus S^*}(z)]| \leq N \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + 3N(N+1) \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Game G_{B6} (Randomized c_{seed}^B). In this game, we modify the computation of $c_{\text{seed}}^B = K_{\text{ENC}}^B \oplus \text{seed}_B$ in π^{Resp} by replacing it with $\hat{c}_{\text{seed}}^B = K_{\text{ENC}}^B \oplus \hat{\text{seed}}_B$, where $\hat{\text{seed}}_B \leftarrow \{0, 1\}^\lambda$ is sampled uniformly and independently of seed_B . Importantly, after Game G_{B5} , the initiator session π^{Init} accepts the session key if and only if it is the matching session of π^{Resp} ; otherwise, π^{Init} aborts. In particular, π^{Init} does not need to recover the seed from the received value c_{seed}^B in any execution relevant to the event FIND. Since K_{ENC}^B is derived from the hidden value $G(K_A^{\text{Test}}, K_B^{\text{Test}})$, while $\mathcal{A}$ only has access to $G' \setminus S^*$, the key K_{ENC}^B is uniformly random and independent of the adversary's view in the execution of $A_{G_{B5}}^{G' \setminus S^*}(z)$. Therefore, the values $K_{\text{ENC}}^B \oplus \text{seed}_B$ and $K_{\text{ENC}}^B \oplus \hat{\text{seed}}_B$ are identically distributed from the adversary's view. Hence, the executions $A_{G_{B5}}^{G' \setminus S^*}(z)$ and $A_{G_{B6}}^{G' \setminus S^*}(z)$ induce identical distributions on all variables relevant to the event FIND. Therefore,

$$\Pr[\text{FIND} : A_{G_{B5}}^{G' \setminus S^*}(z)] = \Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus S^*}(z)].$$

Game G'_{B6} (Random K^*). In this game, we replace the K^* in z and S^* for $A_{G_{B6}}^{G' \setminus S^*}(z)$ with a uniformly random $\hat{K}^*$. That is, let $\hat{z} = (\text{pk}^*, \text{sk}^*, \text{seed}^*, \text{ct}^*, \hat{K}^*)$ and $\hat{S}^* = \{(K_A, \hat{K}^*) \mid K_A \in \mathcal{K}\}$. To bound $|\Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus \hat{S}^*}(\hat{z})]|$, we construct an IND-ECOA adversary $\mathcal{B}_2$ against AFS-KEM as follows. Specifically, $\mathcal{B}_2$ receives $(\text{pk}_s^*, \text{pk}_e^*, \text{cpk}_s^*, \text{sk}_s^*, \text{ct}^*, K_b^*)$ from its challenger. Adversary $\mathcal{B}_2$ honestly simulates the view of $\mathcal{A}$ as $A_{G_{B6}}^{G' \setminus S^*}(z)$, except that it sets pk_s^* as the public key of P^* and embeds $\text{cpk}_B = \text{cpk}^*$ and $\text{ct}_B = \text{ct}^*$ into sessions π and π' , respectively. Moreover, $\mathcal{B}_2$ uses K_b^* as the KEM key K_B in both π and π' , samples a quantum random oracle G' , and uses the oracle $G' \setminus S_b^*$ to answer adversary (quantum) RO queries, where $S_b^* = \{(K_A, K_b^*) \mid K_A \in \mathcal{K}\}$. If the FIND event occurs, $\mathcal{B}_2$ returns 1. When $b = 0$, we have $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 0] = \Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus S^*}(z)]$. When $b = 1$, we have $\Pr[\mathcal{B}_2 \Rightarrow 1 : b = 1] = \Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus \hat{S}^*}(\hat{z})]$. Therefore,

$$|\Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus S^*}(z)] - \Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus \hat{S}^*}(\hat{z})]| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-ECOA}}(\mathcal{B}_2).$$

Analysis of $\Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus \hat{S}^*}(\hat{z})]$. Since $\hat{K}^*$ is uniformly random from the adversary's view, Lemma 5.10 implies that

$$\Pr[\text{FIND} : A_{G_{B6}}^{G' \setminus \hat{S}^*}(\hat{z})] \leq \frac{4q_G}{|\mathcal{K}|}.$$

Combining all the bounds above completes the proof of Case B. $\square$

11.3 Explicit Authentication Analysis for pMAKE

Theorem 11.5. *Let there be at most N parties and at most S sessions activated in the Init-EA and Resp-EA games. Suppose that the underlying AFS-KEM is δ -correct, γ -spreadness, and satisfies perfect ExtractPK-correctness and ephemeral-key-generation equivalence. Then, for any polynomial-time adversary $\mathcal{A}$ attacking the Init-EA or Resp-EA security of pMAKE, there exist polynomial-time*

adversaries $\mathcal{B}$, $\mathcal{C}$, $\mathcal{D}$, $\mathcal{F}$, and $\mathcal{Z}$, constructed below and running in time comparable to that of $\mathcal{A}$, such that

$$\begin{aligned}\text{Adv}_{\text{pMAKE}}^{\text{Init-EA}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} + 2NS^2\delta \\ &\quad + NS \left(2\text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 4S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \right. \\ &\quad \left. + (N+1)(N+2)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 3\text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \cdot \\ \text{Adv}_{\text{pMAKE}}^{\text{Resp-EA}}(\mathcal{A}) &\leq \frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma} + 2NS^2\delta \\ &\quad + NS \left(2\text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}) + \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}) + 4S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}) \right. \\ &\quad \left. + (N+1)(N+2)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}) + 3\text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) \right) \cdot\end{aligned}$$

Proof. We give a unified analysis for Init-EA and Resp-EA. We begin by defining game G_0 , which is identical to the Init-EA game except for the win condition. Specifically, the adversary is said to win the experiment if the session π output by the adversary satisfies $\text{MalAcc}_{\text{Init}}(\pi) = \text{true}$ or $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$. It is immediate that $\text{Adv}_{\text{pMAKE}}^{\text{Init-EA}}(\mathcal{A}) \leq \text{Adv}^{G_0}(\mathcal{A})$ and $\text{Adv}_{\text{pMAKE}}^{\text{Resp-EA}}(\mathcal{A}) \leq \text{Adv}^{G_0}(\mathcal{A})$. Therefore, it suffices to bound $\text{Adv}^{G_0}(\mathcal{A})$.

We first assume that all honest users possess distinct long-term public keys, and that all honest sessions compute distinct combined public keys cpk , including cpk_A for initiator sessions and cpk_B for responder sessions. Moreover, we assume that every honest responder session that receives an honestly generated cpk_A from an honest initiator session computes a distinct ciphertext ct_A . Similarly, every honest initiator session that receives an honestly generated cpk_B from an honest responder session computes a distinct ciphertext ct_B .

Since long-term public keys have min-entropy at least k_1 , combined public keys generated honestly from a fixed long-term public key have min-entropy at least k_2 , and the ciphertexts of AFS-KEM satisfy γ -spreadness, these assumptions contribute $\frac{N^2}{2^{k_1}} + \frac{S^2}{2^{k_2}} + \frac{S^2}{2^\gamma}$ to the final bound. Under these assumptions, it is impossible for any session to have more than one **PartialMatch** session or more than one **Match** session.

This proof largely mirrors the analysis for the cases $\text{FM}_5 \vee \text{FM}_6$ in Lemma 11.3, except for the treatment of Game G_7 . In the following analysis, we assume that the session π output by the adversary satisfies $\text{MalAcc}_{\text{Init}}(\pi) = \text{true}$ or $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$, otherwise the adversary trivially loses the game.

Game G_1 (Abort if Collision in K_A or K_B). Let π denote the session output by the adversary in the game. Let π' denote the unique session, if it exists, such that $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) = (\pi.\text{cpk}_A, \pi.\text{ct}_A)$ when $\pi.\text{role} = \text{Resp}$, and such that $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) = (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ when $\pi.\text{role} = \text{Init}$.

In this game, we define the **COLL** event as follows. The event **COLL** occurs if π' exists and $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) \neq (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ (resp. $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) \neq (\pi.\text{cpk}_A, \pi.\text{ct}_A)$), but $\pi.K_B = \pi'.K_B$ (resp. $\pi.K_A = \pi'.K_A$), when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). If the **COLL** event occurs, we abort the game. Since Games G_0 and G_1 are identical unless **COLL** occurs, we have $|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq \Pr[\text{COLL} : G_0]$. Using the same argument as in Game G_1 of Lemma 11.3, we obtain

$$|\text{Adv}^{G_0}(\mathcal{A}) - \text{Adv}^{G_1}(\mathcal{A})| \leq 4NS^2 \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{Key-Coll}}(\mathcal{F}).$$

Game G_2 (KEM Correctness). In this game, we define the event **ERR**₁ to occur if the decapsulation of ct_A (resp. ct_B) in π' (if it exists) does not equal the corresponding encapsulated key when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). Moreover, if π' is further a **PartialMatch** session of π , we define the event **ERR**₂ to occur if the decapsulation of ct_B (resp. ct_A) in π' does not equal the corresponding encapsulated key when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). If **ERR**₁ or **ERR**₂ occurs, we abort the game. Using the same argument for δ -correctness property for AFS-KEM as in Game G_2 of Lemma 11.3, we obtain

$$|\text{Adv}^{G_1}(\mathcal{A}) - \text{Adv}^{G_2}(\mathcal{A})| \leq 2NS^2\delta.$$

Game G_3 (Guessing $\pi.\text{peer}$ and π). In this game, we guess the session π output by the adversary and its peer identity $\pi.\text{peer}$, and abort if either guess is incorrect. Formally, we sample P^* uniformly

from the N parties and sample one session instance π^* uniformly over the at most S sessions. If $P^* \neq \pi.\text{peer}$ or $\pi^* \neq \pi$, where π is the session output by the adversary, the game is aborted. Since the probability of guessing correctly is $1/(NS)$, we have

$$\text{Adv}^{G_2}(\mathcal{A}) \leq NS \cdot \text{Adv}^{G_3}(\mathcal{A}).$$

Game G_4 (New win condition). In this game, we modify the win condition as follows. The adversary $\mathcal{A}$ wins if and only if the guessed session π accepts and satisfies $\text{MalAcc}_{\text{Init}}(\pi) = \text{true}$ or $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$. The game is stopped at the moment when π accepts. Notably, the party P^* remains uncorrupted in G_4 . It is evident that any adversary that wins Game G_3 also wins Game G_4 . Thus,

$$\text{Adv}^{G_3}(\mathcal{A}) \leq \text{Adv}^{G_4}(\mathcal{A}).$$

Game G_5 (Random K_A or K_B). In this game, we replace the KEM key K_A (resp. K_B) in the tested session π with an independently uniform key $\tilde{k} \leftarrow \mathcal{K}$ if $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). Let π' denote the unique session, if it exists, such that $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) = (\pi.\text{cpk}_A, \pi.\text{ct}_A)$ when $\pi.\text{role} = \text{Resp}$, and such that $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) = (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ when $\pi.\text{role} = \text{Init}$. If such a session π' exists, we replace the corresponding KEM key in π' with the same key $\tilde{k}$. Using the same argument as in Game G_5 of Lemma 11.3, we obtain

$$|\text{Adv}^{G_4}(\mathcal{A}) - \text{Adv}^{G_5}(\mathcal{A})| \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{IND-LCK-SCCA}}(\mathcal{B}).$$

Game G_6 (Random K_{SESSION} , K_{ENC}^A , K_{ENC}^B). In this game, we replace the keys K_{SESSION} , K_{ENC}^A , and K_{ENC}^B in the session π with independently sampled uniform keys $\hat{K}_{\text{SESSION}}$, $\hat{K}_{\text{ENC}}^A$, and $\hat{K}_{\text{ENC}}^B$, respectively. If π' exists and is the **PartialMatch** session of π , we use the same uniform keys $\hat{K}_{\text{SESSION}}$, $\hat{K}_{\text{ENC}}^A$, and $\hat{K}_{\text{ENC}}^B$ in π' . Otherwise, if π' exists but is not a **PartialMatch** session of π , we use independently sampled uniform keys $\bar{K}_{\text{SESSION}}$, $\bar{K}_{\text{ENC}}^A$, and $\bar{K}_{\text{ENC}}^B$ in π' . Notably, after Game G_1 , if π' exists but is not a **PartialMatch** session of π , then we must have $(\pi'.\text{cpk}_B, \pi'.\text{ct}_B) \neq (\pi.\text{cpk}_B, \pi.\text{ct}_B)$ (resp. $(\pi'.\text{cpk}_A, \pi'.\text{ct}_A) \neq (\pi.\text{cpk}_A, \pi.\text{ct}_A)$), and $\pi.K_B \neq \pi'.K_B$ (resp. $\pi.K_A \neq \pi'.K_A$) when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$).

Using the same argument as in Game G_6 of Lemma 11.3, we can construct an adversary $\mathcal{Z}$ against the dual-PRF security of the PRF such that

$$|\text{Adv}^{G_5}(\mathcal{A}) - \text{Adv}^{G_6}(\mathcal{A})| \leq \text{Adv}_{\text{PRF}}^{\text{dual-PRF}}(\mathcal{Z}).$$

Game G_7 (Abort on BAD). In this game, we define the BAD event as follows: BAD occurs if the guessed session π satisfies $\text{MalAcc}_{\text{Init}}(\pi) = \text{true}$ or $\text{MalAcc}_{\text{Resp}}(\pi) = \text{true}$. If BAD occurs, we abort the game. Since every winning execution is aborted in Game G_7 , we have

$$\text{Adv}^{G_7}(\mathcal{A}) = 0.$$

It follows that $|\text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A})| \leq \Pr[\text{BAD} : G_6]$. We now analyze $\Pr[\text{BAD} : G_6]$. In Game G_6 , the keys $\hat{K}_{\text{ENC}}^A$ and $\hat{K}_{\text{ENC}}^B$ of the session π are uniform from the adversary's view. Moreover, if the **PartialMatch** session $\pi^{\text{PartialMatch}}$ of π exists, then it uses the same keys $\hat{K}_{\text{ENC}}^A$ and $\hat{K}_{\text{ENC}}^B$. We consider the following cases.

Case 1: The **PartialMatch** session $\pi^{\text{PartialMatch}}$ of π does not exist. Here, the keys $\hat{K}_{\text{ENC}}^A$ and $\hat{K}_{\text{ENC}}^B$ of the tested session π are uniform from the adversary's view and are never used in any other session. Therefore, $\text{seed}'_A = \hat{K}_{\text{ENC}}^A \oplus c_{\text{seed}}^A$ (resp. $\text{seed}'_B = \hat{K}_{\text{ENC}}^B \oplus c_{\text{seed}}^B$) computed in π is uniform from the adversary's view when $\pi.\text{role} = \text{Resp}$ (resp. $\pi.\text{role} = \text{Init}$). Using the same argument as in Game G_7 of Lemma 11.3, we can construct an adversary $\mathcal{D}$ against the RSPK-Bind security of AFS-KEM such that $\Pr[\text{BAD} : \text{Case 1}] \leq 2 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$.

Case 2: The unique **PartialMatch** session $\pi^{\text{PartialMatch}}$ of π exists and $\pi.\text{role} = \text{Resp}$. In this case, BAD implies that the adversary has tampered with the third protocol message. That is, $(\text{id}_A, c_{\text{seed}}^A)$ sent by the **PartialMatch** session differs from (P^*, c_{seed}^A) received by π .

- **Case 2.a:** If $\text{id}_A \neq P^*$, we construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_6 for $\mathcal{A}$, except that it first guesses the PartialMatch session $\pi^{\text{PartialMatch}}$ of π uniformly over the S sessions, and guesses the identity id_A uniformly over the N parties. Thus, $\mathcal{C}$ guesses correctly with probability at least $1/(NS)$. Then $\mathcal{C}$ sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into the session $\pi^{\text{PartialMatch}}$. When π receives $(P^*, \text{ct}_B, c_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts c_{seed}^A and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{Verf}}(P^*, \text{pk}'_A) = 1$ and $\text{pk}_{P^*} = \overline{\text{pk}}_s^*$, we have $\text{pk}'_A = \text{pk}_s^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2.a}] \leq NS \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 2.b:** If $\text{id}_A = P^*$ but $c_{\text{seed}}^A \neq c_{\text{seed}}^A$, we again construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_6 for $\mathcal{A}$, except that it first guesses $\pi^{\text{PartialMatch}}$ uniformly over the S sessions, with success probability at least $1/S$. $\mathcal{C}$ then sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into $\pi^{\text{PartialMatch}}$. When π receives $(\text{id}_A, \text{ct}_B, c_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts c_{seed}^A and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{Verf}}(\text{id}_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}_A} = \text{pk}_s^*$, we have $\text{pk}'_A = \text{pk}_s^*$. Moreover, since $c_{\text{seed}}^A \neq c_{\text{seed}}^A$ and the OTP key used to recover the seed is the same, we have $\text{seed}'_A \neq \text{seed}_e^*$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 2.b}] \leq S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

Case 3: The unique PartialMatch session $\pi^{\text{PartialMatch}}$ of π exists and $\pi.\text{role} = \text{Init}$. If the PartialMatch session aborts, then $\hat{K}_{\text{ENC}}^B$ is never used in any other session. Therefore, the value $\text{seed}_B = \hat{K}_{\text{ENC}}^B \oplus c_{\text{seed}}^B$ computed in π is uniform from the adversary's view. Similarly to the analysis in Case 1, this case can be bounded by $\text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D})$.

It remains to consider the case where $\pi^{\text{PartialMatch}}$ accepts. In this case, BAD implies that the adversary has tampered with either the third or the fourth protocol message between π and $\pi^{\text{PartialMatch}}$. Specifically, the tuple $(\text{id}_A, c_{\text{seed}}^A, \text{id}_B = P^*, c_{\text{seed}}^B)$ maintained by π differs from $(\text{id}'_A, c_{\text{seed}}^A, \text{id}'_B, c_{\text{seed}}^B)$ maintained by $\pi^{\text{PartialMatch}}$. We consider the following subcases.

- **Case 3.a:** If $\text{id}_A \neq \text{id}'_A$, this analysis is similar to Case 2.a. We construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_6 for $\mathcal{A}$, except that it first guesses the session $\pi^{\text{PartialMatch}}$ uniformly over the S sessions, and guesses the identities id_A and id'_A uniformly over the N parties. Notably, $\mathcal{C}$ guesses correctly with probability at least $1/(N^2S)$. Then $\mathcal{C}$ sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and $\text{pk}_{\text{id}'_A} = \overline{\text{pk}}_s^*$. Moreover, $\mathcal{C}$ embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into the session π . When $\pi^{\text{PartialMatch}}$ receives $(\text{id}'_A, \text{ct}_B, c_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts c_{seed}^A and outputs the corresponding seed'_A to its challenger. Similarly to Case 2.a, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins its static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 3.a}] \leq N^2S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 3.b:** If $\text{id}_B \neq \text{id}'_B$, this analysis is similar to Case 2.a. We also construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM. Since $\text{id}_B = P^*$ has already been guessed in G_3 , $\mathcal{C}$ only needs to guess $\pi^{\text{PartialMatch}}$ and id'_B , with success probability at least $1/(NS)$. Thus, $\Pr[\text{BAD} : \text{Case 3.b}] \leq NS \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.
- **Case 3.c:** If $\text{id}_A = \text{id}'_A$ and $c_{\text{seed}}^A \neq c_{\text{seed}}^A$, this analysis is similar to Case 2.b. We also construct an adversary $\mathcal{C}$ against the SPK-Bind security of AFS-KEM as follows. Adversary $\mathcal{C}$ receives the challenge tuple $(\text{cpk}^*, \text{seed}_e^*, \text{pk}_s^*, \text{sk}_s^*, \overline{\text{pk}}_s^*, \overline{\text{sk}}_s^*)$ and perfectly simulates Game G_6 for $\mathcal{A}$, except that it first guesses $\pi^{\text{PartialMatch}}$ uniformly over the S sessions and guesses id_A uniformly over the N parties, with success probability at least $1/(NS)$. $\mathcal{C}$ then sets $\text{pk}_{\text{id}_A} = \text{pk}_s^*$ and embeds $\text{seed}_A = \text{seed}_e^*$ and $\text{cpk}_A = \text{cpk}^*$ into π . When $\pi^{\text{PartialMatch}}$ receives $(\text{id}_A, \text{ct}_B, c_{\text{seed}}^A)$, $\mathcal{C}$ honestly decrypts c_{seed}^A and outputs the corresponding seed'_A to its challenger. Let $(\text{pk}_e^{A'}, \text{sk}_e^{A'}) \leftarrow \text{EKeyGen}_c(\text{cpk}^*, \text{seed}'_A)$ and $\text{pk}'_A \leftarrow \text{ExtractPK}(\text{cpk}^*, \text{pk}_e^{A'})$. Since $\text{CA}_{\text{Verf}}(\text{id}_A, \text{pk}'_A) = 1$ and $\text{pk}_{\text{id}_A} = \text{pk}_s^*$, we have $\text{pk}'_A = \text{pk}_s^*$. Moreover, since $c_{\text{seed}}^A \neq c_{\text{seed}}^A$ and the OTP key used to recover the seed is the same, we have

$\text{seed}'_A \neq \text{seed}^*_c$. Therefore, if the BAD event occurs in this case and $\mathcal{C}$ guesses correctly, then $\mathcal{C}$ wins the static public-key binding game. Hence, $\Pr[\text{BAD} : \text{Case 3.c}] \leq NS \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

- **Case 3.d:** If $\text{id}_B = \text{id}'_B$ and $c_{\text{seed}}^B \neq c_{\text{seed}}'^B$, the analysis is analogous to Case 3.c. Since $\text{id}_B = P^*$ has already been guessed, $\mathcal{C}$ only needs to guess $\pi^{\text{PartialMatch}}$, with success probability at least $1/S$. Thus, $\Pr[\text{BAD} : \text{Case 3.d}] \leq S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C})$.

For simplicity, we reuse the notation $\mathcal{C}$ for the SPK-Bind reductions in the above cases. Combining the above cases, we obtain

$$|\text{Adv}^{G_6}(\mathcal{A}) - \text{Adv}^{G_7}(\mathcal{A})| \leq 3 \text{Adv}_{\text{AFS-KEM}}^{\text{RSPK-Bind}}(\mathcal{D}) + (N+1)(N+2)S \cdot \text{Adv}_{\text{AFS-KEM}}^{\text{SPK-Bind}}(\mathcal{C}).$$

Combining all the bounds derived above completes the proof. $\square$

References

- ADPS16. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016: 25th USENIX Security Symposium*, pages 327–343, Austin, TX, USA, August 10–12, 2016. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>.
- AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 269–295, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-26951-7_10.
- Alb17. Martin R. Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 103–129, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-56614-6_4.
- AWK⁺25. Samy Amer, Yingchen Wang, Hunter Kippen, Thinh Dang, Daniel Genkin, Andrew Kwong, Alexander Nelson, and Arkady Yerukhimovich. PQ-hammer: End-to-end key recovery attacks on post-quantum cryptography using rowhammer. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 3567–3582, San Francisco, CA, USA, May 12–15, 2025. IEEE Computer Society Press. <https://doi.org/10.1109/SP61157.2025.00048>.
- BBBV97. Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997. <https://doi.org/10.1137/S0097539796300933>.
- BCCT12. Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 326–349, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery. <https://doi.org/10.1145/2090236.2090263>.
- BCD⁺16. Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1006–1018, Vienna, Austria, October 24–28, 2016. ACM Press. <https://doi.org/10.1145/2976749.2978425>.
- BCD⁺24. Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karoline Varner, and Bas Westerbaan. X-Wing. *IACR Communications in Cryptology (CiC)*, 1(1):21, 2024. <https://doi.org/10.62056/a3qj89n4e>.
- BCGP08. Colin Boyd, Yvonne Cliff, Juan González Nieto, and Kenneth G. Paterson. Efficient one-round key exchange in the standard model. In Yi Mu, Willy Susilo, and Jennifer Seberry, editors, *ACISP 08: 13th Australasian Conference on Information Security and Privacy*, volume 5107 of *Lecture Notes in Computer Science*, pages 69–83, Wollongong, Australia, July 7–9, 2008. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-540-70500-0_6.

- BCL⁺26. Hugo Beguinet, Céline Chevalier, Guirec Lebrun, Thomas Legavre, Thomas Ricosset, Maxime Roméas, and Éric Sageloli. DAKE: Bandwidth-efficient (u)AKE from double-KEM. In Shi Bai and Edoardo Persichetti, editors, *Public-Key Cryptography – PKC 2026*, volume 16554 of *Lecture Notes in Computer Science*. Springer, Cham, 2026. https://doi.org/10.1007/978-3-032-26740-5_1.
- BDF⁺11. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-25385-0_3.
- BDK⁺18. Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A cca-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 353–367. IEEE, 2018. <https://doi.org/10.1109/EUROSP.2018.00032>.
- Bel06. Mihir Bellare. New proofs for NMAC and HMAC: Security without collision-resistance. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 602–619, Santa Barbara, CA, USA, August 20–24, 2006. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/11818175_36.
- BHH⁺19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part II*, volume 11892 of *Lecture Notes in Computer Science*, pages 61–90, Nuremberg, Germany, December 1–5, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-36033-7_3.
- BMS19. Colin Boyd, Anish Mathuria, and Douglas Stebila. *Protocols for Authentication and Key Establishment*. Information Security and Cryptography. Springer, Berlin, Heidelberg, 2 edition, 2019. <https://doi.org/10.1007/978-3-662-58146-9>.
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93: 1st Conference on Computer and Communications Security*, pages 62–73, Fairfax, Virginia, USA, November 3–5, 1993. ACM Press. <https://doi.org/10.1145/168588.168596>.
- BR94. Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology – CRYPTO’93*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249, Santa Barbara, CA, USA, August 22–26, 1994. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/3-540-48329-2_21.
- BRS02. John Black, Phillip Rogaway, and Thomas Shrimpton. Encryption-scheme security in the presence of key-dependent messages. In Kaisa Nyberg and Howard M. Heys, editors, *Selected Areas in Cryptography, 9th Annual International Workshop, SAC 2002, St. John’s, Newfoundland, Canada, August 15-16, 2002. Revised Papers*, volume 2595 of *Lecture Notes in Computer Science*, pages 62–75. Springer, 2002. https://doi.org/10.1007/3-540-36492-7_6.
- BS20. Nina Bindel and John M. Schanck. Decryption failure is more likely after success. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 206–225, Paris, France, April 15–17, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-44223-1_12.
- CDM24. Cas Cremers, Alexander Dax, and Niklas Medinger. Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 1046–1060, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press. <https://doi.org/10.1145/3658644.3670283>.
- CF12. Cas J. F. Cremers and Michele Feltz. Beyond eCK: Perfect forward secrecy under actor compromise and ephemeral-key reveal. In Sara Foresti, Moti Yung, and Fabio Martinelli, editors, *ESORICS 2012: 17th European Symposium on Research in Computer Security*, volume 7459 of *Lecture Notes in Computer Science*, pages 734–751, Pisa, Italy, September 10–12, 2012. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-33167-1_42.
- CH98. David Carrel and Dan Harkins. The Internet Key Exchange (IKE). RFC 2409, November 1998. <https://doi.org/10.17487/RFC2409>.
- CK01. Ran Canetti and Hugo Krawczyk. Analysis of key-exchange protocols and their use for building secure channels. In Birgit Pfitzmann, editor, *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 453–474, Innsbruck, Austria, May 6–10, 2001. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/3-540-44987-6_28.

- CK02. Ran Canetti and Hugo Krawczyk. Security analysis of IKE’s signature-based key-exchange protocol. In Moti Yung, editor, *Advances in Cryptology – CRYPTO 2002*, volume 2442 of *Lecture Notes in Computer Science*, pages 143–161, Santa Barbara, CA, USA, August 18–22, 2002. Springer Berlin Heidelberg, Germany. URL: <https://eprint.iacr.org/2002/120/>, https://doi.org/10.1007/3-540-45708-9_10.
- CMST25. Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of MATZOV’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part I*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-032-01855-7_15.
- CN11. Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-25385-0_1.
- DB22. Jan-Pieter D’Anvers and Senne Batsleer. Multitarget decryption failure attacks and their application to Saber and Kyber. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13177 of *Lecture Notes in Computer Science*, pages 3–33, Virtual Event, March 8–11, 2022. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-97121-2_1.
- DDGR20. Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 329–358, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-56880-1_12.
- DFM20. Jelle Don, Serge Fehr, and Christian Majenz. The measure-and-reprogram technique 2.0: Multi-round fiat-shamir and more. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 602–631, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-56877-1_21.
- DFMS19. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 356–383, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-26951-7_13.
- DGJ⁺19. Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on IND-CCA secure lattice-based schemes. In Dongdai Lin and Kazue Sako, editors, *PKC 2019: 22nd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 11443 of *Lecture Notes in Computer Science*, pages 565–598, Beijing, China, April 14–17, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-17259-6_19.
- DH76. W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. <https://doi.org/10.1109/TIT.1976.1055638>.
- DHK⁺21. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, and Gregor Seiler. Faster lattice-based KEMs via a generic Fujisaki-Okamoto transform using prefix hashing. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2722–2737, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press. <https://doi.org/10.1145/3460120.3484819>.
- DJK⁺25. Yevgeniy Dodis, Daniel Jost, Shuichi Katsumata, Thomas Prest, and Rolfe Schmidt. Triple ratchet: A bandwidth efficient hybrid-secure signal protocol. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part VIII*, volume 15608 of *Lecture Notes in Computer Science*, pages 302–331, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-91101-9_11.
- DRV20. Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 3–33, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-45727-3_1.
- FO99. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/3-540-48405-1_34.

- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013. <https://doi.org/10.1007/s00145-011-9114-1>.
- FSXY12. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Strongly secure authenticated key exchange from factoring, codes, and lattices. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography*, volume 7293 of *Lecture Notes in Computer Science*, pages 467–484, Darmstadt, Germany, May 21–23, 2012. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-30057-8_28.
- FSXY13. Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Practical and post-quantum authenticated key exchange from one-way secure key encapsulation mechanism. In Kefei Chen, Qi Xie, Weidong Qiu, Ninghui Li, and Wen-Guey Tzeng, editors, *ASIACCS 13: 8th ACM Symposium on Information, Computer and Communications Security*, pages 83–94, Hangzhou, China, May 8–10, 2013. ACM Press. <https://doi.org/10.1145/2484313.2484323>.
- GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part I*, volume 13090 of *Lecture Notes in Computer Science*, pages 637–667, Singapore, December 6–10, 2021. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-92062-3_22.
- GMR89. Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989. <https://doi.org/10.1137/0218012>.
- HHK17. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-70500-2_12.
- HKNW25a. Keitaro Hashimoto, Shuichi Katsumata, Guilhem Niot, and Thom Wiggers. Revisiting PQ WireGuard: A comprehensive security analysis with a new design using reinforced KEMs. Cryptology ePrint Archive, Paper 2025/1758, 2025. To appear at IEEE Symposium on Security and Privacy (S&P) 2026. URL: <https://eprint.iacr.org/2025/1758>.
- HKNW25b. Keitaro Hashimoto, Shuichi Katsumata, Guilhem Niot, and Thom Wiggers. Revisiting PQ WireGuard: A comprehensive security analysis with a new design using reinforced KEMs. Cryptology ePrint Archive, Paper 2025/1758, 2025. URL: <https://eprint.iacr.org/2025/1758>.
- HKSU20. Kathrin Hövelmanns, Eike Kiltz, Sven Schäge, and Dominique Unruh. Generic authenticated key exchange in the quantum random oracle model. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 389–422, Edinburgh, UK, May 4–7, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-45388-6_14.
- HNS⁺21. Andreas Hülsing, Kai-Chun Ning, Peter Schwabe, Fiona Johanna Weber, and Philip R. Zimmermann. Post-quantum WireGuard. In *2021 IEEE Symposium on Security and Privacy*, pages 304–321, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press. <https://doi.org/10.1109/SP40001.2021.00030>.
- HRS16. Andreas Hülsing, Joost Rijneveld, and Fang Song. Mitigating multi-target attacks in hash-based signatures. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016: 19th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 9614 of *Lecture Notes in Computer Science*, pages 387–416, Taipei, Taiwan, March 6–9, 2016. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-662-49384-7_15.
- IEE. IEEE P802.3dj Task Force. IEEE P802.3dj 200 Gb/s, 400 Gb/s, 800 Gb/s, and 1.6 Tb/s Ethernet Task Force. <https://www.ieee802.org/3/dj/index.html>. Accessed: 2026-06-25.
- JZC⁺18. Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 96–125, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-96878-0_4.
- JZM19. Haodong Jiang, Zhenfeng Zhang, and Zhi Ma. Tighter security proofs for generic key encapsulation mechanism in the quantum random oracle model. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 227–248, Chongqing, China, May 8–10, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-25510-7_13.

- KHN⁺14. Charlie Kaufman, Paul E. Hoffman, Yoav Nir, Pasi Eronen, and Tero Kivinen. Internet Key Exchange Protocol Version 2 (IKEv2). RFC 7296, October 2014. <https://doi.org/10.17487/RFC7296>.
- KKPP20. Shuichi Katsumata, Kris Kwiatkowski, Federico Pintore, and Thomas Prest. Scalable ciphertext compression techniques for post-quantum KEMs and their applications. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 289–320, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-64837-4_10.
- KLSS23. Duhyeon Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 549–580, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-38554-4_18.
- Kra05. Hugo Krawczyk. HMQV: A high-performance secure Diffie-Hellman protocol. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 546–566, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/11535218_33.
- LBES25. Patrick Longa, Joppe W. Bos, Stephan Ehlen, and Douglas Stebila. FrodoKEM: key encapsulation from learning with errors. Internet-Draft draft-longa-cfrg-frodokem-01, Internet Engineering Task Force, September 2025. Work in Progress. URL: <https://datatracker.ietf.org/doc/draft-longa-cfrg-frodokem-01/>.
- LHT16. Adam Langley, Mike Hamburg, and Sean Turner. Elliptic Curves for Security. RFC 7748, January 2016. <https://doi.org/10.17487/RFC7748>.
- LL23. You Lyu and Shengli Liu. Two-message authenticated key exchange from public-key encryption. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *ESORICS 2023: 28th European Symposium on Research in Computer Security, Part I*, volume 14344 of *Lecture Notes in Computer Science*, pages 414–434, The Hague, The Netherlands, September 25–29, 2023. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-50594-2_21.
- LLM07. Brian A. LaMacchia, Kristin E. Lauter, and Anton Mityagin. Stronger security of authenticated key exchange. In Willy Susilo, Joseph K. Liu, and Yi Mu, editors, *Provable Security, First International Conference, ProvSec 2007, Wollongong, Australia, November 1-2, 2007, Proceedings*, volume 4784 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2007. https://doi.org/10.1007/978-3-540-75670-5_1.
- LPWZ21. Huanhuan Lian, Tianyu Pan, Huige Wang, and Yunlei Zhao. Identity-based identity-concealed authenticated key exchange. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *Computer Security – ESORICS 2021: 26th European Symposium on Research in Computer Security, ESORICS 2021, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part II*, volume 12973 of *Lecture Notes in Computer Science*, pages 651–675, Cham, 2021. Springer. https://doi.org/10.1007/978-3-030-88428-4_32.
- LS15. Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015. <https://doi.org/10.1007/S10623-014-9938-4>.
- LZS⁺26. Songlin Li, Hengchuan Zou, Shiyu Shen, Yifan Dong, and Yunlei Zhao. BW-KEM: Robust and versatile MLWE-based KEM with Barnes-Wall lattices. In Shi Bai and Edoardo Persichetti, editors, *Public-Key Cryptography – PKC 2026*, volume 16551 of *Lecture Notes in Computer Science*, pages 70–102, Cham, 2026. Springer. https://doi.org/10.1007/978-3-032-26731-3_3.
- MAB⁺25. Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jurjen Bos, Jean-Christophe Deneuville, Arnaud Dion, Philippe Gaborit, Jérôme Lacan, Edoardo Persichetti, Jean-Marc Robert, Pascal Véron, and Gilles Zémor. HQC: Hamming Quasi-Cyclic, 2025. Selected by NIST for standardization on Mar. 11, 2025. <https://pqc-hqc.org/>.
- Nat17. National Institute of Standards and Technology (NIST). Post-quantum cryptography standardization. <https://www.nist.gov/pqcrypto>, 2017. Created January 3, 2017; last updated December 11, 2025.
- Nat24a. National Institute of Standards and Technology. Module-lattice-based digital signature standard (FIPS 204). Federal Information Processing Standards Publication 204, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2024. <https://doi.org/10.6028/NIST.FIPS.204>.
- Nat24b. National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard (FIPS 203). Federal Information Processing Standards Publication 203, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2024. <https://doi.org/10.6028/NIST.FIPS.203>.

- Nat24c. National Institute of Standards and Technology. Stateless hash-based digital signature standard (FIPS 205). Federal Information Processing Standards Publication 205, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2024. <https://doi.org/10.6028/NIST.FIPS.205>.
- NC00. Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2 edition, 2000.
- PWZ23. Jiaxin Pan, Benedikt Wagner, and Runzhi Zeng. Lattice-based authenticated key exchange with tight security. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 616–647, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-38554-4_20.
- PZ23. Jiaxin Pan and Runzhi Zeng. A generic construction of tightly secure password-based authenticated key exchange. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VIII*, volume 14445 of *Lecture Notes in Computer Science*, pages 143–175, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore. https://doi.org/10.1007/978-981-99-8742-9_5.
- Res18. Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, August 2018. <https://doi.org/10.17487/RFC8446>.
- RSA78. R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, February 1978. <https://doi.org/10.1145/359340.359342>.
- SAB⁺20. Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, and Damien Stehlé. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- SFG25. Douglas Stebila, Scott Fluhrer, and Shay Gueron. Hybrid key exchange in TLS 1.3. Internet-Draft draft-ietf-tls-hybrid-design-16, Internet Engineering Task Force, September 2025. Work in Progress. URL: <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/16/>.
- Sho94. Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science*, pages 124–134, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. <https://doi.org/10.1109/SFCS.1994.365700>.
- SSW20. Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1461–1480, Virtual Event, USA, November 9–13, 2020. ACM Press. <https://doi.org/10.1145/3372297.3423350>.
- WS25. Guilin Wang and Valery Smyslov. KEM-based Authentication for IKEv2 with Post-quantum Security. IETF Internet-Draft, draft-wang-ipsecme-kem-auth-ikev2-02, October 2025. Work in Progress, last updated 18 Oct 2025. URL: <https://www.ietf.org/archive/id/draft-wang-ipsecme-kem-auth-ikev2-02.html>.
- XLL⁺18. Haiyang Xue, Xianhui Lu, Bao Li, Bei Liang, and Jingnan He. Understanding and constructing AKE via double-key key encapsulation mechanism. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 158–189, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-03329-3_6.
- YZ10. Andrew Chi-Chih Yao and Yunlei Zhao. Deniable internet key exchange. In Jianying Zhou and Moti Yung, editors, *Applied Cryptography and Network Security, 8th International Conference, ACNS 2010, Beijing, China, June 22–25, 2010. Proceedings*, volume 6123 of *Lecture Notes in Computer Science*, pages 329–348, 2010. https://doi.org/10.1007/978-3-642-13708-2_20.
- YZ13. Andrew Chi-Chih Yao and Yunlei Zhao. OAKE: a new family of implicitly authenticated diffie-hellman protocols. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *2013 ACM SIGSAC Conference on Computer and Communications Security, CCS’13, Berlin, Germany, November 4–8, 2013*, pages 1113–1128. ACM, 2013. <https://doi.org/10.1145/2508859.2516695>.
- YZ14. Andrew Chi-Chih Yao and Yunlei Zhao. Privacy-preserving authenticated key-exchange over internet. *IEEE Trans. Inf. Forensics Secur.*, 9(1):125–140, 2014. <https://doi.org/10.1109/TIFS.2013.2293457>.
- Zha12. Mark Zhandry. Secure identity-based encryption in the quantum random oracle model. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 758–775, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-32009-5_44.

- Zha16. Yunlei Zhao. Identity-concealed authenticated encryption and key exchange. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1464–1479, Vienna, Austria, October 24–28, 2016. ACM Press. <https://doi.org/10.1145/2976749.2978350>.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-26951-7_9.
- Zha20. Yunlei Zhao. Identity-based authenticated encryption with identity confidentiality. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve Schneider, editors, *Computer Security – ESORICS 2020: 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14–18, 2020, Proceedings, Part II*, volume 12309 of *Lecture Notes in Computer Science*, pages 633–653, Cham, 2020. Springer. https://doi.org/10.1007/978-3-030-59013-0_31.
- ZZZ⁺26. Yunlei Zhao, Biming Zhou, Zhixiang Zhao, Yifan Dong, Cheng Huang, and Haodong Jiang. Post-Quantum Internet Key Exchange via Authenticated Forward-Secure KEM. In *Advances in Cryptology – CRYPTO 2026*. Springer, 2026.